
**Information technology — Radio
frequency identification for item
management —**

**Part 6:
Parameters for air interface
communications at 860 MHz to 960 MHz**

**AMENDMENT 1: Extension with Type C and
update of Types A and B**

*Technologies de l'information — Identification par radiofréquence
(RFID) pour la gestion d'objets —*

*Partie 6: Paramètres de communications d'une interface d'air entre
860 MHz et 960 MHz*

*AMENDEMENT 1: Extension avec Type C et mise à jour
des Types A et B*

PDF disclaimer

This PDF file may contain embedded typefaces. In accordance with Adobe's licensing policy, this file may be printed or viewed but shall not be edited unless the typefaces which are embedded are licensed to and installed on the computer performing the editing. In downloading this file, parties accept therein the responsibility of not infringing Adobe's licensing policy. The ISO Central Secretariat accepts no liability in this area.

Adobe is a trademark of Adobe Systems Incorporated.

Details of the software products used to create this PDF file can be found in the General Info relative to the file; the PDF-creation parameters were optimized for printing. Every care has been taken to ensure that the file is suitable for use by ISO member bodies. In the unlikely event that a problem relating to it is found, please inform the Central Secretariat at the address given below.

© ISO/IEC 2006

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from either ISO at the address below or ISO's member body in the country of the requester.

ISO copyright office
Case postale 56 • CH-1211 Geneva 20
Tel. + 41 22 749 01 11
Fax + 41 22 749 09 47
E-mail copyright@iso.org
Web www.iso.org

Published in Switzerland

Foreword

ISO (the International Organization for Standardization) and IEC (the International Electrotechnical Commission) form the specialized system for worldwide standardization. National bodies that are members of ISO or IEC participate in the development of International Standards through technical committees established by the respective organization to deal with particular fields of technical activity. ISO and IEC technical committees collaborate in fields of mutual interest. Other international organizations, governmental and non-governmental, in liaison with ISO and IEC, also take part in the work. In the field of information technology, ISO and IEC have established a joint technical committee, ISO/IEC JTC 1.

International Standards are drafted in accordance with the rules given in the ISO/IEC Directives, Part 2.

The main task of the joint technical committee is to prepare International Standards. Draft International Standards adopted by the joint technical committee are circulated to national bodies for voting. Publication as an International Standard requires approval by at least 75 % of the national bodies casting a vote.

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. ISO and IEC shall not be held responsible for identifying any or all such patent rights.

Amendment 1 to ISO/IEC 18000-6:2004 was prepared by Joint Technical Committee ISO/IEC JTC 1, *Information technology*, Subcommittee SC 31, *Automatic identification and data capture techniques*.

Amendment 1 to ISO/IEC 18000-6:2004 covers the extension of ISO/IEC 18000-6 to Type C, to accommodate the latest development of passive RFID technology in the UHF frequency band from 860 MHz to 960 MHz.

Furthermore, it covers changes in order to achieve an improved collision arbitration and a more robust protocol for Type A.

Copyright International Organization for Standardization
Provided by IHS under license with ISO
No reproduction or networking permitted without license from IHS

Information technology — Radio frequency identification for item management —

Part 6:

Parameters for air interface communications at 860 MHz to 960 MHz

AMENDMENT 1: Extension with Type C and update of Types A and B

Page vii, Introduction

Replace the paragraph after the bulleted list with the following paragraphs:

This International Standard specifies the physical and logical requirements for a passive-backscatter, interrogator-talks-first (ITF), radio frequency identification (RFID) system operating in the 860 MHz to 960 MHz frequency range. The system comprises interrogators, also known as readers, and tags, also known as labels.

An interrogator transmits information to a tag by modulating an RF signal in the 860 MHz to 960 MHz frequency range. The tag receives both information and operating energy from this RF signal. Tags are passive, meaning that they receive all of their operating energy from the interrogator's RF waveform.

An interrogator receives information from a tag by transmitting a continuous-wave (CW) RF signal to the tag; the tag responds by modulating the reflection coefficient of its antenna, thereby backscattering an information signal to the interrogator. The system is ITF, meaning that a tag modulates its antenna reflection coefficient with an information signal only after being directed to do so by an interrogator.

Interrogators and tags are not required to talk simultaneously; rather, communications are half-duplex, meaning that interrogators talk and tags listen, or vice versa.

Page vii, Introduction**Add the following to the table of patent holders:**

Contact details	Patent number	Affected clause(s) in this part of ISO/IEC 18000
Alien Technology Corporation ATTN: Dr. John Stephen Smith 18200 Butterfield Blvd Morgan Hill CA 95037 USA Tel: 1-408-782-3900 Fax: 1-408-782-3910 E-mail: ssmith@alientechonology.com	6,933,848 USA, 10/141,489 USA, 11/029,445 USA, 2003/0019929 USA, 10/160,458 USA, 11/132,085 USA, 11/153,030 USA, (divisional of 6,942,155 US), Not yet assigned, (continuation of 11/153,030 USA), 2005/0114326 USA, 10/982,557 USA, Not yet assigned, (continuation of 10/982,557 USA), Not yet assigned, (continuation of 10/982,557 USA), US04/036991 PCT, WO2005048180, 2003/0137403 USA, 10/267,924 USA, WO03032240, 2820082.9 China, 1636039A China, 2801064.3 EU, 2003- 535135 Japan, 091123291 Taiwan, 60/681,656 USA, 10/915,725 USA, US04/025883 PCT, 10/140,557 US,	9.3.2.1.1.1, 9.3.2.1.2.1, 9.3.2.4.7, 9.3.2.10.3.4, Fig. Amd.1-22, Fig. Amd.1-26, 9.3.2.1.1.1, 9.3.2.1.2.1, 9.3.2.4.7, 9.3.2.10.3.4, Fig. Amd.1-22, Fig. Amd.1-26, 9.3.2.7, 9.3.2.10.1.1, 9.3.2.10.2.4, Table Amd.1-28, 9.3.2.7, 9.3.2.10.1.1, 9.3.2.10.2.4, Table Amd.1-28, 9.3.2.1.1.1, 9.3.2.1.2.1, 9.3.2.2, 9.3.2.3, 9.3.2.4.7, 9.3.2.10.3.4, Table Amd.1-16, Fig. Amd.1-22, Fig. Amd.1-26, 9.3.2.1.1.1, 9.3.2.1.2.1, 9.3.2.2, 9.3.2.3, 9.3.2.4.7, 9.3.2.10.3.4, Table Amd.1-16, Fig. Amd.1-22, Fig. Amd.1-26, 9.3.2.2, 9.3.2.3, 9.3.2.4, 9.3.2.4.1, 9.3.2.4.2, 9.3.2.4.8, 9.3.2.5, 9.3.2.8, 9.3.2.9, 9.3.2.10, 9.3.2.10.1.1, 9.3.2.10.2.1, 9.3.2.10.2.2, 9.3.2.10.2.3, 9.3.2.10.2.4, Table Amd.1-18, Table Amd.1-19, Table Amd.1-20, Fig. Amd.1-21, Fig. Amd.1-22, 9.3.2.10, Table Amd.1-18, 9.3.2.9, Fig. Amd.1-22, 9.3.2.2, 9.3.2.3, 9.3.2.4, 9.3.2.4.1, 9.3.2.4.2, 9.3.2.4.8, 9.3.2.5, 9.3.2.8, 9.3.2.9, 9.3.2.10.1.1, 9.3.2.10.2.1, 9.3.2.10.2.2, 9.3.2.10.2.3, 9.3.2.10.2.4, Table Amd.1-18, Table Amd.1-19, Table Amd.1-20, Fig. Amd.1-21, Fig. Amd.1-22, 9.3.2.1.1.1, 9.3.2.1.2.1, 9.3.2.4.7, 9.3.2.10.1.1, 9.3.2.10.2.4, 9.3.2.10.3.4, 9.3.2.10.3.5, Table Amd.1-39, Fig. Amd.1-22, Fig. Amd.1-26, Fig. Amd.1-27, 9.3.2.1.1.1, 9.3.2.1.2.1, 9.3.2.4.7, 9.3.2.10.1.1, 9.3.2.10.2.4, 9.3.2.10.3.4, 9.3.2.10.3.5, Table Amd.1-39, Fig. Amd.1-22, Fig. Amd.1-26, Fig. Amd.1-27, 9.3.2.1.1.1, 9.3.2.1.2.1, 9.3.2.4.7, 9.3.2.10.1.1, 9.3.2.10.2.4, 9.3.2.10.3.4, 9.3.2.10.3.5, Table Amd.1-39, Fig. Amd.1-22, Fig. Amd.1-26, Fig. Amd.1-27, 9.3.2.1.1.1, 9.3.2.1.2.1, 9.3.2.4.7, 9.3.2.10.1.1, 9.3.2.10.2.4, 9.3.2.10.3.4, 9.3.2.10.3.5, Table Amd.1-39, Fig. Amd.1-22, Fig. Amd.1-26, Fig. Amd.1-27, 9.3.2.10.1.1, 9.3.2.10.2.4, 9.3.1.3.2.3, 9.3.1.3.3, Table Amd.1-12, Fig. Amd.1-15, Fig. Amd.1-16, 9.3.2.2, 9.3.2.3, 9.3.2.10, 9.3.2.10.1.1, Table Amd.1-18, Table Amd.1-19, Table Amd.1-20, Fig. Amd.1-21, 9.3.2.2, 9.3.2.3, 9.3.2.10, 9.3.2.10.1.1, Table Amd.1-18, Table Amd.1-19, Table Amd.1-20, Fig. Amd.1-21, 9.3.2.1

Atmel Germany GmbH ATTN: Dr. Bertram Koch Patent Division IPM Theresienstr. 2 74072 Heilbronn Germany Tel: +49(7131)67-3254 Fax: +49(7131)67-2789 Email: Bertram.Koch@hno.atmel.com	10050878.2-31 DE, 09/929703 US, 1508/01 CH, 0113094 FR, 10138217.0 DE, 10/353298 US, 02767283.1-2210 FR, 02767283.1-2210 FI, 50202435.6- 08 DE, 10335009.8 DE, 10/896670 Us, 04017298.3-1237 EP, 200410054468.5 CN, 102004019309.6 DE, 05007835.1- 2210 EP, 11/104435 US, 102004014563.6 DE, 11/087871 us, 200510059501.8 CN, 10158442.3-09 DE, 10256099.4 DE, 02026665.6.1248 EP, 10/308248 US, 10301451.9 DE, 10/753859 US, 04000202.41248 EP, 2004-4143 JP, 200410002008.8 CN, 10325399.8 DE, 0 012309 3-241 E, 10/855866 US, 2004-157635 JP, 10357665.7 DE, 10/753849 US, 04000203.2-2207 EP, 200410002047.8 CN, 2004-5689 JP, 10356259.1-35 DE, 11/002852 US, 200410098306.1 CN, 102005032590.4- 35 DE, PCT/EP2005/007 496 WO, 102004007106.3 DE, 11/058056 US, 200510008328.9 CN	6.3.1.2.2, 6.3.1.2.3, 6.3.1.2.5, 6.3.1.2.8, 6.3.1.5, 6.3.1.3.1, 6.3.1.3, 6.3.2.10.3, 6.3.2.2, 6.3.2.3
BTG International Ltd ATTN: Mr Mathew Frankel Senior Legal Advisor 10 Fleet Place Limeburner Lane London EC4M 7SB UK Tel: +44 20 7575 0000 Fax: +44 20 7575 0010 Website: www.btgplc.com Email: info@btgplc.com	CHINA 98117396.9; EUROPE 98306849.5; JAPAN 241748.98; UNITED STATES OF AMERICA 09/138560; SOUTH AFRICA 987635 ; CHINA 98117442.6; CHINA 200410045765.3; EUROPE 98306943.6; JAPAN 10-243364; UNITED STATES OF AMERICA 09/143711; SOUTH AFRICA 987838; CHINA 98119594.6; JAPAN 272945/98; UNITED STATES OF AMERICA 09/160354; SOUTH AFRICA 988956; CHINA 99111121.4; EUROPE 99304836.2; JAPAN 171775/1999; UNITED STATES OF AMERICA 09/334151; UNITED STATES OF AMERICA 10/827814; SOUTH AFRICA 985286; SOUTH AFRICA 994046; AUSTRIA 99308924.2; BELGIUM 99308924.2; SWITZERLAND 99308924.2; CHINA 99122449.3; GERMANY 99308924.2; EUROPE 99308924.2; SPAIN 99308924.2; FRANCE 99308924.2; ITALY 99308924.2; JAPAN 318426/1999; NETHERLANDS 99308924.2; UNITED STATES OF AMERICA 435467; CHINA 00104540.7; EUROPE 00300753.1; JAPAN 2000-023891; UNITED STATES OF AMERICA 09/495456; SOUTH AFRICA 2000/0668; CHINA 00121458.6; EUROPE 00306298.1; JAPAN 2000- 222353; UNITED STATES OF AMERICA 09/624006; SOUTH AFRICA 2000/3699; EUROPE 01304980.4; JAPAN 2001-177175; UNITED STATES OF AMERICA 09/877438; SOUTH AFRICA 2001/4484; CHINA 01115964.2; EUROPE 01305116.4; JAPAN 2001-177567; UNITED STATES OF AMERICA 09/878403;	???

	SOUTH AFRICA 2001/4921; CHINA 01129277.6; EUROPE 01305265.9; JAPAN 184904/2001; UNITED STATES OF AMERICA 09/881741; SOUTH AFRICA 2001/4922; CHINA 01145439.3; EUROPE 01310061.5; JAPAN 366552/2001; UNITED STATES OF AMERICA 09/996937; SOUTH AFRICA 2001/9960; EUROPE 02782413.5; SOUTH AFRICA 2004/4464; CHINA ; EUROPE ; JAPAN ; UNITED STATES OF AMERICA ; SOUTH AFRICA 2005/07600 CHINA 1123139; UNITED STATES OF AMERICA 6054925; SOUTH AFRICA 987635; UNITED STATES OF AMERICA 6367697; SOUTH AFRICA 987838; CHINA ZL981195994.6; UNITED STATES OF AMERICA 6198381; SOUTH AFRICA 988956; CHINA 99111121.4; UNITED STATES OF AMERICA 6724895; SOUTH AFRICA 994046; AUSTRIA 1001366; BELGIUM 1001366; SWITZERLAND 1001366; GERMANY 69923645.2; EUROPE 1001366; SPAIN 1001366; FRANCE 1001366; ITALY 1001366; NETHERLANDS 1001366; UNITED STATES OF AMERICA 6480143; UNITED STATES OF AMERICA 6346922; SOUTH AFRICA 2000/0668; UNITED STATES OF AMERICA 6388630; SOUTH AFRICA 2000/3699; UNITED STATES OF AMERICA 6867687 ; SOUTH AFRICA 2001/4484; SOUTH AFRICA 2001/4921; UNITED STATES OF AMERICA 6891466 ; SOUTH AFRICA 2001/4922; CHINA ZL01145439.3; UNITED STATES OF AMERICA 6870460; SOUTH AFRICA 2001/9960	
Impinj, Inc. ATTN: Todd E. Humes, CTO ATTN: Gregory T. Kavounas Sr. P.C. Impinj Patent Licensing Department 701 N. 34th Street, Suite 300 Seattle, WA 98103 USA Tel: +1(206)517-5300 Fax: +1(206)517-5262 Email: todd.humes@impinj.com / greg.kavounas@impinj.com	[USA S/N 10 / 915,930]; [PCT / US 2005/ 028180]; [USA S/N 10 / 890,662]; [EP S/N 5103959.2]; [USA S/N 10 / 824,049]; [PCT / US 2004/ 037668]; [USA S/N 10 / 967,996]; [USA S/N 10 / 985,518]; [PCT / US 2004/ 037387]; [USA S/N 11 / 031,459]; [USA S/N 11 / 031,471]; [USA S/N 11 / 033,028]	9.3.1.2.3; Figure Amd. 1-4; 9.3.1.2.8; Figure Amd. 1-7; Annex J; 9.3.1.3 and subsections (especially 9.3.1.3.2 and 9.3.1.3.3); 9.3.2.10.2.1; 9.3.1.2.8; Figure Amd. 1-7; 9.3.1.3.2.2; Figure Amd. 1-14; Annex J; 9.3.1.3 and subsections (especially 9.3.1.3.2 and 9.3.1.3.3); 9.3.2.10.2.1; 9.3.1.2.8; Figure Amd. 1-7; Annex D; Table D.1; 9.3.2.1; 9.3.2.10.1.1; Table Amd.1-19; 9.3.2.10.3.2; Table Amd. 1-32; 9.3.2.10.3.3; Table Amd. 1-34; 9.3.2.10.3.7; Table Amd. 1-44; 9.3.2.10.3.8; Table Amd. 1-46; 9.3.2.10; Table Amd. 1-18; plus all uses of the header "Code" specified in Table Amd. 1-18 throughout the specification; 2.3, 4.1.5 ("cover-coding"); 9.3.2.5; 9.3.2.9; 9.3.2.10.3.3; Table Amd. 1-34; 9.3.2.10.3.4; Table Amd. 1-36; Figure Amd.1-26; 9.3.2.10.3.6; Table Amd. 1-42; Figure Amd. 1-28

Intermec IP Corp ATTN: Ronald D. Payne Legal Department 6001 36th Avenue West Everett, WA 98203 Tel: +1(425)348.2756 Fax: +1(425)348.2703 Email: ron.payne@intermec.com	5,673,037 USA; 69530547.6 DE; 0702323 EP; EP0702323 FR; EP0702323 GB; 204748 KR; 318306 TW; 6,172,596 USA; 6,400,274 USA; 6,404,325 USA; 4,786,907 USA; 1272787 CA; 0254954 EP; 1435/1996 HK; 2644496 JP; 115902 KR 4,739,328 USA; 5,030,807 USA; 629153 AU; 2033868 CA; 0438250 EP; 0438250 GB; 5,777,561 USA; 5,828,693 USA; 97916151.0 EP; 5,850,181 USA; 5,912,632 USA; 5,995,019 USA; 6,400,274 USA; 6,429,775 USA; 6,288,629 USA; 6,812,841 USA; US03/00755 PC; 03702060.9 EP; 20050179521 USA; 20040189443 USA ; 10/662950 USA;	Type C 9.1.1 Physical layer Table 3: Tag:7e Subcarrier Frequency Tag:7g Subcarrier Modulation Tag:7h Duty Cycle Tag:8 Data Coding Tag:9 Bit Rate 9.3.1.3 Tag-to-interrogator (T=>R) communications 9.3.1.3.2 Data encoding 9.3.1.3.2.3 Miller-modulated subcarrier Figure Amd.1-16 — Subcarrier sequences 9.3.1.3.2.4 Miller subcarrier preamble 9.3.1.3.3 Tag supported Tari values and backscatter link rates Table Amd.1-11 — Tag-to-interrogator link frequencies Annex I Dense- and multiple-interrogator channelised signaling Type A 7.3.1 Tag memory organization 7.4.7 Command codes and CRC 7.8.9 Write_single_block 7.8.10 Write_multiple_blocks 7.8.11 Lock_blocks 7.8.12 Write_AFI 7.8.13 Lock_AFI 7.8.14 Write_DSFD command 7.8.15 Lock_DSFD Type B 8.2.4.2 Detailed Command Processing 8.2.7.6 Command codes and format 8.2.7.9.11 WRITE 8.2.7.9.12 WRITE4BYTE 8.2.7.9.13 LOCK 8.2.7.9.15 WRITE_MULTIPLE 8.2.7.9.16 WRITE4BYTE_MULTIPLE Annex B Memory mapping for Type B Type C 9.1.2 Tag-identification layer Table 4 — Tag inventory and access parameters P:5Write Size P:7Write Transaction Time 9.3.2.1.1 Reserved Memory 9.3.2.1.2 Ull Memory 9.3.2.1.3 TID memory 9.3.2.1.4 User memory 9.3.2.1.1.1 Kill password 9.3.2.1.1.2 Access password 9.3.2.1.2.2 Protocol-control (PC) bits 9.3.2.1.2.1 CRC-16 9.3.2.1.2.3 Ull for EPCglobal™ applications 9.3.2.1.2.4 Ull for non-EPCglobal™ applications 9.3.2.9 Accessing individual tags Table Amd.1-17 — Access commands and tag states in which they are permitted
--	--	---

		9.3.2.10.3.3 Write (mandatory) 9.3.2.10.3.4 Kill (mandatory) 9.3.2.10.3.5 Lock (mandatory) 9.3.2.10.3.7 BlockWrite (optional) 9.3.2.10.3.8 BlockErase (optional) F.10 Command response: Write F.11 Command response: Kill F.12 Command response: Lock F.14 Command response: BlockWrite F.15 Command response: BlockErase Type C 9.1.1 Physical layer Table 3 — Tag-to-interrogator (T=>R) communications: Tag:7e Subcarrier Frequency Tag:7g Subcarrier Modulation Tag:7h Duty Cycle Tag:8 Data Coding Tag:9 Bit Rate 9.3.1.3 Tag-to-interrogator (T=>R) communications 9.3.1.3.2 Data encoding 9.3.1.3.2.3 Miller-modulated subcarrier Figure Amd.1-16 — Subcarrier sequences 9.3.1.3.2.4 Miller subcarrier preamble 9.3.1.3.3 Tag supported Tari values and backscatter link rates Table Amd.1-11 — Tag-to-interrogator link frequencies Annex I Dense- and multiple-interrogator channelised signaling Types A, B, C Table 2 — Interrogator-to-tag (R=>T) communications: Int:1d Frequency Hop Rate (frequency-hopping [FHSS] systems) Int:1e Frequency Hop Sequence (frequency-hopping [FHSS] systems) Table 3 — Tag-to-interrogator (T=>R) communications: Tag:1d Frequency Hop Rate (frequency-hopping [FHSS] systems) Tag:1e Frequency Hop Sequence (frequency-hopping [FHSS] systems) Types A, B 6.4 Frequency hopping carrier rise and fall times Type C 9.3.1.2.9 Frequency-hopping spread-spectrum waveform 9.3.1.2.10 Frequency-hopping spread-spectrum channelisation Annex I Dense- and multiple-interrogator channelised signaling Bibliography: ETSI EN 300 220 ETSI EN 302 208-1
--	--	--

		ETSI EN 302 208-2 US Code of Federal Regulations (CFR) Title 47, Chapter I, Part 15 Types A, B, C Table 2 — Interrogator-to-tag (R=>T) communications: Int:1d Frequency Hop Rate (frequency-hopping [FHSS] systems) Int:1e Frequency Hop Sequence (frequency-hopping [FHSS] systems) Table 3 — Tag-to-interrogator (T=>R) communications: Tag:1d Frequency Hop Rate (frequency-hopping [FHSS] systems) Tag:1e Frequency Hop Sequence (frequency-hopping [FHSS] systems) Types A, B 6.4 Frequency hopping carrier rise and fall times Type C 9.3.1.2.9 Frequency-hopping spread- spectrum waveform 9.3.1.2.10 Frequency-hopping spread-spectrum channelisation Annex I Dense- and multiple- interrogator channelised signaling Bibliography: ETSI EN 300 220 ETSI EN 302 208-1 ETSI EN 302 208-2 US Code of Federal Regulations (CFR) Title 47, Chapter I, Part 15 Type A, B 6.5.5 Message Format 6.5.6 Return preamble Type C 9.3.1.3.3 Tag supported Tari values and backscatter link rates Type C 9.1.1 Physical layer Table 3: Tag:7e Subcarrier Frequency Tag:7g Subcarrier Modulation Tag:7h Duty Cycle Tag:8 Data Coding Tag:9 Bit Rate 9.3.1.3 Tag-to-interrogator (T=>R) communications 9.3.1.3.2 Data encoding 9.3.1.3.2.3 Miller-modulated subcarrier Figure Amd.1-16 — Subcarrier sequences 9.3.1.3.2.4 Miller subcarrier preamble 9.3.1.3.3 Tag supported Tari values and backscatter link rates Table Amd.1-11 — Tag-to-interrogator link frequencies Annex I Dense- and multiple-
--	--	---

		interrogator channelised signaling CLAIMS 11-et seq Types A, B, C Table 2: Int:1d Frequency Hop Rate (frequency-hopping [FHSS] systems) Int:1e Frequency Hop Sequence (frequency-hopping [FHSS] systems) Table 3: Tag:1d Frequency Hop Rate (frequency-hopping [FHSS] systems) Tag:1e Frequency Hop Sequence (frequency-hopping [FHSS] systems) Types A, B 6.4 Frequency hopping carrier rise and fall times Type C 9.3.1.2.9 Frequency-hopping spread- spectrum waveform 9.3.1.2.10 Frequency-hopping spread-spectrum channelisation Annex I Dense- and multiple- interrogator channelised signaling Bibliography: ETSI EN 300 220 ETSI EN 302 208-1 ETSI EN 302 208-2 US Code of Federal Regulations (CFR) Title 47, Chapter I, Part 15 Type A 7.8.9 Write single block 7.8.10 Write multiple blocks 7.8.11 Lock single block 7.8.12 Write AFI 7.8.13 Lock AFI 7.8.14 Write DSFID command 7.8.15 Lock DSFID Type B 8.2.1.3.3 WRITE_OK 8.2.7.9.11 WRITE 8.2.7.9.12 WRITE4BYTE 8.2.7.9.13 LOCK 8.2.7.9.14 QUERY_LOCK 8.2.7.9.15 WRITE_MULTIPLE 8.2.7.9.16 WRITE4BYTE_MULTIPLE Type C 9.3.2.10.3.3 Write (mandatory) 9.3.2.10.3.4 Kill (mandatory) 9.3.2.10.3.5 Lock (mandatory) 9.3.2.10.3.7 BlockWrite (optional) 9.3.2.10.3.8 BlockErase (optional) Annex E State-transition tables Type A Tag state storage Type B 8.2.1.3.2 Data exchange status bit (DE_SB)
--	--	---

		Type C persistent memory or persistent flag 9.3.2.2 Sessions and inventoried flags 9.3.2.3 Selected flag 9.3.2.7 Selecting tag populations 9.3.2.8 Inventorying tag populations 9.3.2.10.1.1 Select (mandatory) 9.3.2.10.2.1 Query (mandatory) 9.3.2.10.2.2 QueryAdjust (mandatory) 9.3.2.10.2.3 QueryRep (mandatory) Annex E State-transition tables Annex F Command-Response Tables
Magellan Technology Pty Limited ATTN: Ms E. J. Angus Company Secretary 65 Johnston St ANNANDALE NSW 2038 Australia Tel: +61 2 9562 9800 Fax: +61 2 9518 7620 E-mail : jeana@magtech.com.au	US10/204159 USA , AU3711301 Australia , WO0165712 International PCT, EP1266458 Europe , 2003526148 Japan	6, 7, 8, 9
OMRON Corporation ATTN: Atsushi Hisano Business Development Group Planning Division Business Management Department Gate City Ohsaki West Tower 14F, 1-11-1, Ohsaki, Shinagawa-ku, Tokyo, 141-0032 Japan Tel: +81(3)5435-2004 Fax: +81(3)5435-2025 Email: atsushi_hisano@omron.co.jp	2949728/ Japan	6.3.1.3.3
Koninklijke Philips Electronics N.V. ATTN: Harald Röggla Triester Strasse 64 1101 Vienna Austria Tel: 43(1)60101 1469 Fax: +43(1)60101 1101 Email: harald.roeggla@philips.com	AT 401 127/ AT;EP 0 669 591 / BE, CH, De, DK, ES, FR, GB, IT, LI, NL, SE;EP 1 038 257 /AT, DE, ES, FR, GB, IT;IN /PCT/00/00034 / IN;US10/382262;JP00-561579 / JP;	2.5.1
Symbol Technologies, Inc. ATTN: Aaron Bernstein, VP of IP, Legal Department One Symbol Plaza, MS A6 Holtsville, NY 11742 USA Tel: 1-800-927-9626 Fax 1-631-738-4110 E-mail aaron.bernstein@symbol.com	6,784,813 (USA); 10/926,269 (USA); 10/932,279 (USA); 761843 (AU); 2,310,623 (CA); 98812462.9 (CN); 1031120 (DE); 1031120 (EP); 1320062 (EP); 1031120 (FR); 1031120 (GB); 1032468 (HK); 136,220 (IL); 1031120 (IT); 2000-521687 (JP); 10/688,535 (USA); 2003282941 (AU); TBA (CN); TBA (EP); TBA (JP); TBA (KR); 10/725,010 (USA); 10/073,000 (USA); 10/072,885 (USA); 10/693,687 (USA); 2003286702 (AU); TBA (CA); TBA (CN); 03777912.1 (EP); TBA (JP); 10- 2005-7007116 (KR); TBA (SG); 10/927,775 (USA); US04/027999(WO); 2,503,407 (CA); TBA (EP); TBA (JP); 0480001172.5 (CN); 1020057007371 (KR); 2004269728 (AU)	Type C

TAGSYS, S.A. ATTN: Alastair McArthur, CTO 180 Chemin de Saint Lambert 13821 La Penne sur Huveaune France Tel: +33(0)4 91 27 57 01 Fax: +33(0)4 91 27 57 00 E-mail: alastair.mcarthur@tagsys.net	US 6,641,036; EP 1232471; WO 02054365; WO 0141043; EP 0953181; US 6,538,564	???
Texas Instruments Incorporated ATTN: Mrs. Scharlene Franks Contract Manager, RFID Business Group 6550 Chase Oaks Boulevard, MS 8470 Plano, Texas 75023 USA Tel: +1(214) 567-8830 Fax: +1(214) 567-2409 Email: s-franks@ti.com	NONE	Not applicable
Zebra Technologies Corporation ATTN: Eric McAlpine IP Counsel Legal Dept. 333 Corporate Woods Parkway Vernon Hills, IL 60061 USA Phone: 1.847.793.5640 Fax: 1.847.955.4514 E-mail: emcalpine@zebra.com	92/10006 AUSTRALIA; BW/A/97/00142 BOTSWANA; 2058692 CANADA; 02017523.8 EPC; 95112753.9 EPC (AT, BE, CH, DE, DK, ES, FR, GB, GR, IT, LU, NL, PT, SE); 92300041.8 EPC (AT, BE, CH, DE, DK, ES, FR, GB, GR, IT, LU, NL, PT, SE); 91/346821 JAPAN; 9609578-1 SINGAPORE; 92/0039 SOUTH AFRICA; 08/976949 USA; 07/816893 USA; 08/976948 USA; 326618 ARGENTINA; 93/50781 AUSTRALIA; BW/A/97/00141 BOTSWANA; PI9304761-4 BRAZIL; 2103288 CANADA; 93121410 CHINA; 01117486.9 EPC; 97104997.8 EPC (AT, BE, CH, DE, DK, ES, FR, GB, GR, IE, IT, NL, PT, SE); 97104998.6 EPC (AT, BE, CH, DK, DE, ES, FR, GB, GR, IE, IT, NL, PT, SE); 93309232.2 EPC (AT, BE, CH, ES, FR, GB, GR, IE, IT, NL, PT, SE); 0598624UKREG GUERNSEY; 98109976.2 HONG KONG; 93/01312 INDIA; 107636 ISRAEL; 47219/03 JAPAN; 93/289591 JAPAN; 0598624UKREG JERSEY; 93/02408 MALAYSIA; 9307214 MEXICO; 250219 NEW ZEALAND; 314270 NEW ZEALAND; 314269 NEW ZEALAND; 19934177 NORWAY; 19993092 NORWAY; 19993091 NORWAY; 93051222 RUSSIAN FEDERATION; 9609092-3 SINGAPORE; 93/8624 SOUTH AFRICA; 93/024833 SOUTH KOREA; 82109793 TAIWAN; 93003717 UKRAINE; 08/154329 USA; 08/580913 USA; 95/16530 AUSTRALIA; 93/44940 AUSTRALIA; BW/A/97/00144 BOTSWANA; 2104829 CANADA; 93118806 CHINA; 93306790.2 EPC (AT, BE, CH, DE, DK, ES, FR, GB, GR, IE, IT, LU, MN, NL, PT, SE); 93/00956 INDIA; 108621 ISRAEL; 93/211454 JAPAN; 93049090 RUSSIAN FEDERATION; 9608519-6	???

	SINGAPORE; 93/6267 SOUTH AFRICA; 94/03632 SOUTH KOREA; 83101248 TAIWAN; 93003224 UKRAINE; 08/665363 USA; 08/111430 USA; 99/10433 AUSTRALIA; 2310241 CANADA; 02021446.6 EPC; 05017862.3 EPC; 98952886.4 EPC (AT, BE, CH, CY, DE, DK, ES, FI, FR, GB, GR, IE, IT, LU, MN, NL, PT, SE); 2000-521391 JAPAN; 504144 NEW ZEALAND; 98/10356 SOUTH AFRICA; 09/570951 USA; 98/74401 AUSTRALIA; PI9808714-2 BRAZIL; 2289207 CANADA; 98805116.8 CHINA; 04012409.1 EPC; 98921613.0 EPC (AT, BE, CH, DE, DK, ES, FI, FR, GB, GR, IE, IT, NL, SE); 02024117.0 EPC (BE, DE, DK, ES, FI, FR, GB, IE, IT, NL, SE); 132297 ISRAEL; 98/548931 JAPAN; 9910381 MEXICO; 337966 NEW ZEALAND; 99126440 RUSSIAN FEDERATION; 9904942-1 SINGAPORE; 98/10257 SOUTH AFRICA; 1999-7010475 SOUTH KOREA; 10/635683 USA; 09/415234 USA; PCT/GB04/004505 PCT; PCT/GB05/000110 PCT; 0400968.4 UNITED KINGDOM; PCT/GB05/000345 PCT; 0402667.0 UNITED KINGDOM; 2,148,145 CANADA; 08/430825 USA BOTSWANA BW/P03/00031; MALAYSIA MY-109809-A; BOTSWANA BW/P/03/00033; TAIWAN NI-84612; TAIWAN NI-67003; SOUTH AFRICA 98/10356; SOUTH AFRICA 98/10257; SOUTH AFRICA 93/8624; SOUTH AFRICA 93/6267; SOUTH AFRICA 92/0039; JERSEY 622; UKRAINE 37182; UKRAINE 44215; SINGAPORE 48262; SINGAPORE 48423; SINGAPORE 55818; SINGAPORE 68358; ISRAEL 107636; ISRAEL 108621; ISRAEL 132297; MEXICO 186161; INDIA 187341; INDIA 188258; NEW ZEALAND 250219; ARGENTINA 250637; NORWAY 307590; SOUTH KOREA 309651; NORWAY 310261; NORWAY 310262; NEW ZEALAND 314269; NEW ZEALAND 314270; SOUTH KOREA 316754; NEW ZEALAND 337966; EPC (AT, BE, CH, DE, DK, ES, FR, GB, GR, IT, LU, NL, PT, SE) 0494114; NEW ZEALAND 504144; EPC (AT, BE, CH, DE, DK, ES, FR, GB, GR, IE, IT, LU, MN, NL, PT, SE) 0585132; EPC (AT, BE, CH, ES, FR, GB, GR, IE, IT, NL, PT, SE) 0598624; GUERNSEY 0598624UKREG; AUSTRALIA 656088; AUSTRALIA 658857; AUSTRALIA 670402; AUSTRALIA 676853; EPC (AT, BE, CH, DE, DK, ES, FR, GB, GR, IT, LU, NL, PT, SE) 0685825; AUSTRALIA
--	--

	745638; AUSTRIA 764975; EPC (AT, BE, CH, DE, DK, ES, FR, GB, GR, IE, IT, NL, PT, SE) 0789253; EPC (AT, BE, CH, DK, DE, ES, FR, GB, GR, IE, IT, NL, PT, SE) 0789254; EPC (AT, BE, CH, DE, DK, ES, FI, FR, GB, GR, IE, IT, NL, SE) 0983569; HONG KONG 1009295; EPC (AT, BE, CH, CY, DE, DK, ES, FI, FR, GB, GR, IE, IT, LU, MN, NL, PT, SE) 1031046; EPC (BE, DE, DK, ES, FI, FR, GB, IE, IT, NL, SE) 1280092; EPC 1291671; CANADA 2058692; CANADA 2103288; CANADA 2104829; RUSSIAN FEDERATION 2126165; RUSSIAN FEDERATION 2156540; RUSSIAN FEDERATION 2238585; JAPAN 3100716; JAPAN 3290003; USA 5519381; USA 5537105; USA 5557280; USA 5680459; USA 5699066; USA 5726630; USA 5966083; USA 5995017; USA 6661336; USA 6784787; BRAZIL 9304761-4; CHINA 93118806.7; CHINA 93121410.6	
--	---	--

Page 1, clause 1**Replace paragraph 2 with the following paragraphs:**

This part of ISO/IEC 18000 contains one mode with three types. All three types are ITF. Type A uses Pulse-Interval Encoding (PIE) in the forward link and an adaptive ALOHA collision-arbitration algorithm. Type B uses Manchester in the forward link and an adaptive binary-tree collision-arbitration algorithm. Type C uses Pulse-Interval Encoding (PIE) in the forward link and a random slotted collision-arbitration algorithm. The detailed technical differences between the three types are shown in the associated parameter tables.

This part of ISO/IEC 18000 specifies

- physical interactions (the signalling layer of the communication link) between interrogators and tags,
- interrogator and tag operating procedures and commands, and
- the collision arbitration scheme used to identify a specific tag in a multiple-tag environment.

Page 1, clause 2**Replace the whole clause with the following clause:****2 Conformance****2.1 Claiming conformance**

To claim conformance with this part of ISO/IEC 18000, an interrogator or tag shall comply with all relevant clauses of this part of ISO/IEC 18000, except those marked as “optional”. The interrogator or tag shall also operate within local radio regulations, which may further restrict operation.

Relevant conformance test methods will be provided in a future Technical Report (ISO/IEC TR 18047-6).

Conformance may also require a license from the owner of any intellectual property utilized by said device.

2.2 Interrogator conformance and obligations

To conform to this part of ISO/IEC 18000, an interrogator shall

- support at least one type of A, B or C – it may optionally support two or all three;
- implement the mandatory commands defined in this part of ISO/IEC 18000;
- modulate/transmit and receive/demodulate a sufficient set of the electrical signals defined in the signalling layer of this part of ISO/IEC 18000 to communicate with conformant tags; and
- operate within applicable local regulations.

To conform to this part of ISO/IEC 18000, an interrogator may

- implement any subset of the optional commands defined in this part of ISO/IEC 18000, and
- implement any proprietary and/or custom commands in conformance with this part of ISO/IEC 18000.

To conform to this part of ISO/IEC 18000, the interrogator shall not

- implement any command that conflicts with this part of ISO/IEC 18000, or
- require using an optional, proprietary, or custom command to meet the requirements of this part of ISO/IEC 18000.

2.3 Tag conformance and obligations

To conform to this part of ISO/IEC 18000, a tag shall

- support at least one type of A, B or C – it may optionally support two or all three;
- operate over the frequency range from 860 MHz to 960 MHz, inclusive;
- implement the mandatory commands defined in this part of ISO/IEC 18000;
- modulate a backscatter signal only after receiving the requisite command from an interrogator; and
- conform to local radio regulations.

To conform to this part of ISO/IEC 18000, a tag may

- implement any subset of the optional commands defined in this part of ISO/IEC 18000; and
- implement any proprietary and/or custom commands as defined in clauses 7, 8 and 9 of this part of ISO/IEC 18000.

To conform to this part of ISO/IEC 18000, a tag shall not

- implement any command that conflicts with this part of ISO/IEC 18000;
- require using an optional, proprietary, or custom command to meet the requirements of this part of ISO/IEC 18000; or
- modulate a backscatter signal unless commanded to do so by an interrogator using the signalling layer defined in ISO/IEC 18000.

Page 2, clause 3

Insert the following paragraph as first paragraph:

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

Page 2, clause 3

Delete the following normative reference and associated footnote:

ISO/IEC 19762-3, *Information technology — Automatic identification and data capture (AIDC) techniques — Harmonized vocabulary — Part 3: Radio frequency identification (RFID)*¹⁾

Page 2, clause 3**Insert the following normative references:**

ISO/IEC 15961, *Information technology — Radio frequency identification (RFID) for item management — Data protocol: application interface*

ISO/IEC 15962, *Information technology — Radio frequency identification (RFID) for item management — Data protocol: data encoding rules and logical memory functions*

ISO/IEC 15963, *Information technology — Radio frequency identification for item management — Unique identification for RF tags*

ISO/IEC 18000-1, *Information technology — Radio frequency identification for item management — Part 1: Reference architecture and definition of parameters to be standardized*

ISO/IEC 19762 (all parts), *Information technology — Automatic identification and data capture (AIDC) techniques — Harmonized vocabulary*

EPCglobal™ Tag Data Standards version 1.3 and above, EPCglobal™

Page 2, clause 4.1**Replace the introductory paragraph with the following:**

For the purposes of this document, the terms and definitions given in ISO/IEC 19762 (all parts) and the following apply.

Page 2, clause 4.1**Insert the following terms and definitions:****4.1.3****command set**

set of commands used to explore and modify a tag population

4.1.4**continuous wave**

typically sinusoid at a given frequency, but more generally any interrogator waveform suitable for powering a passive tag without amplitude and/or phase modulation of sufficient magnitude to be interpreted by a tag as transmitted data

4.1.5**cover-coding**

method by which an interrogator obscures information that it is transmitting to a tag; to cover-code data or a password an interrogator first requests a random number from the tag, then performs a bit-wise EXOR of the data or password with the received random number, and, finally, transmits the cover-coded (also called ciphertext) string to the tag; the tag uncovers the data or password by performing a bit-wise EXOR of the received cover-coded string with the original random number

4.1.6**dense-interrogator environment**

operating environment (4.1.13) within which most or all of the available channels are occupied by active interrogators

EXAMPLE 25 active interrogators operating in 25 available channels.

4.1.7

dense-interrogator mode

set of interrogator-to-tag and tag-to-interrogator signalling parameters used in dense-interrogator environments

4.1.8

EPCglobal™ Application

application whose usage denotes an acceptance of EPCglobal™ standards and policies

NOTE Compare non-EPCglobal™ Application (4.1.12).

4.1.9

inventoried flag

flag that indicates whether a tag may respond to an interrogator; tags maintain a separate **inventoried** flag for each of four sessions, where each flag has symmetric *A* and *B* values within any given session, and in which interrogators typically inventory tags from *A* to *B* followed by a re-inventory of tags from *B* back to *A* (or vice versa)

4.1.10

inventory round

period initiated by a *Query* command and terminated by either a subsequent *Query* command (which also starts a new inventory round) or a *Select* command

4.1.11

multiple-interrogator environment

operating environment (4.1.13) within which a modest number of the available channels are occupied by active interrogators

EXAMPLE 5 active interrogators operating in 25 available channels.

4.1.12

non-EPCglobal™ Application

application whose usage does not denote an acceptance of EPCglobal™ standards and policies

NOTE Compare EPCglobal™ Application (4.1.8).

4.1.13

operating environment

region within which an interrogator's RF transmissions are attenuated by less than 90 dB

NOTE 1 In free space, the operating environment is a sphere whose radius is approximately 1000 m, with the interrogator located at the centre.

NOTE 2 In a building or other enclosure, the size and shape of the operating environment depends on factors such as the material properties and shape of the building, and may be less than 1000 m in certain directions and greater than 1000 m in other directions

4.1.14

operating procedure tag-identification layer

set of functions and commands used by an interrogator to identify and modify tags

4.1.15

permalock or permalocked

memory location whose lock status is unchangeable (i.e. the memory location is permanently locked or permanently unlocked)

4.1.16**persistent memory or persistent flag**

memory or flag value whose state is maintained during a brief loss of tag power

4.1.17**physical layer**

data coding and modulation waveforms used in interrogator-to-tag and tag-to-interrogator signalling

4.1.18**random-slotted collision arbitration**

collision-arbitration algorithm where tags load a random (or pseudo-random) number into a slot counter, decrement this slot counter based on interrogator commands, and reply to the interrogator when their slot counter reaches zero

4.1.19**session**

inventory process comprising an interrogator and an associated tag population; an interrogator chooses one of four sessions and inventories tags within that session; the interrogator and associated tag population operate in one and only one session for the duration of an inventory round (4.1.10); for each session, tags maintain a corresponding **inventoried** flag; sessions allow tags to keep track of their inventoried status separately for each of four possible time-interleaved inventory processes, using an independent **inventoried** flag for each process

4.1.20**single-interrogator environment**

operating environment (4.1.13) within which there is a single active interrogator at any given time

4.1.21**singulation**

identifying an individual tag in a multiple-tag environment

4.1.22**slot**

point in an inventory round at which a tag may respond, which corresponds to the value output by a tag's slot counter

NOTE Tags reply when their slot (i.e. the value in their slot counter) is zero.

Page 2, clause 4.2

Insert the following symbols:

BLF	Backscatter-link frequency ($BLF = 1/T_{pri} = DR/TR_{cal}$)
DR	Divide ratio
FDM	Frequency-division multiplexing
FT	Frequency tolerance
M_h	RF signal envelope ripple (overshoot)
M_{hh}	FHSS signal envelope ripple (overshoot)
M_{hl}	FHSS signal envelope ripple (undershoot)
M_{hs}	FHSS signal level during a hop
M_l	RF signal envelope ripple (undershoot)
M_s	RF signal level when OFF

Q	Slot-count parameter. Q is an integer in the range (0,15)
R=>T	Interrogator-to-tag (reader-to-tag)
RTcal	Interrogator-to-tag (reader-to-tag) calibration symbol
T=>R	Tag-to-interrogator (tag-to-reader)
T ₁	Time from interrogator transmission to tag response
T ₂	Time from tag response to interrogator transmission
T ₃	Time an interrogator waits, after T ₁ , before it issues another command
T ₄	Minimum time between interrogator commands
T _f or T _{f,10-90%}	RF signal envelope fall time
T _{hf}	FHSS signal envelope fall time
T _{hr}	FHSS signal envelope rise time
T _{hs}	Time for an FHSS signal to settle to within a specified percentage of its final value
T _{pri}	Backscatter-link pulse-repetition interval (T _{pri} = 1/BLF = TRcal/DR)
T _r or T _{r,10-90%}	RF signal envelope rise time
TRcal	Tag-to-interrogator (tag-to-reader) calibration symbol
T _s	Time for an RF signal to settle to within a specified percentage of its final value
UII	Unique Item Identifier
x _{fp}	floating-point value
xxxx ₂	binary notation
xxxx _h	hexadecimal notation

Page 2, clause 4.2

Replace the following symbols:

Tari	Reference time interval for a data-0 in interrogator-to-tag signalling
-------------	--

Page 3, clause 4.3

Insert the following abbreviated terms:

Ciphertext	Information that is cover-coded
CW	Continuous wave
dBch	Decibels referenced to the integrated power in the reference channel
DSB	Double sideband
DSB-ASK	Double-sideband amplitude-shift keying
EPC™	Electronic product code
FCC	Federal Communications Commission
Handle	16-bit tag-authentication number
ITF	Interrogator-talks-first (Note: the common usage is RTF (Reader-talks-first) but the more precise term is ITF, which is used throughout the document.)
NSI	Numbering system identifier
Pivot	Decision threshold differentiating an R=>T data-0 symbol from a data-1 symbol

Plaintext	Information that is not cover-coded
ppm	parts per million
PR-ASK	Phase-reversal amplitude shift keying
RFU	Reserved for future use
RN16	16-bit random or pseudo-random number
RNG	Random or pseudo-random number generator
SSB	Single sideband
SSB-ASK	Single-sideband amplitude-shift keying
TID	Tag-identification or tag identifier, depending on context
Word	16 bits

Page 4, before clause 5

Insert the following new subclause:

4.4 Notation

This part of ISO/IEC 18000 uses the following notational conventions for type C:

- States and flags are denoted in bold. Example: **ready**.
- Commands are denoted in italics. Variables are also denoted in italics. Where there might be confusion between commands and variables, this specification will make an explicit statement. Example: *Query*.
- Command parameters are underlined. Example: Pointer.
- For logical negation, labels are preceded by '~'. Example: If **flag** is true, then **~flag** is false.
- The symbol, R=>T, refers to commands or signalling from an interrogator to a tag (reader-to-tag).
- The symbol, T=>R, refers to commands or signalling from a tag to an interrogator (tag-to-reader).

Page 4, clause 5.1

Replace clause 5.1 by the following clause:

5.1 General

This part of ISO/IEC 18000 specifies three communication types: Type A, Type B and Type C.

Figure 1, Figure 2, and Figure 3 show the architecture of each of the three interrogator types. Figure Amd.1-1, Figure Amd.1-2, and Figure Amd.1-3 show the architecture of each of the three tag types.

Table 1 provides an overview of the three communication types showing the protocol blocks from interrogator input (demodulator) to interrogator output (modulator) independent of the communication flow.

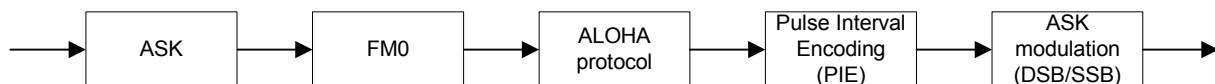


Figure 1 — Type A interrogator architecture

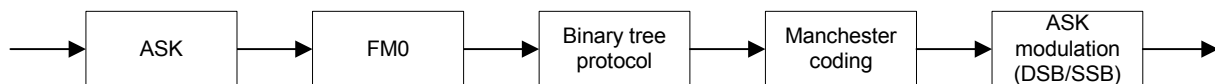


Figure 2 — Type B interrogator architecture

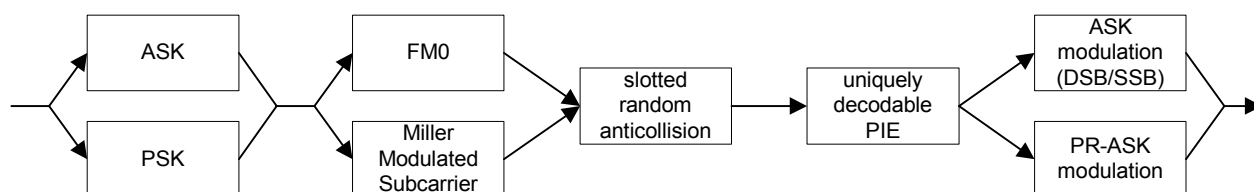


Figure 3 — Type C interrogator architecture

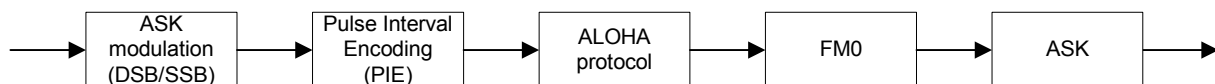


Figure Amd.1-1 — Type A tag architecture

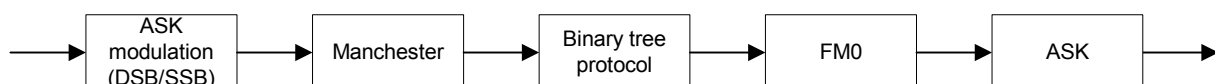


Figure Amd.1-2 — Type B tag architecture

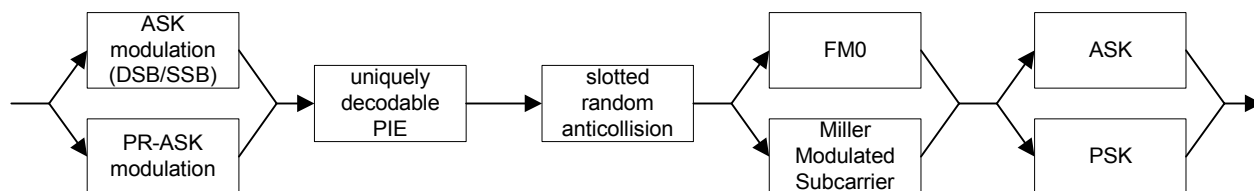


Figure Amd.1-3 — Type C tag architecture

Table 1 — Overview of Types A, B and C

Parameter	Type A	Type B	Type C
Forward link encoding	PIE	Manchester	Uniquely decodeable PIE
Modulation depth	27% to 100%	30.5% or 100%	80% to 100%
Data rate	33 kbit/s (assuming equiprobable data)	10 or 40 kbit/s (according to local regulations)	26.7 kbit/s to 128 kbit/s (assuming equiprobable data)
Return link encoding	FM0	FM0	FM0, Miller subcarrier
Collision arbitration	ALOHA / FST	Binary Tree	random slotted collision-arbitration
Tag unique identifier	64 bits (40 bit SUID)	64 bits	Variable: minimum 16 bits, maximum 496 bits
Memory addressing	Blocks up to 256 bits	Byte blocks, 1,2,3 or 4 byte writes	Bit, word (16-bit), or vendor-defined block boundaries (depending on command)
Error detection, forward link	5 bit CRC for all commands (with an additional 16-bit CRC appended for all long commands)	16-bit CRC	16-bit CRC, except a 5-bit CRC for the <i>Query</i> command
Error detection, return link	16-bit CRC	16-bit CRC	16-bit CRC, except no error check for RN16
Collision arbitration linearity	2048	Up to 2^{256}	Linear up to 2^{15} tags in the interrogator's RF field; above that number, $N \log(N)$ for tags with unique UIDs

Page 6, Table 2

Replace Table 2 with the following table:

Table 2 — Interrogator-to-tag link parameters

Ref.	Parameter Name	Description	Ref Type C
Int:1	Operating Frequency Range	Types A and B: 860 – 960 MHz The interrogator operating frequency range shall be determined by the radio regulations in force in a particular regulatory jurisdiction and by the type approval requirements of the particular jurisdiction. Before an interrogator may be used, it shall meet the local radio regulations of the country in which it is to be used. It is envisaged that there will be more than one version of interrogator having different frequency and power characteristics in order to comply with local regulations. NOTE Performance will vary according to bandwidth and power output permitted by local regulations. Type C: 860 MHz – 960 MHz, as required by local regulations	See clause 9.3.1.1
Int:1a	Default Operating Frequency	Types A and B: In accordance with the local radio regulations. See Int: 1 Type C: Determined by local radio regulations and by the radio-frequency environment at the time of the communication	See clause 9.3.1.1
Int:1b	Operating Channels (spread-spectrum systems)	Types A and B: In accordance with the local radio regulations. See Int: 1 Type C: In accordance with local regulations; if the channelisation is unregulated, then as specified	See clause 9.3.1.2.10, Annex I
Int:1c	Operating Frequency Accuracy	Types A and B: In accordance with the local radio regulations See Int: 1 Type C: As specified	See clause 9.3.1.2.1
Int:1d	Frequency Hop Rate (frequency-hopping [FHSS] systems)	Types A and B: Not applicable for single fixed frequency or channelized Adaptive Frequency Agile systems. Where FHSS is permitted, the hop rate shall be in accordance with the local radio regulations. Type C: In accordance with local regulations	See clause 9.3.1.2.9

Ref.	Parameter Name	Description	Ref Type C
Int:1e	Frequency Hop Sequence (frequency-hopping [FHSS] systems)	Types A and B: In accordance with the local radio regulations. Where not specified by such regulations a pseudo-random hopping sequence shall be used that ensures an even distribution of transmissions over the available channels. Type C: In accordance with local regulations	See clause 9.3.1.2.9
Int:2	Occupied Channel Bandwidth	In accordance with local regulations	
Int:2a	Minimum Receiver Bandwidth	In accordance with local regulations	
Int:3	Interrogator Transmit Maximum EIRP	In accordance with local regulations	
Int:4	Interrogator Transmit Spurious Emissions	Types A and B: In accordance with the local radio regulations. Type C: As specified; tighter emission limits may be imposed by local regulations	See clause 9.3.1.2.11
Int:4a	Interrogator Transmit Spurious Emissions, In-Band (spread-spectrum systems)	Types A and B: In accordance with the local radio regulations. Type C: As specified; tighter emission limits may be imposed by local regulations	See clause 9.3.1.2.11
Int:4b	Interrogator Transmit Spurious Emissions, Out-of-Band	Types A and B: In accordance with the local radio regulations. Type C: As specified; tighter emission limits may be imposed by local regulations	See clause 9.3.1.2.11
Int:5	Interrogator Transmitter Spectrum Mask	Types A and B: As per Int: 2 and Int: 4a. Type C: As specified; tighter emission limits may be imposed by local regulations	Figure Amd.1-9, Figure Amd.1-10
Int:6	Timing	Types A and B: See below Int: 6x. Type C: As specified	See clause 9.3.1.6, Figure Amd.1-19, Table Amd.1-15
Int:6a	Transmit-to-Receive Turn-Around Time	Types A and B: The interrogator transmit/receive settling time shall not exceed 85 μ s. Type C: MAX(RTcal, 10T _{pri}) nominal	See clause 9.3.1.6, Figure Amd.1-19, Table Amd.1-15
Int:6b	Receive-to-Transmit Turn-Around Time	Types A and B: As determined by the communication protocol – refer Tag: 6a. Type C: 3T _{pri} minimum; 20T _{pri} maximum when tag is in reply & acknowledged states; no limit otherwise	See clause 9.3.1.6, Figure Amd.1-19, Table Amd.1-15
Int:6c	Dwell Time or interrogator Transmit Power-On Ramp	1500 μ s, maximum settling time	See clause 9.3.1.2.6, Figure Amd.1-6, Table Amd.1-8
Int:6d	Decay Time or interrogator Transmit Power-Down Ramp	Types A and B: Maximum 1 ms. Type C: 500 μ s, maximum	Figure Amd.1-6, Table Amd.1-9
Int:7	Modulation	Types A and B: Amplitude Modulation. Type C: DSB-ASK, SSB-ASK, or PR-ASK	See clause 9.3.1.2.2

Ref.	Parameter Name	Description	Ref Type C
Int:7a	Spreading Sequence (direct-sequence [DSSS] systems)	Not applicable.	
Int:7b	Chip Rate (spread-spectrum systems)	Not applicable.	
Int:7c	Chip Rate Accuracy (spread-spectrum systems)	Not applicable.	
Int:7d	Modulation Depth	Type A: Nominal 30 % to 100 %. See Table 11 and clause 7.1.1. Type B: Nominal 18% or 100% . Type C: 90% nominal	See clause 9.3.1.2.5, Figure Amd.1-5, Table Amd.1-7
Int:7e	Duty Cycle	Types A and B: In accordance with local regulations. Type C: 48% – 82.3% (time the waveform is high)	See clause 9.3.1.2.3, Figure Amd.1-4, Table Amd.1-7
Int:7f	FM Deviation	Not applicable.	
Int:8	Data Coding	Type A: Pulse Interval Encoding Type B: Manchester bi-phase Type C: PIE	See clause 9.3.1.2.3, Figure Amd.1-4
Int:9	Bit Rate	Type A: 33 kbit/s as constrained by the local radio regulations Type B: 10 kbit/s or 40 kbit/s as constrained by the local radio regulations. Type C: 26.7 kbit/s to 128 kbit/s (assuming equiprobable data)	See clause 9.3.1.2.4
Int:9a	Bit Rate Accuracy	Types A and B: 100 ppm Type C: +/- 1%, minimum	See clause 9.3.1.2.4
Int:10	Interrogator Transmit Modulation Accuracy	Types A and B: Not applicable. Type C: As specified	See clause 9.3.1.2.4
Int:11	Preamble	Type A: has no preamble Type B: See clause 8.1.4.3 Type C: Required	See clause 9.3.1.2.8
Int:11a	Preamble Length	Type A: not applicable. Type B: 9 bits. See clause 8.1.4.3. Type C: As specified	See clause 9.3.1.2.8
Int:11b	Preamble Waveform(s)	Type A: not applicable . Type B: See clause 8.1.4.3. Type C: As specified	See Figure Amd.1-7
Int:11c	Bit Sync Sequence	Type A: not applicable. Type B: see clause 8.1.4.3 Type C: None	
Int:11d	Frame Sync Sequence	Types A and B: Not applicable. Type C: Required	See clause 9.3.1.2.8, Figure Amd.1-7
Int:12	Scrambling (spread-spectrum systems)	Not Applicable.	
Int:13	Bit Transmission Order	MSB is transmitted first	See clause 9.3.1.4

Ref.	Parameter Name	Description	Ref Type C
Int:14	Wake-up process	Types A and B: Presence of an appropriate RF signal at the tag followed by a wake-up command as required by the tag type. See relevant clauses. Type C: As specified	See clause 9.3.1.3.4
Int:15	Polarization	Types A and B: Interrogator dependent. Not defined in this part of ISO/IEC 18000. Type C: Not specified	

Page 8, Table 3**Replace Table 3 with the following table:****Table 3 — Tag-to-interrogator link parameters**

Ref.	Parameter Name	Description	Ref Type C
Tag:1	Operating Frequency Range	860 MHz – 960 MHz, inclusive	See clause 9.3.1.1
Tag:1a	Default Operating Frequency	Types A and B: The tag shall respond to an interrogator signal within the frequency range specified in Tag: 1. Type C: Tags respond to interrogator signals that satisfy Int:1a	See clause 9.3.1.1
Tag:1b	Operating Channels (spread-spectrum systems)	Types A and B: The tag shall respond to an interrogator signal within the frequency range specified in Tag: 1. Type C: Tags respond to interrogator signals that satisfy Int:1b	See clause 9.3.1.2.10
Tag:1c	Operating Frequency Accuracy	Types A and B: The tag shall respond to an interrogator signal within the frequency range specified in Tag: 1. Type C: As specified	See clause 9.3.1.3.3 Table Amd.1-11
Tag:1d	Frequency Hop Rate (frequency-hopping [FHSS] systems)	Types A and B: Not applicable Type C: Tags respond to interrogator signals that satisfy Int:1d	See clause 9.3.1.2.9
Tag:1e	Frequency Hop Sequence (frequency-hopping [FHSS] systems)	Types A and B: Not applicable Type C: Tags respond to interrogator signals that satisfy Int:1e	See clause 9.3.1.2.9
Tag:2	Occupied Channel Bandwidth	In accordance with local regulations	
Tag:3	Transmit Maximum EIRP	In accordance with local regulations	
Tag:4	Transmit Spurious Emissions	In accordance with local regulations	
Tag:4a	Transmit Spurious Emissions, In-Band (spread spectrum systems)	In accordance with local regulations	
Tag:4b	Transmit Spurious Emissions, Out-of-Band	In accordance with local regulations	
Tag:5	Transmit Spectrum Mask	In accordance with local regulations	

Ref.	Parameter Name	Description	Ref Type C
Tag:6a	Transmit-to-Receive Turn-Around Time	Type A: The tag shall open its receive command window within 2 bit periods of the end of its response. Type B: 400 μs Type C: $3T_{pri}$ minimum, $20T_{pri}$ maximum in reply & acknowledged states; no limit otherwise	See clause 9.3.1.6, Figure Amd.1-19, Table Amd.1-15
Tag:6b	Receive-to-Transmit Turn-Around Time	Type A: Range from 150 to 1150 μs (see clause 7.5.3 and Table 28) Type B: Range 85 to 460 μs (see clause 8.1.5.2) Type C: $MAX(RT_{cal}, 10T_{pri})$ nominal	See clause 9.3.1.6, Figure Amd.1-19, Table Amd.1-15
Tag:6c	Dwell Time or Transmit Power-On Ramp	Types A and B: Not applicable. Type C: Receive commands 1.5 ms after power-up	See clause 9.3.1.3.4
Tag:6d	Decay Time or Transmit Power-Down Ramp	Not applicable.	
Tag:7	Modulation	Types A and B: Bi-state amplitude modulated backscatter. Type C: ASK and/or PSK modulation (selected by tag)	See clause 9.3.1.3.1
Tag:7a	Spreading Sequence (direct sequence [DSSS] systems)	Not applicable.	
Tag:7b	Chip Rate (spread spectrum systems)	Not applicable.	
Tag:7c	Chip Rate Accuracy (spread spectrum systems)	Not applicable.	
Tag:7d	On-Off Ratio	Types A and B: The tag Delta RCS (Varying Radar Cross Sectional area) affects system performance. A typical value is greater than $0.005m^2$. Type C: Not specified	
Tag:7e	Subcarrier Frequency	Types A and B: Not applicable. Type C: 40 kHz to 640 kHz	See clause 9.3.1.3.3 Table Amd.1-11
Tag:7f	Subcarrier Frequency Accuracy	Types A and B: Not applicable. Type C: As specified	Table Amd.1-11
Tag:7g	Subcarrier Modulation	Types A and B: Not applicable. Type C: Miller, at the data rate	See clause 9.3.1.3.2.3, Figure Amd.1-16
Tag:7h	Duty Cycle	Types A and B: The tag shall transmit its response when commanded to do so by the interrogator. Type C: FM0: 50%, nominal Subcarrier: 50%, nominal	See clause 9.3.1.3.2.1, 9.3.1.3.2.3
Tag:7i	FM Deviation	Not applicable.	
Tag:8	Data Coding	Types A and B: Bi-phase space (FM0) Type C: Baseband FM0 or Miller-modulated subcarrier (selected by the interrogator)	See clause 9.3.1.3.2

Ref.	Parameter Name	Description	Ref Type C
Tag:9	Bit Rate	Types A and B: Typical 40 kbit/s or 160 kbit/s (subject to tag clock tolerance see Table 9), The return bit rate selection for 160 kbit/s for Type B is defined in clause 8.1.4.4.5. Type C: FMO: 40 kbps to 640 kbps Subcarrier modulated: 5 kbps to 320 kbps	See clause 9.3.1.3.3 Table Amd.1-11
Tag:9a	Bit Rate Accuracy	Types A and B: +/- 15% (refer to Table 9) Type C: Same as Subcarrier Frequency Accuracy; see Tag:7f	See clause 9.3.1.3.3, Table Amd.1-11, Table Amd.1-12
Tag:10	Tag Transmit Modulation Accuracy (frequency-hopping [FHSS] systems)	Not applicable.	
Tag:11	Preamble	Types A and B: The preamble is defined in clause 6.5.6 Type C: Required	See clause 9.3.1.3.2.2, See clause 9.3.1.3.2.4
Tag:11a	Preamble Length	Types A and B: 16 bits made up of a quiet period, followed by sync, followed by a code violation followed by an orthogonal code. Type C: As specified	See Table Amd.1-11, See Figure Amd.1-18
Tag:11b	Preamble Waveform	Types A and B: Bi-phase encoded data '1'. Type C: As specified	See Table Amd.1-11, See Figure Amd.1-18
Tag:11c	Bit-Sync Sequence	Types A and B: Included in the preamble. Type C: None	
Tag:11d	Frame-Sync Sequence	Types A and B: Included in the preamble. Type C: None	
Tag:12	Scrambling (spread-spectrum systems)	Not applicable.	
Tag:13	Bit Transmission Order	MSB is transmitted first	See clause 9.3.1.4
Tag:14	Reserved	Deliberately left blank.	
Tag:15	Polarization	Types A and B: Product design feature. Not defined in this part of ISO/IEC 18000. Type C: Tag dependent; not specified by this document	
Tag:16	Minimum Tag Receiver Bandwidth	Types A and B: 860 – 960 MHz Type C: Tag dependent; not specified by this document.	

Page 9, Table 4

Replace Table 4 with the following table:

Table 4 — Protocol parameters

Ref.	Parameter Name	Description	Ref Type C
P:1	Who talks first	Interrogator	See clause 9.3
P:2	Tag addressing capability	Type A see clause 7.3.1 Type B see clause 8.2.2 Type C: As specified	See clause 9.3.2.1
P:3	Tag Ull	Types A and B: Contained in tag memory and accessible by means of a command. Type C: Contained in Tag memory	See clause 9.3.2.1
P:3a	Ull Length	Type A: 64 bits. An SUID of 40 bits is used during census or collision arbitration transactions. Type B: 64 bits Type C: As specified	9.3.2.1.2.2
P:3b	Ull Format	See clause 7.2.1 for type A . See clause B.2 for type B . Type C: NSI < 100 _n : As specified in EPCglobal™ Tag Data Standards (Version 1.3 and above), NSI ≥ 100 _n : As specified in ISO/IEC 15961	9.3.2.1.2.2 9.3.2.1.2.3 9.3.2.1.2.4
P:4	Read size	Type A: Addressable in blocks of up to 256 bits, but always an integer multiple of bytes. Type B: Addressable in byte blocks. Type C: Multiples of 16 bits	See clause 9.3.2.10.3.2 Table Amd.1-32
P:5	Write Size	Type A: Addressable in blocks of up to 256 bits, but always an integer multiple of bytes. Type B: Addressable in byte blocks. Writing in blocks of 1, 2, 3 or 4 bytes. See details in relevant clauses. Type C: Multiples of 16 bits	See clause 9.3.2.10.3.3, Table Amd.1-34, See clause 9.3.2.10.3.7, Table Amd.1-44
P:6	Read Transaction Time	Types A and B: A single tag can typically be identified and have its first 128 bits of user memory read in less than 10 ms. This time may vary depending on the data rate used as constrained by the local radio regulations. Type C: Varied with R=>T and T=>R link rate and number of bits being read	See clause 9.3.2.10.3.2
P:7	Write Transaction Time	Types A and B: Once a tag has been identified and selected, a 32-bit data block can typically be written in less than 20 ms. This time may vary depending on the data rate used as constrained by the local radio regulations. Type C: 20 ms (maximum) after end of <i>Write</i> command	See clause 9.3.2.10.3.3, See clause 9.3.2.10.3.7, Figure Amd.1-25

Ref.	Parameter Name	Description	Ref Type C
P:8	Error detection	Type A: Interrogator to tag: 16-bit commands: CRC-5. Commands of more than 16 bits have an additional CRC-16, bit protection. Tag to interrogator: CRC-16 Type B: Interrogator to tag: CRC-16 Tag to interrogator: CRC-16 Type C: Interrogator-to-tag: <i>Select</i> command: CRC-16 <i>Query</i> command: CRC-5 Other Inventory commands: Command length Access commands: CRC-16 Tag-to-interrogator: PC, UII: CRC-16 RN16: None or CRC-16 (varies with command) <i>handle</i>: CRC-16 All other: CRC-16	See clause 9.3.2.10 and its subsections
P:9	Error correction	Types A and B: No forward error correction code used. Errors are handled by signalling an error to the interrogator that then repeats its last transmission. Type C: None	
P:10	Memory size	Types A and B: No minimum user memory size is specified, but if user memory is provided it shall be an integer multiples of 4 bytes. Type C: Tag dependent, extensible (size is neither limited nor specified by this document)	
P:11	Command structure and extensibility	Type A: Several command codes are reserved for future use. In addition, a Protocol extension flag allows for further extensions. Type B: Several command codes are reserved for future use. Type C: As specified	Table Amd.1-18

Page 10, Table 5**Replace Table 5 with the following table:****Table 5 — Anti-collision parameters**

Ref.	Parameter Name	Description	Ref Type C
A:1	Type (Probabilistic or Deterministic)	Probabilistic	See clause 9.3.2.6
A:2	Linearity	Type A: Essentially linear using adaptive slot allocation up to 256 slots for 250 tags in an interrogator zone. Type B: Essentially linear up to 2^{256} tags depending on size of data content. Type C: Linear up to 2^{15} tags in the interrogator's RF field; above that number, $N\log N$ for tags with unique UIDs	See clause 9.3.2.8
A:3	Tag inventory capacity	Types A and B: The algorithm permits the reading of not less than 250 tags in the reading zone of the interrogator. Type C: Unlimited for tags with unique UIDs	See clause 9.3.2.8

Page 11, Figure 4**Move the figure into clause 6.2 as it is referenced in clause 6.2.****Page 11, Table 6****Move the table into clause 6.2 as it is referenced in clause 6.2.****Page 13, Clause 6.5.3****Replace the wording of clause 6.5.3 with the following wording:**

The return link datarate shall be 40 or 160 kbit/s, which may be selected at the time of tag configuration. The interrogator shall be able to read and decode the tag reply at either datarate without the need to have prior knowledge of the tag configuration in order to handle mixed populations.

Page 13, Table 9**Replace Table 9 with the following table:****Table 9 — Return link parameters**

Data rate	Trlb	Tolerance	Note
40kbit/s	25 µs	+/-15%	Chip set to 40kbit/s Return Link Data Rate
160kbit/s	6.25 µs	+/-15%	Chip set to 160kbit/s Return Link Data Rate

Page 19, Table 12**Replace Table 12 with the following table:****Table 12 — Reference interval timing**

Tari	Stability
10 to 20 μ s	± 100 ppm

Page 28, Table 20**Replace the text in row "7", column "Round size" with the following text:**

1024

Page 28, Table 21**Replace Table 21 with the following modified table:****Table 21 — Command classes**

Code	Class	Number of possible codes
'00', '02', '04', '06', '0A', and '0C' – '0F'	Mandatory	10
'01', '03', '05', '07', '08', '09', '0B' and '10' – '27', '38', '39'	Optional	32
'28' – '37'	Custom	16
'3A' – '3F'	Proprietary	6

Page 28, Clause 7.4.6.3**Replace the first paragraph with the following:**

The optional command codes are '01', '03', '05', '07', '08', '09', '0B', '10' to '27', '38' and '39'.

Page 29, Clause 7.4.6.5**Replace the first paragraph with the following:**

The proprietary command codes range from '3A' to '3F'. Proprietary commands may be enabled by this part of ISO/IEC 18000, but they shall not be specified in this part of ISO/IEC 18000. A proprietary command shall not solely duplicate the functionality of any mandatory or optional command defined in this part of ISO/IEC 18000 by a different method.

Page 30, Table 22**Replace the text in row "Next_Slot", column "Comments" with the following text:**

The signature may be one of 16 values between 0x00 and 0x0F as transmitted by the tag in its last transmission.

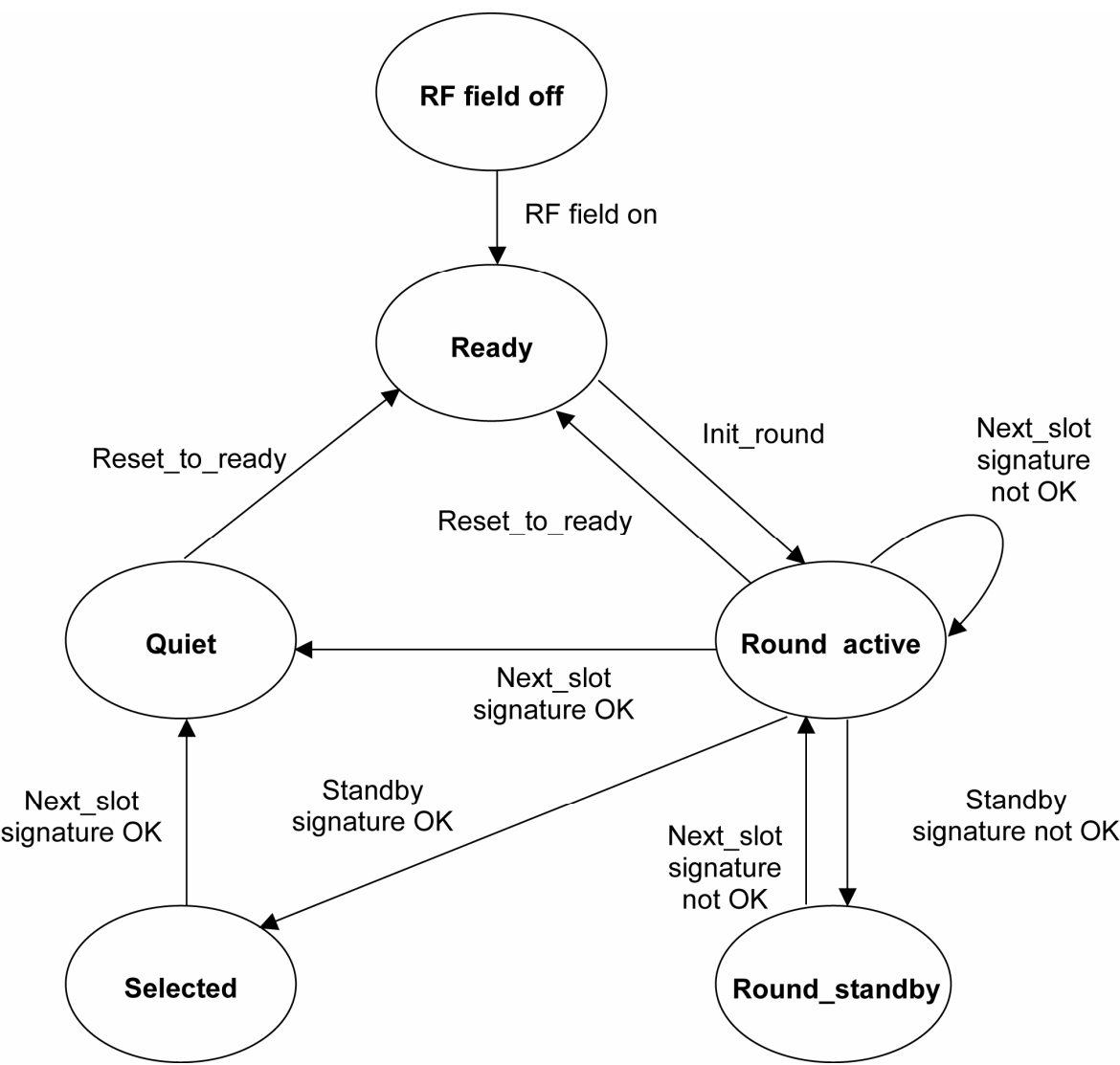
Page 30, Table 22

Insert the following row:

Command 6 bits	Op code	Parameter / flags 4 bits		CRC-5	Extended parameters	CRC-16	Comments
RFU	0x38	RFU	RFU	5 bits			
Init_Fast_Slot	0x39	SUID 1 bit	Round Size 3 bits	5 bits			SUID=0 tag responds with 1 st page of user data; SUID=1 tag responds with SUID

Page 35, Figure 16

Replace Figure 16 with the following new Figure 16.



NOTE This state diagram shows some of the main transitions. State transitions are specified in detail in each command description.

Figure 16 — Tag state transition diagram

Page 37, Clause 7.4.11***Insert the following text at the end of clause 7.4.11.***

Tag manufacturer may optionally configure the tag to power-up in the FST mode. When the tag is configured to power up in FST mode, and enters the energizing field of an interrogator, it goes through a power on reset sequence and then moves to the Round_Standby state until it receives a Next_Slot, Close_Slot, New_Round or Init_Fast_Slot command, at which time it commences a Fast Slot Mode collision arbitration sequence.

Each slot has a duration at least as long as the duration of a tag preamble. The actual duration of the slot is determined by the tag and is equal to 16 tag bit times. If a tag has selected the current slot in which to transmit its reply, the Slot length is increased for that tag to the duration of a message length so that the tag can send its complete message. In order to prevent other tags (those that have not yet started their replies) from transmitting during the first tag's reply slot, the interrogator issues a MUTE command to place the tag into the Round_Standby state. After the active tag has finished transmitting its message, and if the interrogator has successfully read the tag it issues a Next_Slot command synchronously with the tag's signature. If the tag message was not successfully read then the interrogator issues a Close_Slot command, which will cause all the tags currently in the Round_Standby state to re-enter the Round_Active state.

The number of slots in a round, referred to as round size, is determined by the interrogator and is signalled to the tag in the Init_Fast_Slot or New_Round command. In the FST mode the tag elects a default roundsize of 16, which may be overridden by an interrogator command, however the FST mode is able to operate without any round initialising command. During the subsequent collision arbitration process the interrogator dynamically chooses an optimum round size for the following rounds based on the number of collisions and/or unproductive time in a round. The number of collisions is a function of the number of tags in the Round_Active state present in the interrogator field and the current round size. The interrogator signals a change in round size to tags by sending a New_Round command containing the required round size.

The tag on entering the Round_Active State or on re-entering the Round_Active state having completed a round, selects a pseudo slot at random in which to reply. Pseudo slots are equal to tag preamble in duration. If the tag has selected the first pseudo slot, it will transmit immediately, if not it will hold off until it has reached the selected pseudo-slot and then transmit.

On receiving and recognizing a valid tag transmission preamble, the interrogator sends a MUTE command (SOF), which tells all tags that have not yet started transmitting, to move to the Round_Standby state. When the interrogator receives the tag Reply without error, it sends a Next_Slot command containing the signature of the tag that it has just received.

When the tag has reached the end of a round, it will self-trigger a new round, randomly select a new slot in which to transmit and it will transmit its identity or data when it reaches the selected slot. The process continues until the tag has been successfully read and acknowledged by a valid Next_Slot command or removed from the RF energizing field.

Page 39, Clause 7.5.5.3***Replace clause 7.5.5.3. with the following text:*****7.5.5.3 Tag**

When a tag, in the Round active state (and not in the FST mode), has just responded in the current slot and receives a Next_slot command, which was not received within the window specified in clause 7.5.5, the tag shall not move to the quiet state, but shall remain in the Round active state and shall increment its slot counter as specified in Clause 7.7.2 and Table 31.

When a tag in the Round active state, has just responded in the current slot and receives a Standby_round command, which was not received within the window specified in clause 7.5.5, the tag shall not move to the selected state, but shall transition to the Round_standby state as specified in Clause 7.7.3 and Table 33.

Page 41, Clause 7.7.2***Insert the following paragraph at the end of clause 7.7.2:***

In the Fast Slot Mode the Next_Slot command instructs all tags in the Round_Standby state to switch to the Active state.

Page 41, Clause 7.7.3***Replace clause 7.7.3 with the following text:*****7.7.3 Standby_round****Command code = '04'**

The Standby_round command has two functions:

- It acknowledges a valid response from a tag and instructs this tag to enter the Selected state if the tag supports the optional Selected state (individual commands can then be sent to this tag such as Read and Write), or
- It instructs all other tags in the Round_active state to enter the Round_standby state. For these tags, their participation in the round is suspended. tags that do not support the optional Selected state will move from the Round_active state to the Round_standby state irrespective of whether the Standby_round command acknowledged a valid response from the tag or not. The round will be resumed by a Next_slot command, or a Close_slot command. In addition, a new round will be initiated by tags in the Round_standby state by a New_round command, an Init_round command or an Init_round_all command.

The command contains:

The Standby_round command code, and

The tag signature.

No flags are used by this command.

Table 32 shows the Standby_round command format.

Table 32 — Standby_round command format

Protocol Extension	Standby _round	Tag signature	CRC-5
1 bit	6 bits	4 bits	5 bits

Table 33 — Tag state transition for Standby_round

Command : Standby_round			
Current state	Criteria	Action	New state
Ready	None	None	Ready
Quiet	None	None	Quiet
Selected	None	None	Quiet
Round_active	Tag has answered in previous slot, AND Signature matches, AND Next_slot was received in the acknowledgement time window. See clause 7.5.5, AND Tag supports the optional SELECTED state.	None	Selected
	Tag has not answered in previous slot, OR Signature does not match OR Next_slot was not received in the acknowledgement time window. See clause 7.5.5, OR Tag does not support the optional SELECTED state.	None	Round_standby
Round_standby	None	None	Round_standby

Page 47, Clause 7.8.3***Insert the following paragraph after the first paragraph:***

In the Fast Slot Mode the Close_Slot command instructs all tags in the Round_Standby state to switch to the Round_Active state.

Page 48, Table 45

Replace Table 45 with the following table:

Table 45 — Tag state transition for Close_slot

Command : Close_slot			
Current state	Criteria	Action	New state
Ready	None	None	Ready
Quiet	None	None	Quiet
Selected	None	None	Quiet
Round_active	Long_Slot Mode	The tag shall increment its slot counter and send its response if slot counter matches chosen slot.	Round_active
	Fast_Slot Mode	The tag will automatically increment its slot counter at internally determined intervals and send its response if slot counter matches chosen slot.	Round_active
Round_standby	Long_Slot Mode	The tag shall increment its slot counter and send its response if slot counter matches chosen slot.	Round_active
	Fast_Slot Mode	The tag will automatically increment its slot counter at internally determined intervals and send its response if slot counter matches chosen slot.	Round_active

Page 76, Clause 7.8.16

Insert the following clause after clause 7.8.16:

7.8.17 Init_Fast_Slots

Command code = '39'

This command is functionally equivalent to the Init_round command, with AFI set to 00.

The command contains:

The Init_Fast_Slots command code of 6 bits,

The SUID Flag of 1 bit, and

The Round size of 3 bits.

Table Amd.1-1 shows the Init_Fast_Slots command format.

Table Amd.1-1 — Init_Fast_Slots command format

Protocol extension	Init_Fast_Slots	SUID flag	Round size	CRC-5
1 bit	6 bits	1 bit	3 bits	5 bits

The response, as shown in Table Amd.1-2 or Table Amd.1-3 shall contain:

- The DSFID if the SUID flag is set in the command,
- The tag signature,
- The tag type (battery-less or battery-assisted),
- The battery status flags,
- The SUID if the SUID flag is set in the command, and
- The first n bits of the tag memory if the SUID flag is NOT set in the command.

If the tag detects an error, it shall remain silent.

Table Amd.1-2— Init_Fast_Slots response format when the SUID flag is NOT set

Preamble	Flags see Table 24	Tag type see Table Amd.1-4	Battery status see Table Amd.1-5	Signature	Random number See NOTES 1 and 2	First n bits of memory See NOTE 3	CRC-16
	2 bits	1 bit	1 bit	4 bits	8 bits	n = 32, 64, 96 or 128 bits	16 bits

NOTE 1 The purpose of the random number is to increase the probability of detecting a collision between two or more tags whose responses contain the same data. Combined with the 4 bit signature, it represents a 12 bits random number. It is possible that if the SUID flag is NOT set, and the tag returns the first “page” of data (could be 32,64,96 or 128 bits), multiple tags could be programmed with the same memory content. To ensure a collision is detected, the 4 bit signature is extended by 8 bits, giving 12 bits, in which a collision could be detected.

NOTE 2 The DSFID field is not included since in situations where the first “page” of memory is read, it is expected that no further exchange with the tag will take place. The DSFID can nevertheless be determined using other commands.

NOTE 3 The tag shall return the first n bits of user data according to its memory size, with a maximum of 128 bits. If there is no user memory, the tag shall return the SUID.

This International Standard does not specify how the 8 bit random number shall be generated (It could for instance be the last 8 significant bits of the UID, or a random number, etc.).

The generation of the signature and the random number shall be independent of each other.

Table Amd.1-3— Init_Fast_Slots response format when the SUID flag is set

Preamble	Flags see Table 24	Tag type see Table Amd.1-4	Battery status see Table Amd.1-5	Signature	DSFID	SUID	CRC-16
	2 bits	1 bit	1 bit	4 bits	8 bits	40 bits	16 bits

NOTE The 8 bits random number is not required when an SUID is sent back since the collisions will be detected using the combination of the signature and SUID.

Table Amd.1-4 shows the Tag type field, while Table Amd.1-5 shows the Battery Status field.

Table Amd.1-4 — Tag type (battery-assisted or not)

Tag type	Meaning
0	Tag is NOT battery-assisted
1	Tag is battery-assisted

Table Amd.1-5 — Battery status (for battery-assisted tag)

Battery status	Meaning
0	Battery is low. A non-battery-assisted tag shall set this bit to 0.
1	Battery is good.

The tag shall perform the actions specified in Table Amd.1-6.

Table Amd.1-6 — Tag state transition for Init_Fast_Slots

Command : Init_Fast_Slots			
Current state	Criteria	Action	New state
Ready	None	The tag shall choose the slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active
Quiet	None	The tag shall discard the command and remain silent.	Quiet
Selected	None	The tag shall discard the command and transition to the Quiet state.	Quiet
Round_active	None	The tag shall reset the previously chosen slot and chose a new slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active
Round_standby	None	The tag shall reset the previously chosen slot and chose a new slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active

NOTE The slots are numbered from 1 to Round_size.

Page 111, Clause 8.2.7.9.1**Add the following paragraph after the first paragraph:**

For memory lock functionality a tag shall provide the opportunity to mark an address lockable. An ADDRESS is marked lockable by commands as described in this clause. No ADDRESS shall be marked lockable after the tag has been in the POWER-OFF state. A tag shall not support more than one ADDRESS to be marked lockable at the same time.

Page 114, Table 154**Replace Table 154 with the following table:****Table 154 — READ_VARIABLE response in the case of NO error**

Preamble	(Length + 1) * BYTE_DATA	CRC-16
	(Length + 1) * 8 bits	16 bits

Page 118, Clause 8.2.7.9.14**Replace the second paragraph with the following paragraph:**

On receiving a QUERY_LOCK command, a tag shall read its UID and compare it with the ID sent by the interrogator. In the case that the UID is equal to ID, the ADDRESS is within the valid address range, then the tag shall move into the DATA_EXCHANGE state. Further, the tag shall read the lock bit for the memory byte at ADDRESS. In the case that this memory is not locked, then it shall response ACKNOWLEDGE_OK if WRITE_OK is set and ACKNOWLEDGE_NOK if WRITE_OK is cleared. In the case that this memory is locked, then it shall respond ERROR_OK if WRITE_OK is set and ERROR_NOK if WRITE_OK is cleared. Further, the tag shall mark the byte of ADDRESS lockable unless it is already locked.

Page 121, Clause 8**Insert the following new clause after Clause 8:****9 Type C****9.1 Protocol overview****9.1.1 Physical layer**

An interrogator sends information to one or more tags by modulating an RF carrier using double-sideband amplitude shift keying (DSB-ASK), single-sideband amplitude shift keying (SSB-ASK), or phase-reversal amplitude shift keying (PR-ASK) using a pulse-interval encoding (PIE) format. Tags receive their operating energy from this same modulated RF carrier.

An interrogator receives information from a tag by transmitting an unmodulated RF carrier and listening for a backscattered reply. Tags communicate information by backscatter modulating the amplitude and/or phase of the RF carrier. The encoding format, selected in response to interrogator commands, is either FM0 or Miller-modulated subcarrier. The communications link between interrogators and tags is half-duplex, meaning that tags shall not be required to demodulate interrogator commands while backscattering. A tag shall not respond to a mandatory or optional command using full-duplex communications.

9.1.2 Tag-identification layer

An interrogator manages tag populations using three basic operations:

Select. The operation of choosing a tag population for inventory and access. A *Select* command may be applied successively to select a particular tag population based on user-specified criteria. This operation is analogous to selecting records from a database.

Inventory. The operation of identifying tags. An interrogator begins an inventory round by transmitting a *Query* command in one of four sessions. One or more tags may reply. The interrogator detects a single tag reply and requests the PC, UII, and CRC-16 from the tag. Inventory comprises multiple commands. An inventory round operates in one and only one session at a time.

Access. The operation of communicating with (reading from and/or writing to) a tag. An individual tag must be uniquely identified prior to access. Access comprises multiple commands, some of which employ one-time-pad based cover-coding of the R=>T link.

9.2 Command types and command structure

9.2.1 General

This clause defines 4 command types: Mandatory, optional, custom, and proprietary.

All commands defined in this International Standard are either mandatory or optional, as specified in the definition of the command itself. Custom or proprietary commands are vendor-defined.

9.2.2 Mandatory

Conformant tags shall support all mandatory commands.

Conformant interrogators shall support all mandatory commands.

9.2.3 Optional

Tags may or may not support optional commands.

Interrogators may or may not support optional commands.

If an optional command is used, it shall be implemented in the manner specified in this document.

9.2.4 Custom

Custom commands are not specified in this document.

An interrogator shall issue a custom command only after (i) singulating a tag, and (ii) reading (or having prior knowledge of) the tag manufacturer's identification in the tag's TID memory. An interrogator shall use a custom command only in accordance with the specifications of the tag manufacturer identified in the tag ID.

A custom command shall not solely duplicate the functionality of any mandatory or optional command defined in this International Standard by a different method.

9.2.5 Proprietary

Proprietary commands are not specified in this document.

All proprietary commands shall be capable of being permanently disabled. Proprietary commands are intended for manufacturing purposes and shall not be used in field-deployed RFID systems.

9.3 Description of operating procedure

The operating procedure defines the physical and logical requirements for an ITF, random-slotted collision arbitration, RFID system operating in the 860 MHz – 960 MHz frequency range.

9.3.1 Signalling

The signalling interface between an interrogator and a tag may be viewed as the physical layer in a layered network communication system. The signalling interface defines frequencies, modulation, data coding, RF envelope, data rates, and other parameters required for RF communications.

9.3.1.1 Operational frequencies

Tags shall receive power from and communicate with interrogators within the frequency range from 860 MHz to 960 MHz, inclusive. An interrogator's choice of operational frequency will be determined by local radio regulations and by the local radio-frequency environment. Interrogators that are claimed to operate in dense-interrogator environments shall support, but are not required to always use, the optional dense-interrogator mode described in Annex I.

9.3.1.2 Interrogator-to-Tag (R=>T) communications

An interrogator communicates with one or more tags by modulating an RF carrier using DSB-ASK, SSB-ASK, or PR-ASK with PIE encoding. Interrogators shall use a fixed modulation format and data rate for the duration of an inventory round, where "inventory round" is defined in 4.1. The interrogator sets the data rate by means of the preamble that initiates the round.

The high values in Figure Amd.1-4, Figure Amd.1-5, Figure Amd.1-6, Figure Amd.1-7 and Figure Amd.1-8, correspond to emitted CW (i.e. an interrogator delivering power to the tag or tags) whereas the low values correspond to attenuated CW.

9.3.1.2.1 Interrogator frequency accuracy

Interrogators that claim to operate in single- or multiple-interrogator environments shall have a frequency accuracy that meets local regulations. Interrogators that are claimed to operate in dense-interrogator environments shall have a frequency accuracy of ± 10 ppm over the nominal temperature range (-25°C to $+40^{\circ}\text{C}$), and ± 20 ppm over the extended temperature range (-40°C to $+65^{\circ}\text{C}$) while transmitting, unless local regulations specify tighter accuracy, in which case the interrogator frequency accuracy shall meet the local regulations.

9.3.1.2.2 Modulation

Interrogators shall communicate using DSB-ASK, SSB-ASK, or PR-ASK modulation, detailed in Annex J. Tags shall demodulate all three modulation types.

9.3.1.2.3 Data encoding

The R=>T link shall use PIE, shown in Figure Amd.1-4. T_{ari} is the reference time interval for interrogator-to-tag signalling, and is the duration of a data-0. High values represent transmitted CW; low values represent attenuated CW. Pulse modulation depth, rise time, fall time, and PW shall be as specified in Table Amd.1-7, and shall be the same for a data-0 and a data-1. Interrogators shall use a fixed modulation depth, rise time, fall time, PW, T_{ari} , data-0 length, and data-1 length for the duration of an inventory round. The RF envelope shall be as specified in Figure Amd.1-5.

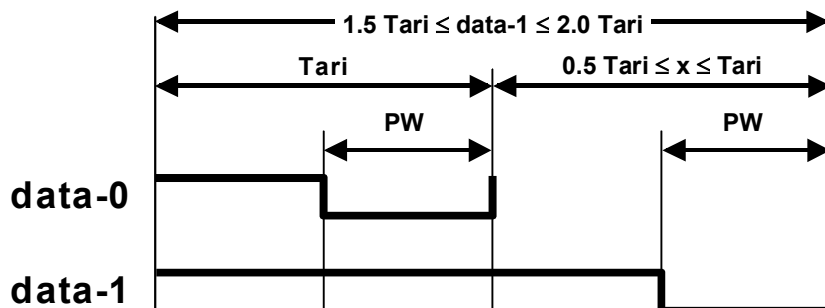


Figure Amd.1-4 — PIE Symbols

9.3.1.2.4 Tari values

Interrogators shall communicate using Tari values in the range of 6.25µs to 25µs. Interrogator compliance shall be evaluated using at least one Tari value between 6.25µs and 25µs with at least one value of the parameter x. The tolerance on all parameters specified in units of Tari shall be $\pm 1\%$. The choice of Tari value and x shall be in accordance with local radio regulations.

9.3.1.2.5 R=>T RF envelope

The R=>T RF envelope shall comply with Figure Amd.1-5 and Table Amd.1-7. The electric field strength A is the maximum amplitude of the RF envelope. Tari is defined in Figure Amd.1-4. The pulsewidth is measured at the 50% point on the pulse. An interrogator shall not change the R=>T modulation type (i.e. shall not switch between DSB-ASK, SSB-ASK, or PR-ASK) without first powering down its RF waveform (see 9.3.1.2.7).

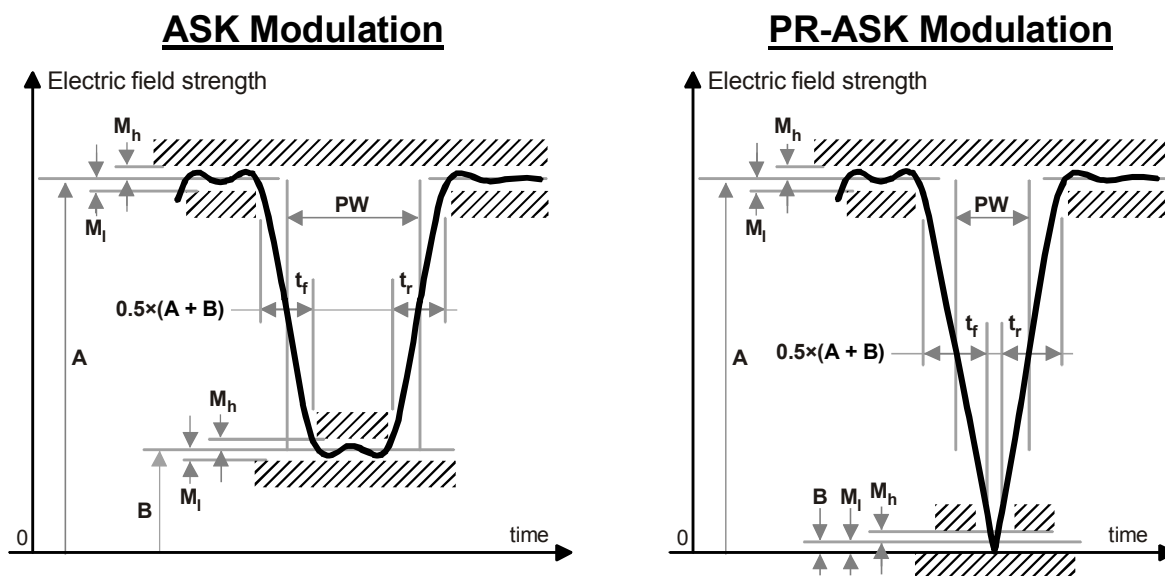


Figure Amd.1-5 — Interrogator-to-tag RF Envelope

Table Amd.1-7 — RF envelope parameters

Tari	Parameter	Symbol	Minimum	Nominal	Maximum	Units
6.25 μ s to 25 μ s	Modulation Depth	$(A-B)/A$	80	90	100	%
	RF Envelope Ripple	$M_h = M_l$	0		$0.05(A-B)$	V/m
	RF Envelope Rise Time	$t_{r,10-90\%}$	0		$0.33T_{ari}$	μ s
	RF Envelope Fall Time	$t_{f,10-90\%}$	0		$0.33T_{ari}$	μ s
	RF Pulsewidth	PW	$\text{MAX}(0.265T_{ari}, 2)$		$0.525T_{ari}$	μ s

9.3.1.2.6 Interrogator power-up waveform

The interrogator power-up RF envelope shall comply with Figure Amd.1-6 and Table Amd.1-8. Once the carrier level has risen above the 10% level, the power-up envelope shall rise monotonically until at least the ripple limit M_l . The RF envelope shall not fall below the 90% point in Figure Amd.1-6 during interval T_s . Interrogators shall not issue commands before the end of the maximum settling-time interval in Table Amd.1-8 (i.e. before T_s).

9.3.1.2.7 Interrogator power-down waveform

The interrogator power-down RF envelope shall comply with Figure Amd.1-6 and Table Amd.1-9. Once the carrier level has fallen below the 90% level, the power-down envelope shall fall monotonically until the power-off limit M_s . Once powered off, an interrogator shall remain powered off for a least 1ms before powering up again.

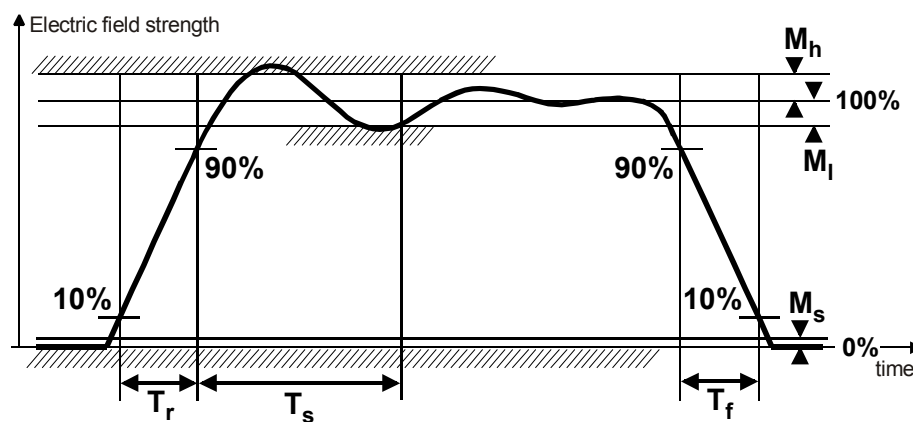


Figure Amd.1-6 — Interrogator power-up and power-down RF envelope

Table Amd.1-8 — Interrogator power-up waveform parameters

Parameter	Definition	Minimum	Nominal	Maximum	Units
T_r	Rise time	1		500	μ s
T_s	Settling time			1500	μ s
M_s	Signal level when OFF			1	% full scale
M_l	Undershoot			5	% full scale
M_h	Overshoot			5	% full scale

Table Amd.1-9 — Interrogator power-down waveform parameters

Parameter	Definition	Minimum	Nominal	Maximum	Units
T_f	Fall time	1		500	μs
M_s	Signal level when OFF			1	% full scale
M_l	Undershoot			5	% full scale
M_h	Overshoot			5	% full scale

9.3.1.2.8 R=>T preamble and frame-sync

An interrogator shall begin all R=>T signalling with either a preamble or a frame-sync, both of which are shown in Figure Amd.1-7. A preamble shall precede a *Query* command (see 9.3.2.10.2.1) and denotes the start of an inventory round. All other signalling shall begin with a frame-sync. The tolerance on all parameters specified in units of T_{ari} shall be $\pm 1\%$. PW shall be as specified in Table Amd.1-7. The RF envelope shall be as specified in Figure Amd.1-5.

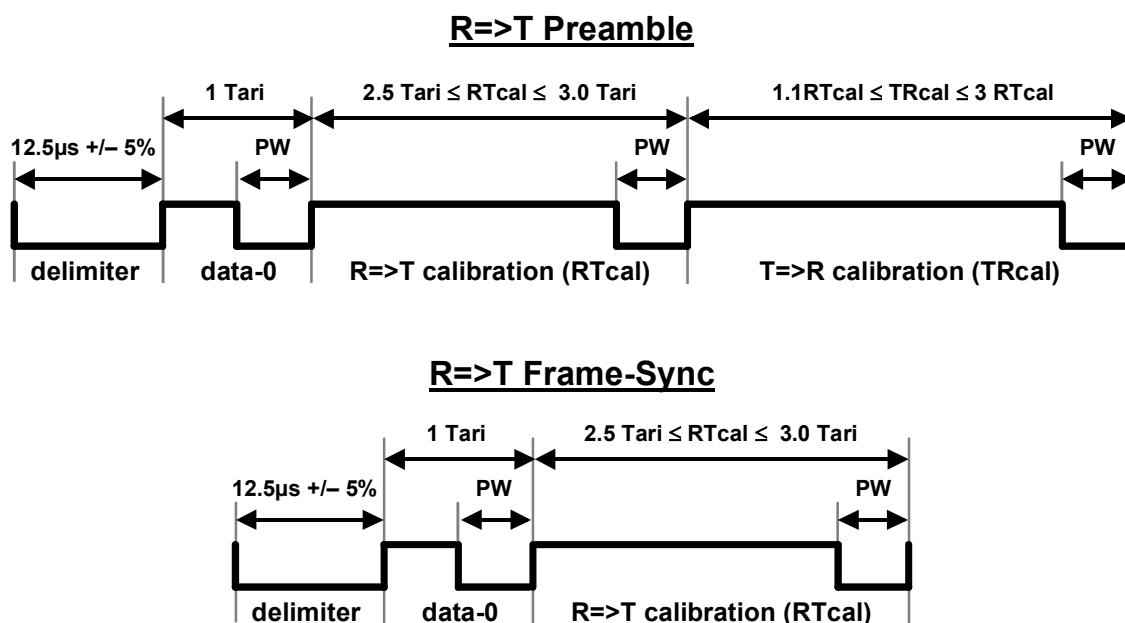


Figure Amd.1-7 — R=>T preamble and frame-sync

A preamble shall comprise a fixed-length start delimiter, a data-0 symbol, an R=>T calibration (RTcal) symbol, and a T=>R calibration (TRcal) symbol.

RTcal: An interrogator shall set RTcal equal to the length of a data-0 symbol plus the length of a data-1 symbol ($RTcal = 0_{\text{length}} + 1_{\text{length}}$). A tag shall measure the length of RTcal and compute $pivot = RTcal / 2$. The tag shall interpret subsequent interrogator symbols shorter than $pivot$ to be data-0s, and subsequent interrogator symbols longer than $pivot$ to be data-1s. The tag shall interpret symbols longer than 4 RTcal to be bad data. Prior to changing RTcal, an interrogator shall transmit CW for a minimum of 8 RTcal.

TRcal: An interrogator shall specify a tag's backscatter link frequency (its FM0 datarate or the frequency of its Miller subcarrier) using the TRcal and divide ratio (DR) in the preamble and payload, respectively, of a *Query* command that initiates an inventory round. Equation (1) specifies the relationship between the backscatter link frequency (BLF), TRcal, and DR. A tag shall measure the length of TRcal, compute BLF, and adjust its T=>R link rate to be equal to BLF (Table Amd.1-11).

shows BLF values and tolerances). The TRcal and RTcal that an interrogator uses in any inventory round shall meet the constraints in Equation (2):

$$\text{BLF} = \frac{\text{DR}}{\text{TRcal}} \quad (1)$$

$$1.1 \times \text{RTcal} \leq \text{TRcal} \leq 3 \times \text{RTcal} \quad (2)$$

A frame-sync is identical to a preamble, minus the TRcal symbol. An interrogator, for the duration of an inventory round, shall use the same length RTcal in a frame-sync as it used in the preamble that initiated the round.

9.3.1.2.9 Frequency-hopping spread-spectrum waveform

When an interrogator uses frequency-hopping spread spectrum (FHSS) signalling, the interrogator's RF envelope shall comply with Figure Amd.1-8 and Table Amd.1-10. The RF envelope shall not fall below the 90% point in Figure Amd.1-8 during interval T_{hs} . Interrogators shall not issue commands before the end of the maximum settling-time interval in Table Amd.1-10 (i.e. before T_{hs}). The maximum time between frequency hops and the minimum RF-off time during a hop shall meet local regulatory requirements.

9.3.1.2.10 Frequency-hopping spread-spectrum channelisation

Interrogators that are claimed to operate in single-interrogator environments shall meet local regulations for spread-spectrum channelisation. Interrogators that are claimed to operate in multiple- or dense-interrogator environments shall meet local regulations for spread-spectrum channelisation, unless the channelisation is unregulated, in which case interrogators shall adopt the channelisation described by the algorithm in Figure I.1 (Annex I describes multiple- and dense-interrogator channelised signalling).

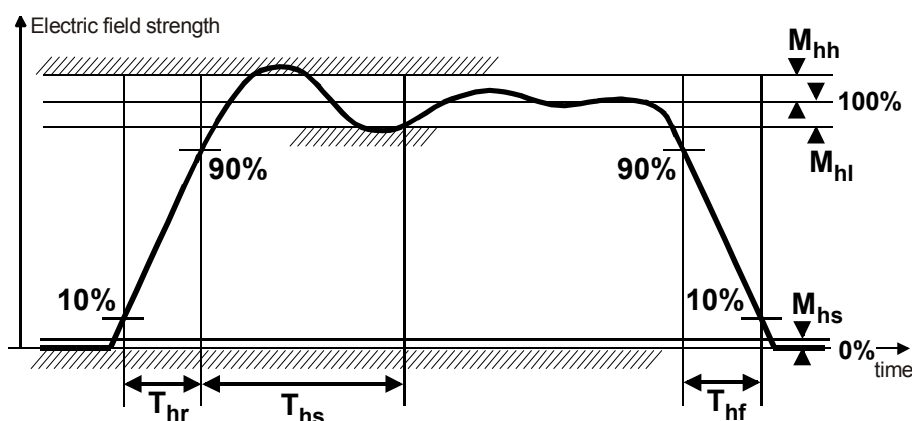


Figure Amd.1-8 — FHSS interrogator RF envelope

Table Amd.1-10 — FHSS waveform parameters

Parameter	Definition	Minimum	Nominal	Maximum	Units
T_{hr}	Rise time			500	μs
T_{hs}	Settling time			1500	μs
T_{hf}	Fall time			500	μs
M_{hs}	Signal level during hop			1	% full scale
M_{hl}	Undershoot			5	% full scale
M_{hh}	Overshoot			5	% full scale

9.3.1.2.11 Transmit mask

Interrogators that are claimed to operate according to this International Standard shall meet local regulations for out-of-channel and out-of-band spurious radio-frequency emissions.

Interrogators that are claimed to operate in multiple-interrogator environments, in addition to meeting local regulations, shall also meet the Multiple-Interrogator Transmit Mask specified in this International Standard:

Multiple-Interrogator Transmit Mask: For an interrogator transmitting random data in channel R , and any other channel $S \neq R$, the ratio of the integrated power $P()$ in channel S to that in channel R shall not exceed the specified values:

$$|R - S| = 1: 10\log_{10}(P(S) / P(R)) < -20 \text{ dB}$$

$$|R - S| = 2: 10\log_{10}(P(S) / P(R)) < -50 \text{ dB}$$

$$|R - S| = 3: 10\log_{10}(P(S) / P(R)) < -60 \text{ dB}$$

$$|R - S| > 3: 10\log_{10}(P(S) / P(R)) < -65 \text{ dB}$$

Where $P()$ denotes the total integrated power in the specified channel. This mask is shown graphically in Figure Amd.1-9, with dBch defined as dB referenced to the integrated power in the reference channel. For any transmit channel R , two exceptions to the mask are permitted, provided that

neither exception exceeds -50 dBch, and

neither exception exceeds local regulatory requirements.

An exception occurs when the integrated power in a channel S exceeds the mask. Each channel that exceeds the mask shall be counted as an exception.

Interrogators that are claimed to operate in dense-interrogator environments shall meet both local regulations and the Transmit Mask shown in Figure Amd.1-9 of this International Standard, except when operating in the optional dense-interrogator mode described in Annex I, in which case they shall instead meet the Dense-interrogator Transmit Mask described below and shown in Figure Amd.1-10. Regardless of the mask used, interrogators certified for operation in dense-interrogator environments shall not be permitted the two exceptions to the transmit mask that are allowed for interrogators certified for operation in multiple-interrogator environments.

Dense-Interrogator Transmit Mask: For interrogator transmissions centred at a frequency f_c , a $2.5/T_{\text{ari}}$ bandwidth R_{BW} also centred at f_c , an offset frequency $f_o = 2.5/T_{\text{ari}}$, and a $2.5/T_{\text{ari}}$ bandwidth S_{BW} centred at $(n \times f_o) + f_c$ (integer n), the ratio of the integrated power $P()$ in S_{BW} to that in R_{BW} with the Interrogator transmitting random data shall not exceed the specified values:

$$|n| = 1: 10\log_{10}(P(S_{BW}) / P(R_{BW})) < -30 \text{ dB}$$

$$|n| = 2: 10\log_{10}(P(S_{BW}) / P(R_{BW})) < -60 \text{ dB}$$

$$|n| > 2: 10\log_{10}(P(S_{BW}) / P(R_{BW})) < -65 \text{ dB}$$

Where $P()$ denotes the total integrated power in the $2.5/T_{\text{ari}}$ reference bandwidth. This mask is shown graphically in Figure Amd.1-10, with dBch defined as dB referenced to the integrated power in the reference channel.

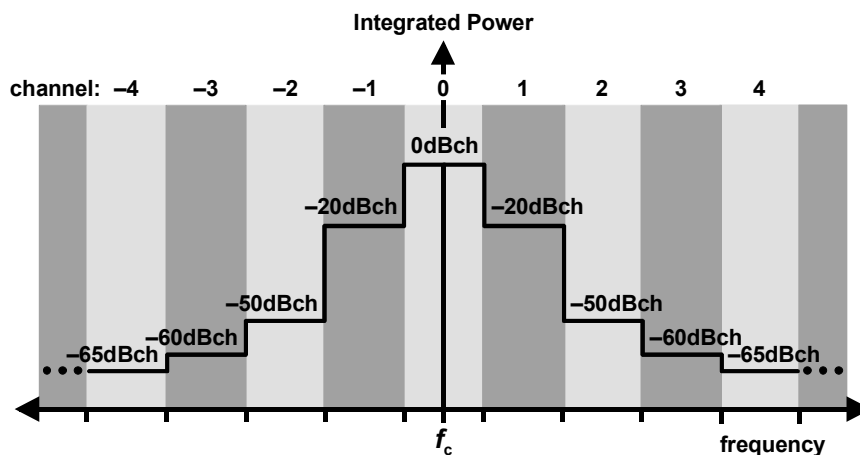


Figure Amd.1-9 — Transmit mask for multiple-interrogator environments

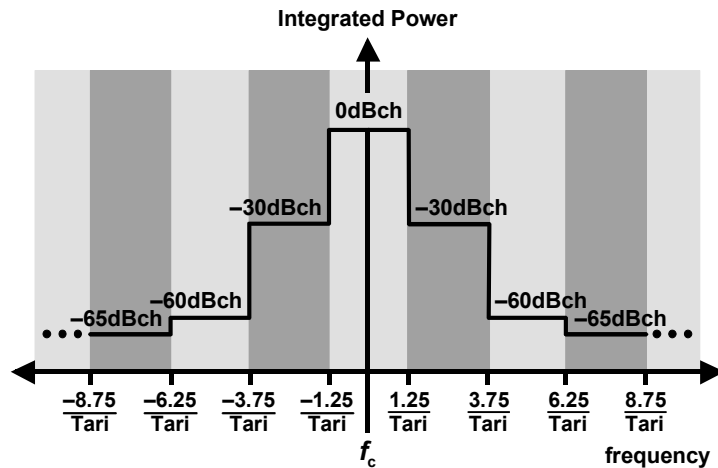


Figure Amd.1-10 — Transmit mask for dense-interrogator environments

9.3.1.3 Tag-to-interrogator (T=>R) communications

A tag communicates with an interrogator using backscatter modulation, in which the tag switches the reflection coefficient of its antenna between two states in accordance with the data being sent.

A tag shall backscatter using a fixed modulation format, data encoding, and data rate for the duration of an inventory round, where “inventory round” is defined in 9.3.2.8. The tag selects the modulation format; the interrogator selects the encoding and data rate by means of the *Query* command that initiates the round. The low values in Figure Amd.1-12, Figure Amd.1-13, Figure Amd.1-14, Figure Amd.1-16, Figure Amd.1-17 and Figure Amd.1-18 correspond to the antenna-reflectivity state the tag exhibits during the CW period prior to a T=>R preamble (e.g. an ASK tag absorbing power), whereas the high values correspond to the antenna-reflectivity state the tag exhibits during the first high pulse of a T=>R preamble (e.g. an ASK tag reflecting power).

9.3.1.3.1 Modulation

Tag backscatter shall use ASK and/or PSK modulation. The tag vendor selects the modulation format. Interrogators shall demodulate both modulation types.

9.3.1.3.2 Data encoding

Tags shall encode the backscattered data as either FM0 baseband or Miller modulation of a subcarrier at the data rate. The interrogator commands the encoding choice.

9.3.1.3.2.1 FM0 baseband

Figure Amd.1-11 shows basis functions and a state diagram for generating FM0 (bi-phase space) encoding. FM0 inverts the baseband phase at every symbol boundary; a data-0 has an additional mid-symbol phase inversion. The state diagram in Figure Amd.1-11 maps a logical data sequence to the FM0 basis functions that are transmitted. The state labels, S_1 – S_4 , indicate four possible FM0-encoded symbols, represented by the two phases of each of the FM0 basis functions. The state labels also represent the FM0 waveform that is transmitted upon entering the state. The labels on the state transitions indicate the logical values of the data sequence to be encoded. For example, a transition from state S_2 to S_3 is disallowed because the resulting transmission would not have a phase inversion on a symbol boundary.

Figure Amd.1-12 shows generated baseband FM0 symbols and sequences. The duty cycle of a 00 or 11 sequence, measured at the modulator output, shall be a minimum of 45% and a maximum of 55%, with a nominal value of 50%. FM0 encoding has memory; consequently, the choice of FM0 sequences in Figure Amd.1-12 depends on prior transmissions. FM0 signalling shall always end with a “dummy” data-1 bit at the end of a transmission, as shown in Figure Amd.1-13.

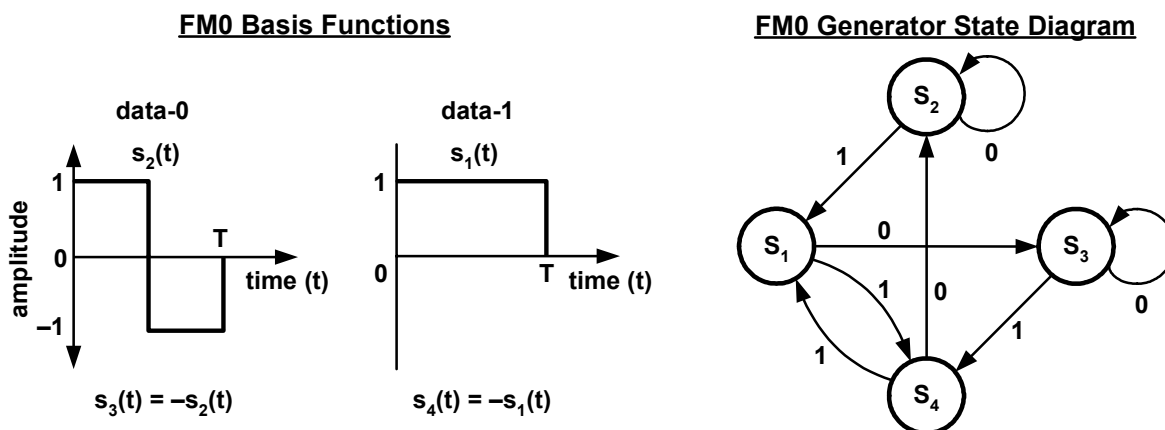


Figure Amd.1-11 — FM0 basis functions and generator state diagram

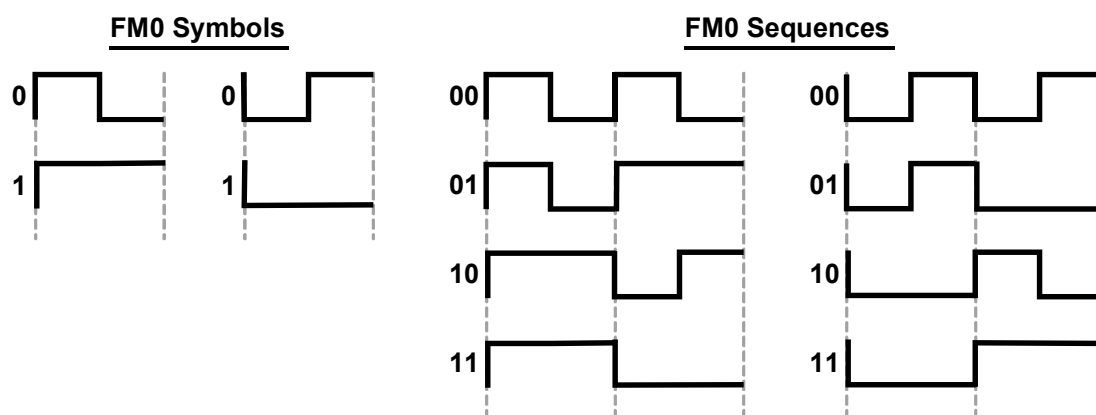


Figure Amd.1-12 — FM0 symbols and sequences

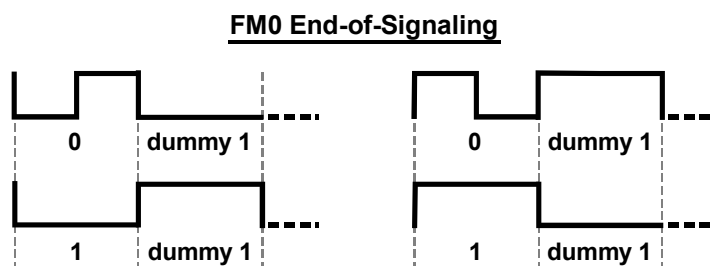


Figure Amd.1-13 — Terminating FM0 transmissions

9.3.1.3.2.2 FM0 preamble

T=>R FM0 signalling shall begin with one of the two preambles shown in Figure Amd.1-14. The choice depends on the value of the TRext bit specified in the *Query* command that initiated the inventory round,

unless a tag is replying to a command that writes to memory, in which case a tag shall use the extended preamble regardless of T_{Rext} (i.e. the tag replies as if T_{Rext}=1 regardless of the T_{Rext} value specified in the *Query*—see 9.3.2.10.3). The “v” shown in Figure Amd.1-14 indicates an FM0 violation (i.e. a phase inversion should have occurred but did not).

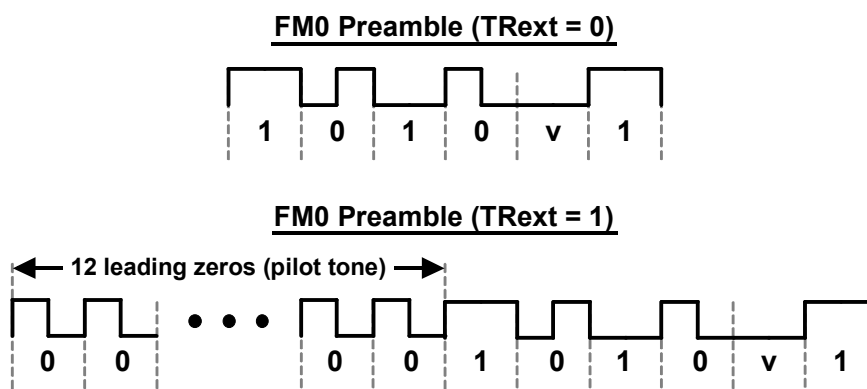


Figure Amd.1-14 — FM0 T=>R preamble

9.3.1.3.2.3 Miller-modulated subcarrier

Figure Amd.1-15 shows basis functions and a state diagram for generating Miller encoding. Baseband Miller inverts its phase between two data-0s in sequence. Baseband Miller also places a phase inversion in the middle of a data-1 symbol. The state diagram in Figure Amd.1-15 maps a logical data sequence to baseband Miller basis functions. The state labels, S_1 – S_4 , indicate four possible Miller-encoded symbols, represented by the two phases of each of the Miller basis functions. The state labels also represent the baseband Miller waveform that is generated upon entering the state. The transmitted waveform is the baseband waveform multiplied by a square-wave at M times the symbol rate. The labels on the state transitions indicate the logical values of the data sequence to be encoded. For example, a transition from state S_1 to S_3 is disallowed because the resulting transmission would have a phase inversion on a symbol boundary between a data-0 and a data-1.

Figure Amd.1-16 shows Miller-modulated subcarrier sequences; the Miller sequence shall contain exactly two, four, or eight subcarrier cycles per bit, depending on the M value specified in the *Query* command that initiated the inventory round (see Figure Amd.1-15). The duty cycle of a 0 or 1 symbol, measured at the modulator output, shall be a minimum of 45% and a maximum of 55%, with a nominal value of 50%. Miller encoding has memory; consequently, the choice of Miller sequences in Figure Amd.1-16 depends on prior transmissions. Miller signalling shall always end with a “dummy” data-1 bit at the end of a transmission, as shown in Figure Amd.1-17.

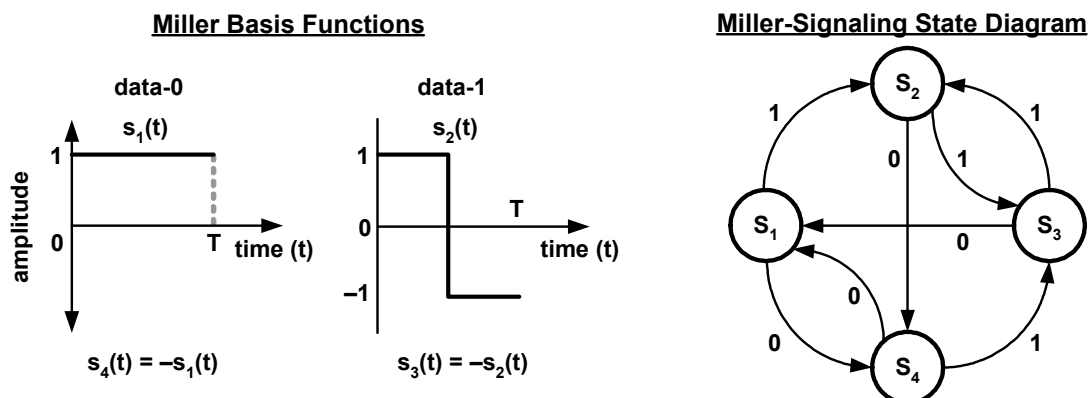


Figure Amd.1-15 — Miller basis functions and generator state diagram

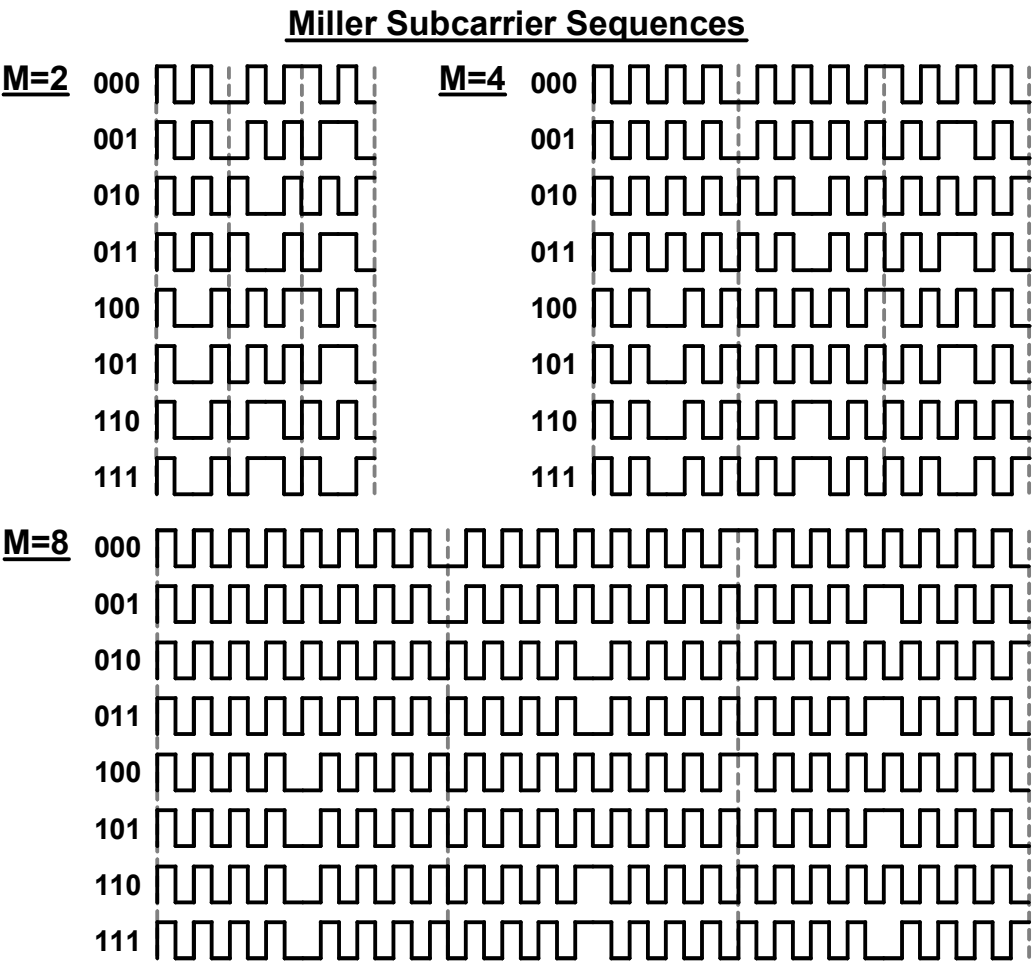


Figure Amd.1-16 — Subcarrier sequences

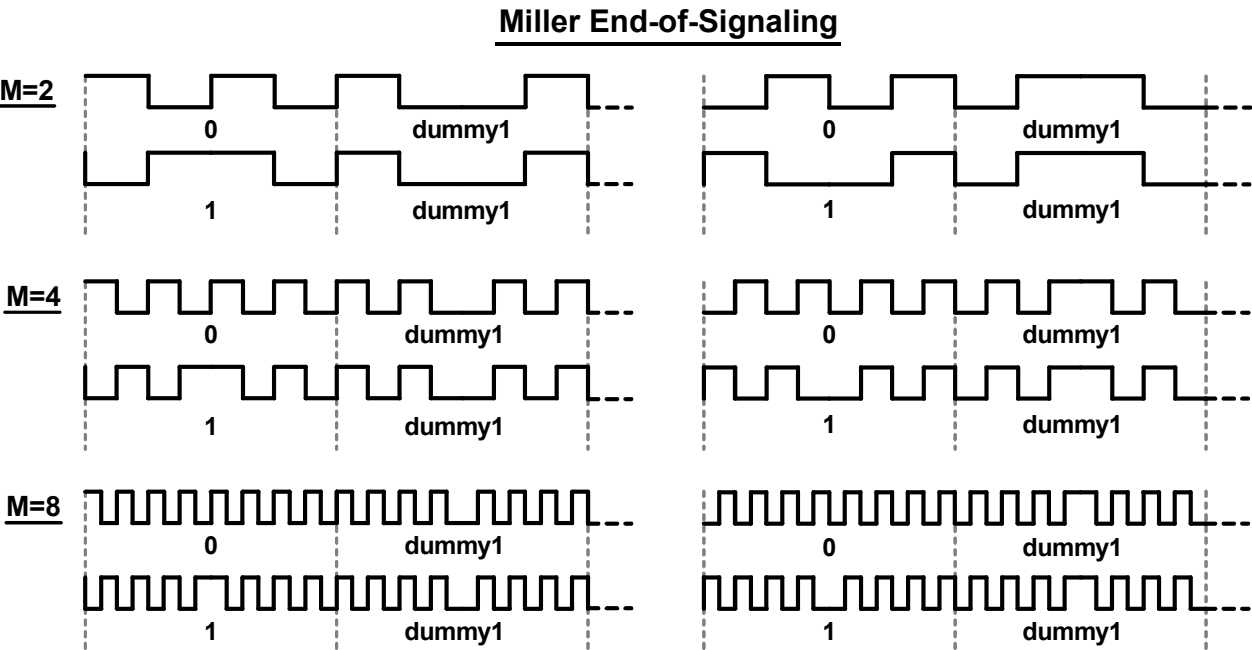


Figure Amd.1-17 — Terminating subcarrier transmissions

9.3.1.3.2.4 Miller subcarrier preamble

T=>R subcarrier signalling shall begin with one of the two preambles shown in Figure Amd.1-18. The choice depends on the value of the TRe_{tx} bit specified in the *Query* command that initiated the inventory round, unless a tag is replying to a command that writes to memory, in which case a tag shall use the extended preamble regardless of TRe_{tx} (i.e. the tag replies as if TRe_{tx}=1 regardless of the TRe_{tx} value specified in the *Query*—see 9.3.2.10.3).

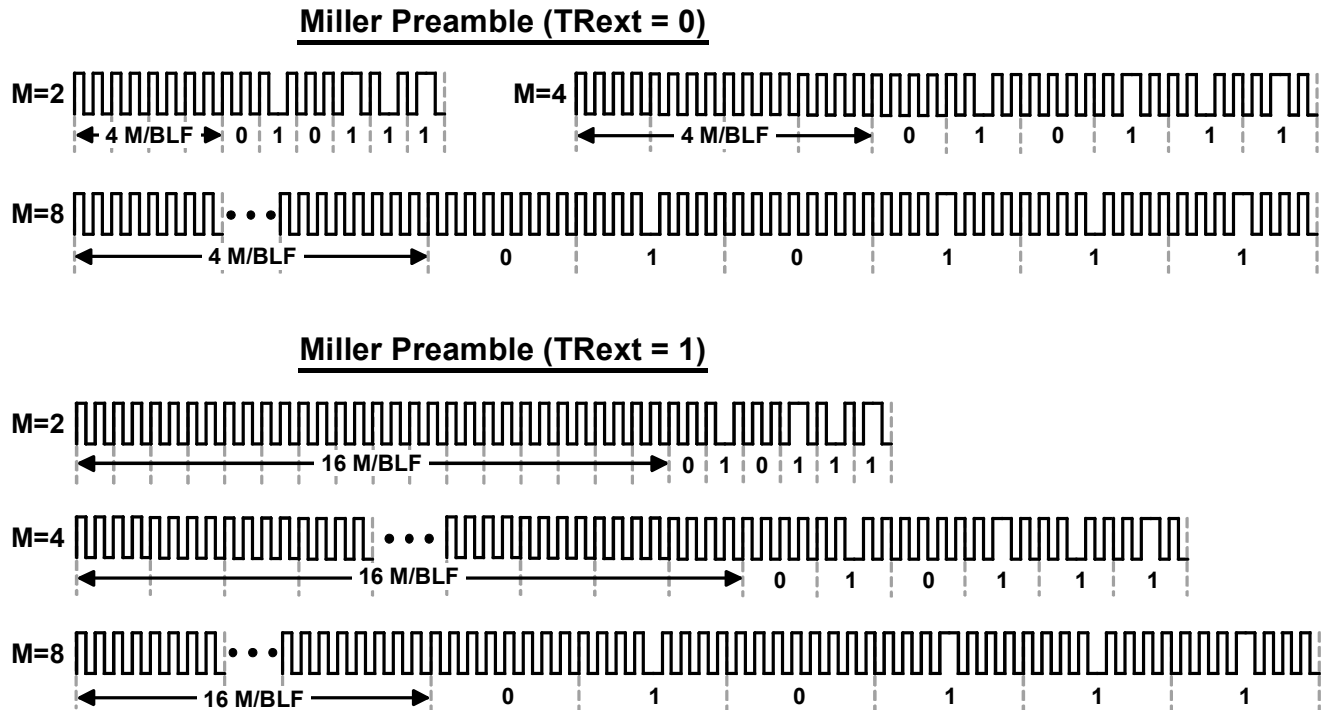


Figure Amd.1-18 — Subcarrier T=>R preamble

9.3.1.3.3 Tag supported Tari values and backscatter link rates

Tags shall support all R=>T Tari values in the range of 6.25µs to 25µs, over all parameters allowed by 9.3.1.2.3.

Tags shall support the T=>R link frequencies and tolerances specified in Table Amd.1-11, and the T=>R data rates specified in Table Amd.1-12. The frequency-variation requirement in Table Amd.1-11 includes both frequency drift and short-term frequency variation during tag response to an interrogator command. The *Query* command that initiates an inventory round specifies DR in Table Amd.1-11 and M in Table Amd.1-12; the preamble that precedes the *Query* specifies TRcal. BLF is computed using Eq. (1). These four parameters together define the backscatter frequency, modulation type (FM0 or Miller), and T=>R data rate for the round (see also 9.3.1.2.8).

Table Amd.1-11 — Tag-to-interrogator link frequencies

DR: Divide Ratio	TRcal ^a (μs $\pm$ 1%)	BLF: Link Frequency (kHz)	Frequency Tolerance FT (-25 °C to 40 °C)	Frequency Tolerance FT (-40 °C to 65 °C)	Frequency variation during backscatter
64/3	33.3	640	+ / – 15%	+ / – 15%	+ / – 2.5%
	33.3 < TRcal < 66.7	320 < BLF < 640	+ / – 22%	+ / – 22%	+ / – 2.5%
	66.7	320	+ / – 10%	+ / – 15%	+ / – 2.5%
	66.7 < TRcal < 83.3	256 < BLF < 320	+ / – 12%	+ / – 15%	+ / – 2.5%
	83.3	256	+ / – 10%	+ / – 10%	+ / – 2.5%
	83.3 < TRcal $\leq$ 133.3	160 $\leq$ BLF < 256	+ / – 10%	+ / – 12%	+ / – 2.5%
	133.3 < TRcal $\leq$ 200	107 $\leq$ BLF < 160	+ / – 7%	+ / – 7%	+ / – 2.5%
	200 < TRcal $\leq$ 225	95 $\leq$ BLF < 107	+ / – 5%	+ / – 5%	+ / – 2.5%
8	17.2 $\leq$ TRcal < 25	320 < BLF $\leq$ 465	+ / – 19%	+ / – 19%	+ / – 2.5%
	25	320	+ / – 10%	+ / – 15%	+ / – 2.5%
	25 < TRcal < 31.25	256 < BLF < 320	+ / – 12%	+ / – 15%	+ / – 2.5%
	31.25	256	+ / – 10%	+ / – 10%	+ / – 2.5%
	31.25 < TRcal < 50	160 < BLF < 256	+ / – 10%	+ / – 10%	+ / – 2.5%
	50	160	+ / – 7%	+ / – 7%	+ / – 2.5%
	50 < TRcal $\leq$ 75	107 $\leq$ BLF < 160	+ / – 7%	+ / – 7%	+ / – 2.5%
	75 < TRcal $\leq$ 200	40 $\leq$ BLF < 107	+ / – 4%	+ / – 4%	+ / – 2.5%
^A Allowing two different TRcal values (with two different DR values) to specify the same BLF offers flexibility in specifying Tari and RTcal.					

Table Amd.1-12 — Tag-to-interrogator data rates

M: Number of subcarrier cycles per symbol	Modulation type	Data rate (kbit/s)
1	FM0 baseband	BLF
2	Miller subcarrier	BLF/2
4	Miller subcarrier	BLF/4
8	Miller subcarrier	BLF/8

9.3.1.3.4 Tag power-up timing

Tags energized by an interrogator shall be capable of receiving and acting on interrogator commands within a period not exceeding the maximum settling-time interval specified in Table Amd.1-8 or Table Amd.1-10, as appropriate (i.e. within T_s or T_{hs} , respectively).

9.3.1.3.5 Minimum operating field strength and backscatter strength

The tag manufacturer shall specify:

— the free-space sensitivity,

- the minimum relative backscattered modulated power (ASK modulation) or change in radar cross-section or equivalent (phase modulation), and
 - the manufacturer's normal operating conditions,
- for the tag mounted on one or more manufacturer-selected materials.

9.3.1.4 Transmission order

The transmission order for all R=>T and T=>R communications shall be most-significant bit (MSB) first.

Within each message, the most-significant word shall be transmitted first

Within each word, the MSB shall be transmitted first.

9.3.1.5 Cyclic Redundancy Check (CRC)

A CRC is a cyclic-redundancy check that a tag uses to ensure the validity of certain R=>T commands, and an interrogator uses to ensure the validity of certain backscattered T=>R replies. This International Standard uses two CRC types: (i) a CRC-16 and (ii) a CRC-5.

To generate a CRC-16 a tag or interrogator shall first generate the CRC-16 precursor shown in Table Amd.1-13, and then take the ones-complement of the generated precursor to form the CRC-16.

A tag or interrogator shall verify the integrity of a received message that uses a CRC-16. The tag or interrogator may use of the methods describe in Annex A to verify the CRC-16.

The CRC-16 that protects a tag's Ull is computed and stored by a tag at power up - See Clause 9.3.2.10.

Tag shall append a CRC-16 to those replies that use a CRC-16. – See Clause 9.3.2.10 for command specific reply formats.

To generate a CRC-5 an interrogator shall use the definition in Table Amd.1-14.

A tag shall verify the integrity of a received message that uses a CRC-5. The tag may use the method as described in Annex A to verify a CRC-5.

Interrogator shall append the appropriate CRC to R=>T transmissions as specified in Table Amd.1-18.

Table Amd.1-13 — CRC-16 precursor

CRC-16 precursor				
CRC Type	Length	Polynomial	Preset	Residue
ISO/IEC 13239	16 bits	$x^{16} + x^{12} + x^5 + 1$	FFFF _h	1D0F _h

Table Amd.1-14 — CRC-5 definition

CRC-5 Definition				
CRC Type	Length	Polynomial	Preset	Residue
—	5 bits	$x^5 + x^3 + 1$	01001 ₂	00000 ₂

9.3.1.6 Link timing

Figure Amd.1-19 illustrates R=>T and T=>R link timing. The figure (not drawn to scale) defines interrogator interactions with a tag population. Table Amd.1-15 shows the timing requirements for Figure Amd.1-19, while 9.3.2.10 describes the commands. Tags and interrogators shall meet all timing requirements shown in

Table Amd.1-15. RTcal is defined in 9.3.1.2.8; T_{pri} is the T=>R link period ($T_{pri} = 1 / BLF$). As described in 9.3.1.2.8, an interrogator shall use a fixed R=>T link rate for the duration of an inventory round; prior to changing the R=>T link rate, an interrogator shall transmit CW for a minimum of 8 RTcal.

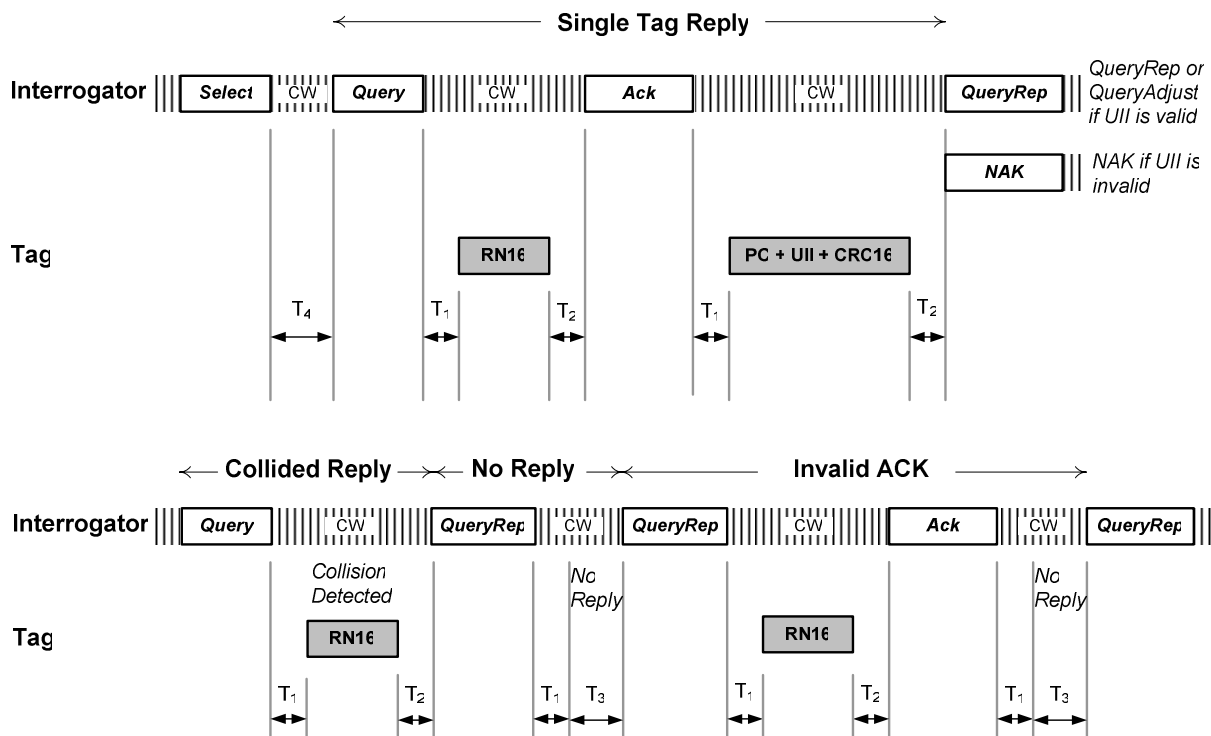


Figure Amd.1-19 — Link timing

Table Amd.1-15 — Link timing parameters

Parameter	Minimum	Nominal	Maximum	Description
T_1	$\text{MAX}(\text{RTcal}, 10T_{pri}) \times (1 - FT) - 2\mu\text{s}$	$\text{MAX}(\text{RTcal}, 10T_{pri})$	$\text{MAX}(\text{RTcal}, 10T_{pri}) \times (1 + FT) + 2\mu\text{s}$	Time from interrogator transmission to tag response (specifically, the time from the last rising edge of the last bit of the interrogator transmission to the first rising edge of the tag response), measured at the tag's antenna terminals.
T_2	$3.0T_{pri}$		$20.0T_{pri}$	Interrogator response time required if a tag is to demodulate the interrogator signal, measured from the last falling edge of the last bit of the tag response to the first falling edge of the interrogator transmission.
T_3	$0.0T_{pri}$			Time an interrogator waits, after T_1 , before it issues another command
T_4	2.0 RTcal			Minimum time between interrogator commands

The following items apply to the requirements specified in Table Amd.1-15:

- 1) T_{pri} denotes either the commanded period of an FM0 symbol or the commanded period of a single subcarrier cycle, as appropriate.

- 2) A tag may exceed the maximum value for T_1 when responding to commands that write to memory — see, for example, 9.3.2.10.3.3.
- 3) The maximum value for T_2 shall apply only to tags in the **reply** or **acknowledged** states (see 9.3.2.4.3 and 9.3.2.4.4). For a tag in the **reply** or **acknowledged** states, if T_2 expires (i.e. reaches its maximum value):
 - Without the tag receiving a valid command, the tag shall transition to the **arbitrate** state (see 9.3.2.4.2),
 - During the reception of a valid command, the tag shall execute the command,
 - During the reception of an invalid command, the tag shall transition to **arbitrate** upon determining that the command is invalid.
 - In all other states the maximum value for T_2 shall be unrestricted. “Invalid command” is defined in 9.3.2.10.
- 4) An interrogator may transmit a new command prior to interval T_2 (i.e. during a tag response). In this case the responding tag is not required to demodulate or otherwise act on the new command, and may undergo a power-on reset.
- 5) FT is the frequency tolerance specified in Table Amd.1-11.
- 6) $T_1 + T_3$ shall not be less than T_4 .

9.3.2 Tag selection, inventory, and access

Tag selection, inventory, and access may be viewed as the lowest level in the data link layer of a layered network communication system.

9.3.2.1 Tag memory

Tag memory shall be logically separated into four distinct banks, each of which may comprise zero or more memory words. A logical memory map is shown in Figure Amd.1-20. The memory banks are:

- **Reserved memory** shall contain the kill and/or access passwords, if passwords are implemented on the tag. The kill password shall be stored at memory addresses 00_h to $1F_h$; the access password shall be stored at memory addresses 20_h to $3F_h$. See 9.3.2.1.1.
- **UII memory** shall contain a CRC-16 at memory addresses 00_h to $0F_h$, Protocol-Control (PC) bits at memory addresses 10_h to $1F_h$, and a code (such as an UII, and hereafter referred to as an UII) that identifies the object to which the tag is or will be attached beginning at address 20_h . See 9.3.2.1.2.
- **TID memory** shall contain an 8-bit ISO/IEC 15963 allocation class identifier at memory locations 00_h to 07_h . TID memory shall contain sufficient identifying information above 07_h for an interrogator to uniquely identify the custom commands and/or optional features that a tag supports. See 9.3.2.1.3.
- **User memory** allows user-specific data storage. See 9.3.2.1.4.

The logical addressing of all memory banks shall begin at zero (00_h). The physical memory map is vendor-specific. Commands that access memory have a MemBank parameter that selects the bank, and an address parameter, specified using the EBV format described in Annex D, to select a particular memory location within that bank. When tags backscatter memory contents, this backscatter shall fall on word boundaries (except in the case of a truncated reply – see 9.3.2.10.1.1).

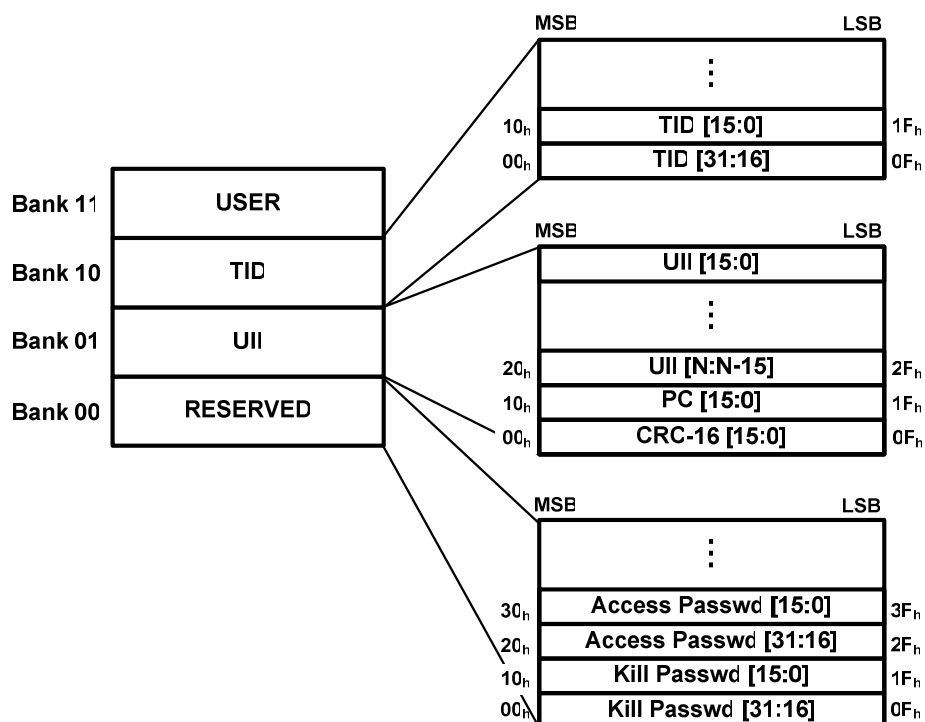


Figure Amd.1-20 — Logical memory map

MemBank is defined as follows:

- 00₂ Reserved
- 01₂ UII
- 10₂ TID
- 11₂ User

Operations in one logical memory bank shall not access memory locations in another bank.

Memory writes, detailed in 9.3.2.9, involve the transfer of 16-bit words from interrogator to tag. A *Write* command writes 16 bits (i.e. one word) at a time, using link cover-coding to obscure the data during R=>T transmission. The optional *BlockWrite* command writes one or more 16-bit words at a time, without link cover-coding. The optional *BlockErase* command erases one or more 16-bit words at a time. A *Write*, *BlockWrite*, or *BlockErase* shall not alter a tag's killed status regardless of the memory address (whether valid or invalid) specified in the command.

Interrogators may lock, permanently lock, unlock, or permanently unlock memory, thereby preventing or allowing subsequent changes (as appropriate). See 9.3.2.9 and 9.3.2.10.3.5 for a detailed description of memory locking and unlocking. The kill and access passwords are individually lockable, as are UII, TID, and User memory. If the kill and/or access passwords are locked they are usable by only the *Kill* and *Access* commands, respectively, and are rendered both unwriteable and unreadable by any other command. The UII, TID, and User memory banks are always readable regardless of their lock status.

9.3.2.1.1 Reserved Memory

Reserved memory contains the kill (see 9.3.2.1.1.1) and/or access (see 9.3.2.1.1.2) passwords, if passwords are implemented on the tag. If a tag does not implement the kill and/or access password(s), the tag shall logically operate as though it has zero-valued password(s) that are permanently read/write locked (see 9.3.2.10.3.5), and the corresponding physical memory locations in Reserved memory need not exist.

9.3.2.1.1.1 Kill password

The kill password is a 32-bit value stored in Reserved memory 00_h to 1F_h, MSB first. The default (unprogrammed) value shall be zero. An interrogator shall use a kill password once, to kill the tag and render it nonresponsive thereafter. A tag shall not execute a kill operation if its kill password is zero. A tag that does

not implement a kill password operates as if it has a zero-valued kill password that is permanently read/write locked.

9.3.2.1.1.2 Access password

The access password is a 32-bit value stored in Reserved memory 20_h to $3F_h$, MSB first. The default (unprogrammed) value shall be zero. A tag with a nonzero access password shall require an interrogator to issue this password before transitioning to the **secured** state. A tag that does not implement an access password operates as if it has a zero-valued access password that is permanently read/write locked.

9.3.2.1.2 Ull Memory

Ull memory contains a CRC-16 at memory addresses 00_h to $0F_h$, PC bits at memory locations 10_h to $1F_h$, and a Ull beginning at address 20_h . The CRC-16, PC, and Ull shall be stored MSB first (the Ull's MSB is stored in location 20_h).

The CRC-16 is described in 9.3.2.1.2.1.

The PC, as described in 9.3.2.1.2.2, is subdivided into a Ull length field in memory locations 10_h to 14_h , RFU bits in memory locations 15_h and 16_h , and a Numbering System Identifier (NSI) in memory locations 17_h to $1F_h$.

The Ull is a code that identifies the object to which a tag is affixed. The Ull for EPCglobal™ Applications is described in 9.3.2.1.2.3; the Ull for non-EPCglobal™ Applications is described in 9.3.2.1.2.4. Interrogators may issue a *Select* command that includes all or part of the Ull in the mask. Interrogators may issue an *ACK* command to cause a tag to backscatter its PC, Ull, and CRC-16 (under certain circumstances the tag may truncate its reply — see 9.3.2.10.1.1). An Interrogator may issue a *Read* command to read all or part of the Ull.

9.3.2.1.2.1 CRC-16

The PC and Ull are protected by the CRC-16 that a tag backscatters during an inventory operation. Because interrogators may issue a *Select* command that includes all or part of this CRC-16 in the mask, and may issue a *Read* command to cause the tag to backscatter this CRC-16, this CRC-16 is logically mapped into Ull memory. At power-up a tag shall compute this CRC-16 over Ull memory location 10_h to the end of the Ull (not necessarily to the end of Ull memory, but to the end of the Ull specified by the length field in the PC — see 9.3.2.1.2.2) and map the computed CRC-16 into Ull memory 00_h to $0F_h$, MSB first. Because the {PC+ Ull} is stored in Ull memory on word boundaries, this CRC-16 shall be computed on word boundaries. tags shall finish this CRC-16 computation and memory mapping by the end of interval T_s or T_{hs} (as appropriate) in Figure Amd.1-6 or Figure Amd.1-8, respectively. tags shall not recalculate this CRC-16 for a truncated reply (see 9.3.2.10.1.1).

9.3.2.1.2.2 Protocol-control (PC) bits

The PC bits contain physical-layer information that a tag backscatters with its Ull during an inventory operation. There are 16 PC bits, stored in Ull memory at addresses 10_h to $1F_h$, with bit values defined as follows:

Bits $10_h - 14_h$: The length of the (PC + Ull) that a tag backscatters, in words:

00000₂: One word (addresses 10_h to $1F_h$ in Ull memory).

00001₂: Two words (addresses 10_h to $2F_h$ in Ull memory).

00010₂: Three words (addresses 10_h to $3F_h$ in Ull memory).

⋮

11111₂: 32 words (addresses 10_h to $20F_h$ in Ull memory).

Bits $15_h - 16_h$: RFU (shall be set to 00₂).

Bits $17_h - 1F_h$: A numbering system identifier (NSI). The MSB of the NSI is stored in memory location 17_h . If bit 17_h contains a logical 0, then the Application is referred to as an EPCglobal™ Application and PC bits $18_h - 1F_h$ shall be as defined in the EPCglobal™ Tag Data Standards. If bit 17_h contains a logical 1, then the Application is referred to as a non-EPCglobal™ Application and PC bits $18_h - 1F_h$

shall contain the entire AFI defined in ISO/IEC 15961. The default value for bits $18_h - 1F_h$ is 00000000_2

The default (unprogrammed) PC value shall be 0000_h .

During truncated replies a tag substitutes 00000_2 for the PC bits — see 9.3.2.10.1.1.

If an interrogator modifies the UII length during a memory write, and it wishes the tag to subsequently backscatter the modified UII, then it must write the length of the new or updated (PC + UII) into the first 5 bits of the tag's PC. A tag shall backscatter an error code (see Annex K) if an interrogator attempts to write a (PC + UII) length that is not supported by the tag to the first 5 bits of the tag's PC.

At power-up a tag shall compute its CRC-16 over the number of (PC + UII) words designated by the first 5 bits of the PC rather than over the length of the entire UII memory (see 9.3.2.1.2.1).

9.3.2.1.2.3 UII for an EPCglobal™ Application

The UII structure for an EPCglobal™ Application shall be as defined in the EPCglobal™ Tag Data Standards.

9.3.2.1.2.4 UII for a non-EPCglobal™ Application

The UII structure for a non-EPCglobal™ Application shall be as defined in ISO/IEC 15961.

9.3.2.1.3 TID memory

TID memory locations 00_h to 07_h shall contain one of two ISO/IEC 15963 class-identifier values — either $E0_h$ or $E2_h$. TID memory locations above 07_h shall be defined according to the registration authority defined by this class-identifier value and shall contain, at a minimum, sufficient identifying information for an Interrogator to uniquely identify the custom commands and/or optional features that a tag supports. TID memory may also contain tag- and vendor-specific data (for example, a tag serial number).

NOTE The tag manufacturer assigns the class-identifier value (i.e. $E0_h$ or $E2_h$), for which ISO/IEC 15963 defines the registration authorities. The class-identifier does not specify the Application. If the class identifier is $E0_h$, TID memory locations 08_h to $0F_h$ contain an 8-bit manufacturer identifier, TID memory locations 10_h to $3F_h$ contain a 48-bit tag serial number (assigned by the tag manufacturer), the composite 64-bit tag ID (i.e. TID memory 00_h to $3F_h$) is unique among all classes of tags defined in ISO/IEC 15963, and TID memory is permalocked at the time of manufacture. If the class identifier is $E2_h$, TID memory locations 08_h to 13_h contain a 12-bit tag mask-designer identifier (obtainable from the registration authority), TID memory locations 14_h to $1F_h$ contain a vendor-defined 12-bit tag model number, and the usage of tag memory locations above $1F_h$ is defined in version 1.3 and above of the EPCglobal™ Tag Data Standards.

9.3.2.1.4 User memory

A tag may contain User memory. User memory allows user-specific data storage.

9.3.2.1.4.1 User memory for an EPCglobal™ Application

If User memory is included on a tag then its encoding shall be as defined in the EPCglobal™ Tag Data Standards.

9.3.2.1.4.2 User memory for a non-EPCglobal™ Application

If User memory is included on a tag then User memory locations 00_h to 07_h shall be the DSFID as defined in ISO/IEC 15961. The encoding of User memory locations above 07_h shall be as defined in ISO/IEC 15962.

9.3.2.2 Sessions and inventoried flags

Interrogators shall support and tags shall provide 4 sessions (denoted S0, S1, S2, and S3). Tags shall participate in one and only one session during an inventory round. Two or more interrogators can use

sessions to independently inventory a common tag population. The sessions concept is illustrated in Figure Amd.1-21.

Tags shall maintain an independent **inventoried** flag for each session. Each of the four **inventoried** flags has two values, denoted *A* and *B*. At the beginning of each and every inventory round an interrogator chooses to inventory either *A* or *B* tags in one of the four sessions. Tags participating in an inventory round in one session shall neither use nor modify the **inventoried** flag for a different session. The **inventoried** flags are the only resource a tag provides separately and independently to a given session; all other tag resources are shared among sessions.

After singulating a tag an interrogator may issue a command that causes the tag to invert its **inventoried** flag for that session (i.e. $A \rightarrow B$ or $B \rightarrow A$).

The following example illustrates how two interrogators can use sessions and **inventoried** flags to independently and completely inventory a common tag population, on a time-interleaved basis:

Interrogator #1 powers-on, then

It initiates an inventory round during which it singulates *A* tags in session S2 to *B*,

It powers off.

Interrogator #2 powers-on, then

It initiates an inventory round during which it singulates *B* tags in session S3 to *A*,

It powers off.

This process repeats until interrogator #1 has placed all tags in session S2 into *B*, after which it inventories the tags in session S2 from *B* back to *A*. Similarly, interrogator #2 places all tags in session S3 into *A*, after which it inventories the tags in session S3 from *A* back to *B*. By this multi-step procedure each interrogator can independently inventory all tags in its field, regardless of the initial state of their **inventoried** flags.

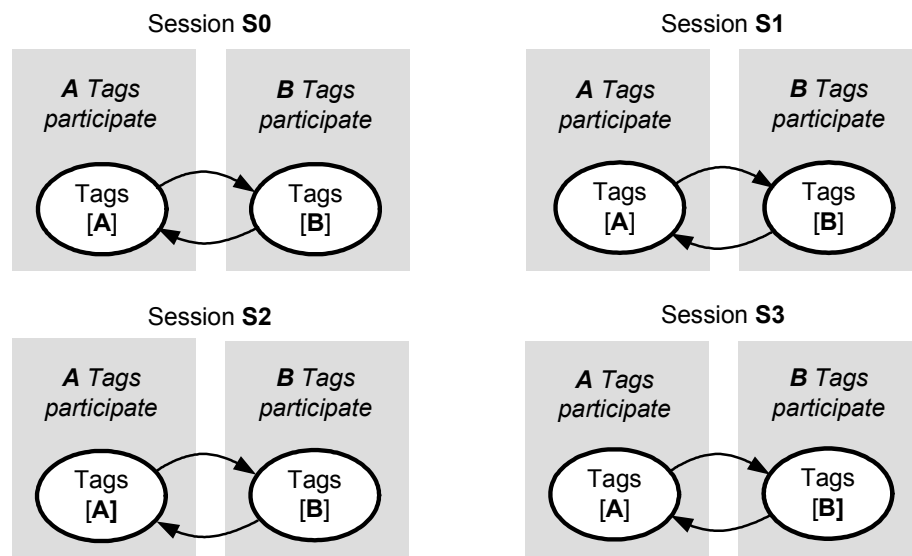


Figure Amd.1-21 — Session diagram

A tag's **inventoried** flags shall have the persistence times shown in Table Amd.1-16. A tag shall power-up with its **inventoried** flags set as follows:

The S0 **inventoried** flag shall be set to *A*.

The S1 **inventoried** flag shall be set to either *A* or *B*, depending on its stored value, unless the flag was set longer in the past than its persistence time, in which case the tag shall power-up with its S1 **inventoried** flag set to *A*. Because the S1 **inventoried** flag is not automatically refreshed, it may revert from *B* to *A* even when the tag is powered.

The S2 **inventoried** flag shall be set to either *A* or *B*, depending on its stored value, unless the tag has lost power for a time greater than its persistence time, in which case the tag shall power-up with the S2 **inventoried** flag set to *A*.

The S3 **inventoried** flag shall be set to either *A* or *B*, depending on its stored value, unless the tag has lost power for a time greater than its persistence time, in which case the tag shall power-up with its S3 **inventoried** flag set to *A*.

A tag shall set any of its inventoried flags to either *A* or *B* in 2 ms or less, regardless of the initial flag value. A tag shall refresh its S2 and S3 flags while powered, meaning that every time a tag loses power its S2 and S3 **inventoried** flags shall have the persistence times shown in Table Amd.1-16. The value of the S1 **inventoried** flag shall not change as a result of a persistence timeout while a tag is participating in an inventory round. If the S1 persistence time expires during an inventory round then the tag shall change its S1 flag to *A* at the end of the round.

9.3.2.3 Selected flag

The tag shall implement a **selected** flag, **SL**, which an interrogator may assert or de-assert using a *Select* command. The *Sel* parameter in the *Query* command allows an interrogator to inventory tags that have **SL** either asserted or de-asserted (i.e. **SL** or **~SL**), or to ignore the flag and inventory tags regardless of their **SL** value. **SL** is not associated with any particular session; **SL** may be used in any session and is common to all sessions.

A tag's **SL** flag shall have the persistence times shown in Table Amd.1-16. A tag shall power-up with its **SL** flag either asserted or de-asserted, depending on the stored value, unless the tag has lost power for a time greater than the **SL** persistence time, in which case the tag shall power-up with its **SL** flag de-asserted (set to **~SL**). A tag shall be capable of asserting or de-asserting its **SL** flag in 2 ms or less, regardless of the initial flag value. A tag shall refresh its **SL** flag when powered, meaning that every time a tag loses power its **SL** flag shall have the persistence times shown in Table Amd.1-16.

Table Amd.1-16 — Tag flags and persistence values

Flag	Required persistence
S0 inventoried flag	Tag energized: Indefinite Tag not energized: None
S1 inventoried flag ^a	Tag energized: –25 °C to +40 °C: 500ms < persistence < 5s –40 °C to +65 °C: Not specified Tag not energized: –25 °C to +40 °C: 500ms < persistence < 5s –40 °C to +65 °C: Not specified
S2 inventoried flag ^a	Tag energized: Indefinite Tag not energized: –25 °C to +40 °C: 2s < persistence –40 °C to +65 °C: Not specified
S3 inventoried flag ^a	Tag energized: Indefinite Tag not energized: –25 °C to +40 °C: 2s < persistence –40 °C to +65 °C: Not specified
Selected (SL) flag ^a	Tag energized: Indefinite Tag not energized: –25 °C to +40 °C: 2s < persistence –40 °C to +65 °C: Not specified

The following requirement applies to those items denoted with a superscript “a” in Table Amd.1-16: For a randomly chosen and sufficiently large tag population, 95% of the tag persistence times shall meet the persistence requirements in Table Amd.1-16, with a 90% confidence interval.

9.3.2.4 Tag states and slot counter

Tags shall implement the states and the slot counter shown in Figure Amd.1-22. Annex E shows the associated state-transition tables; Annex F shows the associated command-response tables.

9.3.2.4.1 Ready state

Tags shall implement a **ready** state. **Ready** can be viewed as a “holding state” for energized tags that are neither killed nor currently participating in an inventory round. Upon entering an energizing RF field a tag that is not killed shall enter **ready**. The tag shall remain in **ready** until it receives a *Query* command (see 9.3.2.10.2.1) whose inventoried parameter (for the session specified in the *Query*) and sel parameter match its current flag values. Matching tags shall draw a Q-bit number from their RNG (see 9.3.2.5), load this number into their slot counter, and transition to the **arbitrate** state if the number is nonzero, or to the **reply** state if the number is zero. If a tag in any state except **killed** loses power it shall return to **ready** upon regaining power.

9.3.2.4.2 Arbitrate state

Tags shall implement an **arbitrate** state. **Arbitrate** can be viewed as a “holding state” for tags that are participating in the current inventory round but whose slot counters (see 9.3.2.4.8) hold nonzero values. A tag in **arbitrate** shall decrement its slot counter every time it receives a *QueryRep* command (see 9.3.2.10.2.3) whose session parameter matches the session for the inventory round currently in progress, and it shall transition to the **reply** state and backscatter an RN16 when its slot counter reaches 0000_h. Tags that return to **arbitrate** (for example, from the **reply** state) with a slot value of 0000_h shall decrement their slot counter from 0000_h to 7FFF_h at the next *QueryRep* (with matching session) and, because their slot value is now nonzero, shall remain in **arbitrate**.

9.3.2.4.3 Reply state

Tags shall implement a **reply** state. Upon entering **reply** a tag shall backscatter an RN16. If the tag receives a valid acknowledgement (*ACK*) it shall transition to the **acknowledged** state, backscattering its PC, UII and CRC-16. If the tag fails to receive an *ACK* within time $T_{2(max)}$, or receives an invalid *ACK* or an *ACK* with an erroneous RN16, it shall return to **arbitrate**. Tag and interrogator shall meet all timing requirements specified in Table Amd.1-15.

9.3.2.4.4 Acknowledged state

Tags shall implement an **acknowledged** state. A tag in **acknowledged** may transition to any state except **killed**, depending on the received command (see Figure Amd.1-22). If a tag in the **acknowledged** state receives a valid *ACK* containing the correct RN16 it shall re-backscatter its PC, UII, and CRC-16. If a tag in the **acknowledged** state fails to receive a valid command within time $T_{2(max)}$ it shall return to **arbitrate**. Tag and interrogator shall meet all timing requirements specified in Table Amd.1-15.

9.3.2.4.5 Open state

Tags shall implement an **open** state. A tag in the **acknowledged** state whose access password is nonzero shall transition to **open** upon receiving a *Req_RN* command, backscattering a new RN16 (denoted handle) that the interrogator shall use in subsequent commands and the tag shall use in subsequent replies. Tags in the **open** state can execute all access commands except *Lock*. A tag in **open** may transition to any state except **acknowledged**, depending on the received command (see Figure Amd.1-22). If a tag in the **open** state receives a valid *ACK* containing the correct handle it shall re-backscatter its PC, UII, and CRC-16. Tag and interrogator shall meet all timing requirements specified in Table Amd.1-15 except $T_{2(max)}$; in the **open** state the maximum delay between tag response and interrogator transmission is unrestricted.

9.3.2.4.6 Secured state

Tags shall implement a **secured** state. A tag in the **acknowledged** state whose access password is zero shall transition to **secured** upon receiving a *Req_RN* command, backscattering a new RN16 (denoted handle) that the interrogator shall use in subsequent commands and the tag shall use in subsequent replies. A tag in the **open** state whose access password is nonzero shall transition to **secured** upon receiving a valid *Access* command sequence, maintaining the same handle that it previously backscattered when it transitioned from the **acknowledged** to the **open** state. Tags in the **secured** state can execute all access commands. A tag in **secured** may transition to any state except **open** or **acknowledged**, depending on the received command (see Figure Amd.1-22). If a tag in the **secured** state receives a valid *ACK* containing the correct handle it shall re-backscatter its PC, UII, and CRC-16. Tag and interrogator shall meet all timing requirements specified in Table Amd.1-15 except $T_{2(max)}$; in the **secured** state the maximum delay between tag response and interrogator transmission is unrestricted.

9.3.2.4.7 Killed state

Tags shall implement a **killed** state. A tag in either the **open** or **secured** states shall enter the **killed** state upon receiving a *Kill* command sequence (see 9.3.2.10.3.4) with a valid nonzero kill password and valid handle. *Kill* permanently disables a tag. Upon entering the **killed** state a tag shall notify the interrogator that the kill operation was successful, and shall not respond to an interrogator thereafter. Killed tags shall remain in the **killed** state under all circumstances, and shall immediately enter killed upon subsequent power-ups. A kill operation is not reversible.

9.3.2.4.8 Slot counter

Tags shall implement a 15-bit slot counter. Upon receiving a *Query* or *QueryAdjust* command a tag shall preload into its slot counter a value between 0 and 2^Q-1 , drawn from the tag's RNG (see 9.3.2.5). Q is an integer in the range (0,15). A *Query* specifies Q ; a *QueryAdjust* may modify Q from the prior *Query*.

Tags in the **arbitrate** state shall decrement their slot counter every time they receive a *QueryRep*, transitioning to the **reply** state and backscattering an RN16 when their slot counter reaches 0000_h . Tags whose slot counter reached 0000_h , who replied, and who were not acknowledged (including tags that responded to an original *Query* and were not acknowledged) shall return to **arbitrate** with a slot value of 0000_h and shall decrement this slot value from 0000_h to $7FFF_h$ at the next *QueryRep*. The slot counter shall be capable of continuous counting, meaning that, after the slot counter rolls over to $7FFF_h$ it begins counting down again, thereby effectively preventing subsequent replies until the tag loads a new random value into its slot counter. See also Annex L.

9.3.2.5 Tag random or pseudo-random number generator

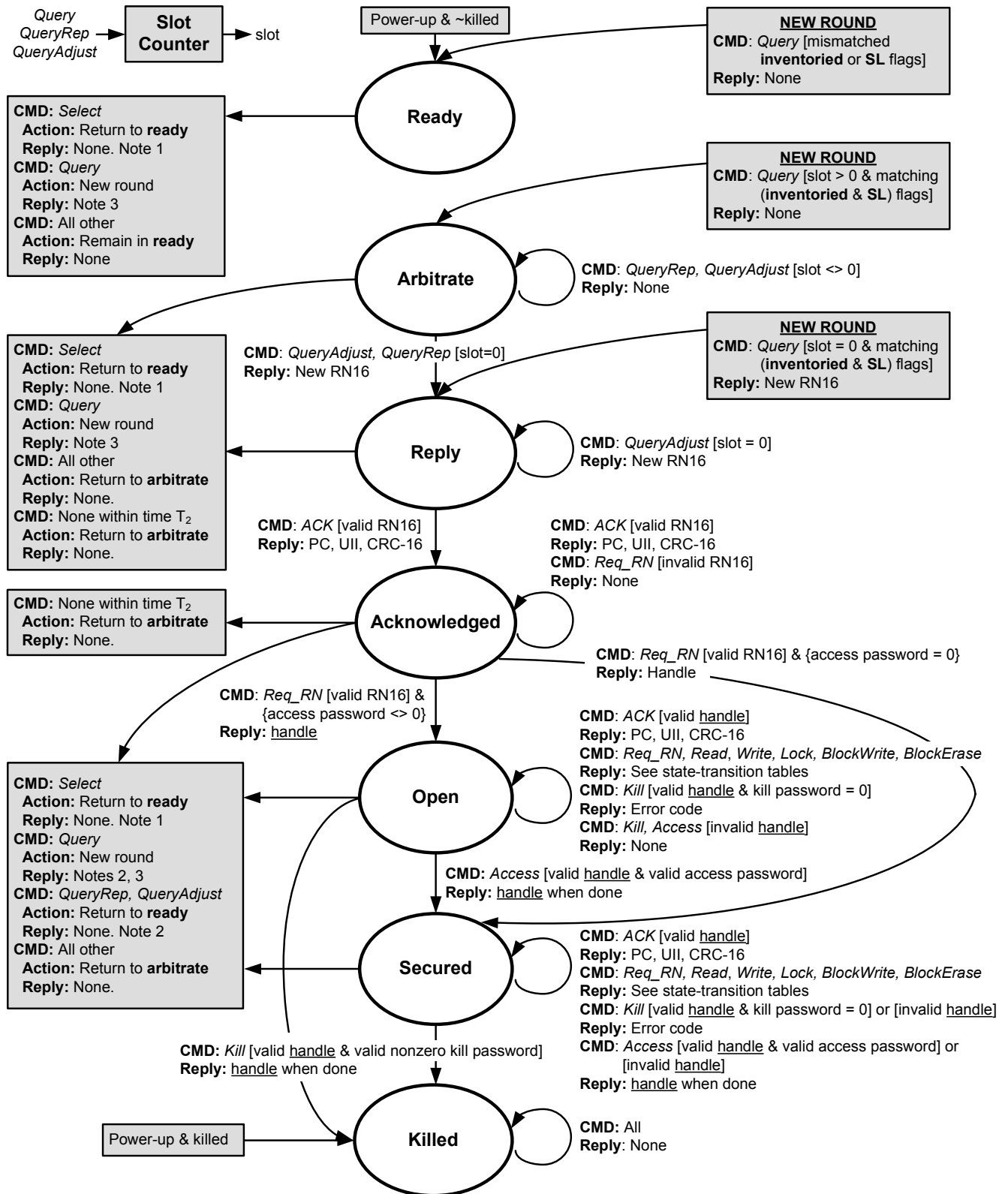
Tags shall implement a random or pseudo-random number generator (RNG). The RNG shall meet the following randomness criteria independent of the strength of the energizing field, the $R \Rightarrow T$ link rate, and the data stored in the tag (including the PC, UII, and CRC-16). Tags shall generate 16-bit random or pseudo-random numbers (RN16) using the RNG, and shall have the ability to extract Q -bit subsets from an RN16 to preload the tag's slot counter (see 9.3.2.4.8). Tags shall have the ability to temporarily store at least two RN16s while powered, to use, for example, as a handle and a 16-bit cover-code during password transactions (see Figure Amd.1-26 or Figure Amd.1-28).

Probability of a single RN16: The probability that any RN16 drawn from the RNG has value $RN16 = j$, for any j , shall be bounded by $0.8/2^{16} < P(RN16 = j) < 1.25/2^{16}$.

Probability of simultaneously identical sequences: For a tag population of up to 10,000 tags, the probability that any two or more tags simultaneously generate the same sequence of RN16s shall be less than 0.1%, regardless of when the tags are energized.

Probability of predicting an RN16: An RN16 drawn from a tag's RNG 10ms after the end of T_r in Figure Amd.1-6 shall not be predictable with a probability greater than 0.025% if the outcomes of prior draws from the RNG, performed under identical conditions, are known.

This document recommends that interrogators wait 10ms after T_r in Figure Amd.1-6 or T_{hr} in Figure Amd.1-8 before issuing passwords to tags.

**NOTES**

1. *Select*: Assert/deassert **SL** or set **inventoried** to A or B.
2. *Query*: $A \rightarrow B$ or $B \rightarrow A$ if the new session matches the prior session; otherwise no change to the **inventoried** flag.
QueryRep/QueryAdjust: $A \rightarrow B$ or $B \rightarrow A$ if the session matches the prior *Query*; otherwise, the command is invalid and ignored by the Tag.
3. *Query* starts a new round and may change the session. Tags may go to **ready**, **arbitrate**, or **reply**.

Figure Amd.1-22 — Tag state diagram

9.3.2.6 Managing tag populations

Interrogators manage tag populations using the three basic operations shown in Figure Amd.1-23. Each of these operations comprises one or more commands. The operations are defined as follows:

Select: The process by which an interrogator selects a tag population for inventory and access. Interrogators may use one or more *Select* commands to select a particular tag population prior to inventory.

Inventory: The process by which an interrogator identifies tags. An interrogator begins an inventory round by transmitting a *Query* command in one of four sessions. One or more tags may reply. The interrogator detects a single tag reply and requests the PC, Ull, and CRC-16 from the tag. An inventory round operates in one and only one session at a time. Annex H shows an example of an interrogator inventorying and accessing a single tag.

Access: The process by which an interrogator transacts with (reads from or writes to) individual tags. An individual tag must be uniquely identified prior to access. Access comprises multiple commands, some of which employ one-time-pad based cover-coding of the R=>T link.

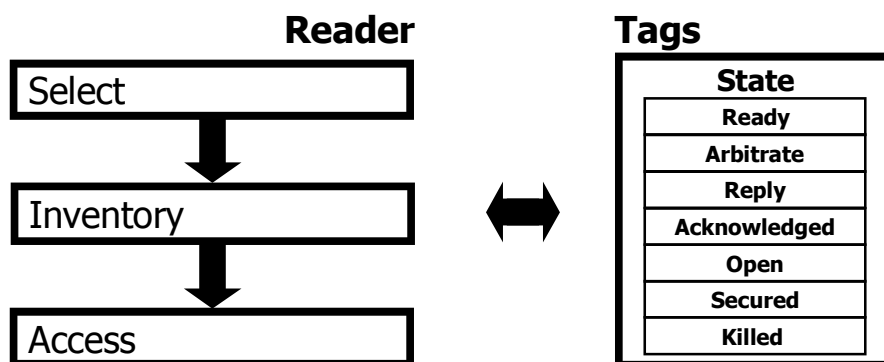


Figure Amd.1-23 — Interrogator/tag operations and tag state

9.3.2.7 Selecting tag populations

The selection process employs a single command, *Select*, which an interrogator may apply successively to select a particular tag population based on user-defined criteria, enabling union (U), intersection ($\cap$), and negation ($\sim$) based tag partitioning. Interrogators perform $\cap$ and U operations by issuing successive *Select* commands. *Select* can assert or de-assert a tag's **SL** flag, or it can set a tag's **inventoried** flag to either *A* or *B* in any one of the four sessions. *Select* contains the parameters Target, Action, MemBank, Pointer, Length, Mask, and Truncate.

Target and Action indicate whether and how a *Select* modifies a tag's **SL** or **inventoried** flag, and in the case of the **inventoried** flag, for which session. A *Select* that modifies **SL** shall not modify **inventoried**, and vice versa.

MemBank specifies if the mask applies to Ull, TID, or User memory. *Select* commands apply to a single memory bank. Successive *Selects* may apply to different memory banks.

Pointer, Length, and Mask: Pointer and Length describe a memory range. Mask, which must be Length bits long, contains a bit string that a tag compares against the specified memory range.

Truncate specifies whether a tag backscatters its entire Ull, or only that portion of the Ull immediately following Mask. Truncated replies are always followed by the CRC-16 in Ull memory 00_n to 0F_n; a tag does not recompute this CRC for a truncated reply.

By issuing multiple identical *Select* commands an interrogator can asymptotically single out all tags matching the selection criteria even though tags may undergo short-term RF fades.

A *Query* command uses **inventoried** and **SL** to decide which tags participate in an inventory. Interrogators may inventory and access **SL** or $\sim$ **SL** tags, or they may choose to ignore the **SL** flag entirely.

9.3.2.8 Inventorying tag populations

The inventory command set includes *Query*, *QueryAdjust*, *QueryRep*, *ACK*, and *NAK*. *Query* initiates an inventory round and decides which tags participate in the round (“inventory round” is defined in clause 4.1).

Query contains a slot-count parameter Q . Upon receiving a *Query* participating tags pick a random value in the range $(0, 2^Q - 1)$, inclusive, and load this value into their slot counter. Tags that pick a zero transition to the **reply** state and reply immediately. Tags that pick a nonzero value transition to the **arbitrate** state and await a *QueryAdjust* or a *QueryRep* command. Assuming that a single tag replies, the query-response algorithm proceeds as follows:

The tag backscatters an RN16 as it enters **reply**,

The interrogator acknowledges the tag with an *ACK* containing this same RN16,

The acknowledged tag transitions to the **acknowledged** state, backscattering its PC, UII, and CRC-16,

The interrogator issues a *QueryAdjust* or *QueryRep*, causing the identified tag to invert its **inventoried** flag (i.e. $A \rightarrow B$ or $B \rightarrow A$) and transition to **ready**, and potentially causing another tag to initiate a query-response dialog with the interrogator, starting in step (a), above.

If the tag fails to receive the *ACK* in step (b) within time T_2 (see Figure Amd.1-19), or receives the *ACK* with an erroneous RN16, it returns to **arbitrate**.

If multiple tags reply in step (a) but the interrogator, by detecting and resolving collisions at the waveform level, can resolve an RN16 from one of the tags, the interrogator can *ACK* the resolved tag. Unresolved tags receive erroneous RN16s and return to **arbitrate** without backscattering their PC, UII, and CRC-16.

If the interrogator sends a valid *ACK* (i.e. an *ACK* containing the correct RN16) to the tag in the **acknowledged** state the tag re-backscatters its PC, UII, and CRC-16.

At any point the interrogator may issue a *NAK*, in response to which all tags in the inventory round that receive the *NAK* return to **arbitrate** without changing their **inventoried** flag.

After issuing a *Query* to initiate an inventory round, the interrogator typically issues one or more *QueryAdjust* or *QueryRep* commands. *QueryAdjust* repeats a previous *Query* and may increment or decrement Q , but does not introduce new tags into the round. *QueryRep* repeats a previous *Query* without changing any parameters and without introducing new tags into the round. An inventory round can contain multiple *QueryAdjust* or *QueryRep* commands. At some point the interrogator will issue a new *Query*, thereby starting a new inventory round.

Tags in the **arbitrate** or **reply** states that receive a *QueryAdjust* first adjust Q (increment, decrement, or leave unchanged), then pick a random value in the range $(0, 2^Q - 1)$, inclusive, and load this value into their slot counter. Tags that pick zero transition to the **reply** state and reply immediately. Tags that pick a nonzero value transition to the **arbitrate** state and await a *QueryAdjust* or a *QueryRep* command.

Tags in the **arbitrate** state decrement their slot counter every time they receive a *QueryRep*, transitioning to the **reply** state and backscattering an RN16 when their slot counter reaches 0000_h . Tags whose slot counter reached 0000_h , who replied, and who were not acknowledged (including tags that responded to the original *Query* and were not acknowledged) return to **arbitrate** with a slot value of 0000_h and decrement this slot value from 0000_h to $7FFF_h$ at the next *QueryRep*, thereby effectively preventing subsequent replies until the tag loads a new random value into its slot counter.

Although tag inventory is based on a random protocol, the Q -parameter affords network control by allowing an interrogator to regulate the probability of tag responses. Q is an integer in the range $(0, 15)$; thus, the associated tag-response probabilities range from $2^0 = 1$ to $2^{-15} = 0.000031$.

Annex G describes an exemplary interrogator algorithm for choosing Q .

The scenario outlined above assumed a single interrogator operating in a single session. However, as described in 9.3.2.2, an interrogator can inventory a tag population in one of four sessions. Furthermore, as

described in 9.3.2.10.2, the *Query*, *QueryAdjust*, and *QueryRep* commands each contain a session parameter. How a tag responds to these commands varies with the command, session parameter, and tag state, as follows:

Query: A *Query* command starts an inventory round and chooses the session for the round. Tags in any state except **killed** execute a *Query*, starting a new round in the specified session and transitioning to **ready**, **arbitrate**, or **reply**, as appropriate (see Figure Amd.1-22).

If a tag in the **acknowledged**, **open**, or **secured** states receives a *Query* whose session parameter matches the prior session it inverts its **inventoried** flag (i.e. $A \rightarrow B$ or $B \rightarrow A$) for the session before it evaluates whether to transition to **ready**, **arbitrate**, or **reply**.

If a tag in the **acknowledged**, **open**, or **secured** states receives a *Query* whose session parameter does not match the prior session it leaves its **inventoried** flag for the prior session unchanged as it evaluates whether to transition to **ready**, **arbitrate**, or **reply**.

QueryAdjust, QueryRep: Tags in any state except **ready** or **killed** execute a *QueryAdjust* or *QueryRep* command if, and only if, (i) the session parameter in the command matches the session parameter in the *Query* that started the round, and (ii) the tag is not in the middle of a *Kill* or *Access* command sequence (see 9.3.2.10.3.4 or 9.3.2.10.3.6 respectively). Tags ignore a *QueryAdjust* or *QueryRep* with mismatched session.

If a tag in the **acknowledged**, **open**, or **secured** states receives a *QueryAdjust* or *QueryRep* whose session parameter matches the session parameter in the prior *Query*, and the tag is not in the middle of a *Kill* or *Access* command sequence (see 9.3.2.10.3.4 or 9.3.2.10.3.6 respectively), it inverts its **inventoried** flag (i.e. $A \rightarrow B$ or $B \rightarrow A$) for the current session then transitions to **ready**.

To illustrate an inventory operation, consider a specific example: Assume a population of 64 powered tags in the **ready** state. An interrogator first issues a *Select* to select a subpopulation of tags. Assume that 16 tags match the selection criteria. Further assume that 12 of the 16 selected tags have their **inventoried** flag set to *A* in session *S0*. The interrogator issues a *Query* specifying (**SL**, *Q* = 4, *S0*, *A*). Each of the 12 tags picks a random number in the range (0,15) and loads the value into its slot counter. Tags that pick a zero respond immediately. The *Query* has 3 possible outcomes:

No tags reply: The interrogator may issue another *Query*, or it may issue a *QueryAdjust* or *QueryRep*.

One tag replies (see Figure Amd.1-24): The tag transitions to the **reply** state and backscatters an RN16. The interrogator acknowledges the tag by sending an *ACK*. If the tag receives the *ACK* with a correct RN16 it backscatters its PC, Ull, and CRC-16 and transitions to the **acknowledged** state. If the tag receives the *ACK* with an incorrect RN16 it transitions to **arbitrate**. Assuming a successful *ACK*, the interrogator may either access the acknowledged tag or issue a *QueryAdjust* or *QueryRep* with matching session parameter to invert the tag's **inventoried** flag from $A \rightarrow B$ and send the tag to **ready** (a *Query* with matching prior-round session parameter will also invert the **inventoried** flag from $A \rightarrow B$).

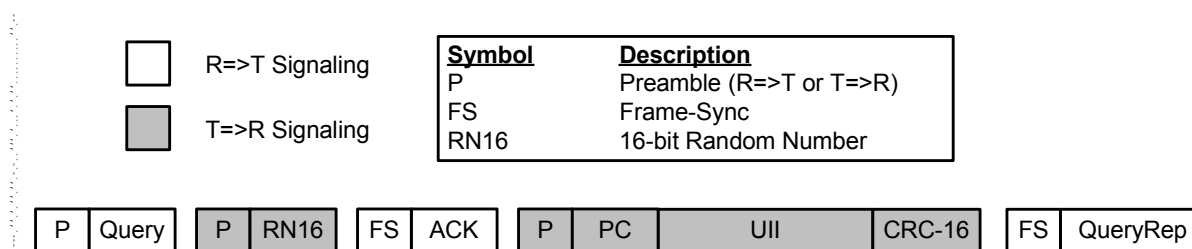


Figure Amd.1-24 — One tag reply

Multiple tags reply: The interrogator observes a backscattered waveform comprising multiple RN16s. It may try to resolve the collision and issue an *ACK*; not resolve the collision and issue a *QueryAdjust*, *QueryRep*, or *NAK*; or quickly identify the collision and issue a *QueryAdjust* or *QueryRep* before the collided tags have finished backscattering. In the latter case the collided tags, not observing a valid reply within T_2 (see Figure Amd.1-19), return to **arbitrate** and await the next *Query* or *QueryAdjust* command.

9.3.2.9 Accessing individual tags

After acknowledging a tag, an interrogator may choose to access it. The access command set comprises *Req_RN*, *Read*, *Write*, *Kill*, *Lock*, *Access*, *BlockWrite*, and *BlockErase*. The tag executes access commands in the states according to Table Amd.1-17.

Table Amd.1-17 — Access commands and tag states in which they are permitted

Command	Acknowledged State	Open State	Secured State	Remark
<i>Req_RN</i>	Permitted	Permitted	Permitted	
<i>Read</i>		Permitted	Permitted	
<i>Write</i>		Permitted	Permitted	Requires prior <i>Req_RN</i>
<i>Kill</i>		Permitted	Permitted	Requires prior <i>Req_RN</i>
<i>Lock</i>			Permitted	
<i>Access</i>		Permitted	Permitted	Optional command; Requires prior <i>Req_RN</i>
<i>BlockWrite</i>		Permitted	Permitted	Optional command
<i>BlockErase</i>		Permitted	Permitted	Optional command

An interrogator accesses a tag in the **acknowledged** state as follows:

Step 1. The interrogator issues a *Req_RN* to the acknowledged tag.

Step 2. The tag generates and stores a new RN16 (denoted handle), backscatters the handle, and transitions to the **open** state if its access password is nonzero, or to the **secured** state if its access password is zero. The interrogator may now issue further access commands.

All access commands issued to a tag in the **open** or **secured** states include the tag's handle as a parameter in the command. When in either of these two states, tags verify that the handle is correct prior to executing an access command, and ignore access commands with an incorrect handle. The handle value is fixed for the entire duration of a tag access.

Tags in the **open** state can execute all access commands except *Lock*. Tags in the **secured** state can execute all access commands. A tag's response to an access command includes, at a minimum, the tag's handle; the response may include other information as well (for example, the result of a *Read* operation).

An interrogator may issue an *ACK* to a tag in the **open** or **secured** states, causing the tag to backscatter its PC, UII, and CRC-16.

Interrogator and tag can communicate indefinitely in the **open** or **secured** states. The interrogator may terminate the communications at any time by issuing a *Query*, *QueryAdjust*, *QueryRep*, or a *NAK*. The tag's response to a *Query*, *QueryAdjust*, or *QueryRep* is described in 9.3.2.8. A *NAK* causes all tags in the inventory round to return to **arbitrate** without changing their **inventoried** flag.

The *Write*, *Kill*, and *Access* commands send 16-bit words (either data or half-passwords) from interrogator to tag. These commands use one-time-pad based link cover-coding to obscure the word being transmitted, as follows:

Step 1. The interrogator issues a *Req_RN*, to which the tag responds by backscattering a new RN16. The interrogator then generates a 16-bit ciphertext string comprising a bit-wise EXOR of the 16-bit word to be transmitted with this new RN16, both MSB first, and issues the command with this ciphertext string as a parameter.

Step 2. The tag decrypts the received ciphertext string by performing a bit-wise EXOR of the received 16-bit ciphertext string with the original RN16.

If an interrogator issues a command containing cover-coded data or a half-password and fails to receive a response from the tag, the interrogator may reissue the command unchanged. If the interrogator issues a command with new data or half-password, then it shall first issue a *Req_RN* to obtain a new RN16 and shall use this RN16 for the cover-coding.

To reduce security risks, this document recommends that (1) tags use unique kill passwords, and (2) memory writes be performed in a secure location.

The *BlockWrite* command (see 9.3.2.10.3.7) communicates multiple 16-bit words from interrogator to tag. Unlike *Write*, *BlockWrite* does not use link cover-coding.

A tag responds to a command that writes or erases memory (i.e. *Write*, *Kill*, *Lock*, *BlockWrite*, and *BlockErase*) by backscattering its handle, indicating that the operation was successful, or by backscattering an error code (see Annex K), indicating that the operation was unsuccessful. A tag reply to all access commands that write memory (i.e. *Write*, *Kill*, *Lock*, *BlockWrite*, and *BlockErase*) uses the extended preamble shown in Figure Amd.1-14 or Figure Amd.1-18, as appropriate (i.e. the tag replies as if T_{RExt}=1 regardless of the T_{RExt} value specified in the *Query* command that initiated the inventory round).

Killing a tag is a multi-step procedure, described in 9.3.2.10.3.4 and outlined in Figure Amd.1-26.

Issuing an access password to a tag is a multi-step procedure, described in 9.3.2.10.3.6 and outlined in Figure Amd.1-28.

Tag memory may be unlocked or locked. The lock status may be changeable or permalocked (i.e. permanently unlocked or permanently locked). An interrogator may write to unlocked memory from either the **open** or **secured** states. An interrogator may write to locked memory that is not permalocked from the **secured** state only. See Table Amd.1-41 for a detailed description of memory lock, permalock, and the tag state required to modify memory.

This protocol recommends that interrogators avoid powering-off while a tag is in the **reply**, **acknowledged**, **open** or **secured** states. Rather, interrogators should end their dialog with a tag before powering off, leaving the tag in either the **ready** or **arbitrate** state.

9.3.2.10 Interrogator commands and tag replies

Interrogator-to-tag commands shall have the format shown in Table Amd.1-18.

QueryRep and *ACK* have 2-bit command codes beginning with 0₂.

Query, *QueryAdjust*, and *Select* have 4-bit command codes beginning with 10₂.

All other base commands have 8-bit command codes beginning with 110₂.

All extended commands have 16-bit command codes beginning with 1110₂.

QueryRep, *ACK*, *Query*, *QueryAdjust*, and *NAK* have the unique command lengths shown in Table Amd.1-18. No other commands shall have these lengths. If a tag receives one of these commands with an incorrect length it shall ignore the command.

Query, *QueryAdjust*, and *QueryRep* contain a session parameter.

Query is protected by a CRC-5, shown in Table Amd.1-14 and detailed in Annex A.

Select, *Req_RN*, *Read*, *Write*, *Kill*, *Lock*, *Access*, *BlockWrite* and *BlockErase* are protected by a CRC-16, defined in 9.3.1.5 and detailed in Annex A.

R=>T commands begin with either a preamble or a frame-sync, as described in 9.3.1.2.8. The command-code lengths specified in Table Amd.1-18 do not include the preamble or frame-sync.

Tags shall ignore invalid commands. In general, "invalid" means a command that (1) is incorrect given the current tag state, (2) is unsupported by the tag, (3) has incorrect parameters, (4) has a CRC error, (5)

specifies an incorrect session, or (6) is in any other way not recognized or not executable by the tag. The actual definition of “invalid” is state-specific and defined, for each tag state, in Annex E and Annex F.

Table Amd.1-18 — Commands

Command	Code	Length (bits)	Mandatory?	Protection
<i>QueryRep</i>	00	4	Yes	Unique command length
<i>ACK</i>	01	18	Yes	Unique command length
<i>Query</i>	1000	22	Yes	Unique command length and a CRC-5
<i>QueryAdjust</i>	1001	9	Yes	Unique command length
<i>Select</i>	1010	> 44	Yes	CRC-16
<i>Reserved for future use</i>	1011	—	—	—
<i>NAK</i>	11000000	8	Yes	Unique command length
<i>Req_RN</i>	11000001	40	Yes	CRC-16
<i>Read</i>	11000010	> 57	Yes	CRC-16
<i>Write</i>	11000011	> 58	Yes	CRC-16
<i>Kill</i>	11000100	59	Yes	CRC-16
<i>Lock</i>	11000101	60	Yes	CRC-16
<i>Access</i>	11000110	56	No	CRC-16
<i>BlockWrite</i>	11000111	> 57	No	CRC-16
<i>BlockErase</i>	11001000	> 57	No	CRC-16
<i>Reserved for future use</i>	11001001 ... 11011111	—	—	—
<i>Reserved for custom commands</i>	11100000 00000000 ... 11100000 11111111	—	—	Manufacturer specified
<i>Reserved for proprietary commands</i>	11100001 00000000 ... 11100001 11111111	—	—	Manufacturer specified
<i>Reserved for future use</i>	11100010 00000000 ... 11101111 11111111	—	—	—

9.3.2.10.1 Select commands

The Select command set comprises a single command: *Select*.

9.3.2.10.1.1 *Select* (mandatory)

Select selects a particular tag population based on user-defined criteria, enabling union (U), intersection ($\cap$), and negation ($\sim$) based tag partitioning. Interrogators perform $\cap$ and U operations by issuing successive *Select* commands. *Select* can assert or de-assert a tag's **SL** flag, which applies across all four sessions, or it can set a tag's **inventoried** flag to either *A* or *B* in any one of the four sessions.

Interrogators and tags shall implement the *Select* command shown in Table Amd.1-19. Target shall indicate whether the *Select* modifies a tag's **SL** or **inventoried** flag, and in the case of the **inventoried** flag, for which

session. Action shall elicit the tag response shown in Table Amd.1-20. The criteria for determining whether a tag is matching or non-matching are specified in the MemBank, Pointer, Length and Mask fields. Truncate indicates whether a tag's backscattered reply shall be truncated to include only those UII and CRC-16 bits following Mask. Select passes the following parameters from interrogator to tags:

Target indicates whether the Select command modifies a tag's **SL** flag or its **inventoried** flag, and in the case of **inventoried** it further specifies one of four sessions. A Select command that modifies **SL** does not modify **inventoried**, and vice versa. Tags shall ignore Select commands whose Target is 101₂, 110₂, or 111₂.

Action indicates whether matching tags assert or de-assert **SL**, or set their **inventoried** flag to *A* or to *B*. Tags conforming to the contents of the MemBank, Pointer, Length, and Mask fields are considered matching. Tags not conforming to the contents of these fields are considered non-matching.

MemBank specifies whether Mask applies to UII, TID, or User memory. Select commands shall apply to a single memory bank. Successive Selects may apply to different banks. MemBank shall not specify Reserved memory; if a tag receives a Select specifying MemBank = 00₂ it shall ignore the Select. MemBank parameter value 00₂ is reserved for future use (RFU).

Pointer, Length, and Mask: Pointer and Length describe a memory range. Pointer references a memory bit address (Pointer is not restricted to word boundaries) and uses EBV formatting (see Annex D). Length is 8 bits, allowing Masks from 0 to 255 bits in length. Mask, which is Length bits long, contains a bit string that a tag compares against the memory location that begins at Pointer and ends Length bits later. If Pointer and Length reference a memory location that does not exist on the tag then the tag shall consider the Select to be non-matching. If Length is zero then all tags shall be considered matching, unless Pointer references a memory location that does not exist on the tag or Truncate = 1 and Pointer is outside the UII specified in the PC bits, in which case the tag shall consider the Select to be non-matching.

Truncate: If an interrogator asserts Truncate, and if a subsequent Query specifies Sel=10 or Sel=11, then matching tags shall truncate their reply to an ACK to that portion of the UII immediately following Mask, followed by the CRC-16 stored in UII memory 00_n to 0F_n. If an interrogator asserts Truncate, it shall assert it:

- in the last Select that the interrogator issues prior to sending a Query,
- only if the Select has Target = 100₂, and
- only if Mask ends in the UII.

These constraints *do not* preclude an interrogator from issuing multiple Select commands that target the **SL** and/or **inventoried** flags. They *do* require that an interrogator that is requesting tags to truncate their replies assert Truncate in the last Select, and that this last Select targets the **SL** flag. Tags shall power-up with Truncate de-asserted.

Tags shall decide whether to truncate their backscattered UII on the basis of the most recently received Select. If a tag receives a Select with Truncate=1 but Target≠100₂ the tag shall ignore the Select. If a tag receives a Select in which Truncate=1 but MemBank≠01, the tag shall consider the Select to be invalid. If a tag receives a Select in which Truncate=1, MemBank=01, but Mask ends outside the UII specified in the PC bits, the tag shall consider the Select to be not matching.

Mask may end at the last bit of the UII, in which case a selected tag shall backscatter its CRC-16.

Truncated replies never include PC bits, because Mask must end in the UII.

A tag shall preface its truncated reply with five leading zeros (00000₂) inserted between the preamble and the truncated reply. A tag does not recalculate the CRC-16 for a truncated reply.

Interrogators can use a Select command to reset all tags in a session to **inventoried** state *A*, by issuing a Select with Action = 000₂ and a Length value of zero.

Interrogators shall prepend a Select with a frame-sync (see 9.3.1.2.8). The CRC-16 is calculated over the first command-code bit to the Truncate bit.

Tags shall not reply to a Select.

Table Amd.1-19 — Select command

	Command	Target	Action	MemBank	Pointer	Length	Mask	Truncate	CRC-16
# of bits	4	3	3	2	EBV	8	Variable	1	16
description	1010	000: Inventoried (S0) 001: Inventoried (S1) 010: Inventoried (S2) 011: Inventoried (S3) 100: SL 101: RFU 110: RFU 111: RFU	See Table Amd.1-20	00: RFU 01: UII 10: TID 11: User	Starting <u>Mask</u> address	<u>Mask</u> length (bits)	<u>Mask</u> value	0: Disable truncation 1: Enable truncation	

Table Amd.1-20 — Tag response to Action parameter

Action	Matching	Non-Matching
000	assert SL or inventoried → A	de-assert SL or inventoried → B
001	assert SL or inventoried → A	do nothing
010	do nothing	de-assert SL or inventoried → B
011	negate SL or (A → B, B → A)	do nothing
100	de-assert SL or inventoried → B	assert SL or inventoried → A
101	de-assert SL or inventoried → B	do nothing
110	do nothing	assert SL or inventoried → A
111	do nothing	negate SL or (A → B, B → A)

9.3.2.10.2 Inventory commands

The inventory command set comprises *Query*, *QueryAdjust*, *QueryRep*, *ACK*, and *NAK*.

9.3.2.10.2.1 Query (mandatory)

Interrogators and tags shall implement the *Query* command shown in Table Amd.1-21. *Query* initiates and specifies an inventory round. *Query* includes the following fields:

DR (TRcal divide ratio) sets the T=>R link frequency as described in 9.3.1.2.8 and Table Amd.1-11.

M (cycles per symbol) sets the T=>R data rate and modulation format as shown in Table Amd.1-12.

TRext chooses whether the T=>R preamble is pre-pended with a pilot tone as described in 9.3.1.3.2.2 and 9.3.1.3.2.4, however, a tag's reply to access commands that write to memory (i.e. *Write*, *Kill*, *Lock*, *BlockWrite* and *BlockErase*) always use a pilot tone regardless of the value of TRext.

Sel chooses which tags respond to the *Query* (see 9.3.2.10.1.1 and 9.3.2.8).

Session chooses a session for the inventory round (see 9.3.2.8).

Target selects whether tags whose **inventoried** flag is A or B participate in the inventory round. Tags may change their inventoried flag from A to B (or vice versa) as a result of being singulated.

Q sets the number of slots in the round (see 9.3.2.8).

Interrogators shall prepend a *Query* with a preamble (see 9.3.1.2.8).

The CRC-5 is calculated over the first command-code bit to the last Q bit. If a tag receives a *Query* with a CRC-5 error it shall ignore the command.

Upon receiving a *Query*, tags with matching Sel and Target shall pick a random value in the range $(0, 2^Q - 1)$, inclusive, and shall load this value into their slot counter. If a tag, in response to the *Query*, loads its slot counter with zero, then its reply to a *Query* shall be as shown in Table Amd.1-22; otherwise the tag shall remain silent.

A *Query* may initiate an inventory round in a new session, or in the prior session. If a tag in the **acknowledged**, **open**, or **secured** states receives a *Query* whose session parameter matches the prior session it shall invert its **inventoried** flag (i.e. $A \rightarrow B$ or $B \rightarrow A$) for the session before it evaluates whether to transition to **ready**, **arbitrate**, or **reply**. If a tag in the **acknowledged**, **open**, or **secured** states receives a *Query* whose session parameter does not match the prior session it shall leave its **inventoried** flag for the prior session unchanged when beginning the new round.

Tags shall support all DR and M values specified in Table Amd.1-11 and Table Amd.1-12, respectively.

Tags in any state other than **killed** shall execute a *Query* command, starting a new round in the specified session and transitioning to **ready**, **arbitrate**, or **reply**, as appropriate (see Figure Amd.1-22). Tags in the **killed** state shall ignore a *Query*.

Table Amd.1-21 — *Query* command

	Command	DR	M	TRExt	Sel	Session	Target	Q	CRC-5
# of bits	4	1	2	1	2	2	1	4	5
description	1000	0: DR=8 1: DR=64/3	00: M=1 01: M=2 10: M=4 11: M=8	0: No pilot tone 1: Use pilot tone	00: All 01: All 10: ~SL 11: SL	00: S0 01: S1 10: S2 11: S3	0: A 1: B	0–15	

Table Amd.1-22 — Tag reply to a *Query* command

	Response
# of bits	16
description	RN16

9.3.2.10.2.2 *QueryAdjust* (mandatory)

Interrogators and tags shall implement the *QueryAdjust* command shown in Table Amd.1-23. *QueryAdjust* adjusts Q (i.e. the number of slots in an inventory round – see 9.3.2.8) without changing any other round parameters.

QueryAdjust includes the following fields:

Session corroborates the session number for the inventory round (see 9.3.2.8 and 9.3.2.10.2.1). If a tag receives a *QueryAdjust* whose session number is different from the session number in the *Query* that initiated the round it shall ignore the command.

UpDn determines whether and how the tag adjusts Q, as follows:

110: Increment Q (i.e. $Q = Q + 1$).

000: No change to Q.

011: Decrement Q (i.e. $Q = Q - 1$).

If a tag receives a *QueryAdjust* with an UpDn value different from those specified above it shall ignore the command. If a tag whose Q value is 15 receives a *QueryAdjust* with UpDn = 110 it shall change UpDn to 000 prior to executing the command; likewise, if a tag whose Q value is 0 receives a *QueryAdjust* with UpDn = 011 it shall change UpDn to 000 prior to executing the command.

Tags shall maintain a running count of the current Q value. The initial Q value is specified in the *Query* command that started the inventory round; one or more subsequent *QueryAdjust* commands may modify Q.

A *QueryAdjust* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

Upon receiving a *QueryAdjust* tags first update Q , then pick a random value in the range $(0, 2^Q - 1)$, inclusive, and load this value into their slot counter. If a tag, in response to the *QueryAdjust*, loads its slot counter with zero, then its reply to a *QueryAdjust* shall be shown in Table Amd.1-24; otherwise, the tag shall remain silent. Tags shall respond to a *QueryAdjust* only if they received a prior *Query*.

Tags in any state except **ready** or **killed** shall execute a *QueryAdjust* command if, and only if, (i) the session parameter in the command matches the session parameter in the *Query* that started the round, and (ii) the tag is not in the middle of a *Kill* or *Access* command sequence (see 9.3.2.10.3.4 or 9.3.2.10.3.6 respectively).

Tags in the **acknowledged**, **open**, or **secured** states that receive a *QueryAdjust* whose session parameter matches the session parameter in the prior *Query*, and who are not in the middle of a *Kill* or *Access* command sequence (see 9.3.2.10.3.4 or 9.3.2.10.3.6 respectively), shall invert their **inventoried** flag (i.e. $A \rightarrow B$ or $B \rightarrow A$, as appropriate) for the current session and transition to **ready**.

Table Amd.1-23 — *QueryAdjust* command

	Command	Session	UpDn
# of bits	4	2	3
description	1001	00: S0 01: S1 10: S2 11: S3	110: $Q = Q + 1$ 000: No change to Q 011: $Q = Q - 1$

Table Amd.1-24 — Tag reply to a *QueryAdjust* command

	Response
# of bits	16
description	RN16

9.3.2.10.2.3 *QueryRep* (mandatory)

Interrogators and tags shall implement the *QueryRep* command shown in Table Amd.1-25. *QueryRep* instructs tags to decrement their slot counters and, if slot = 0 after decrementing, to backscatter an RN16 to the interrogator.

QueryRep includes the following field:

Session corroborates the session number for the inventory round (see 9.3.2.8 and 9.3.2.10.2.1). If a tag receives a *QueryRep* whose session number is different from the session number in the *Query* that initiated the round it shall ignore the command.

A *QueryRep* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

If a tag, in response to the *QueryRep*, decrements its slot counter and the decremented slot value is zero, then its reply to a *QueryRep* shall be as shown in Table Amd.1-26; otherwise the tag shall remain silent. Tags shall respond to a *QueryRep* only if they received a prior *Query*.

Tags in any state except **ready** or **killed** shall execute a *QueryRep* command if, and only if, (i) the session parameter in the command matches the session parameter in the *Query* that started the round, and (ii) the tag is not in the middle of a *Kill* or *Access* command sequence (see 9.3.2.10.3.4 or 9.3.2.10.3.6 respectively).

Tags in the **acknowledged**, **open**, or **secured** states that receive a *QueryRep* whose session parameter matches the session parameter in the prior *Query*, and who are not in the middle of a *Kill* or *Access* command sequence (see 9.3.2.10.3.4 or 9.3.2.10.3.6 respectively), shall invert their **inventoried** flag (i.e. $A \rightarrow B$ or $B \rightarrow A$, as appropriate) for the current session and transition to **ready**.

Table Amd.1-25 — *QueryRep* command

	Command	Session
# of bits	2	2
description	00	00: S0 01: S1 10: S2 11: S3

Table Amd.1-26 — Tag reply to a *QueryRep* command

	Response
# of bits	16
description	RN16

9.3.2.10.2.4 ACK (mandatory)

Interrogators and tags shall implement the *ACK* command shown in Table Amd.1-27. An interrogator sends an *ACK* to acknowledge a single tag. *ACK* echoes the tag's backscattered RN16.

If an interrogator issues an *ACK* to a tag in the **reply** or **acknowledged** states, then the echoed RN16 shall be the RN16 that the tag previously backscattered as it transitioned from the **arbitrate** state to the **reply** state. If an interrogator issues an *ACK* to a tag in the **open** or **secured** states, then the echoed RN16 shall be the tag's handle (see 9.3.2.10.3.1).

An *ACK* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

The tag reply to a successful *ACK* shall be as shown in Table Amd.1-28. As described in 9.3.2.10.1.1, the reply may be truncated, in which case the tag reply is 00000₂ followed by the truncated UII followed by the CRC-16 computed by the tag at power-up. A tag that receives an *ACK* with an incorrect RN16 or an incorrect handle (as appropriate) shall return to **arbitrate** without responding, unless the tag is in **ready** or **killed**, in which case it shall ignore the *ACK* and remain in its present state.

Table Amd.1-27 — *ACK* command

	Command	RN
# of bits	2	16
description	01	Echoed RN16 or <u>handle</u>

Table Amd.1-28 — Tag reply to a successful *ACK* command

	Response
# of bits	21 to 528
description	{PC, UII, CRC-16} OR {00000 ₂ , truncated UII, CRC-16}

9.3.2.10.2.5 NAK (mandatory)

Interrogators and tags shall implement the *NAK* command shown in Table Amd.1-29. Any tag that receives a *NAK* shall return to the **arbitrate** state without changing its **inventoried** flag, unless the tag is in **ready** or **killed**, in which case it shall ignore the *NAK* and remain in its current state.

A *NAK* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

Tags shall not reply to a *NAK*.

Table Amd.1-29 — *NAK* command

	Command
# of bits	8
description	11000000

9.3.2.10.3 Access commands

The set of access commands comprises *Req_RN*, *Read*, *Write*, *Kill*, *Lock*, *Access*, *BlockWrite*, and *BlockErase*. As described in 9.3.2.9, tags execute *Req_RN* from the **acknowledged**, **open**, or **secured** states. Tags execute *Read*, *Write*, *BlockWrite*, and *BlockErase* from the **secured** state; if allowed by the lock status of the memory location being accessed, they may also execute these commands from the **open** state. Tags execute *Access* and *Kill* from the **open** or **secured** states. Tags execute *Lock* only from the **secured** state.

All access commands issued to a tag in the **open** or **secured** states include the tag's handle as a parameter in the command. When in either of these two states, the tag shall verify that the handle is correct prior to executing an access command, and the tag shall ignore access commands with an incorrect handle. The handle value is fixed for the entire duration of a tag access.

A tag's reply to all access commands that read or write memory (i.e. *Read*, *Write*, *Kill*, *Lock*, *BlockWrite*, and *BlockErase*) includes a 1-bit header. Header=0 indicates that the operation was successful and the reply is valid; header=1 indicates that the operation was unsuccessful and the reply is an error code.

A tag's reply to all access commands that write memory (i.e. *Write*, *Kill*, *Lock*, *BlockWrite*, and *BlockErase*) shall use the extended preamble shown in Figure Amd.1-14 or Figure Amd.1-18, as appropriate (i.e. the tag shall reply as if *TRExt*=1 regardless of the *TRExt* value specified in the *Query* command that initiated the inventory round).

After an interrogator writes part or all of a tag's PC or Ull, the CRC-16 stored in tag Ull memory 00_h to 0F_h will not be valid until the interrogator first powers-down and then re-powers-up its energizing RF field, because the tag computes the CRC-16 stored in tag Ull memory 00_h to 0F_h at power-up.

If an Interrogator attempts to write directly to Ull memory 00_h to 0F_h using either a *Write* or a *BlockWrite* command, the tag shall respond with an error code (see Annex L for error-code definitions and for the reply format).

Req_RN, *Read*, *Write*, *Kill*, and *Lock* are required commands; *Access*, *BlockWrite*, and *BlockErase* are optional. A tag shall ignore optional access commands that it does not support.

See Annex M for an example of a data-flow exchange during which an interrogator accesses a tag and reads its kill password.

9.3.2.10.3.1 *Req_RN* (mandatory)

Interrogators and tags shall implement the *Req_RN* command shown in Table Amd.1-30. *Req_RN* instructs a tag to backscatter a new RN16. Both the interrogator's command, and the tag's response, depend on the tag's state:

Acknowledged state: When issuing a *Req_RN* command to a tag in the **acknowledged** state, an interrogator shall include the tag's last backscattered RN16 as a parameter in the *Req_RN*. The *Req_RN* command is protected by a CRC-16 calculated over the command bits and the RN16. If the tag receives the *Req_RN* with a valid CRC-16 and a valid RN16 it shall generate and store a new RN16 (denoted handle), backscatter this handle, and transition to the **open** or **secured** state. The choice of ending state depends on the tag's access password, as follows:

Access password $\neq$ 0: Tag transitions to **open** state.

Access password = 0: Tag transitions to **secured** state.

If the tag receives the *Req_RN* command with a valid CRC-16 but an invalid RN16 it shall ignore the *Req_RN* and remain in the **acknowledged** state.

Open or **secured** states: When issuing a *Req_RN* command to a tag in the **open** or **secured** states, an interrogator shall include the tag's handle as a parameter in the *Req_RN*. The *Req_RN* command is protected by a CRC-16 calculated over the command bits and the handle. If the tag receives the *Req_RN* with a valid CRC-16 and a valid handle it shall generate and backscatter a new RN16. If the tag receives the *Req_RN* with a valid CRC-16 but an invalid handle it shall ignore the *Req_RN*. In either case the tag shall remain in its current state (**open** or **secured**, as appropriate).

If an interrogator wishes to ensure that only a single tag is in the **acknowledged** state it may issue a *Req_RN*, causing the tag or tags to each backscatter a handle and transition to the **open** or **secured** state (as appropriate). The interrogator may then issue an *ACK* with handle as a parameter. Tags that receive an *ACK* with an invalid handle shall return to **arbitrate** (Note: If a tag receives an *ACK* with an invalid handle it returns to **arbitrate**, whereas if it receives an access command with an invalid handle it ignores the command).

The first bit of the backscattered RN16 shall be denoted the MSB; the last bit shall be denoted the LSB.

A *Req_RN* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

The tag reply to a *Req_RN* shall be as shown in Table Amd.1-31. The RN16 or handle are protected by a CRC-16.

Table Amd.1-30 — *Req_RN* command

	Command	RN	CRC-16
# of bits	8	16	16
description	11000001	Prior RN16 or <u>handle</u>	

Table Amd.1-31 — Tag reply to a *Req_RN* command

	RN	CRC-16
# of bits	16	16
description	<u>handle</u> or new RN16	

9.3.2.10.3.2 Read (mandatory)

Interrogators and tags shall implement the *Read* command shown in Table Amd.1-32. *Read* allows an interrogator to read part or all of a tag's Reserved, Ull, TID, or User memory. *Read* has the following fields:

MemBank specifies whether the *Read* accesses Reserved, Ull, TID, or User memory. *Read* commands shall apply to a single memory bank. Successive *Reads* may apply to different banks.

WordPtr specifies the starting word address for the memory read, where words are 16 bits in length. For example, WordPtr = 00_h specifies the first 16-bit memory word, WordPtr = 01_h specifies the second 16-bit memory word, etc. WordPtr uses EBV formatting (see Annex D).

WordCount specifies the number of 16-bit words to be read. If WordCount = 00_h the tag shall backscatter the contents of the chosen memory bank starting at WordPtr and ending at the end of the bank, unless MemBank = 01, in which case the tag shall backscatter the Ull memory contents starting at WordPtr and ending at the length of the Ull specified by the first 5 bits of the PC if WordPtr lies within the Ull, and shall backscatter the Ull memory contents starting at WordPtr and ending at the end of Ull memory if WordPtr lies outside the Ull.

The *Read* command also includes the tag's handle and a CRC-16. The CRC-16 is calculated over the first command-code bit to the last handle bit.

If a tag receives a *Read* with a valid CRC-16 but an invalid handle it shall ignore the *Read* and remain in its current state (**open** or **secured**, as appropriate).

A *Read* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

If all of the memory words specified in a *Read* exist and none are read-locked, the tag reply to the *Read* shall be as shown in Table Amd.1-33. The tag responds by backscattering a header (a 0-bit), the requested memory words, and its handle. The reply includes a CRC-16 calculated over the 0-bit, memory words, and handle.

If one or more of the memory words specified in the *Read* command either do not exist or are read-locked, the tag shall backscatter an error code, within time T_1 in Table Amd.1-15, rather than the reply shown in Table Amd.1-33 (see Annex K for error-code definitions and for the reply format).

Table Amd.1-32 — Read command

	Command	MemBank	WordPtr	WordCount	RN	CRC-16
# of bits	8	2	EBV	8	16	16
description	11000010	00: Reserved 01: Ull 10: TID 11: User	Starting address pointer	Number of words to read	<u>handle</u>	

Table Amd.1-33 — Tag reply to a successful Read command

	Header	Memory Words	RN	CRC-16
# of bits	1	Variable	16	16
description	0	Data	<u>handle</u>	

9.3.2.10.3.3 Write (mandatory)

Interrogators and tags shall implement the *Write* command shown in Table Amd.1-34. *Write* allows an interrogator to write a word in a tag's Reserved, Ull, TID, or User memory. *Write* has the following fields:

MemBank specifies whether the *Write* occurs in Reserved, Ull, TID, or User memory.

WordPtr specifies the word address for the memory write, where words are 16 bits in length. For example, WordPtr = 00_n specifies the first 16-bit memory word, WordPtr = 01_n specifies the second 16-bit memory word, etc. WordPtr uses EBV formatting (see Annex D).

Data contains a 16-bit word to be written. Before each and every *Write* the interrogator shall first issue a *Req_RN* command; the tag responds by backscattering a new RN16. The interrogator shall cover-code the data by EXORing it with this new RN16 prior to transmission.

The *Write* command also includes the tag's handle and a CRC-16. The CRC-16 is calculated over the first command-code bit to the last handle bit.

If a tag in the **open** or **secured** states receives a *Write* with a valid CRC-16 but an invalid handle, or it receives a *Write* before which the immediately preceding command was not a *Req_RN*, it shall ignore the *Write* and remain in its current state.

A *Write* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

After issuing a *Write* an interrogator shall transmit CW for the lesser of T_{REPLY} or 20ms, where T_{REPLY} is the time between the interrogator's *Write* command and the tag's backscattered reply. An interrogator may observe several possible outcomes from a *Write*, depending on the success or failure of the tag's memory-write operation:

The *Write* succeeds: After completing the *Write* a tag shall backscatter the reply shown in Table Amd.1-35 and Figure Amd.1-25 comprising a header (a 0-bit), the tag's handle, and a CRC-16 calculated over the 0-bit and handle. If the interrogator observes this reply within 20 ms then the *Write* completed successfully.

The tag encounters an error: The tag shall backscatter an error code during the CW period rather than the reply shown in Table Amd.1-35 (see Annex K for error-code definitions and for the reply format).

The *Write* does not succeed: If the interrogator does not observe a reply within 20ms then the *Write* did not complete successfully. The interrogator may issue a *Req_RN* command (containing the tag's handle) to verify that the tag is still in the interrogator's field, and may reissue the *Write* command.

Upon receiving a valid *Write* command a tag shall write the commanded Data into memory. The tag's reply to a *Write* shall use the extended preamble shown in Figure Amd.1-14 or Figure Amd.1-18, as appropriate (i.e. a tag shall reply as if T_{Rext}=1 regardless of the T_{Rext} value in the *Query* that initiated the round).

Table Amd.1-34 — *Write* command

	Command	MemBank	WordPtr	Data	RN	CRC-16
# of bits	8	2	EBV	16	16	16
description	11000011	00: Reserved 01: Ull 10: TID 11: User	Address pointer	RN16 ⊗ word to be written	<u>handle</u>	

Table Amd.1-35 — Tag reply to a successful *Write* command

	Header	RN	CRC-16
# of bits	1	16	16
description	0	<u>handle</u>	



Figure Amd.1-25 — Successful Write sequence

9.3.2.10.3.4 Kill (mandatory)

Interrogators and tags shall implement the *Kill* command shown in Table Amd.1-36. *Kill* allows an interrogator to permanently disable a tag.

Kill contains 3 RFU bits. Interrogators shall set these bits to 000₂. Tags shall ignore these bits. Higher-functionality tags may use these bits to expand the functionality of the *Kill* command (for example, to kill a tag to a recycled state rather than killing it dead).

To kill a tag, an interrogator shall follow the multi-step kill procedure outlined in Figure Amd.1-26. Briefly, an interrogator issues two *Kill* commands, the first containing the 16 MSBs of the tag's kill password EXORed with an RN16, and the second containing the 16 LSBs of the tag's kill password EXORed with a different RN16. Each EXOR operation shall be performed MSB first (i.e. the MSB of each half-password shall be EXORed with the MSB of its respective RN16). Just prior to issuing each *Kill* the interrogator first issues a *Req_RN* to obtain a new RN16.

Tags shall incorporate the necessary logic to successively accept two 16-bit subportions of a 32-bit kill password. Interrogators shall not intersperse commands other than *Req_RN* between the two successive *Kill* commands. If a tag, after receiving a first *Kill*, receives any valid command other than *Req_RN* before the second *Kill* it shall return to **arbitrate**, unless the intervening command is a *Query*, in which case the tag shall execute the *Query* (inverting its **inventoried** flag if the session parameter in the *Query* matches the prior session).

Tags whose kill password is zero do not execute a kill operation; if such a tag receives a *Kill* it ignores the command and backscatters an error code (see Figure Amd.1-26).

The tag reply to the first *Kill* shall be as shown in Table Amd.1-37. The reply shall use the T_{RExt} value specified in the *Query* command that initiated the round.

After issuing the second *Kill* an interrogator shall transmit CW for the lesser of T_{REPLY} or 20ms, where T_{REPLY} is the time between the interrogator's second *Kill* command and the tag's backscattered reply. An interrogator may observe several possible outcomes from a *Kill*, depending on the success or failure of the tag's kill operation:

The *Kill* succeeds: After completing the *Kill* the tag shall backscatter the reply shown in Table Amd.1-37 and Figure Amd.1-25 comprising a header (a 0-bit), the tag's handle, and a CRC-16 calculated over the 0-bit and handle. Immediately after this reply the tag shall render itself silent and shall not respond to an interrogator thereafter. If the interrogator observes this reply within 20 ms then the *Kill* completed successfully.

The tag encounters an error: The tag shall backscatter an error code during the CW period rather than the reply shown in Table Amd.1-37 (see Annex K for error-code definitions and for the reply format).

The *Kill* does not succeed: If the interrogator does not observe a reply within 20ms then the *Kill* did not complete successfully. The interrogator may issue a *Req_RN* command (containing the tag's handle) to verify that the tag is still in the interrogator's field, and may reinitiate the multi-step kill procedure outlined in Figure Amd.1-26.

A *Kill* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

Upon receiving a valid *Kill* command sequence a tag shall render itself killed. The tag's reply to the second *Kill* command shall use the extended preamble shown in Figure Amd.1-14 or Figure Amd.1-18, as appropriate (i.e. a tag shall reply as if T_{RExt}=1 regardless of the T_{RExt} value in the *Query* that initiated the round).

Table Amd.1-36 — *Kill* command

	Command	Password	RFU	RN	CRC-16
# of bits	8	16	3	16	16
description	11000100	(½ kill password) ⊗ RN16	000 ₂	<u>handle</u>	

Table Amd.1-37 – Tag reply to the first *Kill* command

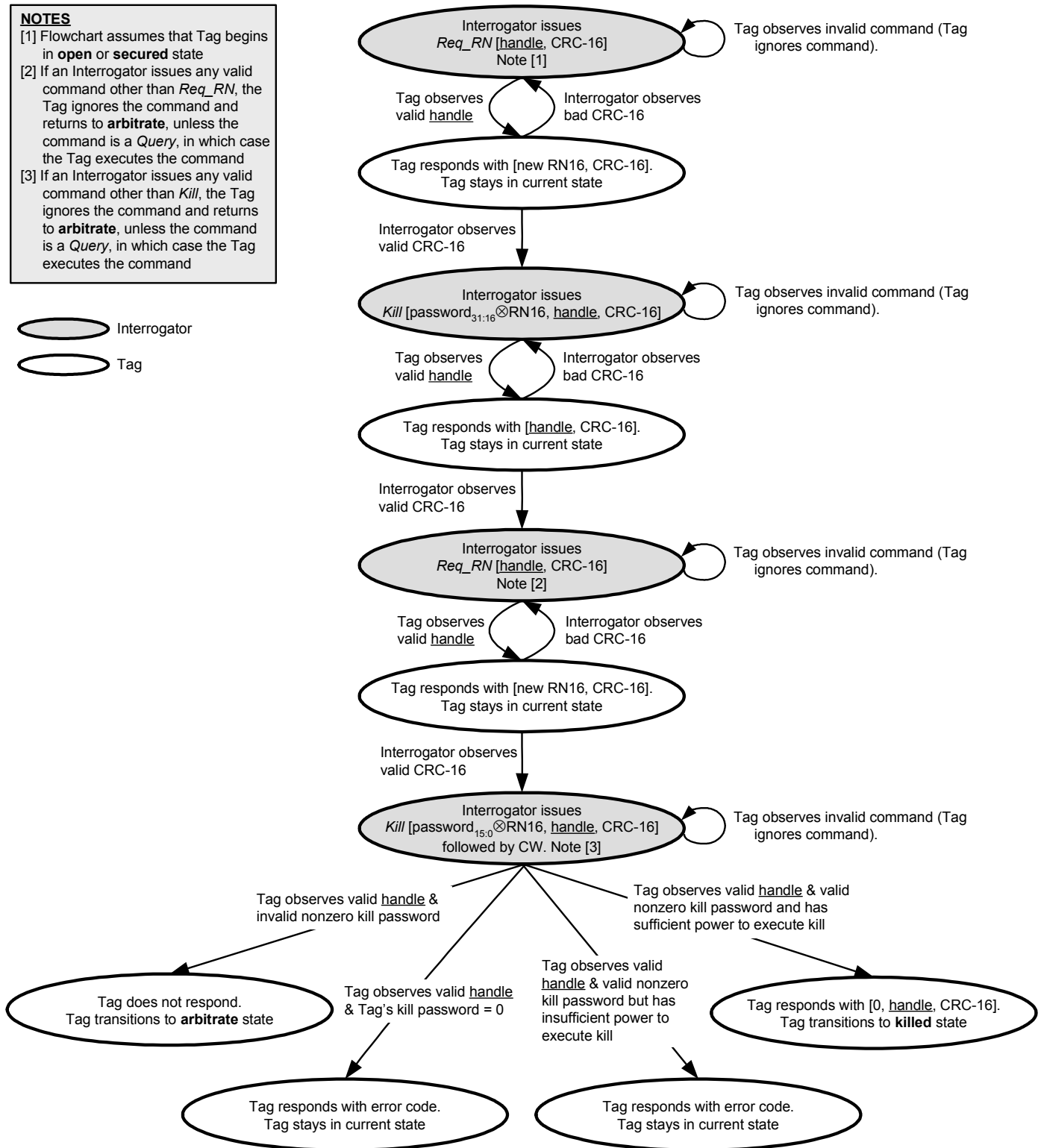
	RN	CRC-16
# of bits	16	16
description	<u>handle</u>	

Table Amd.1-38 — Tag reply to a successful *Kill* procedure

	Header	RN	CRC-16
# of bits	1	16	16
description	0	<u>handle</u>	

NOTES

- [1] Flowchart assumes that Tag begins in **open** or **secured** state
- [2] If an Interrogator issues any valid command other than *Req_RN*, the Tag ignores the command and returns to **arbitrate**, unless the command is a *Query*, in which case the Tag executes the command
- [3] If an Interrogator issues any valid command other than *Kill*, the Tag ignores the command and returns to **arbitrate**, unless the command is a *Query*, in which case the Tag executes the command

**Figure Amd.1-26 — Kill procedure**

9.3.2.10.3.5 Lock (mandatory)

Interrogators and tags shall implement the *Lock* command shown in Table Amd.1-39 and Figure Amd.1-27. Only tags in the **secured** state shall execute a *Lock* command. *Lock* allows an interrogator to:

- Lock individual passwords, thereby preventing or allowing subsequent reads and/or writes of that password,
- Lock individual memory banks, thereby preventing or allowing subsequent writes to that bank, and
- Permalock (make permanently unchangeable) the lock status for a password or memory bank.

Lock contains a 20-bit payload defined as follows:

The first 10 payload bits are Mask bits. A tag shall interpret these bit values as follows:

Mask = 0: Ignore the associated Action field and retain the current lock setting.

Mask = 1: Implement the associated Action field and overwrite the current lock setting.

The last 10 payload bits are Action bits. A tag shall interpret these bit values as follows:

Action = 0: De-assert lock for the associated memory location.

Action = 1: Assert lock or permalock for the associated memory location.

The functionality of the various Action fields is described in Table Amd.1-41.

The payload of a *Lock* command shall always be 20 bits in length.

If an interrogator issues a *Lock* command whose Mask and Action fields attempt to change the lock status of a nonexistent memory bank or nonexistent password, the tag shall ignore the entire *Lock* command and instead backscatter an error code (see Annex K).

Permalock bits, once asserted, cannot be de-asserted. If a tag receives a *Lock* whose payload attempts to de-assert a previously asserted permalock bit, the tag shall ignore the *Lock* and backscatter an error code (see Annex K). If a tag receives a *Lock* whose payload attempts to reassert a previously asserted permalock bit, the tag shall simply ignore this particular Action field and implement the remainder of the *Lock* payload.

A tag's lock bits cannot be read directly; they can be inferred by attempting to perform other memory operations.

All tags shall implement memory locking, and all tags shall implement the *Lock* command. However, tags need not support all the Action fields shown in Figure Amd.1-27, depending on whether the password location or memory bank associated with an Action field exists and is lockable and/or unlockable. Specifically, if a tag receives a *Lock* it cannot execute because one or more of the passwords or memory banks do not exist, or one or more of the Action fields attempt to change a previously permalocked value, or one or more of the passwords or memory banks are either not lockable or not unlockable, the tag shall ignore the entire *Lock* and instead backscatter an error code (see Annex K). The only exception to this general rule relates to tags whose only lock functionality is to permanently lock **all** memory (i.e. all memory banks and all passwords) at once; these tags shall execute a *Lock* whose payload is FFFFFF_h, and shall backscatter an error code for any payload other than FFFFFF_h.

A *Lock* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

After issuing a *Lock* an interrogator shall transmit CW for the lesser of T_{REPLY} or 20ms, where T_{REPLY} is the time between the interrogator's *Lock* command and the tag's backscattered reply. An interrogator may observe several possible outcomes from a *Lock*, depending on the success or failure of the tag's memory-write operation:

The *Lock* succeeds: After completing the *Lock* the tag shall backscatter the reply shown in Table Amd.1-40 and Figure Amd.1-25 comprising a header (a 0-bit), the tag's handle, and a CRC-16 calculated over the 0-bit and handle. If the interrogator observes this reply within 20 ms then the *Lock* completed successfully.

The tag encounters an error: The tag shall backscatter an error code during the CW period rather than the reply shown in Table Amd.1-40 (see Annex K for error-code definitions and for the reply format).

The *Lock* does not succeed: If the interrogator does not observe a reply within 20ms then the *Lock* did not complete successfully. The interrogator may issue a *Req_RN* command (containing the tag's handle) to verify that the tag is still in the interrogator's field, and may reissue the *Lock*.

Upon receiving a valid *Lock* command a tag shall perform the commanded lock operation. The tag's reply to a *Lock* shall use the extended preamble shown in Figure Amd.1-14 or Figure Amd.1-18, as appropriate (i.e. a tag shall reply as if T_{Text}=1 regardless of the T_{Text} value in the *Query* that initiated the round).

Table Amd.1-39 — Lock command

	Command	Payload	RN	CRC-16
# of bits	8	20	16	16
description	11000101	<u>Mask</u> and <u>Action</u> Fields	<u>handle</u>	

Table Amd.1-40 — Tag reply to a Lock command

	Header	RN	CRC-16
# of bits	1	16	16
description	0	<u>handle</u>	

Lock-Command Payload

19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
Kill	Access	Ull				TID		User		Kill	Access		Ull		TID		User		
Mask	Mask	Mask				Mask		Mask		Action	Action		Action		Action		Action		

Masks and Associated Action Fields

	Kill pwd		Access pwd		Ull memory		TID memory		User memory	
	19	18	17	16	15	14	13	12	11	10
<i>Mask</i>	skip/ write	skip/ write	skip/ write	skip/ write	skip/ write	skip/ write	skip/ write	skip/ write	skip/ write	skip/ write
<i>Action</i>	9	8	7	6	5	4	3	2	1	0
	pwd read/ write	perma lock	pwd read/ write	perma lock	pwd write	perma lock	pwd write	perma lock	pwd write	perma lock

Figure Amd.1-27 — Lock payload and usage

Table Amd.1-41 — Lock Action-field functionality

pwd-write	permalock	Description
0	0	Associated memory bank is writeable from either the open or secured states.
0	1	Associated memory bank is permanently writeable from either the open or secured states and may never be locked.
1	0	Associated memory bank is writeable from the secured state but not from the open state.
1	1	Associated memory bank is not writeable from any state.
pwd-read/write	permalock	Description
0	0	Associated password location is readable and writeable from either the open or secured states.
0	1	Associated password location is permanently readable and writeable from either the open or secured states and may never be locked.
1	0	Associated password location is readable and writeable from the secured state but not from the open state.
1	1	Associated password location is not readable or writeable from any state.

9.3.2.10.3.6 Access (optional)

Interrogators and tags may implement an *Access* command; if they do, the command shall be as shown in Table Amd.1-42. *Access* causes a tag with a nonzero-valued access password to transition from the **open** to the **secured** state (a tag with a zero-valued access password is never in the **open** state — see Figure Amd.1-22) or, if the tag is already in the **secured** state, to remain in **secured**.

To access a tag, an interrogator shall follow the multi-step procedure outlined in Figure Amd.1-28. Briefly, an interrogator issues two *Access* commands, the first containing the 16 MSBs of the tag's access password EXORed with an RN16, and the second containing the 16 LSBs of the tag's access password EXORed with a different RN16. Each EXOR operation shall be performed MSB first (i.e. the MSB of each half-password shall be EXORed with the MSB of its respective RN16). Just prior to issuing each *Access* command the interrogator first issues a *Req_RN* to obtain a new RN16.

Tags shall incorporate the necessary logic to successively accept two 16-bit subportions of a 32-bit access password. Interrogators shall not intersperse commands other than *Req_RN* between the two successive *Access* commands. If a tag, after receiving a first *Access*, receives any valid command other than *Req_RN* before the second *Access* it shall return to **arbitrate**, unless the intervening command is a *Query*, in which case the tag shall execute the *Query* (inverting its **inventoried** flag if the session parameter in the *Query* matches the prior session).

An *Access* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

The tag reply to an *Access* command shall be as shown in Table Amd.1-43. If the *Access* is the first in the sequence, then the tag backscatters its handle to acknowledge that it received the command. If the *Access* is the second in the sequence and the entire received 32-bit access password is correct, then the tag backscatters its handle to acknowledge that it has executed the command successfully and has transitioned to the **secured** state; otherwise the tag does not reply and returns to **arbitrate**. The reply includes a CRC-16 calculated over the handle.

Table Amd.1-42 — Access command

	Command	Password	RN	CRC-16
# of bits	8	16	16	16
description	11000110	(½ access password) ⊗ RN16	<u>handle</u>	

Table Amd.1-43 — Tag reply to an Access command

	RN	CRC-16
# of bits	16	16
description	<u>handle</u>	

NOTES

- [1] Flowchart assumes that Tag begins in **open** state or **secured** state
- [2] If an Interrogator issues any valid command other than *Req_RN*, the Tag ignores the command and returns to **arbitrate**, unless the command is a *Query*, in which case the Tag executes the command
- [3] If an Interrogator issues any valid command other than *Access*, the Tag ignores the command and returns to **arbitrate**, unless the command is a *Query*, in which case the Tag executes the command

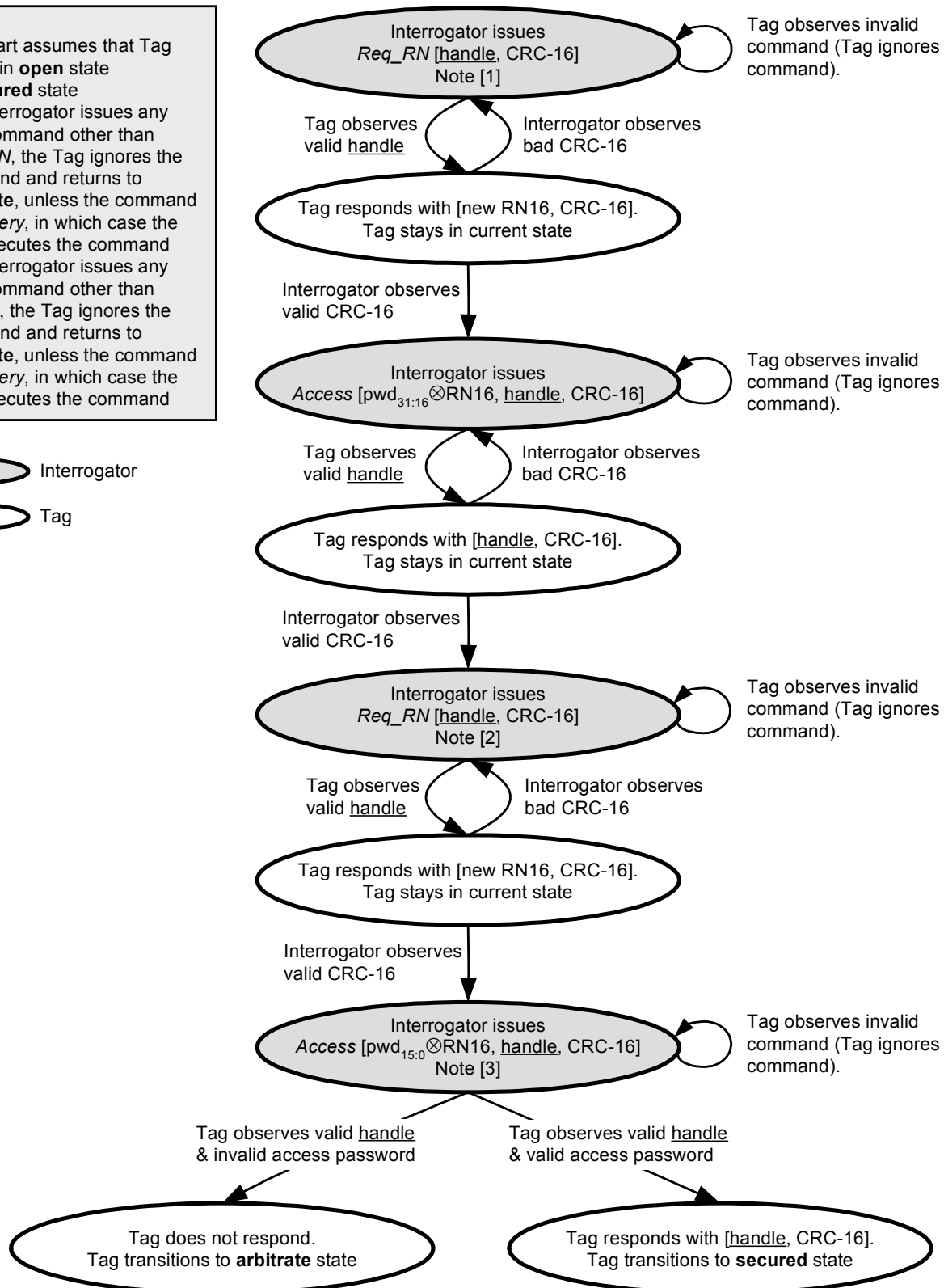


Figure Amd.1-28 — Access procedure

9.3.2.10.3.7 *BlockWrite* (optional)

Interrogators and tags may implement a *BlockWrite* command; if they do, they shall implement it as shown in Table Amd.1-44. *BlockWrite* allows an interrogator to write multiple words in a tag's Reserved, UII, TID, or User memory using a single command. *BlockWrite* has the following fields:

MemBank specifies whether the *BlockWrite* occurs in Reserved, UII, TID, or User memory. *BlockWrite* commands shall apply to a single memory bank. Successive *BlockWrites* may apply to different banks.

WordPtr specifies the starting word address for the memory write, where words are 16 bits in length. For example, WordPtr = 00_h specifies the first 16-bit memory word, WordPtr = 01_h specifies the second 16-bit memory word, etc. WordPtr uses EBV formatting (see Annex D).

WordCount specifies the number of 16-bit words to be written. If WordCount = 00_h the tag shall ignore the *BlockWrite*. If WordCount = 01_h the tag shall write a single data word.

Data contains the 16-bit words to be written, and shall be 16×WordCount bits in length. Unlike a *Write*, the data in a *BlockWrite* are not cover-coded, and an interrogator need not issue a *Req_RN* before issuing a *BlockWrite*.

The *BlockWrite* command also includes the tag's handle and a CRC-16. The CRC-16 is calculated over the first command-code bit to the last handle bit.

If a tag receives a *BlockWrite* with a valid CRC-16 but an invalid handle it shall ignore the *BlockWrite* and remain in its current state (**open** or **secured**, as appropriate).

A *BlockWrite* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

After issuing a *BlockWrite* an interrogator shall transmit CW for the lesser of T_{REPLY} or 20ms, where T_{REPLY} is the time between the interrogator's *BlockWrite* command and the tag's backscattered reply. An interrogator may observe several possible outcomes from a *BlockWrite*, depending on the success or failure of the tag's memory-write operation:

The *BlockWrite* succeeds: After completing the *BlockWrite* a tag shall backscatter the reply shown in Table Amd.1-45 and Figure Amd.1-25 comprising a header (a 0-bit), the tag's handle, and a CRC-16 calculated over the 0-bit and handle. If the interrogator observes this reply within 20 ms then the *BlockWrite* completed successfully.

The tag encounters an error: The tag shall backscatter an error code during the CW period rather than the reply shown in Table Amd.1-45 (see Annex K for error-code definitions and for the reply format).

The *BlockWrite* does not succeed: If the interrogator does not observe a reply within 20ms then the *BlockWrite* did not complete successfully. The interrogator may issue a *Req_RN* command (containing the tag's handle) to verify that the tag is still in the interrogator's field, and may reissue the *BlockWrite*.

Upon receiving a valid *BlockWrite* command a tag shall write the commanded Data into memory. The tag's reply to a *BlockWrite* shall use the extended preamble shown in Figure Amd.1-14 or Figure Amd.1-18, as appropriate (i.e. a tag shall reply as if T_{Text}=1 regardless of the T_{Text} value in the *Query* that initiated the round).

Table Amd.1-44 — *BlockWrite* command

	Command	MemBank	WordPtr	WordCount	Data	RN	CRC-16
# of bits	8	2	EBV	8	Variable	16	16
description	11000111	00: Reserved 01: UII 10: TID 11: User	Starting address pointer	Number of words to write	Data to be written	<u>handle</u>	

Table Amd.1-45 — Tag reply to a successful *BlockWrite* command

	Header	RN	CRC-16
# of bits	1	16	16
description	0	<u>handle</u>	

9.3.2.10.3.8 **BlockErase (optional)**

Interrogators and tags may implement a *BlockErase* command; if they do, they shall implement it as shown in Table Amd.1-46. *BlockErase* allows an interrogator to erase multiple words in a tag's Reserved, Ull, TID, or User memory using a single command. *BlockErase* has the following fields:

MemBank specifies whether the *BlockErase* occurs in Reserved, Ull, TID, or User memory. *BlockErase* commands shall apply to a single memory bank. Successive *BlockErases* may apply to different banks.

WordPtr specifies the starting word address for the memory erase, where words are 16 bits in length. For example, WordPtr = 00_h specifies the first 16-bit memory word, WordPtr = 01_h specifies the second 16-bit memory word, etc. WordPtr uses EBV formatting (see Annex D).

WordCount specifies the number of 16-bit words to be erased. If WordCount = 00_h the tag shall ignore the *BlockErase*. If WordCount = 01_h the tag shall erase a single data word.

The *BlockErase* command also includes the tag's handle and a CRC-16. The CRC-16 is calculated over the first command-code bit to the last handle bit.

If a tag receives a *BlockErase* with a valid CRC-16 but an invalid handle it shall ignore the *BlockErase* and remain in its current state (**open** or **secured**, as appropriate).

A *BlockErase* shall be pre-pended with a frame-sync (see 9.3.1.2.8).

After issuing a *BlockErase* an interrogator shall transmit CW for the lesser of T_{REPLY} or 20ms, where T_{REPLY} is the time between the interrogator's *BlockErase* command and the tag's backscattered reply. An interrogator may observe several possible outcomes from a *BlockErase*, depending on the success or failure of the tag's memory-erase operation:

The *BlockErase* succeeds: After completing the *BlockErase* a tag shall backscatter the reply shown in Table 47 and Figure Amd.1-25 comprising a header (a 0-bit), the tag's handle, and a CRC-16 calculated over the 0-bit and handle. If the interrogator observes this reply within 20 ms then the *BlockErase* completed successfully.

The tag encounters an error: The tag shall backscatter an error code during the CW period rather than the reply shown in Table 47 (see Annex K for error-code definitions and for the reply format).

The *BlockErase* does not succeed: If the interrogator does not observe a reply within 20ms then the *BlockErase* did not complete successfully. The interrogator may issue a *Req_RN* command (containing the tag's handle) to verify that the tag is still in the interrogator's field, and may reissue the *BlockErase*.

Upon receiving a valid *BlockErase* command a tag shall erase the commanded memory words. The tag's reply to a *BlockErase* shall use the extended preamble shown in Figure Amd.1-14 or Figure Amd.1-18, as appropriate (i.e. a tag shall reply as if T_{RExt}=1 regardless of the T_{RExt} value in the *Query* that initiated the round).

Table Amd.1-46 — *BlockErase* command

	Command	MemBank	WordPtr	WordCount	RN	CRC-16
# of bits	8	2	EBV	8	16	16
description	11001000	00: Reserved 01: Ull 10: TID 11: User	Starting address pointer	Number of words to erase	<u>handle</u>	

Table 47 — Tag reply to a successful *BlockErase* command

	Header	RN	CRC-16
# of bits	1	16	16
description	0	<u>Handle</u>	

Annex A (informative)

Calculation of 5-bit and 16-bit cyclic redundancy checks

A.1 Example CRC-5 encoder/decoder

An exemplary schematic diagram for a CRC-5 encoder/decoder is shown in Figure A.1, using the polynomial and preset defined in Table Amd.1-14.

To compute a CRC-5, first preload the entire CRC register (i.e. Q[4:0], Q4 being the MSB and Q0 the LSB) with the value 01001₂ (see Table A.1), then clock the data bits to be encoded into the input labelled DATA, MSB first. After clocking in all the data bits, Q[4:0] holds the CRC-5 value.

To check a CRC-5, first preload the entire CRC register (C[4:0]) with the value 01001₂, then clock the received data and CRC-5 {data, CRC-5} bits into the input labelled DATA, MSB first. The CRC-5 check passes if the value in Q[4:0] = 00000₂.

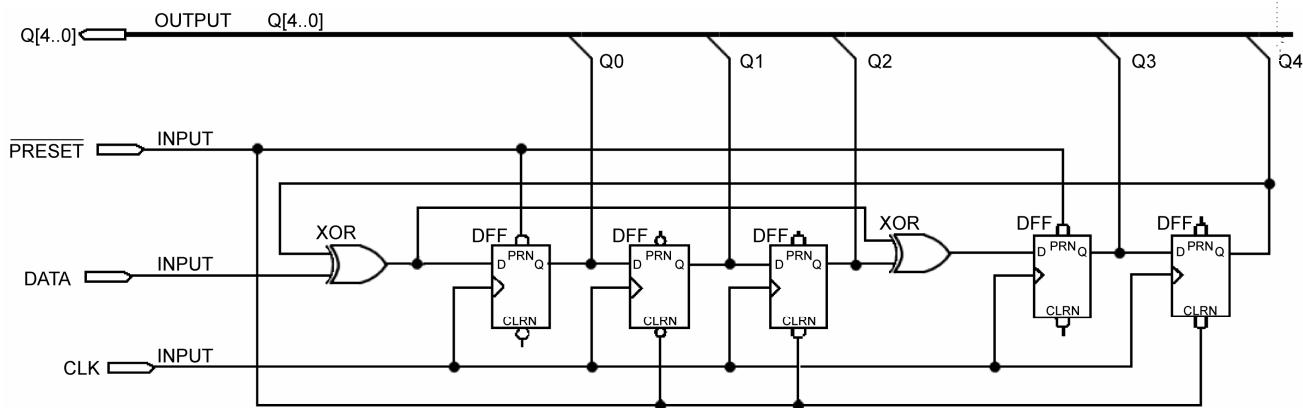


Figure A.1 — Example CRC-5 circuit

Table A.1 — CRC-5 register preload values

Register	Preload value
Q0	1
Q1	0
Q2	0
Q3	1
Q4	0

A.2 Example CRC-16 encoder/decoder

An exemplary schematic diagram for a CRC-16 encoder/decoder is shown in Figure A.2, using the polynomial and preset defined in Table Amd.1-13 (the polynomial used to calculate the CRC-16, $x^{16} + x^{12} + x^5 + 1$, is the CRC-CCITT International Standard).

To compute a CRC-16, first preload the entire CRC register (i.e. Q[15:0], Q15 being the MSB and Q0 the LSB) with the value FFFF_h. Second, clock the data bits to be encoded into the input labelled DATA, MSB first. After clocking in all the data bits, Q[15:0] holds the ones-complement of the CRC-16. Third, invert all the bits of Q[15:0] to produce the CRC-16.

There are two methods to check a CRC-16:

Method 1: First preload the entire CRC register (Q[15:0]) with the value FFFF_h, then clock the received data and CRC-16 {data, CRC-16} bits into the input labelled DATA, MSB first. The CRC-16 check passes if the value in Q[15:0]=1D0F_h.

Method 2: First preload the entire CRC register (Q[15:0]) with the value FFFF_h. Second, clock the received data bits into the input labelled DATA, MSB first. Third, invert all bits of the received CRC-16, and clock the inverted CRC-16 bits into the input labelled DATA, MSB first. The CRC-16 check passes if the value in Q[15:0]=0000_h.

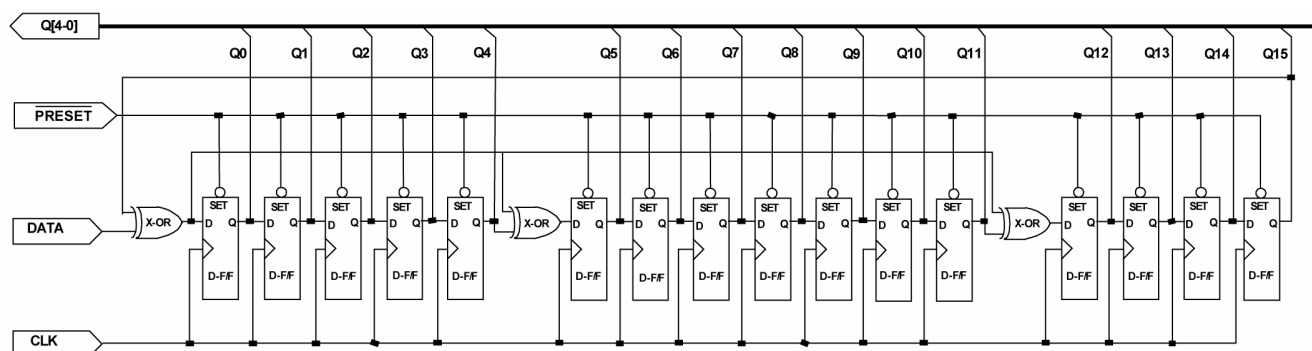


Figure A.2 — Example CRC-16 circuit

A.3 Example CRC-16 calculations

Example 1: This example shows the CRC-16 that a Type C tag would compute at powerup. As shown in Figure Amd.1-20, Ull memory in Type C tags contains a CRC-16 starting at address 00_h, PC bits starting at address 10_h, and zero or more Ull words starting at address 20_h. Table A.2 shows the CRC-16 that a tag would compute and logically map into Ull memory at powerup, for the indicated example PC and Ull word values. In each successive column, one more word of Ull memory is written, with the entire Ull memory written in the rightmost column. The indicated PC values correspond to the number of Ull words written, with PC bits 15_h–1F_h set to zero. Entries marked N/A mean that that word of Ull memory is not included as part of the CRC calculation.

Table A.2 – Ull memory contents for an example Type C tag

Ull word starting address	Ull word contents	Ull word values						
00 _h	CRC-16	E2F0 _h	CCAE _h	968F _h	78F6 _h	C241 _h	2A91 _h	1835 _h
10 _h	PC	0000 _h	0800 _h	1000 _h	1800 _h	2000 _h	2800 _h	3000 _h
20 _h	Ull word 1	N/A	1111 _h	1111	1111 _h	1111 _h	1111 _h	1111 _h
30 _h	Ull word 2	N/A	N/A	2222 _h	2222 _h	2222 _h	2222 _h	2222 _h
20 _h	Ull word 3	N/A	N/A	N/A	3333 _h	3333 _h	3333 _h	3333 _h
40 _h	Ull word 4	N/A	N/A	N/A	N/A	4444 _h	4444 _h	4444 _h
50 _h	Ull word 5	N/A	N/A	N/A	N/A	N/A	5555 _h	5555 _h
60 _h	Ull word 6	N/A	N/A	N/A	N/A	N/A	N/A	6666 _h

Example 2: This example (a) computes the CRC that a Type B interrogator would use when sending a SUCCESS command (code 09_h) to a tag, then (b) performs the check a tag would perform when verifying the received command. The interrogator sends the packet shown in Table A.3; only the SUCCESS command (09_h) is used in the CRC-16 calculation.

Computing the CRC-16: Table A.4 shows the bit values in the 16 CRC registers of Figure A.2 as the command code 09_h is shifted, bit-by-bit, into the CRC circuit. The CRC-16 bits actually transmitted by the interrogator are inverted from those shown in step 8 of Table A.4, namely 8F26_h.

Table A.3 — Example Type B command packet

Preamble detect	Preamble	Start delimiter	SUCCESS command	CRC-16
2 bytes field high	nine Manchester 0's	11 00 11 10 10	09 _h	CRC-16

Table A.4 — Values of Q[15:0] for the command in Table A.3

Step	Bit input (SUCCESS command)	CRC register values Q[15:0]
1	0	EFDF _h
2	0	CF9F _h
3	0	8F1F _h
4	0	0E1F _h
5	1	0C1F _h
6	0	183E _h
7	0	307C _h
8	1	70D9 _h

Checking the CRC-16: As described in A.2, a tag may use choose one of two methods to verify the CRC-16 in a received command. Table A.5 shows the first method, in which the tag clocks the received data and CRC-16 {data, CRC-16} bits into the input labelled DATA, MSB first, and verifies that the value in Q[15:0]=1D0F_h. Table A.5 assumes, at step 0, that the CRC-16 circuit was already preloaded with FFFF_h and the data bits clocked in, leaving the value in Q[15:0]=70D9_h prior to inputting the CRC-16. Table A.5 shows the bit values in the 16 CRC registers of Figure A.2 as the CRC-16 bits are shifted, one-by-one, into the CRC circuit.

Table A.5 — First method for checking a CRC-16 for the {command, CRC-16} from Table A.4

Step	Input (received CRC-16)	CRC register values Q[15:0]
0		70D9 _h
1	1	F193 _h
2	0	F307 _h
3	0	F62F _h
4	0	FC7F _h
5	1	F8FE _h
6	1	F1FC _h
7	1	E3F8 _h
8	1	C7F0 _h
9	0	9FC1 _h
10	0	2FA3 _h
11	1	4F67 _h
12	0	9ECE _h
13	0	2DBD _h
14	1	4B5B _h
15	1	8697 _h
16	0	1D0F _h

Table A.6 shows the second method of checking the CRC, in which the tag clocks the received data bits into the input labelled DATA, MSB first, then inverts and clocks all bits of the received CRC-16 into the input labelled DATA, MSB first, and verifies that the value in Q[15:0]=0000_h. Table A.5 assumes, at step 0, that the CRC-16 circuit was already preloaded with FFFF_h and the data bits clocked in, leaving the value in Q[15:0]=70D9_h prior to inputting the inverted CRC-16. Table A.5 shows the bit values in the 16 CRC registers of Figure A.2 as the inverted CRC-16 bits are shifted, one-by-one, into the CRC circuit.

Table A.6 — Second method for checking a CRC-16 for the {command, CRC-16} from Table A.4

Step	Input (inverted received CRC-16)	CRC register values Q[15:0]
0		70D9 _h
1	0	E1B2 _h
2	1	C364 _h
3	1	86C8 _h
4	1	0D90 _h
5	0	1B20 _h
6	0	3640 _h
7	0	6C80 _h
8	0	D900 _h
9	1	B200 _h
10	1	6400 _h
11	0	C800 _h
12	1	9000 _h
13	1	2000 _h
14	0	4000 _h
15	0	8000 _h
16	1	0000 _h

Annex D
(normative)

Extensible bit vectors (EBV)

An *extensible bit vector* (EBV) is a data structure with an extensible data range.

An EBV is an array of *blocks*. Each block contains a single extension bit followed by a specific number of data bits. If B represents the total number of bits in one block, then a block contains B – 1 data bits. Although a general EBV may contain blocks of varying lengths, tags and interrogators manufactured according to this International Standard shall use blocks of length 8 bits (EBV-8).

The data value represented by an EBV is simply the bit string formed by the data bits as read from left-to-right, ignoring the extension bits.

Tags and interrogators shall use the EBV-8 word format specified in Table D.1.

Table D.1 — EBV-8 word format						
$2^7 - 1$	0	0	0000000			
	1	0	0000001			
	127	0	1111111			
2^7	128	1	0000001	0	0000000	
$2^{14} - 1$	16383	1	1111111	0	1111111	
2^{14}	16384	1	0000001	1	0000000	0 0000000

Because each block has 7 data bits available, the EBV-8 can represent numeric values between 0 and 127 with a single block. To represent the value 128, the extension bit is set to 1 in the first block, and a second block is appended to the EBV-8. In this manner, an EBV-8 can represent arbitrarily large values.

This International Standard uses EBV-8 values to represent memory addresses and mask lengths.

Annex E (normative)

State-transition tables

State-transition Tables E.1 to E.7 shall define a tag's response to interrogator commands. The term "handle" used in the state-transition tables is defined in 9.3.2.4.5; error codes are defined in Table K.2; "slot" is the slot-counter output shown in Figure 19 and detailed in Annex L; "–" in the "Action" column means that a tag neither modifies its **SL** or **inventoried** flags nor backscatters a reply.

E.1 Present state: Ready

Table E.1 — Ready state-transition table

Command	Condition	Action	Next State
<i>Query</i> ^a	slot=0; matching inventoried & SL flags	backscatter new RN16	reply
	slot<>0; matching inventoried & SL flags	–	arbitrate
	otherwise	–	ready
<i>QueryRep</i>	all	–	ready
<i>QueryAdjust</i>	all	–	ready
<i>ACK</i>	all	–	ready
<i>NAK</i>	all	–	ready
<i>Req_RN</i>	all	–	ready
<i>Select</i>	all	assert or de-assert SL , or set inventoried to <i>A</i> or <i>B</i>	ready
<i>Read</i>	all	–	ready
<i>Write</i>	all	–	ready
<i>Kill</i>	all	–	ready
<i>Lock</i>	all	–	ready
<i>Access</i>	all	–	ready
<i>BlockWrite</i>	all	–	ready
<i>BlockErase</i>	all	–	ready
<i>Invalid</i> ^b	all	–	ready
^a <i>Query</i> starts a new round and may change the session. <i>Query</i> also instructs a tag to load a new random value into its slot counter. ^b "Invalid" shall mean an erroneous command, an unsupported command, a command with invalid parameters, a command with a CRC error, or any other command either not recognized or not executable by the tag.			

E.2 Present state: Arbitrate

Table E.2 — Arbitrate state-transition table

Command	Condition	Action	Next State
<i>Query</i> ^{a,b}	slot=0; matching inventoried & SL flags	backscatter new RN16	reply
	slot<>0; matching inventoried & SL flags	—	arbitrate
	otherwise	—	ready
<i>QueryRep</i>	slot=0 after decrementing slot counter	backscatter new RN16	reply
	slot<>0 after decrementing slot counter	—	arbitrate
<i>QueryAdjust</i> ^b	slot=0	backscatter new RN16	reply
	slot<>0	—	arbitrate
<i>ACK</i>	all	—	arbitrate
<i>NAK</i>	all	—	arbitrate
<i>Req_RN</i>	all	—	arbitrate
<i>Select</i>	all	assert or de-assert SL , or set inventoried to <i>A</i> or <i>B</i>	ready
<i>Read</i>	all	—	arbitrate
<i>Write</i>	all	—	arbitrate
<i>Kill</i>	all	—	arbitrate
<i>Lock</i>	all	—	arbitrate
<i>Access</i>	all	—	arbitrate
<i>Erase</i>	all	—	arbitrate
<i>BlockWrite</i>	all	—	arbitrate
<i>BlockErase</i>	all	—	arbitrate
<i>Invalid</i> ^c	all	—	arbitrate
^a <i>Query</i> starts a new round and may change the session. ^b <i>Query</i> and <i>QueryAdjust</i> instruct a tag to load a new random value into its slot counter. ^c “Invalid” shall mean an erroneous command, an unsupported command, a command with invalid parameters, a command with a CRC error, a command (other than a <i>Query</i>) with a <u>session</u> parameter not matching that of the inventory round currently in progress, or any other command either not recognized or not executable by the tag.			

E.3 Present state: Reply

Table E.3 — Reply state-transition table

Command	Condition	Action	Next State
<i>Query</i> ^{a, b}	slot=0; matching inventoried & SL flags	backscatter new RN16	reply
	slot<>0; matching inventoried & SL flags	—	arbitrate
	otherwise	—	ready
<i>QueryRep</i>	all	—	arbitrate
<i>QueryAdjust</i> ^b	slot=0	backscatter new RN16	reply
	slot<>0	—	arbitrate
<i>ACK</i>	valid RN16	backscatter {PC, UII, CRC-16} or {00000 ₂ , truncated UII, CRC-16}	acknowledged
	invalid RN16	—	arbitrate
<i>NAK</i>	all	—	arbitrate
<i>Req_RN</i>	all	—	arbitrate
<i>Select</i>	all	assert or de-assert SL , or set inventoried to A or B	ready
<i>Read</i>	all	—	arbitrate
<i>Write</i>	all	—	arbitrate
<i>Kill</i>	all	—	arbitrate
<i>Lock</i>	all	—	arbitrate
<i>Access</i>	all	—	arbitrate
<i>BlockWrite</i>	all	—	arbitrate
<i>BlockErase</i>	all	—	arbitrate
T ₂ timeout	T ₂ > 20.0T _{pri} (see Figure Amd.1-19)	—	arbitrate
Invalid ^c	all	—	reply

^a *Query* starts a new round and may change the session.

^b *Query* and *QueryAdjust* instruct a tag to load a new random value into its slot counter.

^c “Invalid” shall mean an erroneous command, an unsupported command, a command with invalid parameters, a command with a CRC error, a command (other than a *Query*) with a session parameter not matching that of the inventory round currently in progress, or any other command either not recognized or not executable by the tag.

E.4 Present state: Acknowledged

Table E.4 — Acknowledged state-transition table

Command	Condition	Action	Next State
<i>Query</i> ^a	slot=0; matching inventoried ^b & SL flags	backscatter new RN16; transition inventoried ^b from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	reply
	slot<>0; matching inventoried ^b & SL flags	transition inventoried ^b from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	arbitrate
	otherwise	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	ready
<i>QueryRep</i>	all	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$	ready
<i>QueryAdjust</i>	all	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$	ready
<i>ACK</i>	valid RN16	backscatter {PC, UII, CRC-16} or {00000 ₂ , truncated UII, CRC-16}	acknowledged
	invalid RN16	—	arbitrate
<i>NAK</i>	all	—	arbitrate
<i>Req_RN</i>	valid RN16 & access password<>0	backscatter <u>handle</u>	open
	valid RN16 & access password=0	backscatter <u>handle</u>	secured
	invalid RN16	—	acknowledged
<i>Select</i>	all	assert or de-assert SL , or set inventoried to <i>A</i> or <i>B</i>	ready
<i>Read</i>	all	—	arbitrate
<i>Write</i>	all	—	arbitrate
<i>Kill</i>	all	—	arbitrate
<i>Lock</i>	all	—	arbitrate
<i>Access</i>	all	—	arbitrate
<i>BlockWrite</i>	all	—	arbitrate
<i>BlockErase</i>	all	—	arbitrate
<i>T₂ timeout</i>	$T_2 > 20.0T_{pri}$ (see Figure Amd.1-19)	—	arbitrate
<i>Invalid</i> ^c	all	—	acknowledged

^a *Query* starts a new round and may change the session. *Query* also instructs a tag to load a new random value into its slot counter.

^b As described in 9.3.2.8, a tag transitions its **inventoried** flag prior to evaluating the condition.

^c "Invalid" shall mean an erroneous command, an unsupported command, a command with invalid parameters, a command with a CRC error, a command (other than a *Query*) with a session parameter not matching that of the inventory round currently in progress, or any other command either not recognized or not executable by the tag.

E.5 Present state: Open

Table E.5 — Open state-transition table

Command	Condition	Action	Next State
<i>Query</i> ^a	slot=0; matching inventoried ^b & SL flags	backscatter new RN16; transition inventoried ^b from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	reply
	slot<>0; matching inventoried ^b & SL flags	transition inventoried ^b from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	arbitrate
	otherwise	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	ready
<i>QueryRep</i>	all	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$	ready
<i>QueryAdjust</i>	all	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$	ready
<i>ACK</i>	valid <u>handle</u>	backscatter {PC, UII, CRC-16} or {00000 ₂ , truncated UII, CRC-16}	open
	invalid <u>handle</u>	—	arbitrate
<i>NAK</i>	all	—	arbitrate
<i>Req_RN</i>	valid <u>handle</u>	backscatter new RN16	open
	invalid <u>handle</u>	—	open
<i>Select</i>	all	assert or de-assert SL, or set inventoried to A or B	ready
<i>Read</i>	valid <u>handle</u> & valid memory access	backscatter data and <u>handle</u>	open
	valid <u>handle</u> & invalid memory access	backscatter error code	open
	invalid <u>handle</u>	—	open
<i>Write</i>	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	open
	valid <u>handle</u> & invalid memory access	backscatter error code	open
	invalid <u>handle</u>	—	open
<i>Kill</i> (see also Figure Amd.1-26)	valid <u>handle</u> & valid nonzero kill password	backscatter <u>handle</u> when done	killed
	valid <u>handle</u> & invalid nonzero kill password	—	arbitrate
	valid <u>handle</u> & kill password=0	backscatter error code	open
	invalid <u>handle</u>	—	open
<i>Lock</i>	all	—	open
<i>Access</i> (see also Figure Amd.1-28)	valid <u>handle</u> & valid access password	backscatter <u>handle</u>	secured
	valid <u>handle</u> & invalid access password	—	arbitrate
	invalid <u>handle</u>	—	open
<i>BlockWrite</i>	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	open
	valid <u>handle</u> & invalid memory access	backscatter error code	open
	invalid <u>handle</u>	—	open
<i>BlockErase</i>	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	open
	valid <u>handle</u> & invalid memory access	backscatter error code	open
	invalid <u>handle</u>	—	open
Invalid ^c	all, excluding commands interspersed between successive <i>Kill</i> or <i>Access</i> commands in a kill or access sequence, respectively (see Figure Amd.1-26 and Figure Amd.1-28).	—	open
	valid commands, except <i>Req_RN</i> , interspersed between successive <i>Kill</i> or <i>Access</i> commands in a kill or access sequence, respectively (see Figure Amd.1-26 and Figure Amd.1-28).	—	arbitrate

^a *Query* starts a new round and may change the session. *Query* also instructs a tag to load a new random value into its slot counter.

^b As described in 9.3.2.8, a tag transitions its **inventoried** flag prior to evaluating the condition.

^c "Invalid" shall mean an erroneous command, an unsupported command, a command with invalid parameters, a command with a CRC error, a command (other than a *Query*) with a session parameter not matching that of the inventory round currently in progress, or any other command either not recognized or not executable by the tag.

E.6 Present state: Secured

Table E.6 — Secured state-transition table

Command	Condition	Action	Next State
<i>Query</i> ^a	slot=0; matching inventoried ^b & SL flags	backscatter new RN16; transition inventoried ^b from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	reply
	slot<>0; matching inventoried ^b & SL flags	transition inventoried ^b from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	arbitrate
	otherwise	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	ready
<i>QueryRep</i>	all	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$	ready
<i>QueryAdjust</i>	all	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$	ready
<i>ACK</i>	valid <u>handle</u>	backscatter {PC, UII, CRC-16} or {0000 ₂ , truncated UII, CRC-16}	secured
	invalid <u>handle</u>	—	arbitrate
<i>NAK</i>	all	—	arbitrate
<i>Req_RN</i>	valid <u>handle</u>	backscatter new RN16	secured
	invalid <u>handle</u>	—	secured
<i>Select</i>	all	assert or de-assert SL , or set inventoried to A or B	ready
<i>Read</i>	valid <u>handle</u> & valid memory access	backscatter data and <u>handle</u>	secured
	valid <u>handle</u> & invalid memory access	backscatter error code	secured
	invalid <u>handle</u>	—	secured
<i>Write</i>	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	secured
	valid <u>handle</u> & invalid memory access	backscatter error code	secured
	invalid <u>handle</u>	—	secured
<i>Kill</i> (see also Figure Amd.1-26)	valid <u>handle</u> & valid nonzero kill password	backscatter <u>handle</u> when done	killed
	valid <u>handle</u> & invalid nonzero kill password	—	arbitrate
	valid <u>handle</u> & kill password=0	backscatter error code	secured
	invalid <u>handle</u>	—	secured
<i>Lock</i>	valid <u>handle</u> & valid lock payload	backscatter <u>handle</u> when done	secured
	valid <u>handle</u> & invalid lock payload	backscatter error code	secured
	invalid <u>handle</u>	—	secured
<i>Access</i> (see also Figure Amd.1-28)	valid <u>handle</u> & valid access password	backscatter <u>handle</u>	secured
	valid <u>handle</u> & invalid access password	—	arbitrate
	invalid <u>handle</u>	—	secured
<i>BlockWrite</i>	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	secured
	valid <u>handle</u> & invalid memory access	backscatter error code	secured
	invalid <u>handle</u>	—	secured
<i>BlockErase</i>	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	secured
	valid <u>handle</u> & invalid memory access	backscatter error code	secured
	invalid <u>handle</u>	—	secured
Invalid ^c	all, excluding commands interspersed between successive <i>Kill</i> or <i>Access</i> commands in a kill or access sequence, respectively (see Figure Amd.1-26 and Figure Amd.1-28).	—	secured
	valid commands, except <i>Req_RN</i> , interspersed between successive <i>Kill</i> or <i>Access</i> commands in a kill or access sequence, respectively (see Figure Amd.1-26 and Figure Amd.1-28).	—	arbitrate

^a *Query* starts a new round and may change the session. *Query* also instructs a tag to load a new random value into its slot counter.

^b As described in 9.3.2.8, a tag transitions its **inventoried** flag prior to evaluating the condition.

^c "Invalid" shall mean an erroneous command, an unsupported command, a command with invalid parameters, a command with a CRC error, a command (other than a *Query*) with a session parameter not matching that of the inventory round currently in progress, or any other command either not recognized or not executable by the tag.

E.7 Present state: Killed

Table E.7 — Killed state-transition table

Command	Condition	Action	Next State
<i>Query</i>	all	—	killed
<i>QueryRep</i>	all	—	killed
<i>QueryAdjust</i>	all	—	killed
<i>ACK</i>	all	—	killed
<i>NAK</i>	all	—	killed
<i>Req_RN</i>	all	—	killed
<i>Select</i>	all	—	killed
<i>Read</i>	all	—	killed
<i>Write</i>	all	—	killed
<i>Kill</i>	all	—	killed
<i>Lock</i>	all	—	killed
<i>Access</i>	all	—	killed
<i>BlockWrite</i>	all	—	killed
<i>BlockErase</i>	all	—	killed
Invalid ^a	all	—	killed
^a “Invalid” shall mean an erroneous command, an unsupported command, a command with invalid parameters, a command with a CRC error, or any other command either not recognized or not executable by the tag.			

Annex F (normative) Command-Response Tables

Command-response tables F.1 to F.17 shall define a tag's response to interrogator commands. The term "handle" used in the state-transition tables is defined in 9.3.2.4.5; error codes are defined in Table K.2; "slot" is the slot-counter output shown in Figure Amd.1-22 and detailed in Annex L; "—" in the "Response" column means that a tag neither modifies its **SL** or **inventoried** flags nor backscatters a reply.

F.1 Command response: Power-up

Table F.1 — Power-up command-response table

Starting State	Condition	Response	Next State
ready, arbitrate, reply, acknowledged, open, secured	power-up	—	ready
killed	all	—	killed

F.2 Command response: Query

Table F.2 — Query^a command-response table

Starting State	Condition	Response	Next State
ready, arbitrate, reply	slot=0; matching inventoried & SL flags	backscatter new RN16	reply
	slot<>0; matching inventoried & SL flags	—	arbitrate
	otherwise	—	ready
acknowledged, open, secured	slot=0; matching inventoried ^b & SL flags	backscatter new RN16; transition inventoried ^b from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	reply
	slot<>0; matching inventoried ^b & SL flags	transition inventoried ^b from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	arbitrate
	otherwise	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$ if and only if new <u>session</u> matches prior <u>session</u>	ready
killed	all	—	killed

^a *Query* (in any state other than killed) starts a new round and may change the session; *Query* also instructs a tag to load a new random value into its slot counter.

^b As described in 9.3.2.8, a tag transitions its **inventoried** flag prior to evaluating the condition.

F.3 Command response: *QueryRep*

Table F.3 — *QueryRep* command-response table ^a

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate	slot<>0 after decrementing slot counter	—	arbitrate
	slot=0 after decrementing slot counter	backscatter new RN16	reply
reply	all	—	arbitrate
acknowledged, open, secured	all	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$	ready
killed	all	—	killed

^a See Table F.17 for the tag response to a *QueryRep* whose session parameter does not match that of the current inventory round.

F.4 Command response: *QueryAdjust*

Table F.4 — *QueryAdjust* ^a command-response table ^b

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate, reply	slot<>0	—	arbitrate
	slot=0	backscatter new RN16	reply
acknowledged, open, secured	all	transition inventoried from $A \rightarrow B$ or $B \rightarrow A$	ready
killed	all	—	killed

^a *QueryAdjust*, in the arbitrate or reply states, instructs a tag to load a new random value into its slot counter.
^b See Table F.17 for the tag response to a *QueryRep* whose session parameter does not match that of the current inventory round.

F.5 Command response: *ACK*

Table F.5 — *ACK* command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate	all	—	arbitrate
reply	valid RN16	backscatter {PC, UII, CRC-16} or {00000 ₂ , truncated UII, CRC-16}	acknowledged
	invalid RN16	—	arbitrate
acknowledged	valid RN16	backscatter {PC, UII, CRC-16} or {00000 ₂ , truncated UII, CRC-16}	acknowledged
	invalid RN16	—	arbitrate
open	valid <u>handle</u>	backscatter {PC, UII, CRC-16} or {00000 ₂ , truncated UII, CRC-16}	open
	invalid <u>handle</u>	—	arbitrate
secured	valid <u>handle</u>	backscatter {PC, UII, CRC-16} or {00000 ₂ , truncated UII, CRC-16}	secured
	invalid <u>handle</u>	—	arbitrate
killed	all	—	killed

F.6 Command response: *NAK*Table F.6 — *NAK* command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate, reply, acknowledged, open, secured	all	—	arbitrate
killed	all	—	killed

F.7 Command response: *Req_RN*Table F.7 — *Req_RN* command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate, reply	all	—	arbitrate
acknowledged	valid RN16 & access password<>0	backscatter <u>handle</u>	open
	valid RN16 & access password=0	backscatter <u>handle</u>	secured
	invalid RN16	—	acknowledged
open	valid <u>handle</u>	backscatter new RN16	open
	invalid <u>handle</u>	—	open
secured	valid <u>handle</u>	backscatter new RN16	secured
	invalid <u>handle</u>	—	secured
killed	all	—	killed

F.8 Command response: *Select*Table F.8 — *Select* command-response table

Starting State	Condition	Response	Next State
ready, arbitrate, reply, acknowledged, open, secured	all	assert or de-assert SL , or set inventoried to <i>A</i> or <i>B</i>	ready
killed	all	—	killed

F.9 Command response: *Read*

Table F.9 — *Read* command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate, reply, acknowledged	all	—	arbitrate
open	valid <u>handle</u> & invalid memory access	backscatter error code	open
	valid <u>handle</u> & valid memory access	backscatter data and <u>handle</u>	open
	invalid <u>handle</u>	—	open
secured	valid <u>handle</u> & invalid memory access	backscatter error code	secured
	valid <u>handle</u> & valid memory access	backscatter data and <u>handle</u>	secured
	invalid <u>handle</u>	—	secured
killed	all	—	killed

F.10 Command response: *Write*

Table F.10 — *Write* command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate, reply, acknowledged	all	—	arbitrate
open	valid <u>handle</u> & invalid memory access	backscatter error code	open
	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	open
	invalid <u>handle</u>	—	open
secured	valid <u>handle</u> & invalid memory access	backscatter error code	secured
	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	secured
	invalid <u>handle</u>	—	secured
killed	all	—	killed

F.11 Command response: *Kill*Table F.11 — *Kill*¹ command-response table

Starting State	Condition	Response	Next State
ready	All	—	ready
arbitrate, reply, acknowledged	All	—	arbitrate
open	valid <u>handle</u> & kill password=0	backscatter error code	open
	valid <u>handle</u> & invalid nonzero kill password	—	arbitrate
	valid <u>handle</u> & valid nonzero kill password	backscatter <u>handle</u> when done	killed
	invalid <u>handle</u>	—	open
secured	valid <u>handle</u> & kill password=0	backscatter error code	secured
	valid <u>handle</u> & invalid nonzero kill password	—	arbitrate
	valid <u>handle</u> & valid nonzero kill password	backscatter <u>handle</u> when done	killed
	invalid <u>handle</u>	—	secured
killed	All	—	killed

NOTE 1: See also Figure Amd.1-26

F.12 Command response: *Lock*Table F.12 — *Lock* command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate, reply, acknowledged	all	—	arbitrate
open	all	—	open
secured	valid <u>handle</u> & invalid lock payload	backscatter error code	secured
	valid <u>handle</u> & valid lock payload	backscatter <u>handle</u> when done	secured
	invalid <u>handle</u>	—	secured
killed	all	—	killed

F.13 Command response: Access

Table F.13 — Access¹ command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate, reply, acknowledged	all	—	arbitrate
open	valid <u>handle</u> & invalid access password	—	arbitrate
	valid <u>handle</u> & valid access password	backscatter <u>handle</u>	secured
	invalid <u>handle</u>	—	open
secured	valid <u>handle</u> & invalid access password	—	arbitrate
	valid <u>handle</u> & valid access password	backscatter <u>handle</u>	secured
	invalid <u>handle</u>	—	secured
killed	all	—	killed

NOTE 1: See also Figure Amd.1-28

F.14 Command response: *BlockWrite*

Table F.14 — *BlockWrite* command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate, reply, acknowledged	all	—	arbitrate
open	valid <u>handle</u> & invalid memory access	backscatter error code	open
	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	open
	invalid <u>handle</u>	—	open
secured	valid <u>handle</u> & invalid memory access	backscatter error code	secured
	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	secured
	invalid <u>handle</u>	—	secured
killed	all	—	killed

F.15 Command response: *BlockErase*Table F.15 — *BlockErase* command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate, reply, acknowledged	all	—	arbitrate
open	valid <u>handle</u> & invalid memory access	backscatter error code	open
	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	open
	invalid <u>handle</u>	—	open
secured	valid <u>handle</u> & invalid memory access	backscatter error code	secured
	valid <u>handle</u> & valid memory access	backscatter <u>handle</u> when done	secured
	invalid <u>handle</u>	—	secured
killed	all	—	killed

F.16 Command response: T_2 timeoutTable F.16 — T_2 timeout command-response table

Starting State	Condition	Response	Next State
ready	all	—	ready
arbitrate	all	—	arbitrate
reply, acknowledged	$T_2 > 20.0T_{pri}$ (see Figure Amd.1-19)	—	arbitrate
open	all	—	open
secured	all	—	secured
killed	all	—	killed

F.17 Command response: Invalid command

Table F.17 — Invalid command table

Starting State	Condition	Response	Next State
ready ^a	all	—	ready
arbitrate ^b	all	—	arbitrate
reply ^b	all	—	reply
acknowledged ^b	all	—	acknowledged
open ^b	all, excluding commands interspersed between successive <i>Kill</i> or <i>Access</i> commands in a kill or access sequence, respectively (see Figure Amd.1-26 and Figure Amd.1-28).	—	open
	valid commands, except <i>Req_RN</i> , interspersed between successive <i>Kill</i> or <i>Access</i> commands in a kill or access sequence, respectively (see Figure Amd.1-26 and Figure Amd.1-28).	—	arbitrate
secured ^b	all, excluding commands interspersed between successive <i>Kill</i> or <i>Access</i> commands in a kill or access sequence, respectively (see Figure Amd.1-26 and Figure Amd.1-28).	—	secured
	valid commands, except <i>Req_RN</i> , interspersed between successive <i>Kill</i> or <i>Access</i> commands in a kill or access sequence, respectively (Figure Amd.1-26 and Figure Amd.1-28).	—	arbitrate
killed ^a	all	—	killed
^a "Invalid" shall mean an erroneous command, an unsupported command, a command with invalid parameters, a command with a CRC error, or any other command either not recognized or not executable by the tag. ^b "Invalid" shall mean an erroneous command, an unsupported command, a command with invalid parameters, a command with a CRC error, a command (other than a <i>Query</i>) with a <u>session</u> parameter not matching that of the inventory round currently in progress, or any other command either not recognized or not executable by the tag.			

Annex G

(informative)

Example slot-count (Q) selection algorithm

G.1 Example algorithm an interrogator might use to choose Q

Figure G.1 shows an algorithm an interrogator might use for setting the slot-count parameter Q in a *Query* command. Q_{fp} is a floating-point representation of Q ; an interrogator rounds Q_{fp} to an integer value and substitutes this integer value for Q in the *Query*. Typical values for C are $0.1 < C < 0.5$. An interrogator typically uses small values of C when Q is large, and larger values of C when Q is small.

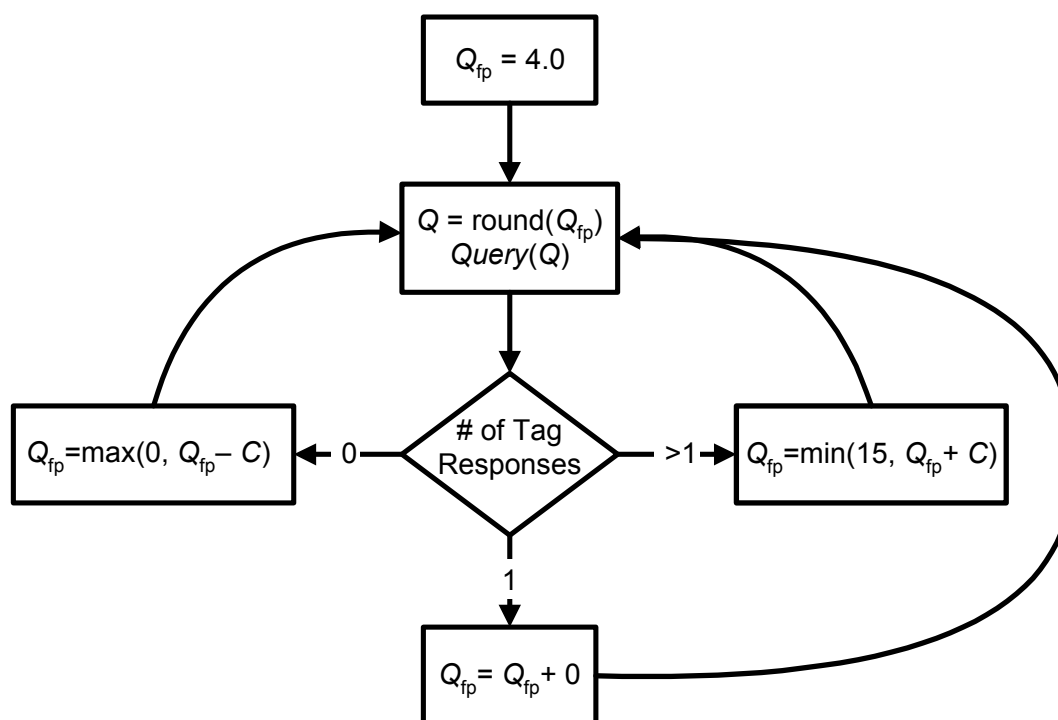


Figure G.1 — Example algorithm for choosing the slot-count parameter Q

Annex H (informative)

Example of tag inventory and access

H.1 Example inventory and access of a single tag

Figure H.1 shows the steps by which an interrogator inventories and accesses a single tag.

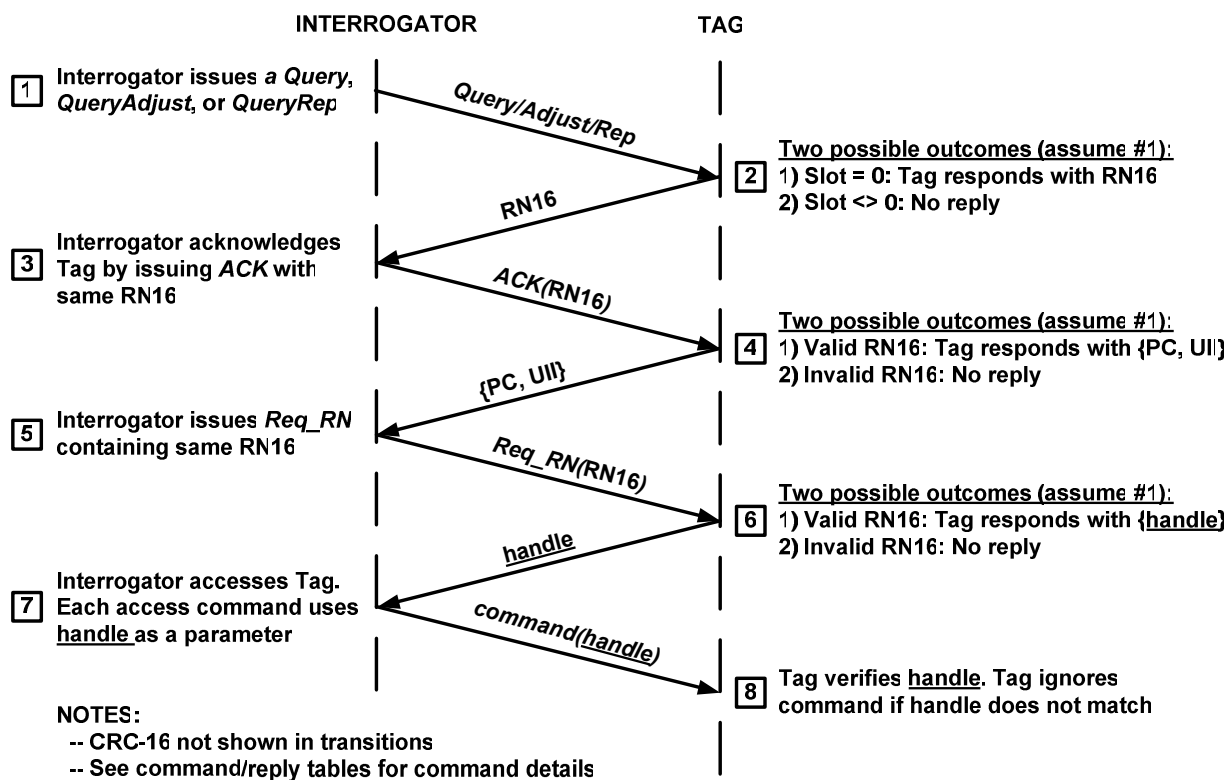


Figure H.1 — Example of tag inventory and access

Annex I (normative)

Dense- and multiple-interrogator channelised signalling

This Annex describes channelised signalling in the optional dense- and multiple-interrogator modes. It provides several alternative methods that interrogators may use, where permitted by local regulatory authorities, to manage frequency-band usage.

I.1 Dense-interrogator modes

In environments containing two and more interrogators, the range and rate at which interrogators singulate tags can be improved by preventing interrogator transmissions from colliding with tag responses, either temporally or spectrally. This Annex describes time-division multiplexing (TDM) and frequency-division multiplexing (FDM) methods that can minimize interrogator-on-tag collisions. If permitted by local regulations, interrogators that are claimed to operate in dense-interrogator environments shall support one of the TDM or FDM methods described below, determined using the algorithm in Figure I.1. Regardless of the choice, Interrogator signalling (both modulated and CW) shall be centred in a channel with the frequency accuracy specified in 9.3.1.2.1, and interrogator transmissions shall satisfy the dense-interrogator transmit mask in Figure Amd.1-10. If an interrogator uses SSB-ASK modulation, the transmit spectrum shall be centred in the channel during R=>T signalling, and the CW shall be centred in the channel during tag backscatter.

TDM: Interrogator transmissions and tag responses shall be separated temporally, with synchronized interrogators first commanding tags, then all interrogators transmitting CW and listening for tag responses.

FDM: Interrogator transmissions and tag responses shall be separated spectrally, using one of the three frequency plans described below.

Channel-boundary backscatter: Interrogator transmissions shall be centred in channels, and tag backscatter shall be situated at channel boundaries.

Adjacent-channel backscatter: Interrogator transmissions shall be centred in odd-numbered channels, and tag backscatter shall be situated in even-numbered channels.

In-channel backscatter: Interrogator transmissions shall be centred in channels, and tag backscatter shall be situated near but within the channel boundaries.

I.1.1 Examples of dense-interrogator mode operation

Figure I.2, shows examples of the single TDM and three FDM dense-Interrogator modes defined in I.1. For optimum performance, this specification recommends that interrogators choose BLF and M to allow a guardband between interrogator signalling and tag responses.

Example 1: TDM

ERC REC 70-03E Annex 1 allows the band from 869.4–869.65 MHz to be used as a single 250kHz channel. By the algorithm in Figure I.1, the dense-interrogator mode will be TDM. Example 1 of Figure I.2 shows one possible operating mode, in which interrogator transmissions use DSB-ASK modulation with $T_{\text{ari}}=25\mu\text{s}$, and tag backscatter is 20 kbit/s data on an 80 kHz subcarrier (BLF=80kHz, M=4).

Example 2: FDM Channel-boundary backscatter

FCC 15.247, dated October 2000, authorizes frequency-hopping operation in the ISM band from 902–928 MHz with 500kHz maximum channel width, and does not prohibit channel-boundary backscatter. By the algorithm in Figure I.1, interrogators will use 500kHz channels with channel-boundary backscatter. Table I.1 shows the channelisation and channel numbering; example 2 of Figure I.2 shows interrogator transmissions using PR-ASK modulation with $T_{\text{ari}}=25\mu\text{s}$, and 62.5 kbit/s tag data backscatter on a 250kHz subcarrier (BLF=250kHz; M=4). Interrogators centre their R=>T signalling in the channels shown in Table I.1, with transmissions unsynchronized in time, hopping among channels.

Example 3: FDM Adjacent-channel backscatter

ERC REC 70-03E Annex 11 specifies fifteen 200kHz channels in the 865 – 868 MHz frequency range, and does not prohibit adjacent-channel backscatter. By the algorithm in Figure I.1, interrogators will use 200kHz channels with adjacent-channel backscatter. Figure I.3 shows the channel numbering; example 3 of Figure I.2 shows interrogator transmissions using SSB-ASK modulation with $T_{\text{ari}}=25\mu\text{s}$, and 50 kbit/s tag data backscatter on a 200kHz subcarrier ($\text{BLF}=200\text{kHz}$, $M=4$).

Example 4: FDM In-channel backscatter

A hypothetical regulatory environment allocates four 500kHz channels and disallows adjacent-channel and channel-boundary backscatter. By the algorithm in Figure I.1, interrogators will use 500kHz channels with in-channel backscatter. Example 4 of Figure I.2 shows interrogator transmissions using PR-ASK modulation with $T_{\text{ari}}=25\mu\text{s}$, and 25 kbit/s tag data backscatter on a 200kHz subcarrier ($\text{BLF}=200\text{kHz}$, $M=8$).

I.2 Channelisation in multiple- and dense-interrogator environments

When Interrogators in multiple- and dense-Interrogator environments instruct tags to use subcarrier backscatter, the Interrogators shall adopt the channelisation determined by the algorithm in I.1. When interrogators in multiple- and dense-interrogator environments instruct tags to use FM0 backscatter, the interrogators shall adopt a channelisation that is in accordance with local regulations. Regardless of the backscatter data encoding, interrogator transmissions shall satisfy the multiple- or dense- interrogator transmit mask in clause 9.3.1.2.11.

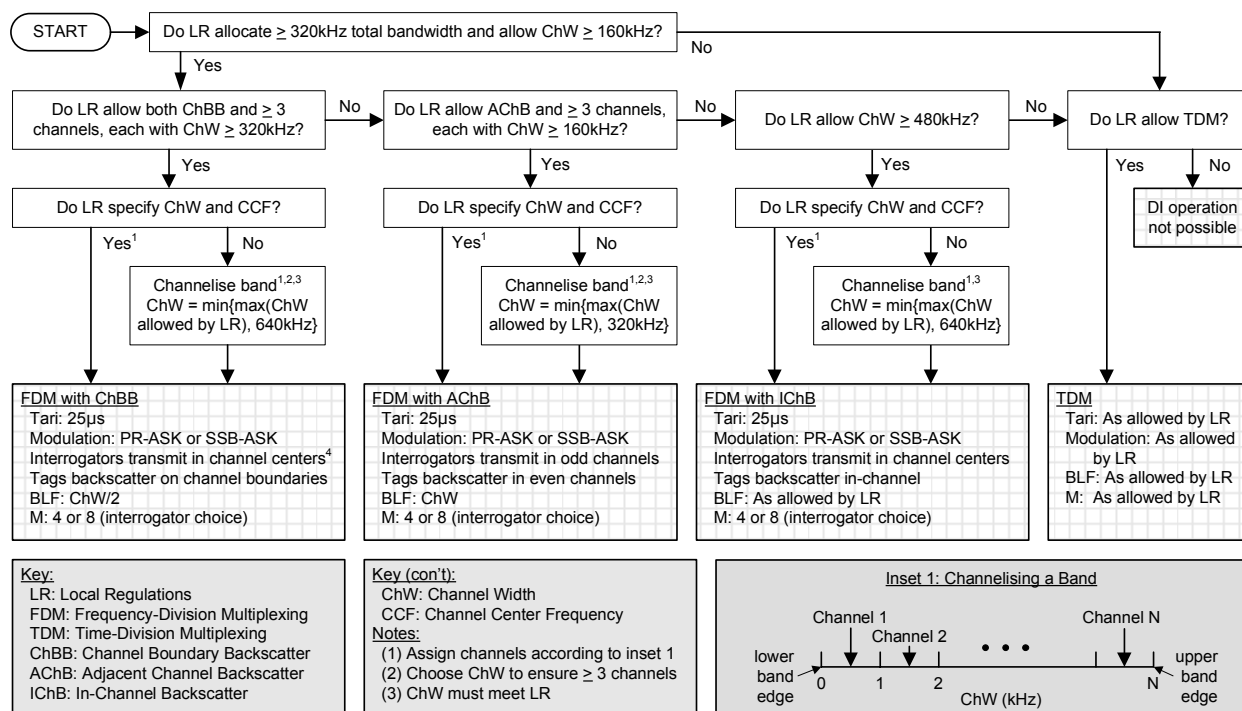
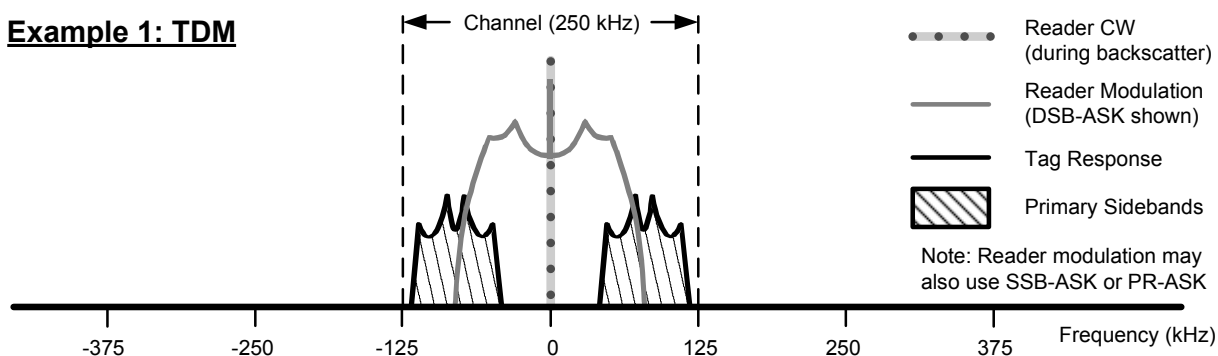
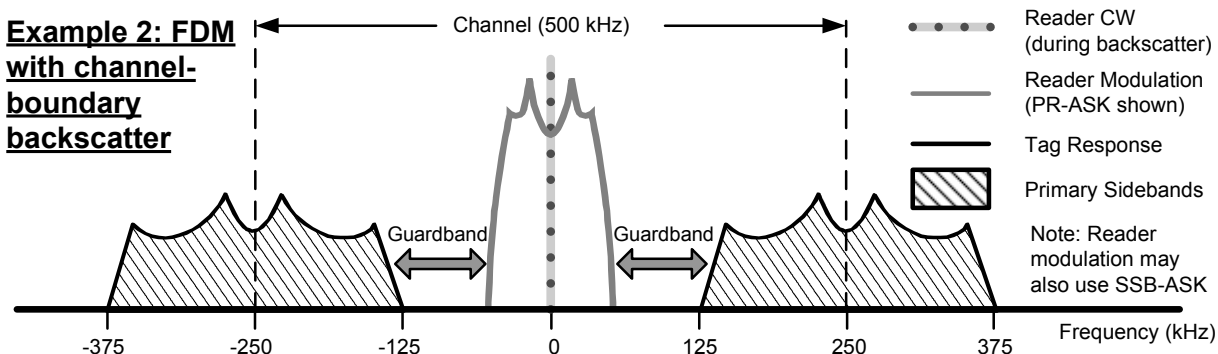


Figure I.1 — Algorithm for determining channelisation and dense-interrogator-mode parameters

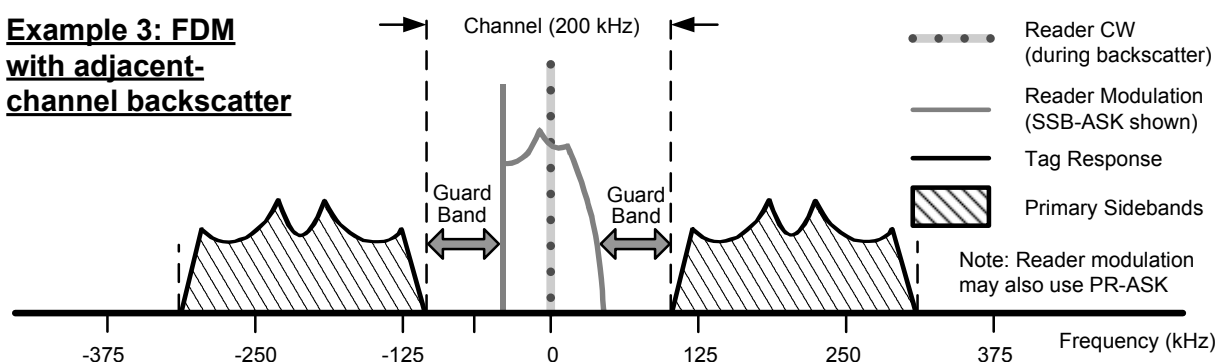
Example 1: TDM



Example 2: FDM with channel-boundary backscatter



Example 3: FDM with adjacent-channel backscatter



Example 4: FDM with in-channel backscatter

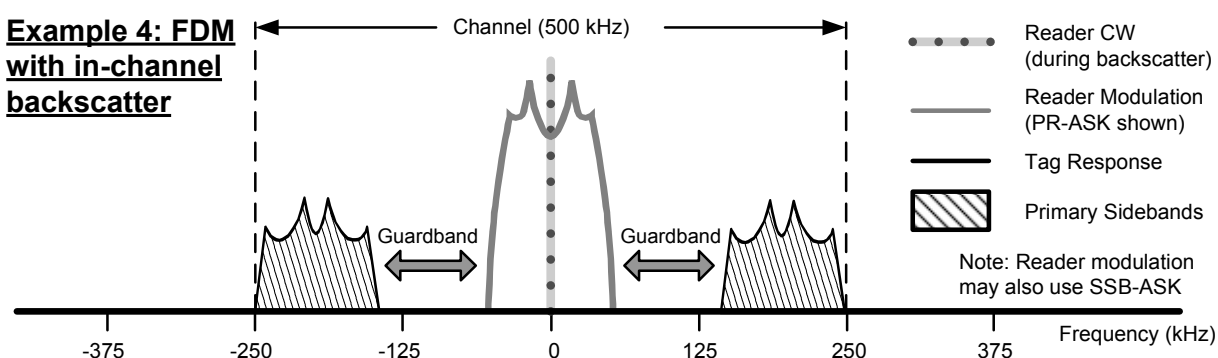


Figure I.2 — Examples of dense-interrogator mode operation

I.2.1 Example channelisation

In the FCC 15.247 environment from example 2 above, Interrogators will use the channelisation in Table I.1.

Table I.1 — Channelisation for Example 2

Commanded tag backscatter format	Channel width	Channel centre frequencies f_c	Guardbands
Subcarrier	500 kHz	Channel 1: 902.75 MHz Channel 2: 903.25 MHz • • Channel 50: 927.25 MHz	Lower bandedge: 902 MHz – 902.5 MHz Upper bandedge: 927.5 MHz – 928 MHz

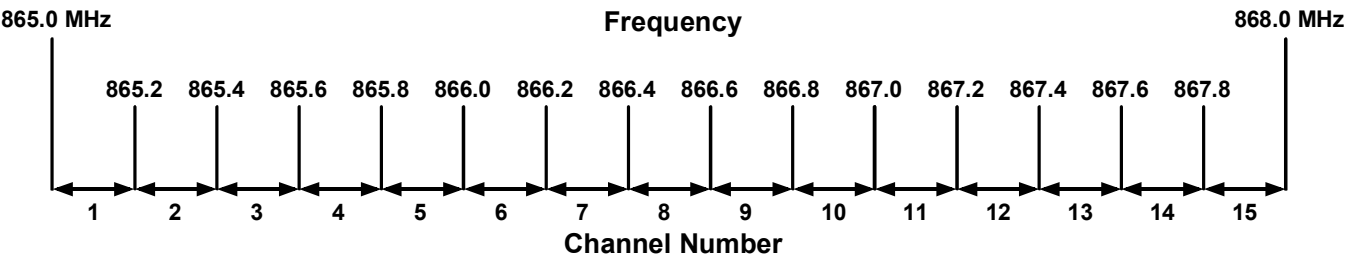


Figure I.3 — Channel numbering for Example 3

Annex J (informative)

Interrogator-to-tag link modulation

J.1 Baseband waveforms, modulated RF, and detected waveforms

Figure J.1 shows R=>T baseband and modulated waveforms as generated by an interrogator, and the corresponding waveforms envelope-detected by a tag, for DSB- or SSB-ASK modulation, and for PR-ASK modulation.

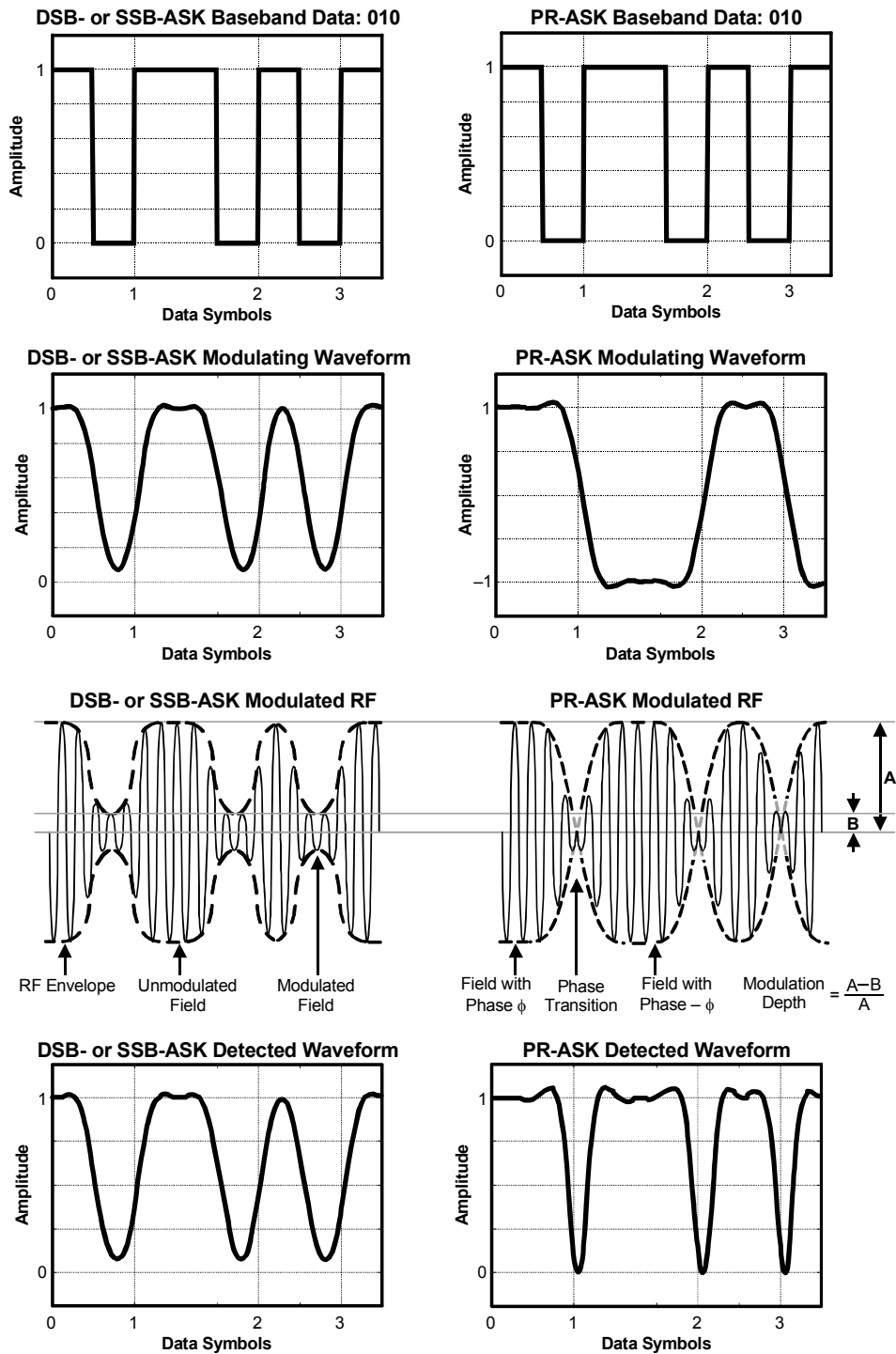


Figure J.1 — Interrogator-to-tag modulation

Annex K (normative)

Error codes

K.1 Tag error codes and their usage

If a tag encounters an error when executing an access command that reads from or writes to memory, and if the command is a handle-based command (i.e. *Read*, *Write*, *Kill*, *Lock*, *BlockWrite*, or *BlockErase*), then the tag shall backscatter an error code as shown in Table K.1 instead of its normal reply.

If the tag supports error-specific codes, it shall use the error-specific codes shown in Table K.2.

If the tag does not support error-specific codes, it shall backscatter error code 00001111₂ (indicating a non-specific error) as shown in Table K.2.

Tags shall backscatter error codes only from the **open** or **secured** states.

A tag shall not backscatter an error code if it receives an invalid access command; instead, it shall ignore the command.

If an error is described by more than one error code, the more specific error code shall take precedence and shall be the code that the tag backscatters.

The header for an error code is a 1-bit, unlike the header for a normal tag response, which is a 0-bit.

Table K.1 — Tag-error reply format

	Header	Error Code	RN	CRC-16
# of bits	1	8	16	16
description	1	Error code	handle	

Table K.2 — Tag error codes

Error-Code Support	Error Code	Error-Code Name	Error Description
Error-specific	00000000 ₂	Other error	Catch-all for errors not covered by other codes
	00000011 ₂	Memory overrun or unsupported PC value	The specified memory location does not exist or the PC value is not supported by the tag
	00000100 ₂	Memory locked	The specified memory location is locked and/or permalocked and is either not writeable or not readable
	00001011 ₂	Insufficient power	The tag has insufficient power to perform the memory-write operation
Non-specific	00001111 ₂	Non-specific error	The tag does not support error-specific codes

Annex L (normative)

Slot counter

L.1 Slot-counter operation

As described in 9.3.2.4.8, tags implement a 15-bit slot counter. As described in 9.3.2.8, interrogators use the slot counter to regulate the probability of a tag responding to a *Query*, *QueryAdjust*, or *QueryRep* command. Upon receiving a *Query* or *QueryAdjust* a tag preloads a Q-bit value, drawn from the tag's RNG (see 9.3.2.5), into its slot counter. Q is an integer in the range (0,15). A *Query* specifies Q; a *QueryAdjust* may modify Q from the prior *Query*.

A tag in the **arbitrate** state shall decrement its slot counter every time it receives a *QueryRep* command, transitioning to the **reply** state and backscattering an RN16 when its slot-counter value reaches 0000_h. A tag whose slot-counter value reached 0000_h, who replied, and who was not acknowledged (including a tag that responded to the original *Query* and was not acknowledged) returns to **arbitrate** with a slot-counter value of 0000_h.

A tag that returns to **arbitrate** with a slot-counter value of 0000_h shall decrement its slot-counter from 0000_h to 7FFF_h (i.e. the slot counter rolls over) at the next *QueryRep* with matching session. Because the slot-counter value is now nonzero, the tag remains in **arbitrate**. Slot counters implements continuous counting, meaning that, after a slot counter rolls over it begins counting down again from 7FFF_h, effectively preventing subsequent tag replies until the tag receives either a *Query* or a *QueryAdjust* and loads a new random value into its slot counter.

Annex E and Annex F contain tables describing a tag's response to interrogator commands; "slot" is a parameter in these tables.

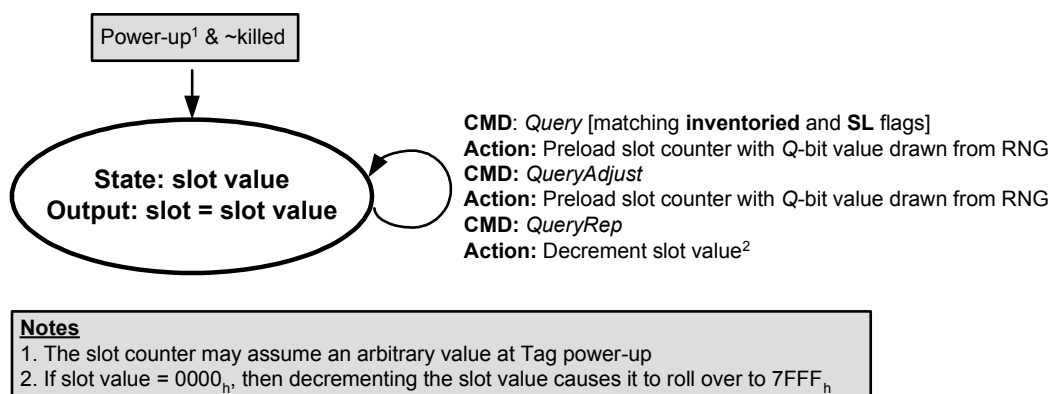


Figure L.1 — Slot-counter state diagram

Annex M (informative)

Example data-flow exchange

M.1 Overview of the data-flow exchange

The following example describes a data exchange, between an interrogator and a single tag, during which the interrogator reads the kill password stored in the tag's Reserved memory. This example assumes that:

The tag has been singulated and is in the **acknowledged** state.

The tag's Reserved memory is locked but not permalocked, meaning that the interrogator must issue the access password and transition the tag to the **secured** state before performing the read operation.

The random numbers the tag generates (listed in sequence, and not random for reasons of clarity) are:

RN16_0	1600 _h	(the RN16 the tag backscattered prior to entering acknowledged)
RN16_1	1601 _h	(will become the <u>handle</u> for the entire access sequence)
RN16_2	1602 _h	
RN16_3	1603 _h	

The tag's UII is 64 bits in length.

The tag's access password is ACCEC0DE_h.

The tag's kill password is DEADC0DE_h.

The 1st half of the access password EXORed with RN16_2 = ACCE_h ⊗ 1602_h = BACC_h.

The 2nd half of the access password EXORed with RN16_3 = C0DE_h ⊗ 1603_h = D6DD_h.

M.2 Tag memory contents and lock-field values

Table M.1 and Table M.2 show the example tag memory contents and lock-field values, respectively.

Table M.1 — Tag memory contents

Memory Bank	Memory Contents	Memory Addresses	Memory Values
TID	TID[15:0]	10 _h –1F _h	54E2 _h
	TID[31:16]	00 _h –0F _h	A986 _h
UII	UII[15:0]	50 _h –5F _h	3210 _h
	UII[31:16]	40 _h –4F _h	7654 _h
	UII[47:32]	30 _h –3F _h	BA98 _h
	UII[63:48]	20 _h –2F _h	FEDC _h
	PC[15:0]	10 _h –1F _h	2000 _h
	CRC-16[15:0]	00 _h –0F _h	as calculated (see Annex A)
Reserved	access password[15:0]	30 _h –3F _h	C0DE _h
	access password[31:16]	20 _h –2F _h	ACCE _h
	kill password[15:0]	10 _h –1F _h	C0DE _h
	kill password[31:16]	00 _h –0F _h	DEAD _h

Table M.2 — Lock-field values

Kill Password		Access Password		UII Memory		TID Memory		User Memory	
1	0	1	0	0	0	0	0	N/A	N/A

M.3 Data-flow exchange and command sequence

The data-flow exchange follows the Access procedure outlined in Figure Amd.1-28 with a *Read* command added at the end. The sequence of interrogator commands and tag replies is:

- Step 1: *Req_RN*[RN16_0, CRC-16]
Tag backscatters RN16_1, which becomes the handle for the entire access sequence
- Step 2: *Req_RN*[handle, CRC-16]
Tag backscatters RN16_2
- Step 3: *Access*[access password[31:16] EXORed with RN16_2, handle, CRC-16]
Tag backscatters handle
- Step 4: *Req_RN*[handle, CRC-16]
Tag backscatters RN16_3
- Step 5: *Access*[access password[15:0] EXORed with RN16_3, handle, CRC-16]
Tag backscatters handle
- Step 6: *Read*[MemBank=Reserved, WordPtr=00_h, WordCount=2, handle, CRC-16]
Tag backscatters kill password

Table M.3 shows the detailed interrogator commands and tag replies. For reasons of clarity, the CRC-16 has been omitted from all commands and replies.

Table M.3 — Interrogator commands and tag replies

Step	Data Flow	Command	Parameter and/or Data	Tag State
1a: <i>Req_RN</i> command	R => T	11000001	0001 0110 0000 0000 (RN16_0=1600 _h)	acknowledged → open
1b: Tag response	T => R		0001 0110 0000 0001 (<u>handle</u> =1601 _h)	
2a: <i>Req_RN</i> command	R => T	11000001	0001 0110 0000 0001 (<u>handle</u> =1601 _h)	open → open
2b: Tag response	T => R		0001 0110 0000 0010 (RN16_2=1602 _h)	
3a: Access command	R => T	11000110	1011 1010 1100 1100 (BACC _h) 0001 0110 0000 0001 (<u>handle</u> =1601 _h)	open → open
3b: Tag response	T => R		0001 0110 0000 0001 (<u>handle</u> =1601 _h)	
4a: <i>Req_RN</i> command	R => T	11000001	0001 0110 0000 0001 (<u>handle</u> =1601 _h)	open → open
4b: Tag response	T => R		0001 0110 0000 0011 (RN16_2=1603 _h)	
5a: Access command	R => T	11000110	1101 0110 1101 1101 (D6DD _h) 0001 0110 0000 0001 (<u>handle</u> =1601 _h)	open → secured
5b: Tag response	T => R		0001 0110 0000 0001 (<u>handle</u> =1601 _h)	
6a: <i>Read</i> command	R => T	11000010	00 (MemBank=Reserved) 00000000 (WordPtr=kill password) 00000010 (WordCount=2) 0001 0110 0000 0001 (<u>handle</u> =1601 _h)	secured → secured
6b: Tag response	T => R		0 (header) 1101 1110 1010 1101 (DEAD _h) 1100 0000 1101 1110 (CODE _h)	

Annex N (informative)

Optional tag features

N.1 General

The following options are available to tags according this International Standard.

N.2 Optional tag passwords

N.2.1 Kill password

A tag may optionally implement a kill password. A tag that does not implement a kill password operates as if it has a zero-valued kill password that is permanently read/write locked. See 9.3.2.1.1.

N.2.2 Access password

A tag may optionally implement an access password. A tag that does not implement an access password operates as if it has a zero-valued access password that is permanently read/write locked. See 9.3.2.1.2

N.3 Optional Tag memory banks and memory-bank sizes

N.3.1 Reserved memory

Reserved memory is optional. If a tag does not implement either a kill password or an access password then the tag need not physically implement Reserved memory. Because a tag with non-implemented passwords operates as if it has zero-valued password(s) that are permanently read/write locked, these passwords must still be logically addressable in Reserved memory at the memory locations specified in 9.3.2.1.1.1 and 9.3.2.1.1.2.

N.3.2 Ull memory

Ull memory is required, but its size is vendor-defined. The minimum size is 32 bits, to contain a 16-bit CRC and 16-bit PC. Ull memory may be larger than 32 bits, to contain an Ull whose vendor-specified length may be 16 bits to 496 bits in 16-bit increments. See 9.3.2.1.2.

N.3.3 TID memory

TID memory is required, but its size is vendor-defined. The minimum-size TID memory contains an 8-bit ISO/IEC 15963 allocation class identifier, as well as sufficient identifying information for an interrogator to uniquely identify the custom commands and/or optional features that a tag supports. TID memory may optionally contain vendor-specific data. See 9.3.2.1.3.

N.3.4 User memory:

User memory is optional. See 9.3.2.1.4, 9.3.2.1.4.1 and 9.3.2.1.4.2.

N.4 Optional tag commands

Proprietary: A tag may support proprietary commands. See 9.2.5.

Custom: A tag may support custom commands. See 9.2.4.

Access: A tag may support the *Access* command. See 9.3.2.10.3.6.

BlockWrite: A tag may support the *BlockWrite* command. See 9.3.2.10.3.7.

BlockErase: A tag may support the *BlockErase* command. See 9.3.2.10.3.8.

N.5 Optional tag error-code reporting format

A tag may support error-specific or non-error-specific error-code reporting. See Annex K.

N.6 Optional tag backscatter modulation format

A tag may support ASK and/or PSK backscatter modulation. See 9.3.1.3.1.

Page 134, Bibliography

Delete the following references, which have been moved to Clause 3:

- [1] ISO/IEC 18000-1, *Information technology — Radio-frequency identification for item management — Part 1: Reference architecture and definition of parameters to be standardized*
- [3] ISO/IEC 15961, *Information technology — Automatic identification and data capture — Radio frequency identification (RFID) for item management — Data protocol: application interface*
- [4] ISO/IEC 15962, *Information technology — Automatic identification and data capture techniques — Radio-frequency identification (RFID) for item management — Data protocol: data encoding rules and logical memory functions*
- [5] ISO/IEC 15963, *Information technology — Automatic identification — Radio-frequency identification for item management — Unique identification for RF tags*

Page 134, Bibliography

Insert the following references and renumber accordingly:

- [1] ARIB STD-T81, *RFID Equipment Using Frequency Hopping System For Specified Low Power Radio Station*
- [2] CEPT/ERC Recommendation 70-03, *Relating to the use of Short Range Devices (SRD), Annex 11*
- [3] ETSI EN 300 220 (all parts), *Electromagnetic compatibility and Radio spectrum Matters (ERM); Short Range Devices (SRD); Radio equipment to be used in the 25 MHz to 1 000 MHz frequency range with power levels ranging up to 500 mW*

- [4] ETSI EN 302 208-1, *Electromagnetic compatibility and radio spectrum matters (ERM) – Radio-frequency identification equipment operating in the band 865 MHz to 868 MHz with power levels up to 2 W, Part 1 – Technical characteristics and test methods*
- [5] ETSI EN 302 208-2, *Electromagnetic compatibility and radio spectrum matters (ERM) – Radio-frequency identification equipment operating in the band 865 MHz to 868 MHz with power levels up to 2 W, Part 2 – Harmonized EN under article 3.2 of the R&TTE directive*
- [6] ISO/IEC 3309, *Information technology — Telecommunications and information exchange between systems — High-level data link control (HDLC) procedures – Frame structure*
- [7] ISO/IEC 18000-2, *Information technology — Radio frequency identification for item management — Part 2: Parameters for air interface communications below 135 kHz*
- [8] ISO/IEC 18000-3, *Information technology — Radio frequency identification for item management — Part 3: Parameters for air interface communications at 13,56 MHz*
- [9] ISO/IEC 18000-4, *Information technology — Radio frequency identification for item management — Part 4: Parameters for air interface communications at 2,45 GHz*
- [10] ISO/IEC 18000-7, *Information technology — Radio frequency identification for item management — Part 7: Parameters for active air interface communications at 433 MHz*
- [11] RCR STD-1, *RFID Equipment For Premises Radio Station*
- [12] RCR STD-29, *RFID Equipment For Specified Low Power Radio Station*
- [13] US Code of Federal Regulations (CFR) Title 47, Chapter I, Part 15. “Radio Frequency Devices”; U.S. Federal Communications Commission
- [14] EPCglobal™: EPC™ Radio-Frequency Identity Protocols, Class-1 Generation-2 UHF RFID, Protocol for Communications at 860 MHz – 960 MHz, Version 1.0.9

Copyright International Organization for Standardization
Provided by IHS under license with ISO
No reproduction or networking permitted without license from IHS

Licensee=etri/9943121100
Not for Resale, 08/01/2006 02:01:00 MDT

**Information technology —
Radio frequency identification for item
management —
Part 6:
Parameters for air interface
communications at 860 MHz to 960 MHz**

*Technologies de l'information — Identification par radiofréquence
(RFID) pour la gestion d'objets —*

*Partie 6: Paramètres pour les communications d'une interface d'air
entre 860 MHz et 960 MHz*

Reference number
ISO/IEC 18000-6:2004(E)



PDF disclaimer

This PDF file may contain embedded typefaces. In accordance with Adobe's licensing policy, this file may be printed or viewed but shall not be edited unless the typefaces which are embedded are licensed to and installed on the computer performing the editing. In downloading this file, parties accept therein the responsibility of not infringing Adobe's licensing policy. The ISO Central Secretariat accepts no liability in this area.

Adobe is a trademark of Adobe Systems Incorporated.

Details of the software products used to create this PDF file can be found in the General Info relative to the file; the PDF-creation parameters were optimized for printing. Every care has been taken to ensure that the file is suitable for use by ISO member bodies. In the unlikely event that a problem relating to it is found, please inform the Central Secretariat at the address given below.

© ISO/IEC 2004

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from either ISO at the address below or ISO's member body in the country of the requester.

ISO copyright office
Case postale 56 • CH-1211 Geneva 20
Tel. + 41 22 749 01 11
Fax + 41 22 749 09 47
E-mail copyright@iso.org
Web www.iso.org

Published in Switzerland

Contents

Page

Foreword	vi
Introduction	vii
1 Scope	1
2 Conformance	1
2.1 Interrogator conformance and obligations	1
2.2 Tag conformance and obligations	1
2.3 Claiming conformance	2
3 Normative references	2
4 Terms, definitions, symbols and abbreviated terms	2
4.1 Terms and definitions	2
4.2 Symbols	2
4.3 Abbreviated terms	3
5 Overview	4
5.1 General	4
5.2 Parameter tables	5
6 Common elements of the physical layer for types A and B	11
6.1 General	11
6.2 Interrogator power-up waveform	11
6.3 Interrogator power-down	11
6.4 Frequency hopping carrier rise and fall times	12
6.5 FM0 return link	13
6.5.1 FM0 return link general	13
6.5.2 Modulation	13
6.5.3 Data rate	13
6.5.4 Data coding	13
6.5.5 Message format	14
6.5.6 Return preamble	14
6.5.7 Cyclic redundancy check (CRC)	15
7 Type A	17
7.1 Physical layer and data coding	17
7.1.1 PIE (Pulse interval encoding) forward link	17
7.2 Data elements	21
7.2.1 Unique identifier (UID)	21
7.2.2 Sub-UID	22
7.2.3 Application family identifier	22
7.2.4 Data storage format identifier (DSFID)	23
7.3 Protocol elements	23
7.3.1 Tag memory organisation	23
7.3.2 Support of battery-assisted tags	23
7.3.3 Block lock status	24
7.3.4 Tag signature	24
7.4 Protocol description	25
7.4.1 Protocol concept	25
7.4.2 Command format	26
7.4.3 Command flags	26
7.4.4 Round size	27
7.4.5 Command code definition and structure	28
7.4.6 Command classes	28

7.4.7	Command codes and CRC	29
7.4.8	Response format	32
7.4.9	Tag states	34
7.4.10	Collision arbitration	36
7.4.11	General explanation of the collision arbitration mechanism	36
7.5	Timing specifications	37
7.5.1	Timing specifications general	37
7.5.2	Tag state storage	37
7.5.3	Forward link to return link handover	37
7.5.4	Return link to forward link handover	38
7.5.5	Acknowledgement time window	38
7.6	Command format examples	40
7.7	Mandatory commands	40
7.7.1	Mandatory commands general	40
7.7.2	Next_slot	40
7.7.3	Standby_round	41
7.7.4	Reset_to_ready	42
7.7.5	Init_round_all	43
7.8	Optional commands	45
7.8.1	Optional commands general	45
7.8.2	Init_round	46
7.8.3	Close_slot	47
7.8.4	New_round	48
7.8.5	Select (by SUID)	49
7.8.6	Read_blocks	51
7.8.7	Get_system_information	55
7.8.8	Begin_round	58
7.8.9	Write_single_block	60
7.8.10	Write_multiple_blocks	62
7.8.11	Lock_blocks	64
7.8.12	Write_AFI	66
7.8.13	Lock_AFI	68
7.8.14	Write_DSFD command	70
7.8.15	Lock_DSFD	72
7.8.16	Get_blocks_lock_status	74
7.9	Custom commands	77
7.10	Proprietary commands	78
8	Type B	78
8.1	Physical layer and data coding	78
8.1.1	Forward link	78
8.1.2	Return link	80
8.1.3	Protocol concept	80
8.1.4	Command format	81
8.1.5	Response format	83
8.1.6	WAIT	83
8.1.7	Examples of a command packet	83
8.1.8	Communication sequences at packet level	84
8.2	Btree protocol and collision arbitration	85
8.2.1	Definition of data elements, bit and byte ordering	85
8.2.2	Tag memory organisation	86
8.2.3	Block security status	87
8.2.4	Overall protocol description, Btree protocol	87
8.2.5	Collision arbitration	92
8.2.6	Commands	94
8.2.7	Command types	94
8.2.8	Transmission errors	121
Annex A	(informative) Cyclic redundancy check (CRC)	122
A.1	Interrogator to tag CRC-5	122
A.2	Interrogator to tag and tag to interrogator CRC-16	123

A.2.1 CRC-16 general 123

A.2.2 CRC calculation examples 125

Annex B (normative) Memory mapping for Type B..... 128

B.1 Unique identifier (normative) 128

B.1.1 Unique identifier general 128

B.1.2 Unique identifier format..... 128

B.1.3 Unique identifier according to ANSI 256 128

B.1.4 Remaining system memory 129

Annex C (informative) Tag Memory Map for Type B 133

C.1 Tag memory map 133

Bibliography 134

Foreword

ISO (the International Organization for Standardization) and IEC (the International Electrotechnical Commission) form the specialized system for worldwide standardization. National bodies that are members of ISO or IEC participate in the development of International Standards through technical committees established by the respective organization to deal with particular fields of technical activity. ISO and IEC technical committees collaborate in fields of mutual interest. Other international organizations, governmental and non-governmental, in liaison with ISO and IEC, also take part in the work. In the field of information technology, ISO and IEC have established a joint technical committee, ISO/IEC JTC 1.

International Standards are drafted in accordance with the rules given in the ISO/IEC Directives, Part 2.

The main task of the joint technical committee is to prepare International Standards. Draft International Standards adopted by the joint technical committee are circulated to national bodies for voting. Publication as an International Standard requires approval by at least 75 % of the national bodies casting a vote.

ISO/IEC 18000-6 was prepared by Technical Committee ISO/IEC JTC 1, *Information technology*, Subcommittee SC 31, *Automatic identification and data capture techniques*.

ISO/IEC 18000 consists of the following parts, under the general title *Information technology — Radio frequency identification for item management*:

- *Part 1: Reference architecture and definition of parameters to be standardized*
- *Part 2: Parameters for air interface communications below 135 kHz*
- *Part 3: Parameters for air interface communications at 13,56 MHz*
- *Part 4: Parameters for air interface communications at 2,45 GHz*
- *Part 6: Parameters for air interface communications at 860 MHz to 960 MHz*
- *Part 7: Parameters for active air interface communications at 433 MHz*

Introduction

This part of ISO/IEC 18000 describes a passive backscatter RFID system that supports the following system capabilities:

- Identification and communication with multiple tags in the field
- Selection of a subgroup of tags for identification or with which to communicate
- Reading from and writing to or rewriting data many times to individual tags
- User-controlled permanently lockable memory
- Data integrity protection
- Interrogator-to-tag communications link with error detection
- Tag-to-interrogator communications link with error detection.
- Support for both passive back-scatter tags with or without batteries.

In this RFID system, the interrogator powers and communicates with the tags that are within range. Tags receive data as amplitude modulation of the power/data signal from the interrogator. During the time that the tag responds to the interrogator, the interrogator transmits at a constant radio frequency power level, while the tag modulates the impedance of its radio frequency load attached to the tag antenna terminals. The interrogator then receives the data back from the tag as a variation in a reflection of its transmitted power.

The International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC) draw attention to the fact that it is claimed that compliance with this document may involve the use of patents concerning radio-frequency identification technology given in the clauses identified below.

ISO and IEC take no position concerning the evidence, validity and scope of these patent rights.

The holders of these patent rights have assured the ISO and IEC that they are willing to negotiate licences under reasonable and non-discriminatory terms and conditions with applicants throughout the world. In this respect, the statements of the holders of these patent rights are registered with ISO and IEC.

Information on the declared patents may be obtained from:

Contact details	Patent number	Affected clause(s) in this part of ISO/IEC 18000
BTG plc ATTN: Mr David Armstrong 10 Fleet Place Limeburner Lane London EC4M 7SB UK Tel: +44 20 7575 0000 Fax: +44 20 7575 0010 Website: www.btgplc.com , Email: david.armstrong@btgplc.com ,	US 5,537,105, US 5,966,083, US 5,995,017, US 5,557,280, US 5,699,066, US 5,519,381, US 5,726,630, EP 0494114B1, EP 0585132B1, EP 0598624B1, WO 98/52142 and WO 99/26081	6, 7

ISO/IEC 18000-6:2004(E)

Contact details	Patent number	Affected clause(s) in this part of ISO/IEC 18000
Intercode 12, Rue des Petits Ruisseaux Z.I. des Godets 91370 Verrières le Buisson France Tel : + 33-1-6975 2170 Fax : + 33.1.60.11.00.31 Email: intercode.sarl@wanadoo.fr	US 5426423, EP 90909459.1, CA 2058 947, US 6177858B1, EP 96402556.3, CA 2191787, US 5923251, EP 96402554.8, CA 21911788, US 5808550, EP 96402555.5 and CA 2191794	7, 8
Intermec Technologies Corporation ATTN: Ronald D. Payne, Vice President, Contracts, 6001 36th Ave, West, Everett, WA 98203 USA	US 5942987, US 5521601, US 5995019, US 5030807, US 5828693, US 5850181, US 4786907, US 5550547, US 5673037, US 5777561 and US 5828318	8
Koninklijke Philips Electronics N.V ATTN: Mr. Harald Röggl Intellectual Property & Standards, Triester Strasse 64 A-1101 Vienna Austria	EP 1034503B, JP 00-560535, US 09/352317, WO 00/04485, JP 03-502778, US 2002/0186789A1 and WO 02/099741 A1	7, 8
Matrics Technology ATTN: Mr Kevin J Powell Senior Director, Product Development 8850 Stanford Blvd, Suite 3000 Columbia, MD 21045 USA +1-410-872-0300 (Voice) +1-443-782-0230 (eFax) kpowell@matrics.com	US 6002344	7, 8
SAMSys Technologies, Inc. ATTN: James Wiley, President, 2525 Meridian Parkway, Suite 60, Durham, NC 27713, USA Tel: +1-919- 281-1541, E-mail: tres.wiley@samsys.com	US 6617962	2
SUPERSENSOR (Pty) Ltd ATTN: Mr. Adelhart Kruger, D.M. Kisch Inc., P O Box 3668, Pretoria 0001, South Africa. Tel: +27-12-460-3203. E-mail: adelhartk@dmkisch.com	ZA 9810199, US 6480143 B1, EP 1001366, JP 200230978 and CN 1255689	6, 7
TAGSYS Australia Pty Ltd, ATTN: Alfio R. Grasso, TECHNICAL MANAGER 212 Pirie Street, ADELAIDE SA 5000 Australia, Tel: +61-8-8100 8324, E-mail: alfio.grasso@tagsys.net	EP 0578701B1, AU 664544, AU PCT AU/00/01493, WO 01/41043, AU PCT AU98/00017, WO 98/32092, US 5523749, AU PCT AU01/01676, WO02/054365, FR FR00/01704, and WO 01/01326	7

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights other than those identified above. ISO and IEC shall not be held responsible for identifying any or all such patent rights.

Information technology — Radio frequency identification for item management —

Part 6: Parameters for air interface communications at 860 MHz to 960 MHz

1 Scope

This part of ISO/IEC 18000 defines the air interface for radio frequency identification (RFID) devices operating in the 860 MHz to 960 MHz Industrial, Scientific, and Medical (ISM) band used in item management applications. Its purpose is to provide a common technical specification for RFID devices that may be used by ISO committees developing RFID application standards. This part of ISO/IEC 18000 is intended to allow for compatibility and to encourage inter-operability of products for the growing RFID market in the international marketplace. This part of ISO/IEC 18000 defines the forward and return link parameters for technical attributes including, but not limited to, operating frequency, operating channel accuracy, occupied channel bandwidth, maximum EIRP, spurious emissions, modulation, duty cycle, data coding, bit rate, bit rate accuracy, bit transmission order, and where appropriate operating channels, frequency hop rate, hop sequence, spreading sequence, and chip rate. It further defines the communications protocol used in the air interface.

This part of ISO/IEC 18000 contains one mode with two types. Both types use a common return link and are reader talks first. Type A uses Pulse Interval Encoding (PIE) in the forward link, and an adaptive ALOHA collision arbitration algorithm. Type B uses Manchester in the forward link and an adaptive binary tree collision arbitration algorithm. The detailed technical differences between the two types are shown in the parameter tables.

2 Conformance

2.1 Interrogator conformance and obligations

To conform to this part of ISO/IEC 18000, the interrogator shall support both communication types. It shall be able to switch from one type to the other.

The interrogator shall be locally programmable by the user to switch from one type to the other and to control the sequence and allocation of the ratio of time between the two types.

The proportion of the total time spent by the interrogator in addressing each type of tag shall be field-programmable from 0% to 100%.

Interrogators shall be set up to operate within local regulations.

2.2 Tag conformance and obligations

To conform to this part of ISO/IEC 18000, the tag shall support at least one type. It may optionally support both.

The tag shall operate over the frequency range of 860 MHz to 960 MHz.

NOTE Depending on the tag antenna characteristics, the operating performance (i.e. operating range) may vary depending on the actual frequency used in the 860-960 MHz band.

When the tag receives a modulated signal from the interrogator that it does not support or recognise, it shall remain silent.

2.3 Claiming conformance

In order to claim conformance with this part of ISO/IEC 18000 it is necessary to comply with all of the relevant clauses of this part of ISO/IEC 18000 except those marked 'optional' and it is also necessary to operate within the local national radio regulations (which may require further restrictions).

Relevant conformance test methods will be given in a future Technical Report (ISO/IEC TR 18047-6).

3 Normative references

ISO/IEC 19762-3, *Information technology — Automatic identification and data capture (AIDC) techniques — Harmonized vocabulary — Part 3: Radio frequency identification (RFID)*¹⁾

ISO/IEC 7816-6, *Identification cards — Integrated circuit cards — Interindustry data elements for interchange*

4 Terms, definitions, symbols and abbreviated terms

4.1 Terms and definitions

For the purposes of this document, the terms, definitions, symbols and abbreviated terms given in ISO/IEC 19762-3 and the following apply.

4.1.1

collision arbitration loop

algorithm used to prepare for and handle a dialogue between an interrogator and a tag. This is also known as collision arbitration.

4.1.2

byte

8 bits of data designated b1 to b8, from the most significant bit (MSB, b8) to the least significant bit (LSB, b1).

4.2 Symbols

Cht Carrier high-level tolerance

Clf Carrier low-level tolerance

D Modulation depth of data coding pulse

f_c Frequency of operating field (carrier frequency)

M Modulation Index

Ma Modulation upper tolerance type B

Mb Modulation lower tolerance type B

1) To be published.

Taq	Quiet time - type A
Tapf	Pulse fall time - type A
Tapr	Pulse rise time - type A
Tapw	Pulse width - type A
Tari	Reference interval time - type A
Tbmf	Manchester fall time – type B
Tbmr	Manchester rise time – type B
Tcf	Carrier fall time
Tcr	Carrier rise time
Tcs	Carrier steady state time
Tf	Fall time
Tfhf	Carrier FHSS fall time
Tfhr	Carrier FHSS rise time
Tfhs	Carrier FHSS steady time
Tr	Rise Time
Trlb	Return link bit time

4.3 Abbreviated terms

AFI	Application family identifier
CRC	Cyclic redundancy check
CRC-5	Five bit CRC used in type a Interrogator to tag commands, used in Type A
CRC-16	Sixteen bit CRC used in both Type A and Type B commands and responses
DSFID	Data storage format identifier
DSSS	Direct Sequence Spread Spectrum
EOF	End of frame
FHSS	Frequency Hopping Spread Spectrum
LSB	Least significant bit
MSB	Most significant bit
NRZ	Non Return to Zero
PIE	Pulse interval encoding, used in Type A
RFU	Reserved for future use

SOF	Start of frame
SUID	Sub unique identifier
TEL	Tag excitation level
UID	Unique identifier

5 Overview

5.1 General

This part of ISO/IEC 18000 specifies two communication types: Type A and Type B.

Figure 1, Figure 2, and Figure 3 show their architecture; Table 1 provides a comparison.

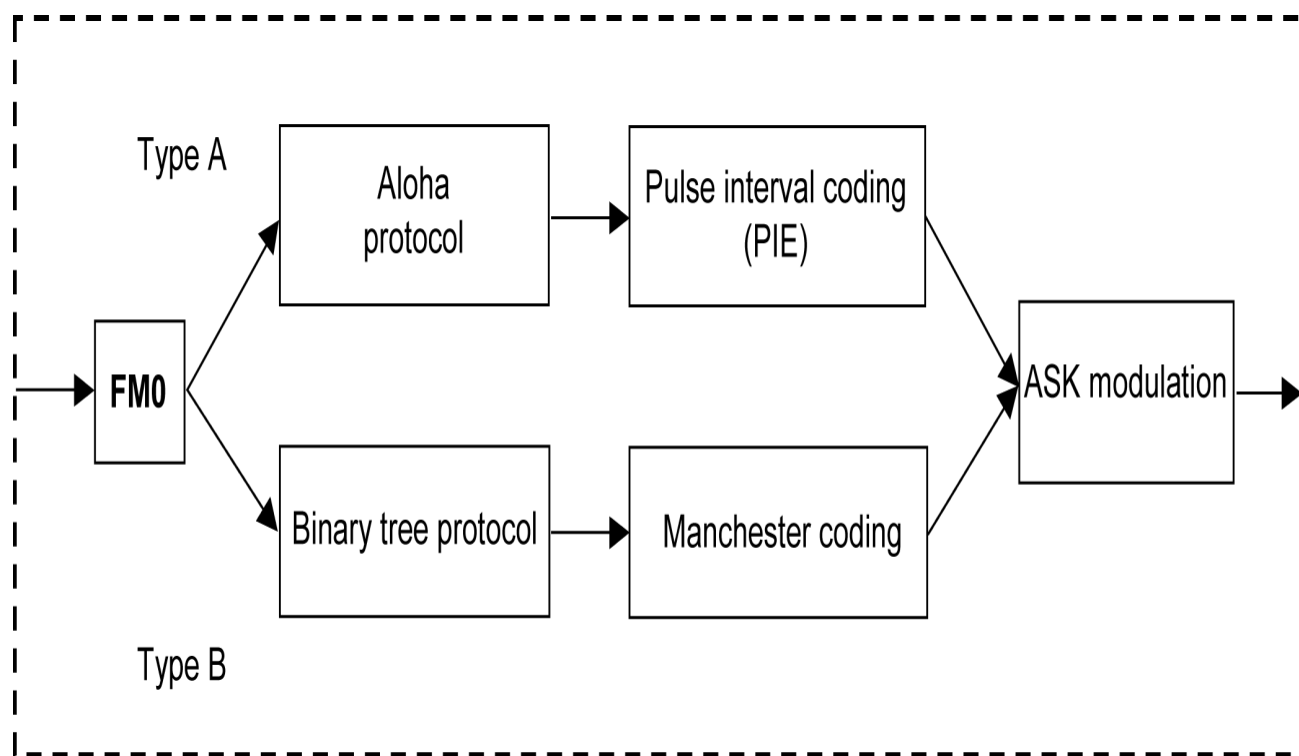


Figure 1 — Interrogator architecture

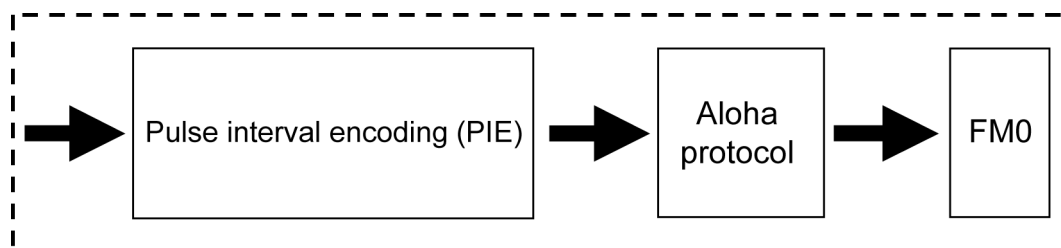


Figure 2 — Type A tag architecture

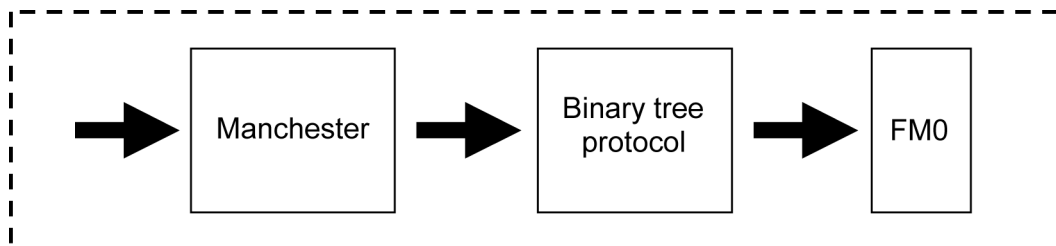


Figure 3— Type B tag architecture

Table 1 — Comparison of Type A and Type B

Parameter	Type A	Type B
Forward link encoding	PIE	Manchester
Modulation index	27% to 100%	18% or 100%
Data rate	33 kbit/s (mean)	10 or 40 kbit/s (according to local regulations)
Return link encoding	FM0	FM0
Collision arbitration	ALOHA	Binary Tree
Tag unique identifier	64 bits (40 bit SUID)	64 bits
Memory addressing	Blocks up to 256 bits	Byte blocks, 1,2,3 or 4 byte writes.
Error detection forward link	5 bit CRC for all commands (with an additional 16 bit CRC appended for all long commands)	16 bit CRC
Error detection return link	16 bit CRC	16 bit CRC
Collision arbitration linearity	Up to 250 tags	Up to 2^{256}

5.2 Parameter tables

Table 2, Table 3, Table 4 and Table 5 contain the parameters for both types A and B in accordance with ISO/IEC 18000-1. Detailed description of the operating modes and parameters are specified in the subsequent clauses.

Table 2 — Interrogator to tag link parameters

Interrogator to tag	Parameter name	Description
Int: 1	Operating frequency range	860 – 960 MHz The interrogator operating frequency range shall be determined by the radio regulations in force in a particular regulatory jurisdiction and by the type approval requirements of the particular jurisdiction. Before an interrogator may be used, it shall meet the local radio regulations of the country in which it is to be used. It is envisaged that there will be more than one version of interrogator having different frequency and power characteristics in order to comply with local regulations. NOTE Performance will vary according to bandwidth and power output permitted by local regulations.
Int: 1a	Default operating frequency	In accordance with the local radio regulations. See Int: 1
Int: 1b	Operating channels	In accordance with the local radio regulations. See Int: 1
Int: 1c	Operating frequency accuracy	In accordance with the local radio regulations See Int: 1
Int: 1d	Frequency hop rate (for frequency hopping [FHSS] systems)	Not applicable for single fixed frequency or channelized Adaptive Frequency Agile systems. Where FHSS is permitted, the hop rate shall be in accordance with the local radio regulations.
Int: 1e	Frequency hop sequence (for frequency hopping [FHSS] systems)	In accordance with the local radio regulations. Where not specified by such regulations a pseudo-random hopping sequence shall be used that ensures an even distribution of transmissions over the available channels.
Int: 2	Occupied channel bandwidth	In accordance with the local radio regulations.
Int:2a	Minimum receiver bandwidth	In accordance with the local radio regulations.
Int: 3	Interrogator transmitter maximum EIRP	In accordance with the local radio regulations
Int: 4	Interrogator transmitter spurious emissions	In accordance with the local radio regulations.
Int: 4a	Interrogator transmitter spurious emissions, in-band (for spread spectrum systems)	In accordance with the local radio regulations.
Int: 4b	Interrogator transmitter spurious emissions, out-of-band	In accordance with the local radio regulations.
Int: 5	Interrogator transmitter spectrum mask	As per Int: 2 and Int: 4a.
Int:6	Timing	See below Int: 6x.
Int: 6a	Transmit to receive turn around time	The interrogator transmit/receive settling time shall not exceed 85µs.
Int: 6b	Receive to transmit turn around time	As determined by the communication protocol – refer Tag: 6a.
Int: 6c	Dwell time or interrogator transmit power on ramp	Maximum 1.5ms.
Int: 6d	Decay time of interrogator transmitter power down ramp	Maximum 1ms.

Table 2 (continued)

Interrogator to tag	Parameter name	Description
Int: 7	Modulation	Amplitude Modulation.
Int: 7a	Spreading sequence (for direct sequence [DSSS] systems)	Not applicable.
Int: 7b	Chip rate (for spread spectrum systems)	Not applicable.
Int: 7c	Chip rate accuracy (for spread spectrum systems)	Not applicable.
Int: 7d	Modulation index	Type A: Nominal 30 % to 100 %. See Table 11 and clause 7.1.1. Type B: Nominal 18% or 100% .
Int: 7e	Duty cycle	In accordance with local regulations.
Int: 7f	FM deviation	Not applicable.
Int: 7g	Transmitter modulation pulse shape (falling and rising slopes)	The modulation pulse fall and rise times shall conform to the detailed specifications in subsequent clauses. Type A see Figure 11 and Table 11, and Type B see Figure 20, Figure 21, Table 93 and Table 94.
Int: 8	Data coding	Type A: Pulse Interval Encoding Type B: Manchester bi-phase
Int: 9	Bit rate	Type A: 33 kbit/s as constrained by the local radio regulations Type B: 10 kbit/s or 40 kbit/s as constrained by the local radio regulations.
Int: 9a	Bit rate accuracy	100 ppm
Int: 10	Interrogator transmit modulation accuracy	Not applicable.
Int: 11	Preamble	Type A : has no preamble Type B: See clause 8.1.4.3
Int:11a	Preamble length	Type A: not applicable. Type B: 9 bits. See clause 8.1.4.3.
Int:11b	Preamble waveform	Type A: not applicable . Type B: See clause 8.1.4.3.
Int: 11c	Bit sync sequence	Type A: not applicable. Type B: see clause 8.1.4.3
Int: 11d	Frame sync sequence	Not applicable.
Int: 12	Scrambling (for spread spectrum systems)	Not Applicable
Int: 13	Bit transmission order	MSB first
Int: 14	Wake-up process	Presence of an appropriate RF signal at the tag followed by a wake-up command as required by the tag type. See relevant clauses.
Int: 15	Polarization	Interrogator dependent. Not defined in this part of ISO/IEC 18000.

Table 3 — Tag to interrogator link parameters

Tag to interrogator	Parameter name	Description
Tag:1	Operating frequency range	860 MHz – 960 MHz
Tag:1a	Default operating frequency	The tag shall respond to an interrogator signal within the frequency range specified in Tag: 1.
Tag:1b	Operating channels (for spread spectrum systems)	The tag shall respond to an interrogator signal within the frequency range specified in Tag: 1.
Tag:1c	Operating frequency accuracy	The tag shall respond to an interrogator signal within the frequency range specified in Tag: 1.
Tag:1d	Frequency hop rate (for frequency hopping [FHSS] systems)	Not applicable
Tag:1e	Frequency hop sequence (for frequency hopping [FHSS] systems)	Not applicable
Tag:2	Occupied channel bandwidth	As permitted by the local radio regulations.
Tag:3	Transmit maximum EIRP	As permitted by the local radio regulations.
Tag:4	Transmit spurious emissions	As permitted by the local radio regulations.
Tag:4a	Transmit spurious emissions, in-band (for spread spectrum systems)	As permitted by the local radio regulations.
Tag:4b	Transmit spurious emissions, out-of-band	As permitted by the local radio regulations.
Tag:5	Transmit spectrum mask	As permitted by the local radio regulations.
Tag:6a	Transmit to receive turn around time	Type A: The tag shall open its receive command window within 2 bit periods of the end of its response. Type B: 400 µs
Tag:6b	Receive to transmit turn around time	Type A: Range from 150 to 1150 µs (see clause 7.5.3 and Table 28) Type B: Range 85 to 460 µs (see clause 8.1.5.2)
Tag:6c	Dwell time or transmit power on ramp	Not applicable.
Tag:6d	Decay time or transmit power down ramp	Not applicable.
Tag:7	Modulation	Bi-state amplitude modulated backscatter.
Tag:7a	Spreading sequence (for direct sequence [DSSS] systems)	Not applicable.
Tag:7b	Chip rate (for spread spectrum systems)	Not applicable.
Tag:7c	Chip rate accuracy (for spread spectrum systems)	Not applicable.
Tag:7d	On-off ratio	The tag Delta RCS (Varying Radar Cross Sectional area) affects system performance. A typical value is greater than 0.005m ² .
Tag:7e	Sub-carrier frequency	Not applicable.
Tag:7f	Sub-carrier frequency accuracy	Not applicable.
Tag:7g	Sub-carrier modulation	Not applicable.

Table 3 (continued)

Tag to interrogator	Parameter name	Description
Tag:7h	Duty cycle	The tag shall transmit its response when commanded to do so by the interrogator.
Tag:7i	FM deviation	Not applicable
Tag:8	Data coding	Bi-phase space (FM0)
Tag:9	Bit rate	Typical 40 kbit/s or 160 kbit/s (subject to tag clock tolerance see Table 9), The return bit rate selection for 160 kbit/s for Type B is defined in clause 8.1.4.4.5.
Tag:9a	Bit rate accuracy	+/- 15% (refer to Table 9)
Tag:10	Tag transmit modulation accuracy (for frequency hopping [fhss] systems)	Not applicable.
Tag:11	Preamble	The preamble is defined in clause 6.5.6
Tag:11a	Preamble length	16 bits made up of a quiet period, followed by sync, followed by a code violation followed by an orthogonal code.
Tag:11b	Preamble waveform	Bi-phase encoded data '1'.
Tag:11c	Bit sync sequence	Included in the preamble.
Tag:11d	Frame sync sequence	Included in the preamble.
Tag:12	Scrambling (for spread spectrum systems)	Not applicable.
Tag:13	Bit transmission order	MSB first.
Tag:14	Reserved	Deliberately left blank.
Tag:15	Polarization	Product design feature. Not defined in this part of ISO/IEC 18000.
Tag:16	Minimum tag receiver bandwidth	860 – 960 MHz

Table 4 — Protocol parameters

Ref.	Parameter name	Description
P:1	Who talks first	Interrogator talks first.
P:2	Tag addressing capability	Type A see clause 7.3.1 Type B see clause 8.2.2
P:3	Tag unique identifier (UID)	Contained in tag memory and accessible by means of a command.
P:3a	UID length	Type A: 64 bits. An SUID of 40 bits is used during census or collision arbitration transactions. Type B: 64 bits
P:3b	UID format	The UID format is different for types A and B. See clause 7.2.1 for type A. See clause B.2 for type B.

Table 4 (continued)

Ref.	Parameter name	Description
P:4	Read size	Type A: Addressable in blocks of up to 256 bits, but always an integer multiple of bytes. Type B: Addressable in byte blocks.
P:5	Write size	Type A: Addressable in blocks of up to 256 bits, but always an integer multiple of bytes. Type B: Addressable in byte blocks. Writing in blocks of 1, 2, 3 or 4 bytes. See details in relevant clauses.
P:6	Read transaction time	A single tag can typically be identified and have its first 128 bits of user memory read in less than 10 ms. This time may vary depending on the data rate used as constrained by the local radio regulations.
P:7	Write transaction time	Once a tag has been identified and selected, a 32-bit data block can typically be written in less than 20 ms. This time may vary depending on the data rate used as constrained by the local radio regulations.
P:8	Error detection	Interrogator to tag Type A: 16-bit commands: CRC-5. Commands of more than 16 bits have an additional CRC-16, bit protection. Type B: CRC-16 Tag to interrogator: CRC-16 for both types.
P:9	Error correction	No forward error correction code used. Errors are handled by signalling an error to the interrogator that then repeats its last transmission.
P:10	Memory size	No minimum user memory size is specified, but if user memory is provided it shall be an integer multiples of 4 bytes.
P:11	Command structure and extensibility	Type A: Several command codes are reserved for future use. In addition, a Protocol extension flag allows for further extensions. Type B: Several command codes are reserved for future use.

Table 5 — Anti-collision parameters

Ref.	Parameter name	Description
A:1	Type (Probabilistic or Deterministic)	Type A: Probabilistic. Type B: Probabilistic.
A:2	Linearity	Type A: Essentially linear using adaptive slot allocation up to 256 slots for 250 tags in an interrogator zone. Type B: Essentially linear up to 2^{256} tags depending on size of data content.
A:3	Tag inventory capacity	The algorithm permits the reading of not less than 250 tags in the reading zone of the interrogator.

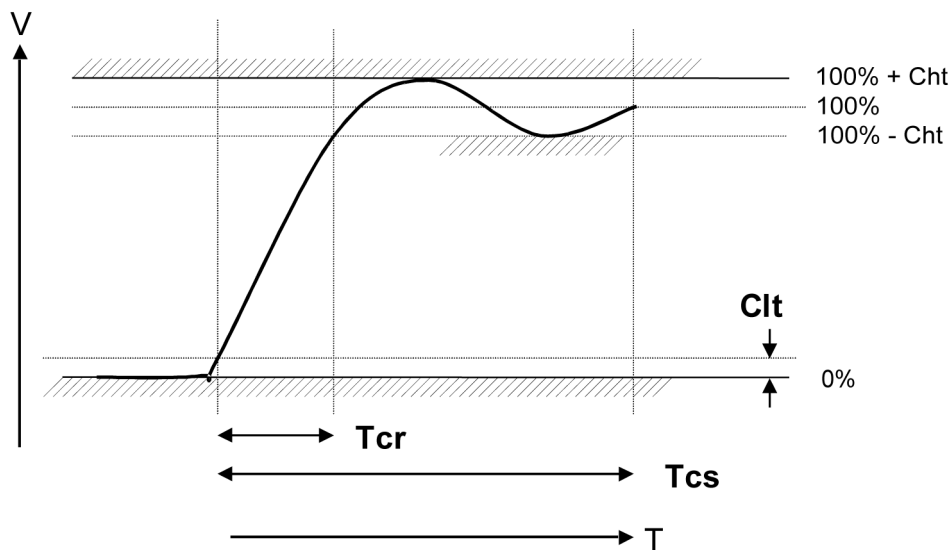


Figure 4 — Interrogator power-up waveform

Table 6 — Interrogator power-up waveform parameter values

Parameter	Min	Max
T_{cs}		1500 μ s
T_{cr}	1 μ s	500 μ s
C_{ht}		10%
Cl_t		1%

6 Common elements of the physical layer for types A and B

6.1 General

Types A and B have several common elements for the physical layers. They are specified in the following sub-clauses.

6.2 Interrogator power-up waveform

The interrogator power-up waveform shall comply with the mask specified in Figure 4 and Table 6.

6.3 Interrogator power-down

Once the carrier level has dropped below the ripple limit C_{ht} , power down shall be monotonic and of duration T_{cf} , as specified in Figure 5 and Table 7.

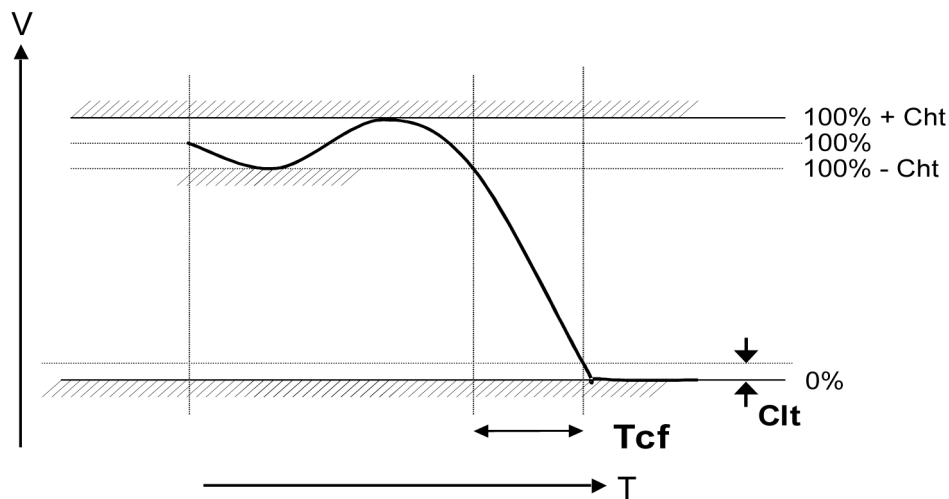


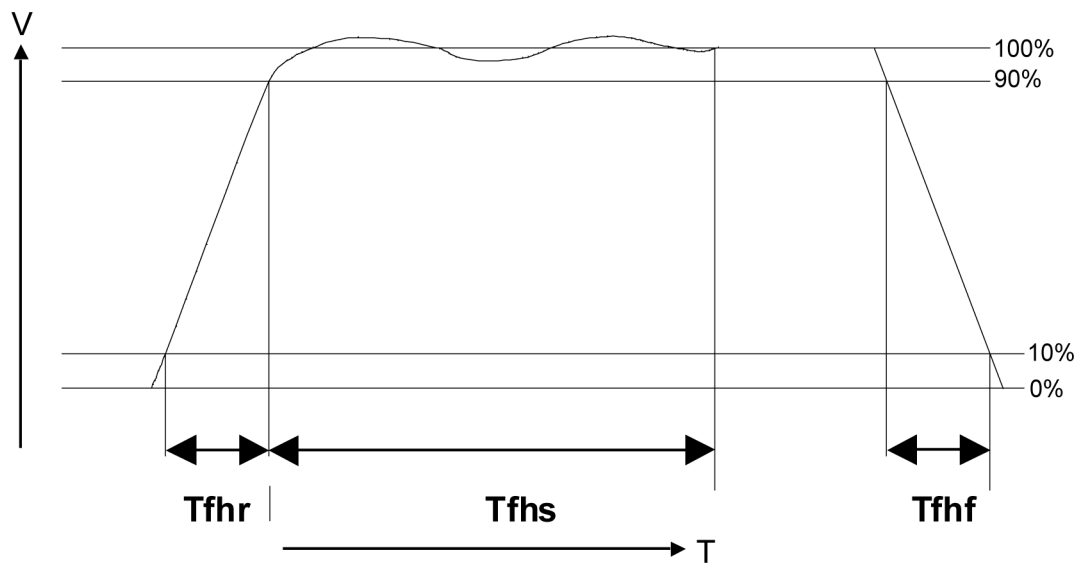
Figure 5 — Interrogator power-down waveform

Table 7 — Interrogator power-down timings

Parameter	Min	Max
T_{cf}	1 μs	500 μs
Cht		$\pm 5\%$ of steady state (100 %) level
Cl_t		1 %

6.4 Frequency hopping carrier rise and fall times

When the interrogator operates in the frequency operating hopping spread spectrum mode (FHSS), the carrier rise and fall times shall conform to the characteristics specified in Figure 6 and Table 8. The interrogator shall complete a frequency hop in a time not exceeding 30 μs (to ensure that the tag is not reset by the frequency hop). The frequency hop is measured from the beginning of T_{fhr} to the end of T_{fhf} .



NOTE Ripple is $\pm 5\%$ of 100 % of steady state level

Figure 6 — FHSS carrier rise and fall characteristics

Table 8 — FHSS carrier rise and fall parameters

Parameter	Min	Max
Tfhr		30 μ s
Tfhs	400 μ s	
Tfhf		30 μ s

6.5 FM0 return link

6.5.1 FM0 return link general

The tag transmits information to the interrogator by modulating the incident energy and reflecting it back to the interrogator (backscatter).

6.5.2 Modulation

The tag switches its reflectivity between two states. The “space” state is the normal condition in which the tag is powered by the interrogator and able to receive and decode the forward link. The “mark” state” is the alternative condition created by changing the antenna configuration or termination.

6.5.3 Data rate

The data rate is 40kbit/s.

6.5.4 Data coding

Data is coded using the FM0 technique, also known as Bi-Phase Space.

One symbol period $Trlb$, as specified in Table 9, is allocated to each bit to be sent. In FM0 encoding, data transitions occur at all bit boundaries. In addition, data transitions occur at the mid-bit of logic 0 being sent.

Table 9 — Return link parameters

Data rate	$Trlb$	Tolerance
40kbit/s	25 μ s	+/-15%

Coding of data is MSB first. Figure 7 illustrates the coding for the 8 bits of 'B1'.

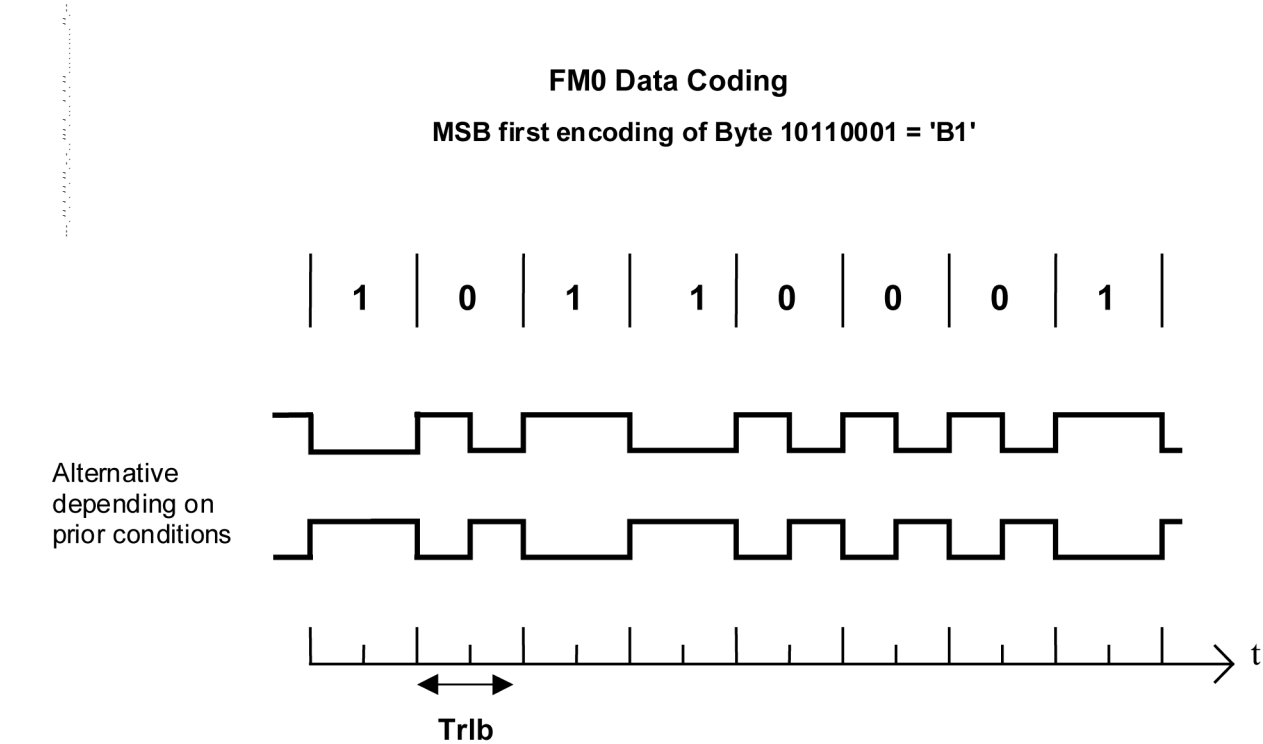


Figure 7 — Tag to interrogator data coding

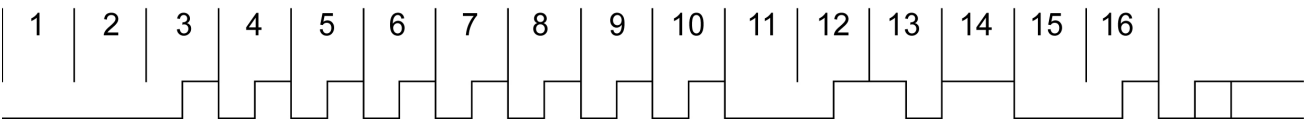
6.5.5 Message format

A return Link Message consists of n data bits preceded by the Preamble. The data bits are sent MSB first.

The Preamble enables the interrogator to lock to the tag data clock and begin decoding of the message. It consists of 16 bits as shown in Figure 8. There are multiple code violations (sequences not conforming to FM0 rules) that act as a frame marker for the transition from Preamble to Data.

6.5.6 Return preamble

The return preamble is a sequence of backscatter modulation specified in Figure 8.



NOTE The high state represents high reflectivity and the low state represents low reflectivity.

Figure 8 — Preamble waveform

Changing the Tag's modulator switch from the high impedance state to the low impedance state causes a change in the incident energy to be back-scattered, see Figure 9.

The tag shall execute backscatter, a half-low and half-high sent by the tag defined as follows:

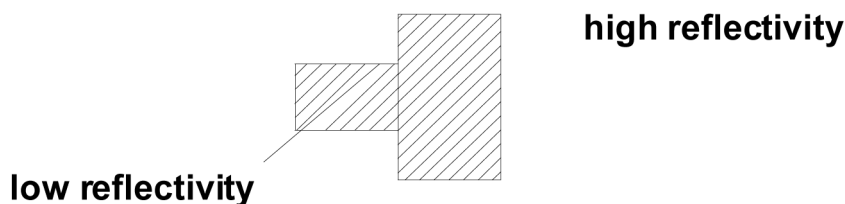


Figure 9 — Return link bit coding

6.5.7 Cyclic redundancy check (CRC)

6.5.7.1 CRC General

Types A and B use the same CRC-16 for the forward and the return links. In addition, Type A uses CRC-5 for short commands when it satisfies the required level of protection against errors.

On receiving a command from the interrogator, the tag shall verify that the checksum or the CRC value is valid. If it is invalid, it shall discard the frame, shall not respond and shall not take any other action.

6.5.7.2 Interrogator to tag 5 bit CRC-5

The 5 bit CRC shall be calculated on all the command bits after the SOF up to but not including the first CRC bit.

The polynomial used to calculate the CRC is $x^5 + x^3 + 1$.

A possible implementation is using a 5-bit shift register as defined in Clause A.1. The 5 bit CRC register is named Q4 to Q0, with Q4 being the MSB and Q0 being the LSB. The 5-bit register must be preloaded with '01001' (MSB to LSB) or in hexadecimal notation 0x09 (HEX), as shown in Table A.1

The 11 bits of data must be clocked through the CRC register, using the MSB first. The 5 CRC bits are then sent, MSB first. After the LSB of the CRC-5 bit is clocked through, the 5-bit CRC register should contain all zero's.

NOTE A schematic of a possible implementation is provided in Annex A.

6.5.7.3 Interrogator to tag 16 bit CRC-16

6.5.7.3.1 Interrogator to tag CRC-16 general

The 16 bit CRC shall be calculated on all the command bits after the SOF up to but not including the first CRC bit.

The polynomial used to calculate the CRC is $x^{16} + x^{12} + x^5 + 1$. The 16-bit register shall be preloaded with 'FFFF'. The resulting CRC value shall be inverted, attached to the end of the packet and transmitted.

The most significant byte shall be transmitted first. The most significant bit of each byte shall be transmitted first.

NOTE A schematic of a possible implementation is provided in Annex A.

The CRC may be implemented in one of two ways:

6.5.7.3.2 Inversion of incoming CRC bits by the tag

At the tag, the incoming CRC bits are inverted and then clocked into the register. After the LSB CRC bit is clocked into the register the 16 bit CRC register should contain all zero's.

6.5.7.3.3 Non-inversion of incoming CRC bits by the tag

If the received CRC bits are not inverted before clocking into the register, then after the LSB CRC bit is clocked into the register the 16 bit CRC register will have the value 0x1D0F (HEX)

6.5.7.4 Tag to interrogator 16 bit CRC-16**6.5.7.4.1 Tag to interrogator to tag CRC-16 general**

The 16 bit CRC shall be calculated on all data bits up to, but not including, the first CRC bit.

The polynomial used to calculate the CRC is $x^{16} + x^{12} + x^5 + 1$. The 16-bit register shall be preloaded with 0xFFFF (HEX). The resulting CRC value shall be inverted, attached to the end of the packet and transmitted.

The most significant byte shall be transmitted first, see Table 10. The most significant bit of each byte shall be transmitted first.

On receiving of a response from the tag, it is recommended that the interrogator verifies that the CRC value is valid. If it is invalid, appropriate remedial action is the responsibility of the interrogator designer.

NOTE A schematic of a possible implementation is provided in Annex A.

Table 10 — CRC-16 bits and bytes transmission rules

MSByte	LSByte
MSB LSB	MSB LSB
CRC-16 (8 bits)	CRC-16 (8 bits)
↑ first transmitted bit of the inverted CRC	

The CRC may be implemented in one of two ways.

6.5.7.4.2 Inversion of incoming CRC bits by the interrogator

At the reader receiver, the incoming CRC bits are inverted and then clocked into the register. After the LSB CRC bit is clocked into the register the 16 bit CRC register should contain all zero's.

6.5.7.4.3 Non-inversion of incoming CRC bits by the interrogator

If the received CRC bits are not inverted before clocking, the CRC register will have the value 0x1D0F (HEX).

7 Type A

7.1 Physical layer and data coding

7.1.1 PIE (Pulse interval encoding) forward link

7.1.1.1 Carrier modulation pulses

The data transmission from the interrogator to the tag is achieved by modulating the carrier amplitude (ASK). The data coding is performed by generating pulses at variable time intervals. The duration of the interval between two successive pulses carries the data coding information.

The tag shall measure the inter-pulse time on the high to low transitions (falling) edges of the pulse as shown in Figure 10.

The carrier modulation pulses are negative-going raised cosine shaped, their characteristics are specified in Figure 11 and Table 11.

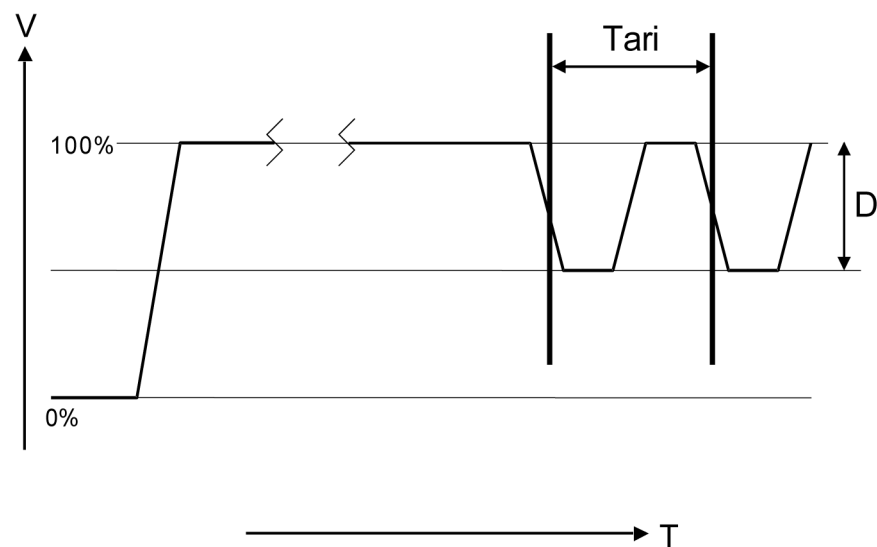
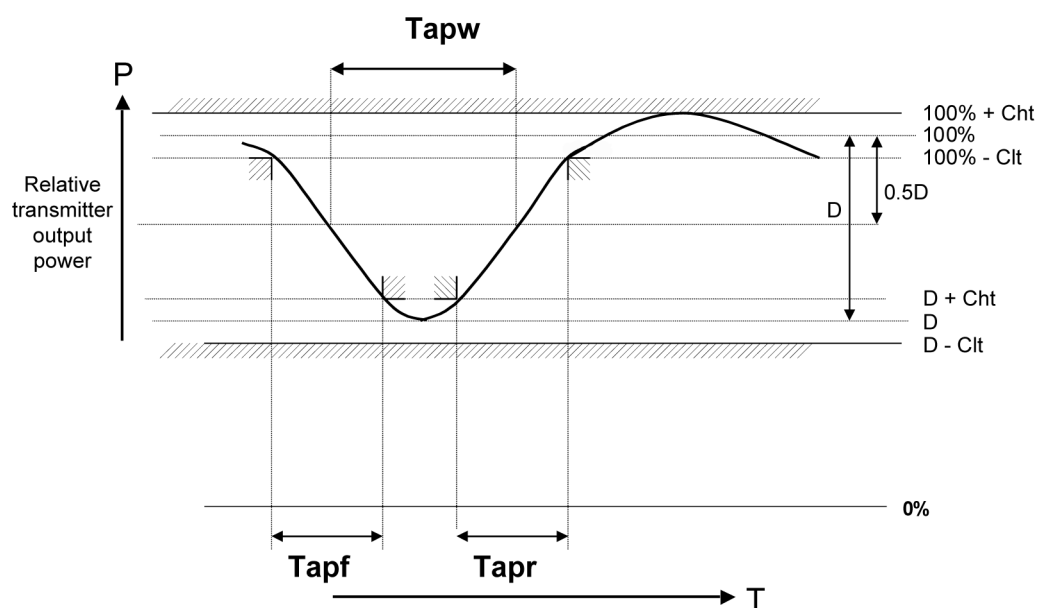


Figure 10 — Inter-pulse mechanism



NOTE The vertical lines occur at the half power (3dB) point on the modulation carrier dip.

Figure 11 — Modulation shaping

Table 11 — Modulation parameters

Parameter	Min	Nominal	Max
Tapw		10µs	
D	27% See NOTE 1		See NOTE 2
Tapf		4µs	
Tapr		4µs	
Cht			0.1 D
Clc			0.1 D
NOTE 1 The minimum modulation depth value is the absolute limit over the interrogator's operating temperature range.			
NOTE 2 The maximum modulation depth is may be in the range of 0.73 to 0 times the steady state value and will depend on the radio regulatory environment.			

The interrogator shall maintain a constant modulation depth during the transmission of a command, within the Cht and Clc tolerances.

7.1.1.2 Data coding and framing

The time Tari specifies the reference interval between the falling edges of two successive pulses representing the symbol '0' as transmitted by the interrogator; its value is specified in Table 12

Table 12 — Reference interval timing

Tari	Tolerance
20 μs	± 100 ppm

Four symbols are defined as shown as shown in Table 13 and Figure 12. The symbols are referenced to the interval Tari and have a duration of 1 Tari, 2 Tari, and 4 Tari being integer multiples of Tari and shall have a tolerance with respect to Tari as defined in Table 13. The Start of Frame, SOF symbol is a combinational symbol made up by concatenating a '0' symbol with a symbol having a duration of 3 Tari, for an overall length of 4 Tari.

Table 13 — PIE symbols, coding

Symbol	Duration	Tolerance with respect to 1 Tari
0	1 Tari	
1	2 Tari	± 100 ppm
SOF	1 Tari followed by 3 Tari	± 100 ppm
EOF	4 Tari	± 100 ppm

Symbol	Number of Tari
'0'	1
'1'	2
SOF	4
EOF	4

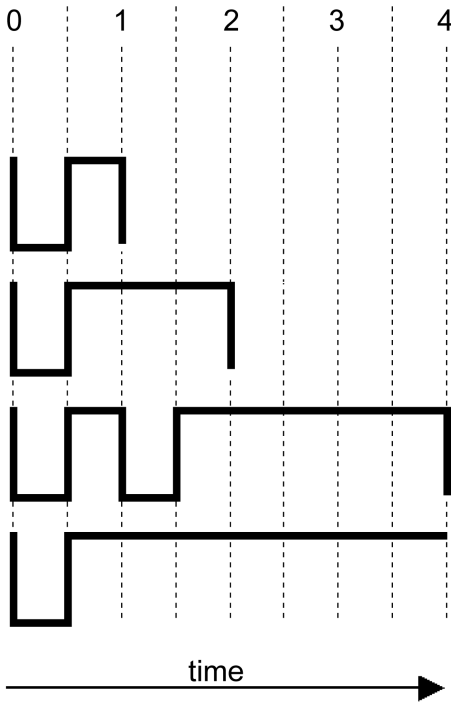


Figure 12 — PIE symbols

7.1.1.3 Decoding of PIE symbols by the tag

The tag shall be able to decode a PIE encoded transmission having symbols of duration as specified in Table 14:

Table 14 — Pie symbols, decoding

Symbol	Mean duration	Limits
0	1 Tari	$1/2 \text{ Tari} < '0' \leq 3/2 \text{ Tari}$
1	2 Tari	$3/2 \text{ Tari} < '1' \leq 5/2 \text{ Tari}$
SOF	1 Tari followed by 3 Tari	Calibration sequence
EOF	4 Tari	$\geq 4 \text{ Tari}$

The reference for the decoder is derived from the SOF and is $= 3/2 \text{ Tari}$

The tag shall interpret an interval between received symbols of $>4 \text{ Tari}$ as an EOF and shall discard the preceding data.

If the tag does not receive data for a period greater than EOF, the tag shall wait for a SOF.

7.1.1.4 Frame format

The bits transmitted by the interrogator to the tag are embedded in a frame as specified in Figure 13.

Before sending the frame, the interrogator shall ensure that it has established an unmodulated carrier for a duration of at least T_{aq} (Quiet time) of $300\mu\text{s}$.

The frame consists of a Start Of Frame (SOF), immediately followed by the data bits and terminated by an End Of Frame (EOF). After having sent the EOF the interrogator shall maintain a steady carrier for the time specified by the protocol so that the tags are powered for transmitting their response.

If the tag does not receive data for a period greater than EOF, then the command decoder shall revert to the ready state and await another SOF.

The tag shall interpret an interval between received symbols of $>4 \text{ Tari}$ as an error and shall discard the preceding data.

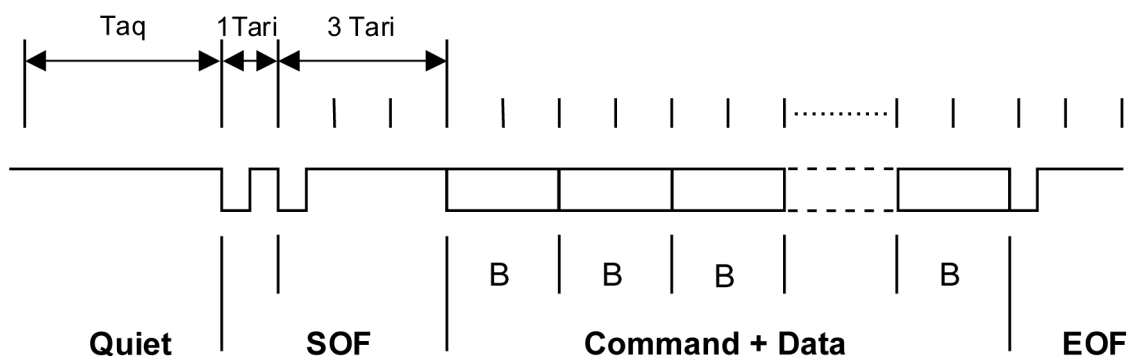


Figure 13 — Forward link frame format

7.1.1.5 Data decoding

The tag shall decode the symbols specified in Table 13.

The tag shall be able to decode an incoming signal having a modulation depth (as specified in clause 7.1.1.1, Figure 11 and Table 11) with respect to the reader steady state carrier of 27 % or greater (ie 27 % to 100 %). In other words the modulation depth may be anywhere in the range of 0.73 to 0 times the steady state carrier amplitude.

If the tag detects an invalid code, it shall discard the frame and wait for an unmodulated carrier of 4Tari duration.

7.1.1.6 Bits and byte ordering

Coding of data into symbols shall be MSB first. The coding for the 8 bits of hex byte 'B1' is shown in Figure 14.

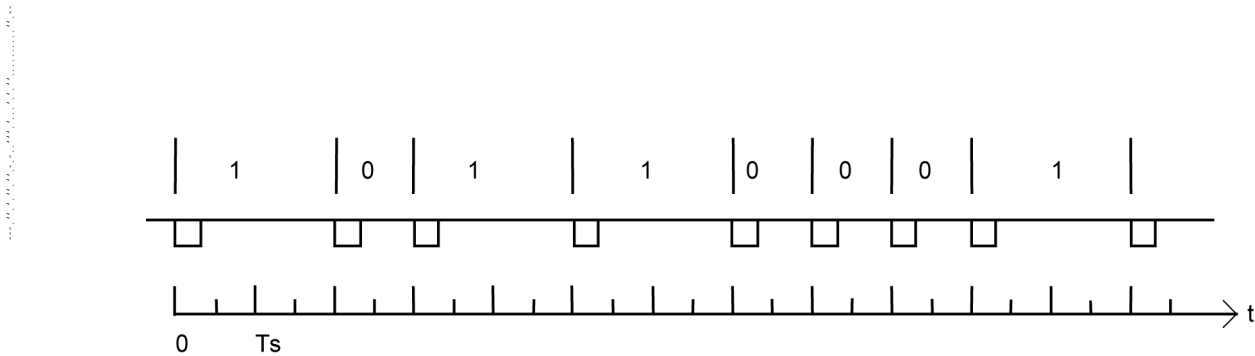


Figure 14 — Example of PIE byte encoding for 'B1'

7.2 Data elements

7.2.1 Unique identifier (UID)

Tags may be uniquely identified in order to provide a one-to-one transaction between the tag and the interrogator and to provide traceability information relating to the transponder chip. When thus used, the tag Unique Identification number (UID) shall be in this form as specified in Table 15 and ISO/IEC 15963.

If used, the UID shall be set permanently by the IC manufacturer in accordance with Table 15.

Table 15 — UID format

MSB			LSB
b64 .. b57	b56 .. b49	b48 .. b33	b32 .. b1
'E0'	IC Mfg code	RFU set to '0'	IC manufacturer serial number

The tag UID shall comprise

- The 8 MSB bits defined as 'E0',
- The IC manufacturer code of 8 bits according to ISO/IEC 7816-6, and
- A unique serial number of 48 bits assigned by the IC manufacturer.

7.2.2 Sub-UID

When the Aloha protocol is used, only a part of the UID, called Sub-UID (SUID) is transmitted in most commands and in the tag response during a collision arbitration process. The only exception is the Get_system_information command that returns the complete UID of 64 bits. See clause 7.8.7.

The SUID consists of 40 bits: the 8 bits manufacturer code followed by the 32 LSBs of the serial number.

The 16 MSBs (bits 33 to 48) of the serial number shall be set to 0, as bits 33 to 48 are ignored in the SUID.

The mapping of the 64 bit UID to the transmitted 40 bits and back is described in Figure 15.

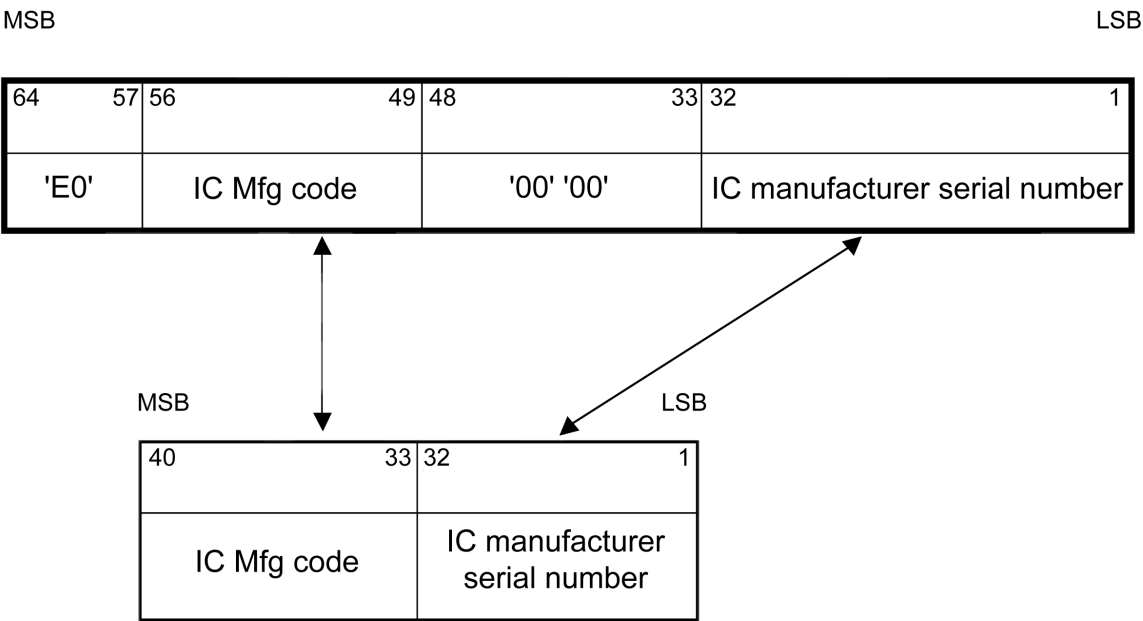


Figure 15 — UID/SUID mapping from 64 to 40

The interrogator shall use the 64-bit format specified in clause 7.2.1 when exchanging the UID with the application. It shall perform the required mapping described in Figure 15.

7.2.3 Application family identifier

An Application Family Identifier (AFI) may optionally be used to group select tags. The AFI shall comprise a byte (8 bits). The AFI is a parameter in the Init_round command, or the first 8 bits in the selection mask of the Begin_round command.

When the AFI is used in the Init_round or the Begin_round command it shall conform to the format specified in ISO/IEC 15961 and ISO/IEC 15962 and shown in Table 16.

When an arbitration sequence is initiated using the Init_round or Begin_round command, the AFI parameter shall contain the value of either 0x00 in which case all tags shall respond or; a specific 8 bit value, in which case only those tags which have a matching value shall respond.

Table 16 — AFI/ASF coding

AFI	Meaning tags respond from	Examples / note
'00'	All families respond	No applicative preselection
'XY'	Tags match 'XY' respond	

NOTE Table 16 is taken from ISO/IEC 15961 and ISO/IEC 15962. Users should refer to the latest version of ISO/IEC 15961 and ISO/IEC 15962, for their reference.

7.2.4 Data storage format identifier (DSFID)

The Data storage format identifier indicates how the data is structured in the tag memory.

It may be programmed and locked by the respective commands. It is coded using one byte. It provides knowledge of the logical organisation of the data.

Its coding shall conform to ISO/IEC 15962.

Where the tag does not support programming of DSFID, it shall respond with the value of DSFID being set to zero ('00').

7.3 Protocol elements

7.3.1 Tag memory organisation

The commands specified in this part of ISO/IEC 18000 assume that the physical memory is organised in blocks of a fixed size.

Up to 256 blocks can be addressed, block sizes can be up to 256 bits, which leads to a maximum memory capacity of up to 8 kBytes (64 kBits).

NOTE The command structure allows for extension of the maximum memory capacity, if required, in revisions of this part of ISO/IEC 18000.

The commands described in this part of ISO/IEC 18000 allow access (read and write) by block(s). There is no implicit or explicit restriction regarding other access methods (e.g., by byte or by logical object in revisions of this part of ISO/IEC 18000, or in custom commands).

7.3.2 Support of battery-assisted tags

This part of ISO/IEC 18000 makes provision for the support of battery-assisted tags.

In normal operation, there is no functional difference between passive and battery-assisted tags.

The mechanisms provided to handle the differences in performance and maintenance of tags that are battery-assisted and those that are not battery-assisted are as follows:

- a) The type of tag and its sensitivity level are returned in the answer to the Get_system_information command. See clause 7.8.7.
- b) The type of tag (battery/no battery) and the battery status is returned in the tag answer during the collision arbitration sequence. See clause 7.7.5.

7.3.3 Block lock status

The block lock status is returned by the tag as a parameter in the response to an interrogator command as specified in clause 7.8.16. The block lock status is coded using two bits. Table 17 shows the 2 bits used for each block diagrammatically, with the actual implementation left to the IC designer.

The block lock status is an element of the protocol. There is no implicit or explicit assumption that the 2 bits are actually implemented in the physical memory structure of the tag.

User locking is performed by the Lock_blocks command.

Factory locking may be performed by a proprietary command.

Table 17 — Block lock status

Bit	Flag name	Value	Description
b1	User_lock_flag	0	Not user locked
		1	User locked
b2	Factory_lock_flag	0	Not factory locked
		1	Factory locked

7.3.4 Tag signature

The tag signature comprises 4 bits. It is used in the Aloha collision arbitration mechanism.

The purpose of the tag signature is to provide a transient session identity that gives differentiation between tags that have answered in the same slot.

The signature is designed to avoid inadvertent isolation, selection or acknowledgement of a tag masked by another tag due to weak signal masking. The tag may generate its signature by a number of mechanisms. For example, by means of a 4 bit pseudo-random number generator, or; by using a part of the tag's UID, or; tag's CRC or; by having a circuit that measures the tag's excitation level [TEL]. The means by which the signature is generated is left to the manufacturer and is not specified in this part of ISO/IEC 18000.

During the collision arbitration process, the tag transmits its signature along with its data or SUID. The interrogator then uses the signature in its communication with the tag by including it in transmitted commands so that only the tag with a matching signature will be responsive to those commands.

On receiving a command requiring a signature, the tag shall test the received signature against the one it has generated and sent. The tag shall behave in accordance with the state diagram, see Figure 16, and the respective state transition tables. For the Next_slot command see Table 31 and for the Standby_round command see Table 33.

7.4 Protocol description

7.4.1 Protocol concept

The transmission protocol defines the mechanism for exchanging commands and data between the interrogator and the tag, in both directions.

It is based on the concept of "interrogator talks first".

This means that the tag shall wait to receive a correctly decoded command from the interrogator before transmitting.

a) The protocol is based on an exchange of

- A command from the interrogator to the tag, and
- A response from the tag(s) to the interrogator.

The conditions under which the tag sends a response are defined in clause 7.7 and 7.8.

b) Each command and each response are contained in a frame. Each command consists of the following fields:

- SOF,
- RFU field (reserved for protocol extension),
- A command code,
- Parameters (or RFU) ,
- CRC-5,
- Optional parameter fields, depending on the command ,
- Application data fields,
- CRC-16, and
- EOF.

c) Each response consists of the following fields:

- SOF,
- Flags,
- Mandatory and optional parameter fields, depending on the command ,
- Application data fields,
- CRC-16, and
- EOF.

d) The protocol is bit-oriented. A single field is transmitted most significant bit (MSB) first.

- e) A multiple-byte field is transmitted most significant byte (MSByte) first, each byte is transmitted most significant bit (MSB) first.
- f) The setting of the flags indicates the presence of the optional fields. When the flag is set (to one), the field is present. When the flag is reset (to zero), the field is absent.
- g) RFU flags shall be set to zero (0).

7.4.2 Command format

7.4.2.1 Type A command format general

There are two forms of commands, short commands of 16 bits, and longer.

7.4.2.2 Short command format

A short command consists of the fields defined in Table 18.

Table 18 — Short command format

SOF	RFU	Command code	Parameters/Flags	CRC-5	EOF
	1 bit	6 bits	4 bits	5 bits	

7.4.2.3 Long command format

A long command consists of the fields defined in Table 19.

Table 19 — Long command format

SOF	RFU	Command code	Parameters or flags	CRC-5	SUID (optional)	Data	Data (optional)	CRC-16	EOF
	1 bit	6 bits	4 bits	5 bits	40 bits	8 bits	8 to n	16 bits	

The upper value of the optional data field length is specified as n, where $n = m + 8$, and where m = the number of bits to program in the tag's memory. Currently, the value of m = 32 is used, but the protocol allows m to be up to 256.

7.4.3 Command flags

7.4.3.1 Command flags

There is only one command flag, and if present is the SUID flag.

7.4.3.2 SUID flag

7.4.3.2.1 SUID flag general

The SUID flag is contained in the most significant bit position of the command parameter field. Its function is modified by the particular command. Refer to Table 22.

7.4.3.2.2 SUID flag and inventory commands

Inventory commands are:

- Init_round,
- Init_round_all,
- Begin_Round,
- New_Round,
- Close_Slot, and
- Next_Slot.

The first four of the six inventory commands are used for round Initialisation. These four commands can select between receiving the first page of user memory or the tag's SUID as the data message received as part of the inventory process.

If the SUID flag is not set ('0'), the tag shall return the first page of user memory. It shall not include the SUID in its response. (A page of user memory may be as little as 0 bits, but if user memory is present it must be a multiple of 4 bytes (32 bits) depending on the specific implementation.)

If the SUID flag is set ('1'), the tag shall return the SUID. See clauses 7.8.2 and 7.7.5

7.4.3.2.3 SUID flag and non-inventory commands

If the SUID flag is present in the Select, Read_blocks, Get_system_information, Write_block, Write_multiple_blocks, Lock_blocks, Write_AFI, Lock_AFI, Write_DSFD, Lock_DSFD or Get_block_lock_status, then only that tag addressed by the SUID will process the command.

When the SUID flag is set ('1'), the command shall contain the SUID of the addressed tag. On receiving a command with the SUID flag set, the tag shall compare the SUID contained in the command with its SUID. If the SUID matches, the tag shall execute the command. If the SUID does not match, the tag shall ignore the command.

When the SUID flag is not set ('0'), the command shall not contain the SUID. On receiving a command the tag shall respond to the command if it is in the select state. (ie. it was previously selected by means of a Standby_round command with valid signature, or by means of the select command.) If the tag is not in the select state it shall ignore the command.

7.4.4 Round size

The round size is coded using 3 bits according to Table 20.

Table 20 — Round size coding

Value	Bit coding		Round size
	MSB	LSB	
'0'	0	0	1
'1'	0	0	8
'2'	0	1	16
'3'	0	1	32
'4'	1	0	64
'5'	1	0	128
'6'	1	1	256
'7'	1	1	RFU

7.4.5 Command code definition and structure

The size of the command code is 6 bits.

7.4.6 Command classes

7.4.6.1 Command classes general

Four sets of commands, as shown in Table 21, are defined: mandatory, optional, custom and proprietary.

Table 21 — Command classes

Code	Class	Number of possible codes
'00', '02', '04', '06', '0A', and '0C' – '0F'	Mandatory	10
'01', '03', '05', '07', '08', '09', '0B' and '10' – '27'	Optional	30
'28' – '37'	Custom	16
'38' – '3F'	Proprietary	8

All tags with the same IC manufacturer code and same IC version number shall behave the same.

7.4.6.2 Mandatory

The mandatory command codes are '02', '04', '06' and '0A'. Command codes '00' and '0C' to '0F', are RFU.

Mandatory commands shall be supported by all tags that claim to be compliant. Interrogators which claim compliance shall support all mandatory commands.

7.4.6.3 Optional

The optional command codes are '01', '03', '05', '07', '08', '09', '0B' and '10' to '27'.

Optional commands are commands that are specified within this part of ISO/IEC 18000. Interrogators shall be technically capable of performing all optional commands that are specified in this part of ISO/IEC 18000 (although need not be set up to do so). Tags may or may not support optional commands.

If an optional command is used, it shall be implemented in the manner specified in this part of ISO/IEC 18000.

If a tag in the selected state does not support the optional command it shall return the error code "command is not supported", see clause 7.4.8.2.

If the command specifies an SUID and the tag with the matching SUID does not support the optional command it shall return the error code "command is not supported", see clause 7.4.8.2.

All other tags shall remain silent.

7.4.6.4 Custom

The custom command codes range from '28' to '37'. Custom commands may be enabled by this part of ISO/IEC 18000, but they shall not be specified in this part of ISO/IEC 18000. A custom command shall not solely duplicate the functionality of any mandatory or optional command defined in this part of ISO/IEC 18000 by a different method.

Tags may support custom commands, at their option, to implement manufacturer specific functions. The function of flags (including reserved bits) shall not be modified. The only fields that can be customised are the parameter and the data fields.

Any custom command contains as its first parameter the IC manufacturer code. This allows IC manufacturers to implement custom commands without risking duplication of command codes and thus misinterpretation.

A tag in the selected state that does not support the Custom command shall return the error code "command is not supported", see clause 7.4.8.2.

If the command specifies an SUID and the tag with the matching SUID, does not support the Custom command, it shall return the error code "command is not supported", see clause 7.4.8.2.

All other tags shall remain silent.

7.4.6.5 Proprietary

The proprietary command codes range from '38' to '3F'. Proprietary commands may be enabled this part of ISO/IEC 18000, but they shall not be specified in this part of ISO/IEC 18000. A proprietary command shall not solely duplicate the functionality of any mandatory or optional command defined in this part of ISO/IEC 18000 by a different method.

These commands are used by IC and tag manufacturers for various purposes such as tests, programming of system information, etc. They are not specified in this part of ISO/IEC 18000. The IC manufacturer may at its option document them or not. This part of ISO/IEC 18000 allows these commands to be disabled after IC and/or tag manufacturing.

7.4.7 Command codes and CRC

All mandatory and optional commands are mandatory for a reader implementation, although the reader may not be set-up to provide them all.

This part of ISO/IEC 18000 currently defines 4 mandatory tag commands, **Next_slot**, **Standby_round**, **Reset_to_ready** and **Init_round_all**, and 6 RFU mandatory commands. All commands are summarised in Table 22, with the 4 mandatory tag commands being highlighted in bold.

Table 22 — Command codes

Command 6 bits	Op code	Parameter / flags 4 bits		CRC-5	Extended parameters				CRC-16	Comments
RFU	00									
Init_round	01	SUID 1 bit	Round size 3 bits	5 bits	AFI 8 bits				16 bits	SUID = 0 tag responds with first page of user data SUID = 1 tag responds with SUID If AFI field = 0x00 (HEX), all tags respond, else if AFI is other value, only tags with matching AFI respond
Next_slot	02	Signature 4 bits		5 bits	none					The signature may be one of 15 values between 0x01 (HEX) and 0x0F (HEX) as transmitted by the tag in its last transmission. A signature value of 0x00 (HEX) will only advance the slot counter (equivalent to close-slot command.)
Close_slot	03	Null		5 bits	none					
Standby_round	04	Signature 4 bits		5 bits	none					The signature may be one of 15 values between 0x01 (HEX) and 0x0F (HEX) as transmitted by the tag in its last transmission.
New_round	05	SUID 1 bit	Round size 3 bits	5 bits	none					
Reset_to_read y	06	Null		5 bits	none					
Select	07	SUID=1 1 bit	Null 3 bits	5 bits	SUID 40 bits				16 bits	
Read_blocks	08	SUID=0 1 bit	Null 3 bits	5 bits	First Block 8 bits	Number of blocks 8 bits			16 bits	
Read_blocks	08	SUID=1 1 bit	Null 3 bits	5 bits	SUID 40 bits	First Block 8 bits	Number of blocks 8 bits		16 bits	
Get_system _information	09	SUID=0 1 bit	Null 3 bits	5 bits						

Table 22 (continued)

Command 6 bits	Op code	Parameter / flags 4 bits		CRC-5	Extended parameters			CRC-16	Comments
Get_system_information	09	SUID=1 1 bit	Null 3 bits	5 bits	SUID 40 bits			16 bits	
Init_round_all	0A	SUID 1 bit	Round size 3 bits	5 bits					SUID = 0 tag responds with first page of user data SUID = 1 tag responds with SUID
Begin_round	0B	SUID=0	Round size 3 bits	5 bits	Mask Length 8 bits	Mask n bits			n=0 to 255
RFU	0C to 0F								
Write_block	10	SUID=0 1 bit	Null 3 bits	5 bits	Block number 8 bits	Data n bits		16 bits	n is currently 32 bits, but can be defined up to 256 bits
Write_block	10	SUID=1 1 bit	Null 3 bits	5 bits	SUID 40 bits	Block number 8 bits	Data n bits	16 bits	n is currently 32 bits, but can be defined up to 256 bits
Write_multiple_blocks	11	SUID=0 1 bit	Null 3 bits	5 bits	First Block 8 bits	Number of blocks 8 bits	Data n bits	16 bits	n is currently 32 bits, but can be defined up to 256 bits B is the number of blocks, with valid numbers being 1 to 256.
Write_multiple_blocks	11	SUID=1 1 bit	Null 3 bits	5 bits	SUID 40 bits	First Block 8 bits	Number of blocks 8 bits	D a t a n b i t s 16 bits	B is the number of blocks, with valid numbers being 1 to 256
Lock_blocks	12	SUID=0 1 bit	Lock Block number 3 bits	5 bits	Lock block data 32 bits			16 bits	
Lock_blocks	12	SUID=1 1 bit	Lock Block number 3 bits	5 bits	SUID 40 bits	Lock block data 32 bits		16 bits	
Write_AFI	13	SUID=0 1 bit	Null 3 bits	5 bits	AFI 8 bits			16 bits	
Write_AFI	13	SUID=1 1 bit	Null 3 bits	5 bits	SUID 40 bits	AFI 8 bits		16 bits	

Table 22 (continued)

Command 6 bits	Op code	Parameter / flags 4 bits		CRC-5	Extended parameters			CRC-16	Comments
Lock_AFI	14	SUID=0 1 bit	Null 3 bits	5 bits					
Lock_AFI	14	SUID=1 1 bit	Null 3 bits	5 bits	SUID 40 bits			16 bits	
Write_DSFID	15	SUID=0 1 bit	Null 3 bits	5 bits	DSFID 8 bits			16 bits	
Write_DSFID	15	SUID=1 1 bit	Null 3 bits	5 bits	SUID 40 bits	DSFID 8 bits		16 bits	
Lock_DSFID	16	SUID=0 1 bit	Null 3 bits	5 bits					
Lock_DSFID	16	SUID 1 bit	Null 3 bits	5 bits	SUID 40 bits			16 bits	
Get_block_lock_status	17	SUID=0 1 bit	Lock Block number 3 bits	5 bits					
Get_block_lock_status	17	SUID=1 1 bit	Lock Block number 3 bits	5 bits	SUID 40 bits			16 bits	
RFU	18-27								
Custom	28 - 37								
Proprietary	38-3F								

7.4.8 Response format

7.4.8.1 Response format general

The response from the tag, as shown in Table 23, consists of the following fields:

- Preamble (See clause 6.5.6),
- Flags (See clause 7.4.8.2),
- One or more parameter fields (Defined in each command),
- Data (Defined in each command), and
- CRC-16 (See clause 6.5.7.4).

Table 23 — General response format

Preamble	Flags	Parameters	Data	CRC-16
----------	-------	------------	------	--------

7.4.8.2 Response error flags

In a response, two flags, as shown in Table 24, Table 25 and Table 26, indicate how the tag has performed actions and whether corresponding fields are present or not.

Table 24 — Response format when Error_flag is set

Preamble	Flags	Error code	Terminator
	2 bits	Optional 4 bits See Table 27	'0' 1 bit

Table 25 — Response format when Error_flag is NOT set, and there is NO appended data.

Preamble	Flags	Terminator
	2 bits	'0' 1 bit

Response parameter:

Error_flag (and Error code if Error_flag is set)

Table 26 — Response flags b1 to b2 definition

Bit	Flag name	Value	Description
b1	Error_flag	0	No error
		1	Error detected. Error code is in the "Error" field.
b2	RFU	0	Shall be set to 0.

7.4.8.3 Response error code

The error code comprises four bits.

When the tag sets the Error_flag, the error code field shall be included and provides information about the error that occurred. Error codes are specified in Table 27.

If the tag does not support specific error code(s) listed in Table 27, it shall answer with the error code 'F' ("Error with no information given").

Table 27 — Response error codes

Code	Description
'0'	RFU
'1'	The command is not supported, i.e. the command code is not recognised
'2'	The command is not recognised, for example: a format error occurred
'3'	The specified block is not available (does not exist)
'4'	The specified block is locked and its content cannot be changed
'5'	The specified block was not successfully programmed and/or locked
'6' – 'A'	RFU
'B'-'E'	Custom command error codes
'F'	Error with no information given or a specific error code is not supported.

7.4.9 Tag states

7.4.9.1 Tag states general

A tag can be in one of the following states:

- RF field off,
- Ready,
- Quiet,
- Selected,
- Round_active, and
- Round_standby.

The transitions between states are shown in Figure 16 and are specified in tables in each command description.

7.4.9.2 RF field off state

The tag is in the RF field off state when it does not receive the required RF field from the interrogator.

For passive tags, it means that the tag is not powered.

For battery-assisted tags, it means that the level of RF excitation is insufficient to turn on the tag circuits.

7.4.9.3 Ready state

The tag is in the Ready state when it is receiving sufficient energy from the interrogator to function correctly. It shall process any command where the Select flag is not set.

7.4.9.4 Quiet state

When in the Quiet state, the tag shall process any command where the SUID flag is set and the SUID matches.

NOTE A tag will move to the Quiet state after it has been correctly identified within its slot by the interrogator and takes no further part within the current or subsequent rounds.

7.4.9.5 Selected state

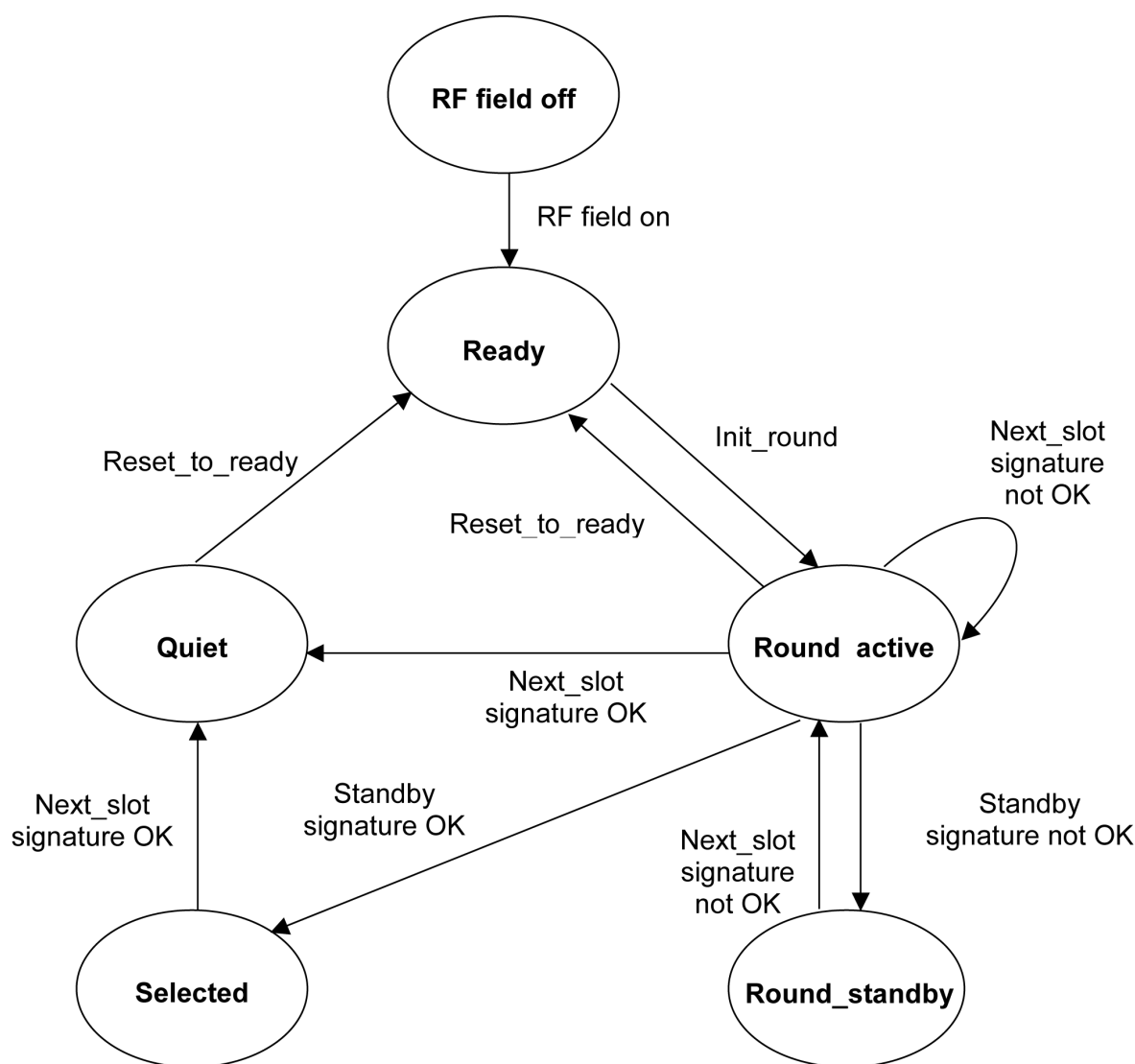
Only a tag in the Selected state shall process commands with the SUID flag set to 0, with the exception of Init_round, Init_round_all, Begin_round and New_round commands, which will be processed even if the SUID flag is set to 1.

7.4.9.6 Round_active state

When in the Round_active state, the tag shall participate in the collision arbitration described in clauses 7.4.10 and 7.4.11.

7.4.9.7 Round_standby state

When in the Round_standby state, the tag shall suspend its participation in the collision arbitration as described in clauses 7.4.10 and 7.4.11.



NOTE This state diagram shows some of the main transitions. State transitions are specified in detail in each command description.

Figure 16 — Tag state transition diagram

7.4.10 Collision arbitration

The purpose of the collision arbitration sequence is to perform a census of the tags present in the interrogator field and to receive information on the tag capabilities and data contents, all in a single sequence. The information that the tag shall return, is specified by the SUID flag, set in the command from the interrogator.

The interrogator is the master of the communication with one or multiple tags.

NOTE This mechanism is also some times referred to as the "wake-up" sequence.

7.4.11 General explanation of the collision arbitration mechanism

The collision arbitration uses a mechanism that allocates tag transmissions into rounds and slots. A round consists of a number of slots. Each slot has a duration, long enough for the interrogator to receive a tag response. The interrogator determines the actual duration of a slot.

In the absence of an RF field, the tags are in the RF_field off state. When the tags enter the energising field of an interrogator, they go through a power on reset sequence and move into the Ready state.

The interrogator initiates a tag census or collision arbitration process by sending an Init_round command, or the Init_round_all command.

Tags receiving an Init_round command randomly select a slot in which to respond but do not immediately start transmitting. The number of slots in a round referred to as round size, is determined by the interrogator and signalled to the tag in the Init_round command. The initial round size is predetermined by the user. During the subsequent collision arbitration process the interrogator dynamically chooses an optimum round size for the next round based on the number of collisions in the round. The number of collisions is a function of the number of tags in the active state present in the interrogator field and the round size.

On receiving an Init_round command, tags select a slot in which to respond. The selection is determined by a pseudo-random number generator. If the tag has selected the first slot it will wait a pseudo-random time delay equal to a time of between 0 and 7 periods and then transmit its response.

The tag includes its four bit tag signature in its response.

If the tag has selected a slot number greater than one, it will retain its slot number and wait for a further command.

After the interrogator has sent the Init_round command there are three possible outcomes:

- 1) The interrogator does not receive a response because either no tag has selected slot one or the interrogator has not detected a tag response. It then issues a Close_slot command because it has not received a response.
- 2) The interrogator detects a collision between two or more tag responses. Collisions may be detected either as contention from the multiple transmissions or by detecting an invalid CRC. Having verified that there are no tags transmitting, the interrogator sends a Close_slot command.
- 3) The interrogator receives a tag response without error, i.e. with a valid CRC. The interrogator sends a Next_slot command containing the signature of the tag just received.

When tags that have not transmitted in the current slot receive a Close_slot or Next_slot command, they increment their slot counters by one. When the slot counter equals the slot number previously selected by the tag, the tag transmits according to the rules above, otherwise the tag waits for another command.

When a tag in the active state receives a Close_slot command, it increments its slot counter.

When a tag that has transmitted its data in the current slot receives a Next_slot command, it:

- verifies that the signature in the command matches the signature it sent in its last response, and
- verifies that the Next_slot command has been received within the time window specified in clause 7.5.5.

If the tag has met these acknowledge conditions it enters the Quiet state. Otherwise, it retains its current state.

The round continues until all slots have been explored.

During a round, the interrogator can suspend the round by sending a Standby_round command. The tag processes the command in a similar way to a Next_slot command, except that if the acknowledge conditions are met, the tag enters the Selected state. If they are not met, the tag enters the Round_standby state.

NOTE The Round_standby mechanism allows the interrogator to conduct a dialogue with a selected tag before continuing the round.

A tag, which has not been acknowledged by a next-slot OK or round-standby OK, keeps track of the slot count.

The interrogator keeps track of the slot count each time it issues a Close_slot or Next_slot command.

When the slot count equals the round-size specified in either of the Init_round or Init_round_all commands, the tag shall select a new slot in which to transmit, select a new random signature and enter a new round.

The detail of the tag state transition is specified for each command in tables detailed below.

7.5 Timing specifications

7.5.1 Timing specifications general

The interrogator and the tag shall comply with the following timing specifications.

7.5.2 Tag state storage

In the case of absent or insufficient energising field, the tag shall retain its state for at least 300µs.

In addition, if the tag is in the Quiet state, it shall retain its Quiet state for at least 2s.

NOTE Implementation of the Quiet state storage may imply that the tag will retain this condition during a time greater than 2s, up to several minutes in low temperature conditions. The Reset_to_ready command allows an exit of the Quiet state in these circumstances.

7.5.3 Forward link to return link handover

After the reception of an EOF from the interrogator, the tag shall wait a time Trs, that starts from the negative going edge of the EOF. See Figure 17 and Table 28.

Table 28 — Forward link to return link handover timing

Minimum	Maximum
150µs	1150µs

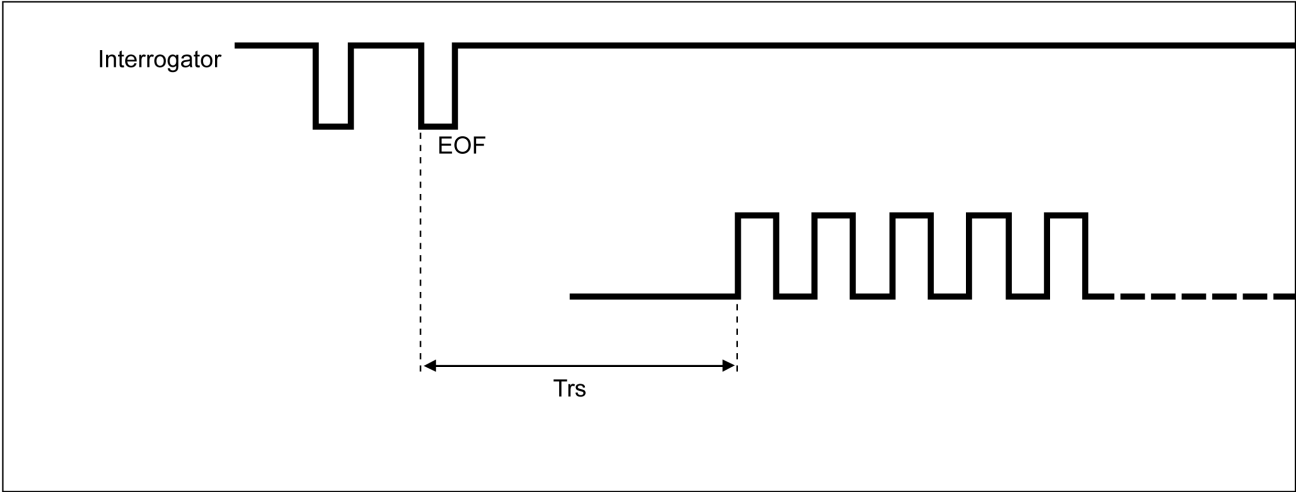


Figure 17 — Forward link to return link handover

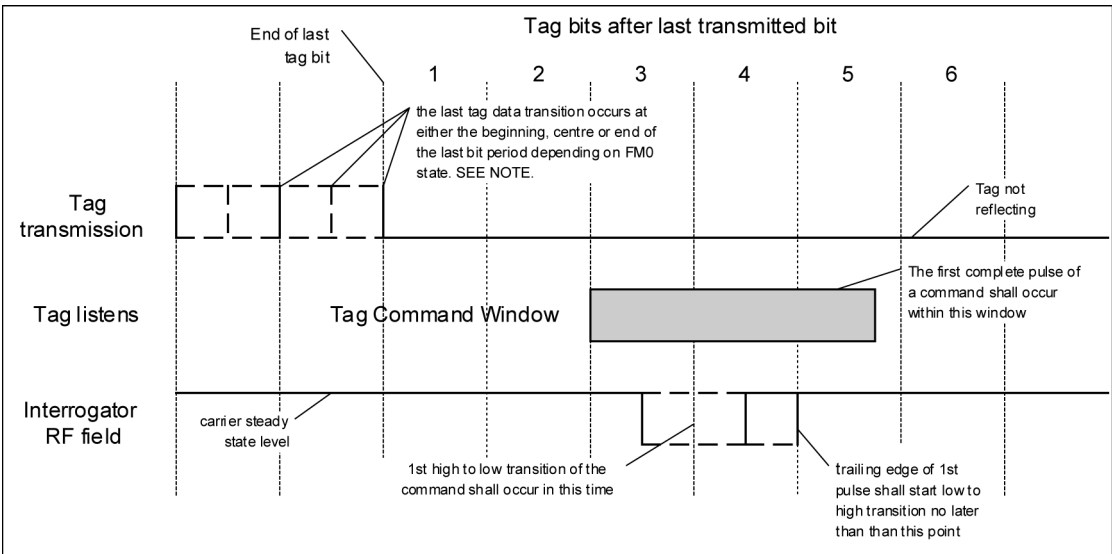
7.5.4 Return link to forward link handover

The protocol is half-duplex. The tag is not required to receive and decode properly a command from the interrogator while it is transmitting.

7.5.5 Acknowledgement time window

7.5.5.1 Acknowledgement time window general

The purpose of the acknowledgement time window, shown in Figure 18, is to ensure that only those tags that receive a synchronised command starting within a command window will respond to those commands. This reduces considerably the possibility that a tag may be incorrectly acknowledged during the collision arbitration or census process.



NOTE The four cases of the FM0 state are detailed in Figure 19.

Figure 18 — Acknowledgement time window

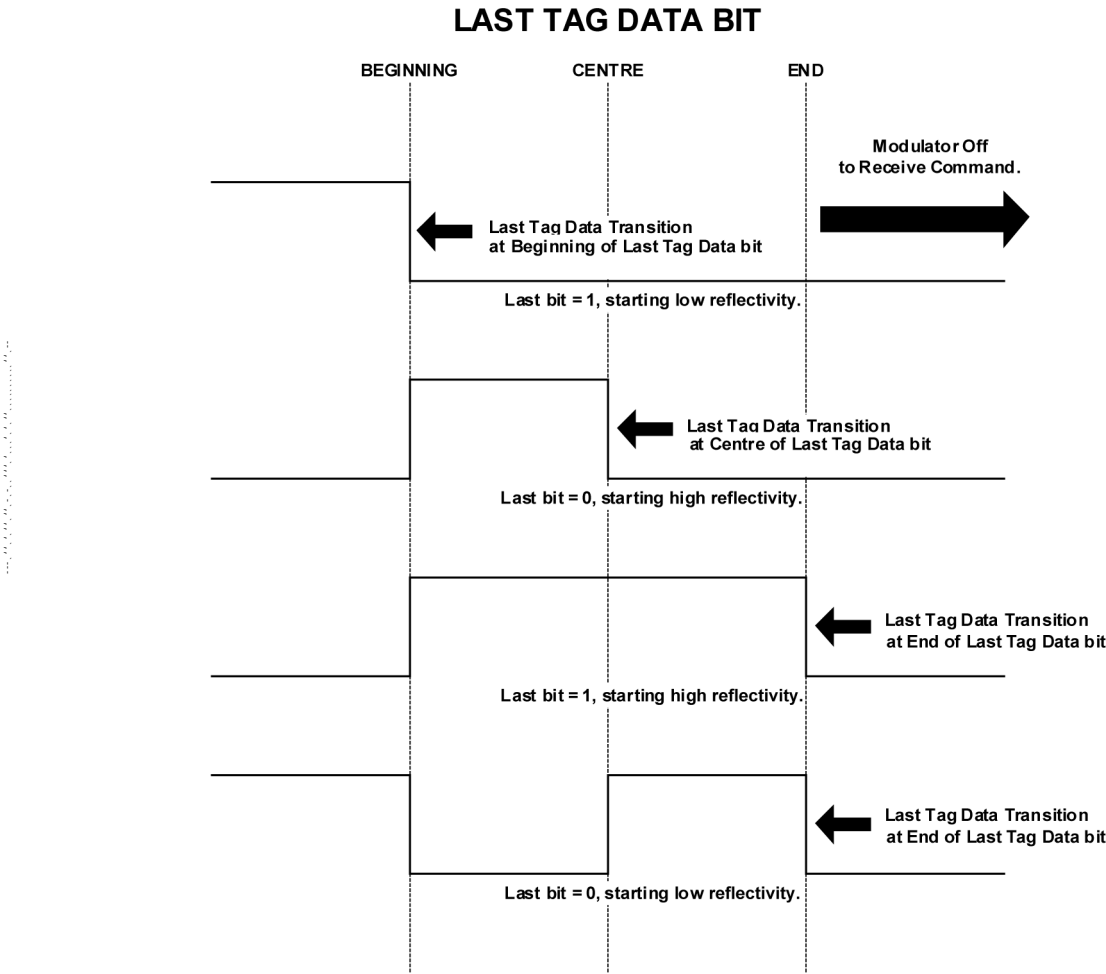


Figure 19 — Details of last FM0 tag data bit

7.5.5.2 Interrogator

The interrogator shall send a Next_slot or a Standby_round command (that acknowledges the answer of a specific tag) within a specific time window. The command window shall be open from the boundary between bits two and three, for a duration of 2.75 tag bit periods, see Figure 18.

The interrogator shall measure the tag clock frequency so that it can start a command in the time window.

The interrogator shall not modulate the carrier for at least 3 tag bit periods prior to sending the command start pulse.

The interrogator shall send the first negative going edge of the command frame within bit period '4'.

7.5.5.3 Tag

When a tag is in the Round active state, has just responded in the current slot and receives a Next_slot command, which was not received within the window specified in clause 7.5.5, the tag shall not move to the quiet state, but shall remain in the Round active state and shall increment its slot counter as specified in Clause 7.7.2 and Table 31.

When a tag is in the Round active state, has just responded in the current slot and receives a Standby_round command, which was not received within the window specified in clause 7.5.5, the tag shall not move to the selected state, but shall transition to the Round_standby state as specified in Clause 7.7.3 and Table 33.

7.6 Command format examples

For examples of the bit pattern for a short command, see 7.7.2 and for a long command see 7.8.6.

7.7 Mandatory commands

7.7.1 Mandatory commands general

There are four mandatory commands, command codes 02, 04, 06 and 0A. Command codes 00 and 0B to 0F are mandatory commands reserved for future use.

Interrogator shall implement all mandatory commands.

Tags shall implement all mandatory commands.

7.7.2 Next_slot

Command code = '02'

The Next_slot command has two functions:

- It acknowledges the tag that has been identified, or
- It instructs all tags in the Round_active state to switch to the next slot by incrementing their slot counter.

The command contains:

- The Next_slot command code, and
- The tag signature.

No flags are used by this command.

Table 29 shows the Next_slot command format.

Table 29 — Next_slot command format

Protocol extension	Next_slot	Tag signature	CRC-5
1 bit	6 bits	4 bits	5 bits

Table 30 is an example of a short command. The complete bit pattern for the Next_slot command, with a tag signature of 6, is as follows:

Table 30 — Example of a short command, Next_slot

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
	PE	Command Code Next slot =02						Tag Signature (eg = 6)				CRC-5 which for this case is 0					
SOF	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	EOF

There is no direct tag response to the Next_slot command. If the tag is in the Round_active state, it shall switch to the next slot by incrementing their slot counter and send its response if its slot counter matches the chosen slot.

The tag shall perform the actions specified in Table 31. It may either enter the Quiet state or switch to the next slot by incrementing its slot counter.

Table 31 — Tag state transition for Next_slot

Command : Next_slot			
Current state	Criteria	Action	New state
Ready	none	none	Ready
Quiet	none	none	Quiet
Selected	none	none	Quiet
Round_active	Tag has answered in previous slot, AND Signature matches AND Next_slot was received in the acknowledgement time window. See clause 7.5.5	none	Quiet
	Tag has not answered in previous slot, OR Signature does not match OR Next_slot was not received in the acknowledgement time window. See clause 7.5.5	The tag shall increment its slot counter and send its response if slot counter matches the chosen slot.	Round_active
Round_standby	none	The tag shall increment its slot counter and send its response if slot counter matches the chosen slot.	Round_active

7.7.3 Standby_round

Command code = '04'

The Standby_round command has two functions:

- It acknowledges a valid response from a tag and instructs this tag to enter the Select state (individual commands can then be sent to this tag such as Read and Write), or
- It instructs all other tags in the Round_active state to enter the Round_standby state. For these tags, their participation in the round is suspended. The round will be resumed by a Next_slot command, or a Close_slot command. In addition, a new round will be initiated by tags in the Round_standby state by a New_round command, an Init_round command or an Init_round_all command.

The command contains:

- The Standby_round command code, and
- The tag signature.

No flags are used by this command.

Table 32 shows the Standby_round command format.

Table 32 — Standby_round command format

Protocol extension	Standby_round	Tag signature	CRC-5
1 bit	6 bits	4 bits	5 bits

There is no tag response to the Standby_round command. The tag shall perform the algorithm described in clauses 7.4.10 and 7.4.11. It may either enter the Select state or the Round_standby state, as shown in Table 33.

Table 33 — Tag state transition for Standby_round

Command : Standby_round			
Current state	Criteria	Action	New state
Ready	None	None	Ready
Quiet	None	None	Quiet
Selected	None	None	Quiet
Round_active	Tag has answered in previous slot, AND Signature matches AND Next_slot was received in the acknowledgement time window. See clause 7.5.5	None	Selected
	Tag has not answered in previous slot, OR Signature does not match OR Next_slot was not received in the acknowledgement time window. See clause 7.5.5.	None	Round_standby
Round_standby	None	None	Round_standby

7.7.4 Reset_to_ready

Command code = '06'

The Reset_to_ready command has one function:

- It instructs all tags to enter the ready state.

The command contains:

- The Reset_to_ready command code, and
- Four RFU bits. These bits shall be set to zero.

On receiving the Reset_to_ready command, the tag shall reset to zero all stored parameters such as the AFI and the slot counter.

This command has the same effect as removing the tag from the RF field for an extended period of time long enough to reset the "quiet" memory and then returning it to the RF field.

Table 34 shows the Reset_to_ready command format.

Table 34 — Reset_to_ready command format

Protocol extension	Reset_to_ready	RFU	CRC-5
1 bit	6 bits	4 bits	5 bits

There is no response to the Reset_to_ready command.

The tag shall perform the actions specified in Table 35.

Table 35 — Tag state transition for Reset_to_ready

Command : Reset_to_ready			
Current state	Criteria	Action	New state
Ready	None	None	Ready
Quiet	None	None	Ready
Selected	None	None	Ready
Round_active	None	None	Ready
Round_standby	None	None	Ready

7.7.5 Init_round_all

Command code = '0A'

This command is functionally equivalent to the Init_round command, with AFI set to 00.

The command contains:

- The Init_round_all command code of 6 bits,
- The SUID Flag of 1 bit, and
- The Round size of 3 bits.

Table 36 shows the Init_round_all command format.

Table 36 — Init_round_all command format

Protocol extension	Init_round_all	SUID flag	Round size	CRC-5
1 bit	6 bits	1 bit	3 bits	5 bits

The response, as shown in Table 37 or Table 38 shall contain:

- The DSFID if the SUID flag is set in the command,
- The tag signature,
- The tag type (battery-less or battery-assisted),
- The battery status flags,
- The SUID if the SUID flag is set in the command, and
- The first n bits of the tag memory if the SUID flag is NOT set in the command.

If the tag detects an error, it shall remain silent.

Table 37 — Init_round_all response format when the SUID flag is NOT set

Preamble	Flags see 7.4.8	Tag type see Table 39	Battery status see Table 40	Signature	Random number See NOTES 1 and 2	First n bits of memory See NOTE 3	CRC-16
	2 bits	1 bit	1 bit	4 bits	8 bits	n = 32, 64, 96 or 128 bits	16 bits

NOTE 1 The purpose of the random number is to increase the probability of detecting a collision between two or more tags whose responses contain the same data. Combined with the 4 bit signature, it represents a 12 bits random number. It is possible that if the SUID flag is NOT set, and the tag returns the first “page” of data (could be 32,64,96 or 128 bits), multiple tags could be programmed with the same memory content. To ensure a collision is detected, the 4 bit signature is extended by 8 bits, giving 12 bits, in which a collision could be detected.

NOTE 2 The DSFID field is not included since in situations where the first “page” of memory is read, it is expected that no further exchange with the tag will take place. The DSFID can nevertheless be determined using other commands, see clause 7.8.7.

NOTE 3 The tag shall return the first n bits of user data according to its memory size, with a maximum of 128 bits. If there is no user memory, the tag shall return the SUID.

This part of ISO/IEC 18000 does not specify how the 8 bit random number shall be generated (It could for instance be the last 8 significant bits of the UID, or a random number, etc.).

The generation of the signature and the random number shall be independent of each other.

Table 38 — Init_round_all response format when the SUID flag is set

Preamble	Flags see 7.4.8	Tag type see Table 39	Battery status see Table 40	Signature	DSFID	SUID	CRC-16
	2 bits	1 bit	1 bit	4 bits	8 bits	40 bits	16 bits

NOTE The 8 bits random number is not required when an SUID is sent back since the collisions will be detected using the combination of the signature and SUID.

Table 39 shows the Tag type field, while Table 40 shows the Battery Status field.

Table 39 — Tag type (battery-assisted or not)

Tag type	Meaning
0	Tag is NOT battery-assisted
1	Tag is battery-assisted

Table 40 — Battery status (for battery-assisted tag)

Battery status	Meaning
0	Battery is low. A non-battery-assisted tag shall set this bit to 0.
1	Battery is good.

The tag shall perform the actions specified in Table 41.

Table 41 — Tag state transition for Init_round_all

Command : Init_round_all			
Current state	Criteria	Action	New state
Ready	None	The tag shall choose the slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active
Quiet	None	The tag shall discard the command and remain silent.	Quiet
Selected	None	The tag shall discard the command and transition to the Quiet state.	Quiet
Round_active	None	The tag shall reset the previously chosen slot and chose a new slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active
Round_standby	None	The tag shall reset the previously chosen slot and chose a new slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active

NOTE The slots are numbered from 1 to Round_size.

7.8 Optional commands

7.8.1 Optional commands general

The following commands are optionally supported by the tag.

If a tag in the selected state does not support the optional command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does not support the optional command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

All other tags shall remain silent.

7.8.2 Init_round

Command code = '01'

The command contains:

- The Init_round command code of 6 bits,
- The SUID flag of 1 bit,
- The Round size of 3 bits, and
- The AFI of 8 bits.

NOTE A tag that is in the Round_active state at the end of a round shall automatically enter a new round

Table 42 shows the Init_round command format.

Table 42 — Init_round command format

Protocol extension	Init_round	SUID	Round size	CRC-5	AFI	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	8 bits	16 bits

The response is the same as that specified for the Init_round_all command, see Clause 7.7.5, Table 37 and Table 38.

The tag shall perform the actions specified in Table 43.

Table 43 — Tag state transition for Init_round

Command : Init_round			
Current state	Criteria	Action	New state
Ready	AFI Matches	The tag shall choose the slot in which it will send its response by generating a random number. It shall reset its slot counter to 1. See NOTE.	Round_active
	AFI does not match	Tag ignores the command.	Ready
Quiet	None	The tag shall discard the command and remain silent.	Quiet
Selected	None	The tag shall discard the command and transition to the Quiet state.	Quiet
Round_active	AFI Matches	The tag shall reset the previously chosen slot and choose a new slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active
	AFI does not match	The tag shall discard the command and returns to the Ready State.	Ready
Round_standby	AFI Matches	The tag shall reset the previously chosen slot and choose a new slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active
	AFI does not match	The tag shall discard the command and returns to the Ready State	Ready

NOTE The slots are numbered from 1 to Round_size.

7.8.3 Close_slot

Command code = '03'

The Close_slot has one function, it instructs all tags in the Round_active state to switch to the next slot by incrementing their slot counter.

The Close_slot command shall be sent by the interrogator when no tag response has been received in the slot or when a collision has been detected. If the interrogator detects a collision, then the interrogator must wait until all tags have replied, and the tag Tx/Rx turnaround time has expired, before it can issue the Close_slot command.

The command contains:

- The Close_slot command code, and
- 4 RFU bits (these bits shall be set to 0).

Table 44 shows the Close_slot command format.

Table 44 — Close_slot command format

Protocol extension	Close_slot	RFU	CRC-5
1 bit	6 bits	4 bits	5 bits

There is no direct tag response to the Close_slot command. If the tag is in the Round_active state, it shall switch to the next slot by incrementing its slot counter and send its response if its slot counter matches the chosen slot.

The tag shall perform the actions specified in Table 45.

Table 45 — Tag state transition for Close_slot

Command : Close_slot			
Current state	Criteria	Action	New state
Ready	None	None	Ready
Quiet	None	None	Quiet
Selected	None	The tag shall discard the command and transition to the Quiet state.	Quiet
Round_active	None	The tag shall increment its slot counter and send its response if its slot counter matches the chosen slot.	Round_active
Round_standby	None	The tag shall increment its slot counter and send its response if its slot counter matches the chosen slot.	Round_active

7.8.4 New_round

Command code = '05'

The New_round command has two functions:

- It instructs the tags that are neither in the Quiet nor in the Selected state to enter a new round, to reset their slot counters to 1 and to enter the Round_active state (tags in the Ready state will not enter a new round), and
- It instructs the tag in the Selected state to enter the Quiet state (tags in the Ready or Quiet state shall remain in their prior states).

The command contains:

- The New_round command code,
- The SUID Flag, and
- The new round size.

All other parameters (such as the AFI/ASF) shall be the same as in the previous round.

Table 46 shows the New_round command format.

Table 46 — New_round command format

Protocol extension	New_round	SUID Flag	Round size	CRC-5
1 bit	6 bits	1 bit	3 bits	5 bits

Assuming that the SUID flag is the same, the response shall be the same as the response to the previous round, with the exception of the signature, that might be different, for instance if based on the TEL or a random-number

If the SUID = 0, the tag shall respond with the first page of user data (32,64,96 or 128 bits) as in Table 37.

If the SUID = 1, the tag shall respond with the SUID as in Table 38.

The tag shall perform the actions specified in Table 47.

Table 47 — Tag state transition for New_round

Command : New_round			
Current state	Criteria	Action	New state
Ready	None	None	Ready
Quiet	None	None	Quiet
Selected	None	None	Quiet
Round_active	None	The tag shall reset the previously chosen slot and chose a new slot in which it will send its response by generating a random number.	Round_active
Round_standby	None	The tag shall reset the previously chosen slot and chose a new slot in which it will send its response by generating a random number.	Round_active

7.8.5 Select (by SUID)

Command code = '07'

The Select command has two functions:

- It instructs the tag specified by its SUID to enter the Selected state from any other state (individual commands, (such as Read and Write), can then be sent to this tag), and
- It instructs all tags in the Round_active state with non-matching SUIDs to enter the Round_standby state. For these tags, their participation in the round is suspended. The round will be resumed following a Next_slot command or a Close_slot command or a new round will be initiated following a New_round command, Init_round or Init_round_all command.

On receiving the Select command:

- If the SUID matches the tag SUID, the tag shall enter the selected state and shall send a response, or
- If the SUID does not match the tag SUID, and if the tag is in the Round_active state, the tag shall enter the Round_standby state, and shall not send a response.

Command parameter:

SUID (mandatory)

Table 48 shows the Select command format.

Table 48 — Select command format

Protocol extension	Select	SUID (always set to 1)	RFU	CRC-5	SUID	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	16 bits

Response format:

If the tag with the matching SUID does not support the Select command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the tag with the matching SUID does support the Select command, and if the error flag is set, the response shall be according to Table 24.

If the tag with the matching SUID does support the Select command, and if the error flag is not set, the response shall be according to Table 25.

All other tags shall remain silent.

The tag shall perform the actions specified in Table 49.

Table 49 — Tag state transition for Select

Command: Select			
Current state	Criteria	Action	New state
Ready	The SUID does not match	None	Ready
	The SUID matches	The tag shall send back its response	Selected
Quiet	The SUID does not match	None	Quiet
	The SUID matches	The tag shall send back its response	Selected
Selected	The SUID does not match	None	Quiet
	The SUID matches	The tag shall send back its response	Selected
Round_active	The SUID does not match	None	Round_standby
	The SUID matches	The tag shall send back its response	Selected
Round_standby	The SUID does not match	None	Round_standby
	The SUID matches	The tag shall send back its response	Selected

7.8.6 Read_blocks

Command code = '08'

On receiving the Read_blocks command, the tag shall respond with the requested data and the block lock status.

The blocks are numbered from '00' to 'FF' (0 to 255).

The number of blocks in the command is one less than the number of blocks that the tag shall return in its response.

For example, a value of '06' in the "Number of blocks" field is a request to read 7 blocks. A value of '00' is a request to read a single block.

There are two formats of the Read_blocks command, one specifying the SUID, see Table 50, and the other that doesn't specify the SUID, see Table 51, but is used to read blocks from the tag in the selected state.

Table 50 — Read_blocks command format SUID specified

Protocol extension	Read_blocks	SUID flag = 1	RFU	CRC-5	SUID	First block number	Number of blocks	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	8 bits	8 bits	16 bits

Table 51 — Read_blocks command format SUID NOT specified

Protocol extension	Read_blocks	SUID flag = 0	RFU	CRC-5	First block number	Number of blocks	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	8 bits	8 bits	16 bits

Command parameter:

(Optional) SUID

First block number

Number of blocks

Table 52 gives an example of a long command showing the bit pattern for the Read_blocks command, with SUID flag = 1, SUID = FEDCBA4321 (hex), First Block = 27 (Hex), Number of blocks to read is 3. Total message length = 88 bits, plus SOF and EOF:

Table 52 — Example of long command, Read_blocks with SUID.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	PE	Command Code Read Blocks =08						*	NULL			CRC-5, in this example is 08				
SOF	0	0	0	1	0	0	0	1	0	0	0	0	1	0	0	0

* SUID Flag = 1 (set)

17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
SUID first 16 bits (40 to 25), In this example the complete SUID is FEDCBA4321 (Hex)															
1	1	1	1	1	1	1	0	1	1	0	1	1	1	0	0

33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
SUID next 16 bits (24 to 9)															
1	0	1	1	1	0	1	0	0	1	0	0	0	0	1	1

49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64
SUID last 8 bits (8 to 1)								First Block number, in this example = 27 hex							
0	0	1	0	0	0	0	1	0	0	1	0	0	1	1	1

65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80
Number of Blocks, in this example = 3								First 8 bits of 16 bit CRC-16 = 70							
0	0	0	0	0	0	1	1	0	1	1	1	0	0	0	0

81	82	83	84	85	86	87	88	
Last 8 bits of 16 bit CRC-16 = F9								
1	1	1	1	1	0	0	1	EOF

Response format:

If a tag in the selected state does not support the Read_blocks command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Read_blocks command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Read_blocks command, and if the error flag is NOT set, the response shall be according to Table 53.

If the command specifies an SUID and the tag with the matching SUID does not support the Read_blocks command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID, does support the Read_blocks command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID, does support the Read_blocks command, and if the error flag is NOT set, the response shall be according to Table 53.

All other tags shall remain silent.

Table 53 — Read_blocks response format when Error_flag is NOT set

Preamble	Flags see 7.4.8	Block security status	Data	CRC-16
	2 bits	2 bits	Block length	16 bits
		Repeated as needed		

The block security status field (2 bits) is defined in Table 17

The tag response shall be ordered such that the most significant bit of the most significant byte of data is transmitted first

Response parameter:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

if Error_flag is not set (the following order shall be respected in the tag response)

Block security status N

Block value N

Block security status N+1

Block value N+1

etc...

where N is the first requested (and returned) block.

The tag shall perform the actions specified in Table 54.

Table 54 — Tag state transition for Read_blocks

Command: Read_blocks			
Current state	Criteria	Action	New state
Ready	The SUID does not match	none	Ready
	The SUID matches	The tag shall send back its response	
Quiet	The SUID does not match	none	Quiet
	The SUID matches	The tag shall send back its response	
Selected	The SUID flag = 0	The tag shall send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	none	Round_active
	The SUID matches	The tag shall send back its response	
Round_standby	The SUID does not match	none	Round_standby
	The SUID matches	The tag shall send back its response	

7.8.7 Get_system_information

Command code = '09'

This command allows for retrieving the system information value from the tag.

There are two formats of the Get_system_information command, one specifying the SUID, shown in Table 55 and the other that doesn't specify the SUID, as shown in Table 56, but is used to get system information from the tag in the selected state.

Table 55 — Get_system_information command format with SUID

Protocol extension	Get_system_info	SUID flag = 1	RFU	CRC-5	SUID	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	16 bits

Table 56 — Get_system_information command format with no SUID

Protocol extension	Get_system_info	SUID flag = 0	RFU	CRC-5
1 bit	6 bits	1 bit	3 bits	5 bits

Command parameter:

(Optional) SUID

Response parameter:

If a tag in the selected state does not support the Get_system_information command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Get_system_information command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Get_system_information command, and if the error flag is NOT set, the response shall be according to Table 57.

If the command specifies an SUID and the tag with the matching SUID does not support the Get_system_information command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Get_system_information command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID, does support the Get_system_information command, and if the error flag is NOT set, the response shall be according to Table 57.

All other tags shall remain silent.

Table 57 — Get_system_information response format when Error_flag is NOT set

Preamble	Flags see clause 7.4.8	Tag type see Table 39	Battery status see Table 40	Signature	UID see NOTE 1	DSFID see NOTE 2	AFI see NOTE 3	System information see Table 58	CRC-16
	2 bits	1 bit	1 bit	4 bits	64 bits	8 bits	8 bits	32 bits	16 bits

NOTE 1 The complete UID on 64 bits shall be returned.

NOTE 2 The field DSFID may be present if and only if supported by the tag

NOTE 3 The field AFI may be present if and only if supported by the tag

Response format:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

If Error_flag is not set, the response contains:

- UID (mandatory),
- DSFID if supported and present,
- AFI if supported and present, and
- System Information, see Table 58.

Table 58 — System information definition

Bit	Flag name	Value	Description
b1	DSFID	0	DSFID is not supported. DSFID field is not present
		1	DSFID is supported. DSFID field is present
b2	AFI	0	AFI is not supported. AFI field is not present
		1	AFI is supported. AFI field is present
b3	Tag memory size	0	Information on tag memory size is not supported. Memory size field is not present.
		1	Information on tag memory size is supported. Memory size field is present.
b4	IC reference	0	Information on IC reference is not supported. IC reference field is not present.
		1	Information on IC reference is supported. IC reference field is present.
b5-b6	Tag sensitivity	00	Tag sensitivity is undetermined.
		01	Tag sensitivity is S1. See Table 59
		10	Tag sensitivity is S2. See Table 59
		11	Tag sensitivity is S3. See Table 59
b7-b8	Tag type	00	Tag is passive backscatter, not battery assisted.
		01	Tag is passive backscatter and battery assisted.
		10	Tag is active.
		11	RFU
b9	RFU	0	RFU
b10	RFU	0	RFU
b11-b16	IC reference		See b4 above, IC Reference is as defined by the IC manufacturer
b17-b32	Tag memory size		See Table 60

Table 59 — Tag sensitivity class

Sensitivity class	Minimum sensitivity (V/m)
S1	10
S2	4
S3	1.5

Table 60 — Tag memory size information

MSB		LSB
b32-b30	b29-b25	b24-b17
RFU	Block size in bytes	Number of blocks

Block size is expressed in number of bytes using 5 bits, allowing specification of up to 32 bytes i.e. 256 bits. It is one less than the actual number of bytes; e.g., a value of '1F' indicates 32 bytes, a value of '00' indicates 1 byte.

Number of blocks is on 8 bits, allowing specification of up to 256 blocks. It is one less than the actual number of blocks; e.g., a value of 'FF' indicates 256 blocks, a value of '00' indicates 1 block.

The three most significant bits are reserved for future use and shall be set to zero.

The IC reference is on 8 bits and its meaning is defined by the IC manufacturer.

The tag shall perform the actions specified in Table 61.

Table 61 — Tag state transition for Get_system_information

Command: Get_system_information			
Current state	Criteria	Action	New state
Ready	The SUID does not match	none	Ready
	The SUID matches	The tag shall send back its response	
Quiet	The SUID does not match	none	Quiet
	The SUID matches	The tag shall send back its response	
Selected	The SUID flag = 0	The tag shall send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	none	Round_active
	The SUID matches	The tag shall send back its response	
Round_standby	The SUID does not match	none	Round_standby
	The SUID matches	The tag shall send back its response	

7.8.8 Begin_round

Command code = '0B'

The Begin_round command, as shown in Table 62, causes all tags whose user data matches the mask starting with the MSB of the data up to the position of the mask length parameter to reply, as shown in Table 63, with their first 32,64,96 or 128 bits of user data (as appropriate) when it is their turn to do so during a collision arbitration, refer to the state transition Table 64.

When the mask length exceeds the size of the data field the tag shall not reply but shall move to the ready state unless the tag was in the quiet state

NOTE The SUID flag is always = 0 because the tag will not act on the UID if present, unless there is no user data, in which case the tag returns the SUID

Table 62 — Begin_round command format

Protocol extension	Begin_round	SUID (always = 0)	Round size	CRC-5	Mask length	Mask	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	8 bits	0 – 255 bits	16 bits

Command parameter:

Mask length

(optional) Mask

Command response

The response shall contain:

- The tag type (battery-less or battery-assisted),
- The battery status flags,
- The tag signature,
- The AFI, and
- The first 32, 64, 96 or 128 bits of the tag memory.

If the tag detects an error, it shall remain silent.

Table 63 — Begin_round response format

Preamble	Flags see 7.4.8.2	Tag type see Table 39	Battery status see Table 40	Signature	AFI	First n bits of memory	CRC-16
	2 bits	1 bit	1 bit	4 bits	8 bits	n = 32, 64, 96 or 128 bits	16 bits

NOTE 1 The tag shall return the first n bits of user data according to its memory size, with a maximum of 128 bits. If there is no user memory, the tag shall return the SUID.

NOTE 2 The selection mask in the begin round command operates on the tag data content which coincides with the tag reply starting at the boundary between first and second bytes ie. after the 4 bit signature value starting with the field currently marked AFI.

Table 64 — Tag state transition for Begin_round

Command : Begin_round			
Current state	Criteria	Action	New state
Ready	Mask length = 0 or the mask matches the appropriate part of the user data.	The tag shall choose the slot in which it will send its response by generating a random number. It shall reset its slot counter to 1. See NOTE.	Round_active
	Mask length > 0 and the mask does not match	The tag shall ignore the command and remain in the ready state.	Ready
Quiet	None	The tag shall discard the command and remain silent.	Quiet
Selected	None	The tag shall discard the command and transition to the Quiet state	Quiet
Round_active	Mask length = 0 or the mask matches the appropriate part of the user data.	The tag shall reset the previously chosen slot and choose a new slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active
	Mask length > 0 and the mask does not match	The tag shall ignore the command and shall move to the ready state.	Ready
Round_standby	Mask length = 0 or the mask matches the appropriate part of the user data.	The tag shall reset the previously chosen slot and choose a new slot in which it will send its response by generating a random number. It shall reset its slot counter to 1.	Round_active
	Mask length > 0 and the mask does not match	The tag shall ignore the command and shall move to the ready state.	Ready

NOTE The slots are numbered from 1 to Round_size.

7.8.9 Write_single_block

Command code = '10'

On receiving the Write_single_block command, the tag shall write the requested block with the data contained in the command and report the success of the operation in the response.

There are two formats of the Write_single_block command, one specifying the SUID, as shown in Table 65 and the other that doesn't specify the SUID, as shown in Table 66, but is used to write block data to the tag in the selected state.

Table 65 — Write_single_block command format, with SUID

Protocol extension	Write_single_block	SUID flag = 1	RFU	CRC-5	SUID	Block number	Data	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	8 bits	n bits	16 bits

Table 66 — Write_single_block command format, with no SUID

Protocol extension	Write_single_block	SUID flag = 0	RFU	CRC-5	Block number	Data	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	8 bits	n bits	16 bits

In Table 65 and Table 66 n is the block size. Currently, the block size is 32 bits, however, the protocol allows a block size of up to 256 bits.

The Data field Table 65 and Table 66 shall be ordered such that the most significant bit of the most significant byte of data is transmitted first

Command parameter:

(Optional) SUID

Block number

Data

Command response

If a tag in the selected state does not support the Write_single_block command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Write_single_block command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Write_single_block command, and if the error flag is NOT set, the response shall be according to Table 25.

If the command specifies an SUID and the tag with the matching SUID, does not support the Write_single_block command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Write_single_block command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID does support the Write_single_block command, and if the error flag is NOT set, the response shall be according to Table 25.

All other tags shall remain silent.

Response parameter:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

The tag shall perform the actions specified in Table 67.

Table 67 — Tag state transition for Write_single_block

Command: Write_single_block			
Current state	Criteria	Action	New state
Ready	The SUID does not match	none	Ready
	The SUID matches	The tag shall process the command and send back its response	
Quiet	The SUID does not match	none	Quiet
	The SUID matches	The tag shall process the command and send back its response	
Selected	The SUID flag = 0	The tag shall process the command and send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall process the command and send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	none	Round_active
	The SUID matches	The tag shall process the command and send back its response	
Round_standby	The SUID does not match	none	Round_standby
	The SUID matches	The tag shall process the command and send back its response	

7.8.10 Write_multiple_blocks

Command code = '11'

On receiving the Write_multiple_block command, the tag shall write the requested block(s) with the data contained in the command and report the success of the operation in the response.

The blocks are numbered from '00' to 'FF' (0 to 255).

The number of blocks in the command is one less than the number of blocks that the tag shall write.

For example, a value of '06' in the "Number of blocks" field requests to write 7 blocks. The "Data" field shall contain 7 blocks. A value of '00' in the "Number of blocks" field requests to write 1 block. The "Data" field shall contain 1 block.

There are two formats of the Write_multiple_blocks command, one specifying the SUID, as shown in Table 68 and the other that doesn't specify the SUID, as shown in Table 69, but is used to write multiple blocks to the tag in the selected state.

Table 68 — Write_multiple_blocks command format – with SUID

PE	Write_multiple_block	SUID flag = 1	RFU	CRC-5	SUID	First block number	Number of blocks	Data	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	8 bits	8 bits	Block length	16 bits
								Repeated as needed	

Table 69 — Write_multiple_blocks command format – with no SUID

PE	Write_multiple_block	SUID flag = 0	RFU	CRC-5	First block number	Number of blocks	Data	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	8 bits	8 bits	Block length	16 bits
							Repeated as needed	

In Table 68 and Table 69 the Block length is the block size. Currently, the block size is 32 bits, however, the protocol allows a block size of up to 256 bits. The data is repeated for the number of blocks. The Data field shall be ordered such that the most significant bit of the most significant byte of the first block is transmitted first, with subsequent blocks following.

Command parameter:

(Optional) UID

First block number

Number of blocks

Block data (repeated as defined in the NOTE after Table 69).

Command response

If a tag in the selected state does not support the Write_multiple_blocks command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Write_multiple_blocks command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Write_multiple_blocks command, and if the error flag is NOT set, the response shall be according to Table 25.

If the command specifies an SUID and the tag with the matching SUID does not support the Write_multiple_blocks command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Write_multiple_blocks command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID does support the Write_multiple_blocks command, and if the error flag is NOT set, the response shall be according to Table 25.

All other tags shall remain silent.

Response parameter:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

The tag shall perform the actions specified in Table 70.

Table 70 — Tag state transition for Write_multiple_blocks

Command: Write_multiple_blocks			
Current state	Criteria	Action	New state
Ready	The SUID does not match	none	Ready
	The SUID matches	The tag shall process the command and send back its response	
Quiet	The SUID does not match	none	Quiet
	The SUID matches	The tag shall process the command and send back its response	
Selected	The SUID flag = 0	The tag shall process the command and send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall process the command and send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	none	Round_active
	The SUID matches	The tag shall process the command and send back its response	
Round_standby	The SUID does not match	none	Round_standby
	The SUID matches	The tag shall process the command and send back its response	

7.8.11 Lock_blocks**Command code = '12'**

On receiving the Lock_blocks command, the tag shall lock permanently the requested block.

The tag “user lock information” is held in blocks of 32 bits. Each block is addressed by means of the command parameter “Lock Block Number”. Each bit in a lock block represents a block of data in the tag; the least significant bit of the lock block representing the lowest order block of tag data (lock bit 1 = tag user block 1). The protocol allows up to 256 blocks, with each block being up to 256 bits. The lock block command specifies a 32 bit quantity to write into the lock area, ie specifies 32 blocks out of the possible 256 blocks. The lock_block_number parameter (3 bits) allows the user to write the 32 bit lock data to one of eight 32 bit lock areas. Eight areas of 32 bits each, equals 256 lock areas representing the 256 blocks. To lock a tag data block, a bit is set in the “block lock data” field when writing to the tag. To not lock a block of data, the corresponding bit in the “block lock data” field shall contain a value ‘0’ (not set). Once a block has been locked, the set condition of the lock bit cannot be reversed.

NOTE The tag "factory lock bits" can only be locked by a proprietary command.

There are two formats of the Lock_blocks command, one specifying the SUID, as shown in Table 71 and the other that doesn't specify the SUID, as shown in Table 72, but is used to lock a block of the tag in the selected state.

Table 71— Lock_blocks command format – with SUID

Protocol extension	Lock_block	SUID Flag = 1	Lock block number	CRC-5	SUID	Block lock data	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	32 bits	16 bits

Table 72 — Lock_blocks command format – with no SUID

Protocol extension	Lock_block	SUID Flag = 0	Lock block number	CRC-5	Block lock data	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	32 bits	16 bits

Command parameter:

(Optional) SUID

Block number

Command response:

If a tag in the selected state does not support the Lock_blocks command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Lock_blocks command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Lock_blocks command, and if the error flag is NOT set, the response shall be according to Table 25.

If the command specifies an SUID and the tag with the matching SUID does not support the Lock_blocks command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Lock_blocks command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID does support the Lock_blocks command, and if the error flag is NOT set, the response shall be according to Table 25.

All other tags shall remain silent.

Response parameter:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

The tag shall perform the actions specified in Table 73.

Table 73 — Tag state transition for Lock_blocks

Command : Lock_blocks			
Current state	Criteria	Action	New state
Ready	The SUID does not match	none	Ready
	The SUID matches	The tag shall process the command and send back its response	
Quiet	The SUID does not match	none	Quiet
	The SUID matches	The tag shall process the command and send back its response	
Selected	The SUID flag = 0	The tag shall process the command and send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall process the command and send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	none	Round_active
	The SUID matches	The tag shall process the command and send back its response	
Round_standby	The SUID does not match	none	Round_standby
	The SUID matches	The tag shall process the command and send back its response	

7.8.12 Write_AFI

Command code = '13'

On receiving the Write_AFI command, the tag shall write the AFI/ASF value into its memory.

There are two formats of the Write_AFI command, one specifying the SUID, as shown in Table 74 and the other that doesn't specify the SUID, as shown in Table 75, but is used to write the AFI/ASF of the tag in the selected state.

Table 74 — Write_AFI command format- with SUID

Protocol extension	Write_AFI	SUID flag = 1	RFU	CRC-5	SUID	AFI	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	8 bits	16 bits

Table 75 — Write_AFI command format- with no SUID

Protocol extension	Write_AFI	SUID flag = 0	RFU	CRC-5	AFI	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	8 bits	16 bits

Command parameter:

(Optional) SUID

AFI

Command response:

If a tag in the selected state does not support the Write_AFI command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Write_AFI command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Write_AFI command, and if the error flag is NOT set, the response shall be according to Table 25.

If the command specifies an SUID and the tag with the matching SUID does not support the Write_AFI command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Write_AFI command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID does support the Write_AFI command, and if the error flag is NOT set, the response shall be according to Table 25.

All other tags shall remain silent.

Response parameter:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

The tag shall perform the actions specified in Table 76.

Table 76 — Tag state transition for Write_AFI

Command : Write_AFI			
Current state	Criteria	Action	New state
Ready	The SUID does not match	none	Ready
	The SUID matches	The tag shall process the command and send back its response	
Quiet	The SUID does not match	none	Quiet
	The SUID matches	The tag shall process the command and send back its response	
Selected	The SUID flag = 0	The tag shall process the command and send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall process the command and send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	none	Round_active
	The SUID matches	The tag shall process the command and send back its response	
Round_standby	The SUID does not match	none	Round_standby
	The SUID matches	The tag shall process the command and send back its response	

7.8.13 Lock_AFI

Command code = '14'

On receiving the Lock_AFI command, the tag shall lock the AFI/ASF value permanently into its memory.

There are two formats of the Lock_AFI command, one specifying the SUID, as shown in Table 77 and the other that doesn't specify the SUID, as shown in Table 78, but is used to lock the AFI/ASF of the tag in the selected state.

Table 77 — Lock_AFI command format – with SUID

Protocol extension	Lock_AFI	SUID Flag = 1	RFU	CRC-5	SUID	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	16 bits

Table 78 — Lock_AFI command format – with no SUID

Protocol extension	Lock_AFI	SUID Flag = 0	RFU	CRC-5
1 bit	6 bits	1 bit	3 bits	5 bits

Command parameter:

(Optional) SUID

Command response:

If a tag in the selected state does not support the Lock_AFI command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Lock_AFI command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Lock_AFI command, and if the error flag is NOT set, the response shall be according to Table 25.

If the command specifies an SUID and the tag with the matching SUID does not support the Lock_AFI command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Lock_AFI command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID does support the Lock_AFI command, and if the error flag is NOT set, the response shall be according to Table 25.

All other tags shall remain silent.

Response parameter:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

The tag shall perform the actions specified in Table 79.

Table 79 — Tag state transition for Lock_AFI

Command : Lock_AFI			
Current state	Criteria	Action	New state
Ready	The SUID does not match	none	Ready
	The SUID matches	The tag shall process the command and send back its response	
Quiet	The SUID does not match	none	Quiet
	The SUID matches	The tag shall process the command and send back its response	
Selected	The SUID flag = 0	The tag shall process the command and send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall process the command and send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	none	Round_active
	The SUID matches	The tag shall process the command and send back its response	
Round_standby	The SUID does not match	none	Round_standby
	The SUID matches	The tag shall process the command and send back its response	

7.8.14 Write_DSFD command

Command code = '15'

On receiving the Write_DSFD command, the tag shall write the DSFD value into its memory.

There are two formats of the Write_DSFD command, one specifying the SUID, as shown in Table 80 and the other that doesn't specify the SUID, as shown in Table 81, but is used to write the DSFD of the tag in the selected state.

Table 80 — Write DSFD command format – with SUID

Protocol extension	Write_DSFD	SUID flag = 1	RFU	CRC-5	SUID	DSFD	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	8 bits	16 bits

Table 81 — Write DSFID command format – with no SUID

Protocol extension	Write DSFID	SUID flag = 0	RFU	CRC-5	DSFID	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	8 bits	16 bits

Command parameter:

(Optional) SUID

DSFID

Command response:

If a tag in the selected state does not support the Write_DSFID command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Write_DSFID command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Write_DSFID command, and if the error flag is NOT set, the response shall be according to Table 25.

If the command specifies an SUID and the tag with the matching SUID does not support the Write_DSFID command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Write_DSFID command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID does support the Write_DSFID command, and if the error flag is NOT set, the response shall be according to Table 25.

All other tags shall remain silent.

Response parameter:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

The tag shall perform the actions specified in Table 82.

Table 82 — Tag state transition for Write DSFID

Command : Write_DSFID			
Current state	Criteria	Action	New state
Ready	The SUID does not match	none	Ready
	The SUID matches	The tag shall process the command and send back its response	
Quiet	The SUID does not match	none	Quiet
	The SUID matches	The tag shall process the command and send back its response	
Selected	The SUID flag = 0	The tag shall process the command and send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall process the command and send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	none	Round_active
	The SUID matches	The tag shall process the command and send back its response	
Round_standby	The SUID does not match	none	Round_standby
	The SUID matches	The tag shall process the command and send back its response	

7.8.15 Lock_DSFIID

Command code = '16'

On receiving the Lock_DSFIID command, the tag shall lock the DSFIID value permanently into its memory.

There are two formats of the Lock_DSFIID command, one specifying the SUID, as shown in Table 83 and the other that doesn't specify the SUID, as shown in Table 84, but is used to lock the DSFIID of the tag in the selected state.

Table 83 — Lock DSFIID command format – with SUID

Protocol extension	Lock_DSFIID	SUID flag = 1	RFU	CRC-5	SUID	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	16 bits

Table 84 — Lock_DSFD command format – with no SUID

Protocol extension	Lock_DSFD	SUID flag = 0	RFU	CRC-5
1 bit	6 bits	1 bit	3 bits	5 bits

Command parameter:

(Optional) SUID

Command response:

If a tag in the selected state does not support the Lock_DSFD command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Lock_DSFD command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Lock_DSFD command, and if the error flag is NOT set, the response shall be according to Table 25.

If the command specifies an SUID and the tag with the matching SUID does not support the Lock_DSFD command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Lock_DSFD command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID does support the Lock_DSFD command, and if the error flag is NOT set, the response shall be according to Table 25.

All other tags shall remain silent.

Response parameter:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

The tag shall perform the actions specified in Table 85.

Table 85 — Tag state transition for Lock_DSFD

Command : Lock_DSFD			
Current state	Criteria	Action	New state
Ready	The SUID does not match	none	Ready
	The SUID matches	The tag shall process the command and send back its response	
Quiet	The SUID does not match	none	Quiet
	The SUID matches	The tag shall process the command and send back its response	
Selected	The SUID flag = 0	The tag shall process the command and send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall process the command and send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	none	Round_active
	The SUID matches	The tag shall process the command and send back its response	
Round_standby	The SUID does not match	none	Round_standby
	The SUID matches	The tag shall process the command and send back its response	

7.8.16 Get_blocks_lock_status

Command code = '17'

On receiving the Get_blocks_lock_status command, the tag shall send back the lock status of the requested blocks.

The Lock block number is numbered from '0' to '7', see Clause 7.8.11.

There are two formats of the Get_block_lock_status command, one specifying the SUID, as shown in Table 86 and the other that doesn't specify the SUID, as shown in Table 87, but is used to get the block lock status of the tag in the selected state.

Table 86 — Get_blocks_lock_status command format, with SUID

Protocol extension	Get_blocks_lock_status	SUID flag = 1	Lock block number	CRC-5	SUID	CRC-16
1 bit	6 bits	1 bit	3 bits	5 bits	40 bits	16 bits

Table 87 — Get_blocks_lock_status command format, with no SUID

Protocol extension	Get_blocks_lock_status	SUID flag = 0	Lock block number	CRC-5
1 bit	6 bits	1 bit	3 bits	5 bits

Command parameter:

(Optional) SUID

First block number

Number of blocks

Command response:

If a tag in the selected state does not support the Get_blocks_lock_status command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Get_blocks_lock_status command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Get_blocks_lock_status command, and if the error flag is NOT set, the response shall be according to Table 88.

If the command specifies an SUID and the tag with the matching SUID does not support the Get_blocks_lock_status command it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Get_blocks_lock_status command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID does support the Get_blocks_lock_status command, and if the error flag is NOT set, the response shall be according to Table 88.

All other tags shall remain silent.

Response parameter:

Error_flag (and Error code if Error_flag is set), see clause 7.4.8.2

If Error_flag is not set

Table 88 — Get_blocks_lock_status response format when Error_flag is NOT set

Preamble	Flags see 7.4.8	Tag type see Table 39	Battery status see Table 40	Signature	SUID	User lock information see Table 89	Factory lock information see Table 89	CRC-16
	2 bits	1 bit	1 bit	4 bits	40 bits	32 bits	32 bits	16 bits

The Lock information comprises 2 bits per block, as defined in clause 7.3.3 and Table 17. Each 32 bit Lock information field contains the lock status of 32 consecutive blocks, starting from the first block number as defined in the command. If the memory does not contain the specified/requested block, the field is zero filled.

Table 89 — Lock information

MSB		LSB	
b32	b31	...	b1
Lock info of 32nd block	Lock info of 31st block		Lock info of first block

The tag shall perform the actions specified in Table 90.

Table 90 — Tag state transition for Get-blocks_lock_status

Command: Get_blocks_lock_status			
Current state	Criteria	Action	New state
Ready	The SUID does not match	None	Ready
	The SUID matches	The tag shall process the command and send back its response	
Quiet	The SUID does not match	None	Quiet
	The SUID matches	The tag shall process the command and send back its response	
Selected	The SUID flag = 0	The tag shall process the command and send back its response	Selected
	The SUID flag = 1 and the SUID matches	The tag shall process the command and send back its response	
	The SUID flag = 1 and the SUID does not match	none	
Round_active	The SUID does not match	None	Round_active
	The SUID matches	The tag shall process the command and send back its response	
Round_standby	The SUID does not match	None	Round_standby
	The SUID matches	The tag shall process the command and send back its response	

7.9 Custom commands

The format of custom command, as shown in Table 91, is generic and allows unambiguous determination of custom command codes by each tag IC manufacturer.

The custom command code is the combination of a custom command code and of the tag IC manufacturer code.

The custom command parameters definition and the state transition diagram are the responsibility of the tag manufacturer.

Table 91 — Custom command format

Protocol extension	Custom	Flags	IC mfg code	Custom command parameters
1 bit	6 bits	4 bits	8 bits	Custom defined

Command parameter:

IC manufacturer code according to ISO/IEC 7816-6.

Command response:

If a tag in the selected state does not support the Custom command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If a tag in the selected state does support the Custom command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If a tag in the selected state does support the Custom command, and if the error flag is NOT set, the response shall be according to Table 92.

If the command specifies an SUID and the tag with the matching SUID does not support the Custom command, it shall return the error code "command is not supported", error code 1, see clauses 7.4.8.2, 7.4.8.3 and Table 27.

If the command specifies an SUID and the tag with the matching SUID does support the Custom command, and if the error flag is set, the response shall be according to clause 7.4.8.2 and Table 24.

If the command specifies an SUID and the tag with the matching SUID does support the Custom command, and if the error flag is NOT set, the response shall be according to Table 92.

All other tags shall remain silent.

Table 92 — Custom response format when Error_flag is NOT set

Flags see clause 7.4.8.2	Custom response parameters
2 bits	Custom defined

Response parameter:

Error_flag (and code if Error_flag is set), see clause 7.4.8.2

If Error_flag is not set

Custom parameters

7.10 Proprietary commands

The format of these commands is not defined in this part of ISO/IEC 18000.

8 Type B

8.1 Physical layer and data coding

8.1.1 Forward link

8.1.1.1 Carrier modulation

The data transmission from the interrogator to the tag is achieved by modulation of the carrier (ASK). The data coding is performed by generating pulses that create a Manchester coding, as shown in Figure 20, Figure 21, Table 93 and Table 94.

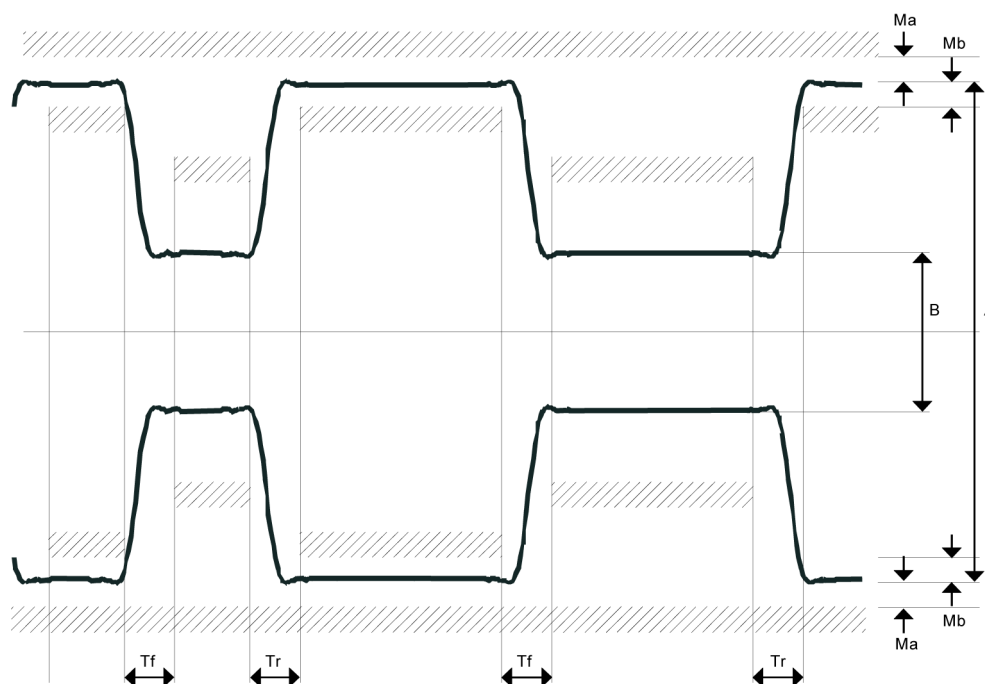
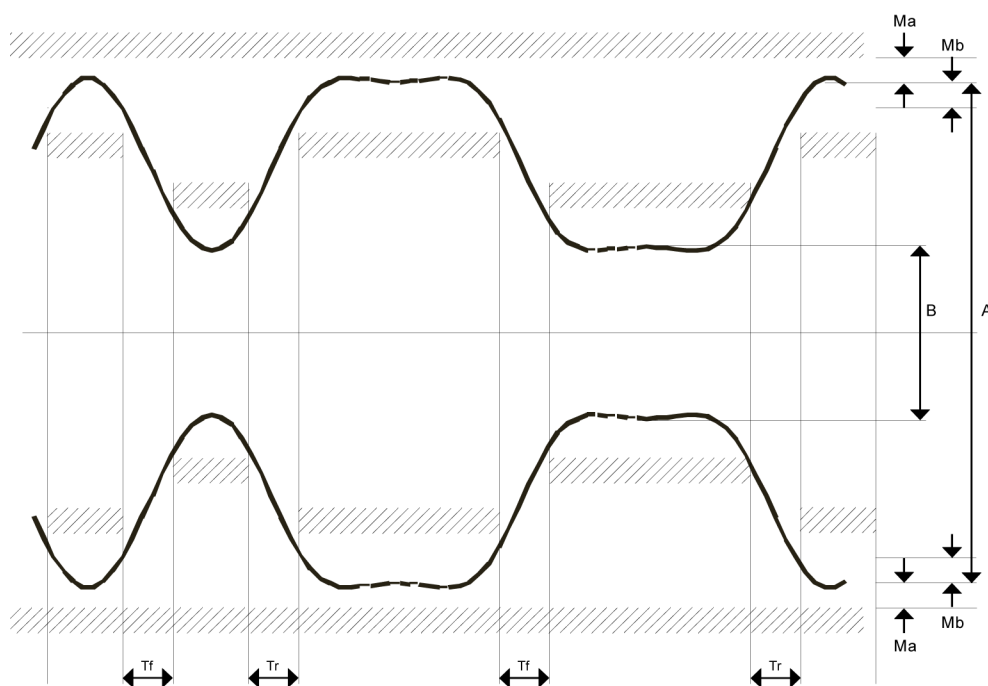


Figure 20 — 100 % modulation (example of 40 kbit/s signal)

Table 93 — Parameter for 100% modulation

Parameter	Minimum	Nominal	Maximum
$M = (A-B)/(A+B)$	90	100	100
Ma	0		0.03 (A-B)
Mb	0		0.03 (A-B)
Tr	0 μ s	1.8 μ s	0.1 / f _{Datarate}
Tf	0 μ s	1.8 μ s	0.1 / f _{Datarate}

NOTE Tr and Tf measured from 10% (A-B) to 90% (A-B)

**Figure 21 — 18% modulation (example of 8 kbit/s signal)****Table 94 — Parameter for 18% modulation**

Parameter	Minimum	Nominal	Maximum
$M = (A-B)/(A+B)$	15%	18%	20%
Ma	0		0.05 (A-B)
Mb	0		0.05 (A-B)
Tr	0 μ s		0.17 / f _{Datarate}
Tf	0 μ s		0.17 / f _{Datarate}

NOTE Tr and Tf measured from 10% (A-B) to 90% (A-B)

8.1.1.2 Bit coding of forward link fields

Data is Manchester encoded as per Figure 22.

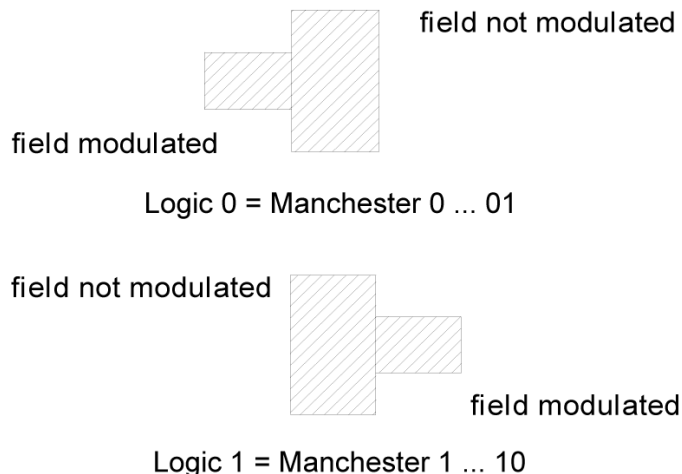


Figure 22 — Forward link bit coding

8.1.2 Return link

See clause 6.5.

8.1.3 Protocol concept

Data is encoded and presented in slightly different ways in the constituent fields. For interrogator-to-tag communication (forward link), data is sent using an on-off key format. The radio frequency field being on corresponds to 1, while the radio frequency field being off corresponds to 0. The modulation index specification is defined in clause 8.1.1.1. In the case of Manchester coding a Manchester 1 is a 1 to 0 transition, while a Manchester 0 is a 0 to 1 transition.

For tag-to-interrogator communication (return link), data is sent using backscatter techniques. This requires that the interrogator provide steady power to the tag during the return link. While the interrogator powers the tag, the tag shall change alternately the effective impedance of the tag front end and thus changing the overall radio frequency reflectivity of the tag as seen by the interrogator. During this time, the interrogator shall not modulate the carrier. During the WAIT field (when tags write data into their memory), the interrogator shall also provide steady power to the tag, and shall not modulate the carrier. The transmission protocol defines the mechanism to exchange instructions and data between the interrogator and the tag, in both directions.

It is based on the concept of "interrogator talks first".

This means that any tag shall not start transmitting (modulating) unless it has received and properly decoded an instruction sent by the interrogator.

The protocol is based on an exchange of a command from the interrogator to the tag and a response from the tag(s) to the interrogator.

The conditions under which the tag sends a response are defined in clause 8.2.7

Each command and each response are contained in a frame. The respective frames are specified in clauses 8.1.4 and 8.1.5. Each command consists of the following fields:

- Preamble Detect (no modulation of the RF carrier),
- Preamble,
- Delimiter,

- Command code,
- Parameter fields (depending on the command),
- Application data fields (depending on the command), and
- CRC-16.

Each response consists of the following fields:

- Quiet (no modulation of the RF carrier),
- Return Preamble,
- Application data fields, and
- CRC-16.

The protocol is bit-oriented. The number of bits transmitted in a frame is a multiple of eight (8), i.e. an integer number of bytes. However, the frame itself is not based on an integer number of bytes, to support the frame detection.

In all byte fields, the MSB shall be transmitted first, proceeding to the LSB. In all word (8-byte) data fields, the MSByte shall be transmitted first.

The MSByte shall be the byte at the specified address. The LSByte shall be the byte at the specified address plus 7 (i.e., bytes are transmitted in incrementing address order).

The byte significance is relevant to data transmission and to the GROUP_SELECT and GROUP_UNSELECT greater than and less than comparisons.

The MSByte of the byte mask shall correspond to the most significant data byte, the byte at the specified address.

Word (8-byte) addresses are not required to be on an 8-word boundary and may be on any byte boundary.

RFU bits and bytes shall be set to zero (0).

8.1.4 Command format

8.1.4.1 Command format general

The command frame, as shown in Figure 23, consist of the following fields:

- Preamble Detect ,
- Preamble,
- Delimiter,
- Command,
- Parameter (and data fields), and
- CRC-16.

Preamble Detect	Preamble	Delimiter	Command	Parameter	Data	CRC-16
-----------------	----------	-----------	---------	-----------	------	--------

Figure 23 — General command format

8.1.4.2 PREAMBLE DETECT field

The preamble detect field consist of a steady carrier (no modulation) during a time of at least 400 µs. This corresponds to 16 bits for a communication rate of 40 kbit/s.

8.1.4.3 PREAMBLE

The preamble is equivalent to 9 bits of Manchester 0 in NRZ format.

0101010101010101

8.1.4.4 Delimiters

8.1.4.4.1 Delimiters general

Four delimiters are defined.

8.1.4.4.2 Start delimiter 1

In NRZ format; includes Manchester errors; spaces ignored

11 00 11 10 10 - Delimiter 1

8.1.4.4.3 Start delimiter 2

In NRZ format; includes Manchester errors; spaces ignored

01 01 11 00 11 - Delimiter 2

Reserved for future use

8.1.4.4.4 Start delimiter 3

In NRZ format; includes Manchester errors; spaces ignored

00 11 10 01 01 - Delimiter 3

Reserved for future use

8.1.4.4.5 Start delimiter 4

In NRZ format; includes Manchester errors; spaces ignored

11 01 11 00 10 1 - Delimiter 4

Delimiter4 supports all commands as delimiter 1, however the return data rate is 4 times the forward link data rate. The supported data rates are defined in clause 5.2.

8.1.4.5 CRC-16

See clauses 6.5.7.3, 6.5.7.4 and Annex A.

8.1.5 Response format

8.1.5.1 Response format general

The response, as shown in Figure 24, consists of the following fields:

- Quiet,
- Return Preamble,
- Data fields, and
- CRC-16.

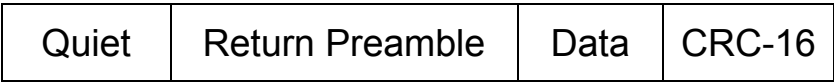


Figure 24 — General response format

The tag shall use a backscatter technique to communicate data to the interrogator. The interrogator shall be steadily powering the tag as well as listening to the tag response throughout the tag-to-interrogator (backscatter) communication. This applies to all fields in the return link.

8.1.5.2 QUIET

The tag shall not backscatter for $16 \cdot T_{\text{return data rate}} - 0,75 \cdot T_{\text{forward data rate}}$. The duration of the quiet time is determined by the communication speed of the return link.

8.1.5.3 CRC-16

See clauses 6.5.7.3, 6.5.7.4 and Annex A.

8.1.6 WAIT

When a tag receives a write command, it shall execute a write operation. (The details of the conditions under which a write will occur are described in clause 8.2.7.9.11. If a write operation is executed, the final field in the overall field sequence shall always be WAIT.

During the WAIT field, when the tag is writing data into the EEPROM, the interrogator must steadily power the tag. On-off key data shall not be sent during this time.

8.1.7 Examples of a command packet

Examples of command packets are given in Figure 25 and Figure 26.

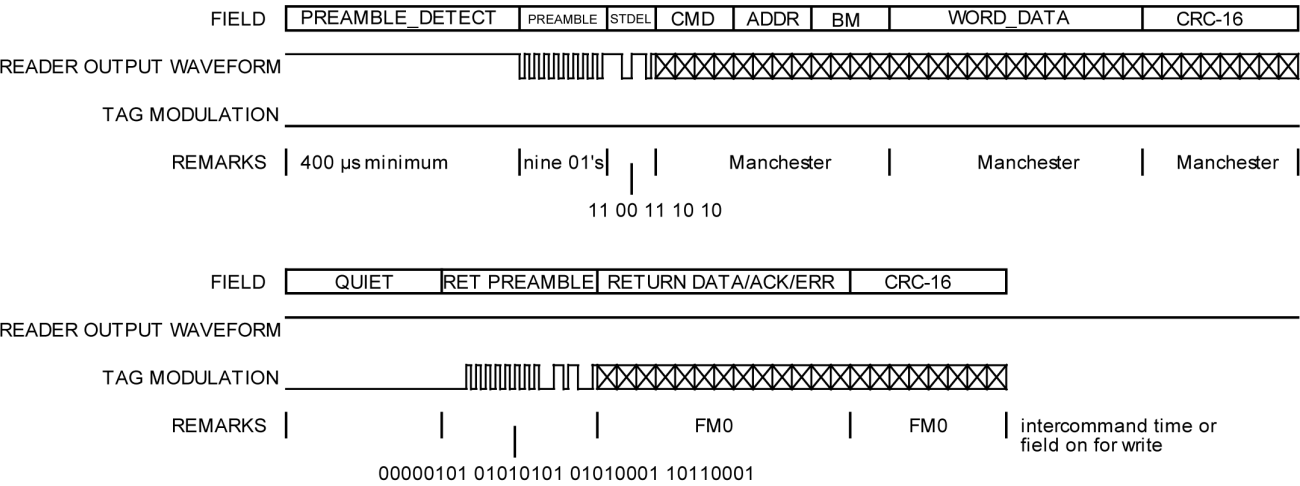


Figure 25 — Sample command/response packets for GROUP_SELECT (40 kbit/s on forward and return link)

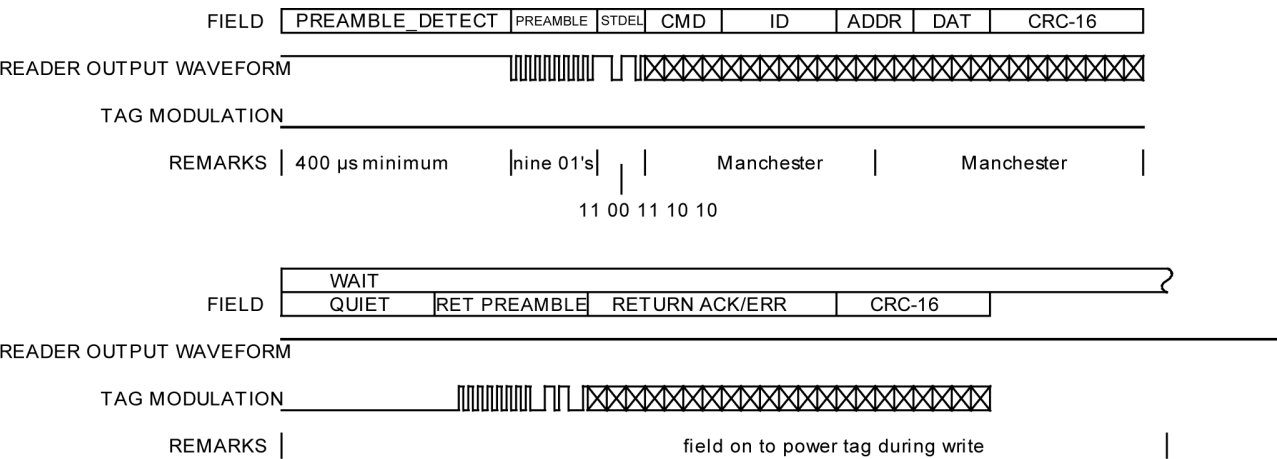


Figure 26 — Sample command/response packets for WRITE (40 kbit/s on forward and return link)

8.1.8 Communication sequences at packet level

Figure 27 and Figure 28 show examples of communication sequences at the packet level. Figure 27 depicts a packet sequence that includes a write command. The sequence includes a wait for write time, which provides the necessary time for the chip to complete its write operation. In addition, following the wait for write time, the interrogator issues a tag resync signal. This signal is composed of 10 consecutive 01 signals. The purpose of the tag resync signal is to initialise the tag data recovery circuitry. It is required after a write because the interrogator may output spurious edges during the wait for write time. Without the tag resync, tags may mis-calibrate as a result of the spurious signals that may be generated.

Figure 28 depicts a packet sequence in which a frequency hop between commands is included. The tag resync signal is again required after the hop because spurious signals may be generated during the hop time.

In order to ensure that tags do not get confused, frequency hops between command and response should be avoided.

Action	COMMAND	RESPONSE	WAIT FOR WRITE	TAG RESYNC	COMMAND	RESPONSE
Component execution action	Interrogator	Tag	Interrogator	Interrogator	Interrogator	Tag
Notes	---	---	15 ms minimum	ten 01's	---	---

Figure 27 — Command sequence (including a write) with no hopping

Action	COMMAND	RESPONSE	HOP	TAG RESYNC	COMMAND	RESPONSE
Component execution action	Interrogator	Tag	Interrogator	Interrogator	Interrogator	Tag
Notes	---	---	< 26 μs	ten 01's	---	---

Figure 28 — Command sequence with a hop between response and next command

8.2 Btree protocol and collision arbitration

8.2.1 Definition of data elements, bit and byte ordering

8.2.1.1 Unique ID

See Annex B and Annex C.

8.2.1.2 CRC-16

See Annex A.

8.2.1.3 FLAGS

8.2.1.3.1 FLAGS general

The tag shall support a field of 8 flags. This field is called FLAGS, and is shown in Table 95.

Table 95 — FLAGS

Bit	Name
FLAG1 (LSB)	DE_SB (Data_Exchange Status Bit)
FLAG2	WRITE_OK
FLAG3	BATTERY_POWERED
FLAG4	BATTERY_OK
FLAG5	0 (RFU)
FLAG6	0 (RFU)
FLAG7	0 (RFU)
FLAG8 (MSB)	0 (RFU)

8.2.1.3.2 Data exchange status bit (DE_SB)

The tag shall set this bit when the tag goes into the DATA_EXCHANGE state and keeps it set unless it moves into the POWER-OFF state.

When the DE_SB is set and the tag comes into the POWER-OFF state, then the tag shall trigger a timer that will reset the DE_SB bit after t_{DE_SB} .

t_{DE_SB} shall be at least 2 seconds in the temperature range $-30\text{ }^{\circ}\text{C}$ to $60\text{ }^{\circ}\text{C}$.

t_{DE_SB} shall be at least 4 seconds in the temperature range $0\text{ }^{\circ}\text{C}$ to $50\text{ }^{\circ}\text{C}$.

When the tag receives the INITIALIZE command, then it shall reset the DE_SB immediately.

8.2.1.3.3 WRITE_OK

The WRITE_OK bit shall be set after a successful write access to the memory. (e.g., WRITE, LOCK)

The WRITE_OK bit is cleared after execution of the command following the write command.

8.2.1.3.4 BATTERY_POWERED

The BATTERY_POWERED bit shall be set when the tag should have a battery. It shall be cleared for passive tags.

8.2.1.3.5 BATTERY_OK

The BATTERY_OK bit shall be set when the battery has enough power to support the tag. It shall be cleared for passive tags.

NOTE BATTERY_POWERED indicates whether the tag should have a battery, while BATTERY_OK reports the status of the battery. BATTERY_POWERED could be hardwired.

8.2.2 Tag memory organisation

The functional memory shall be organised in blocks of one byte.

Up to 256 blocks of one byte can be addressed.

This leads to a maximum memory capacity of up to 2 kbits.

NOTE This structure allows future extensions of the maximum memory capacity, by the use of additional commands to be defined, when required.

8.2.3 Block security status

Each byte shall have a corresponding lock bit. The lock bits may be locked by use of the LOCK command. The status of the lock bit may be read by the QUERY_LOCK command. The tag shall not be allowed to reset any lock bit after leaving the final production site. In most cases this is the production site that defines the unique ID.

8.2.4 Overall protocol description, Btree protocol

8.2.4.1 Tag states

The tag has four major states, as shown in Figure 29:

- POWER-OFF**
- The tag is in the POWER-OFF state when the interrogator cannot activate it. (For battery-assisted tags, it means that the level of RF excitation is insufficient to turn on the tag circuits.)
- READY**
- The tag is in the READY state when the interrogator first powers it up.
- ID**
- The tag is in the ID state when it is trying to identify itself to the interrogator.
- DATA_EXCHANGE**
- The tag is in the DATA_EXCHANGE state, when it is known to the interrogator and was selected.

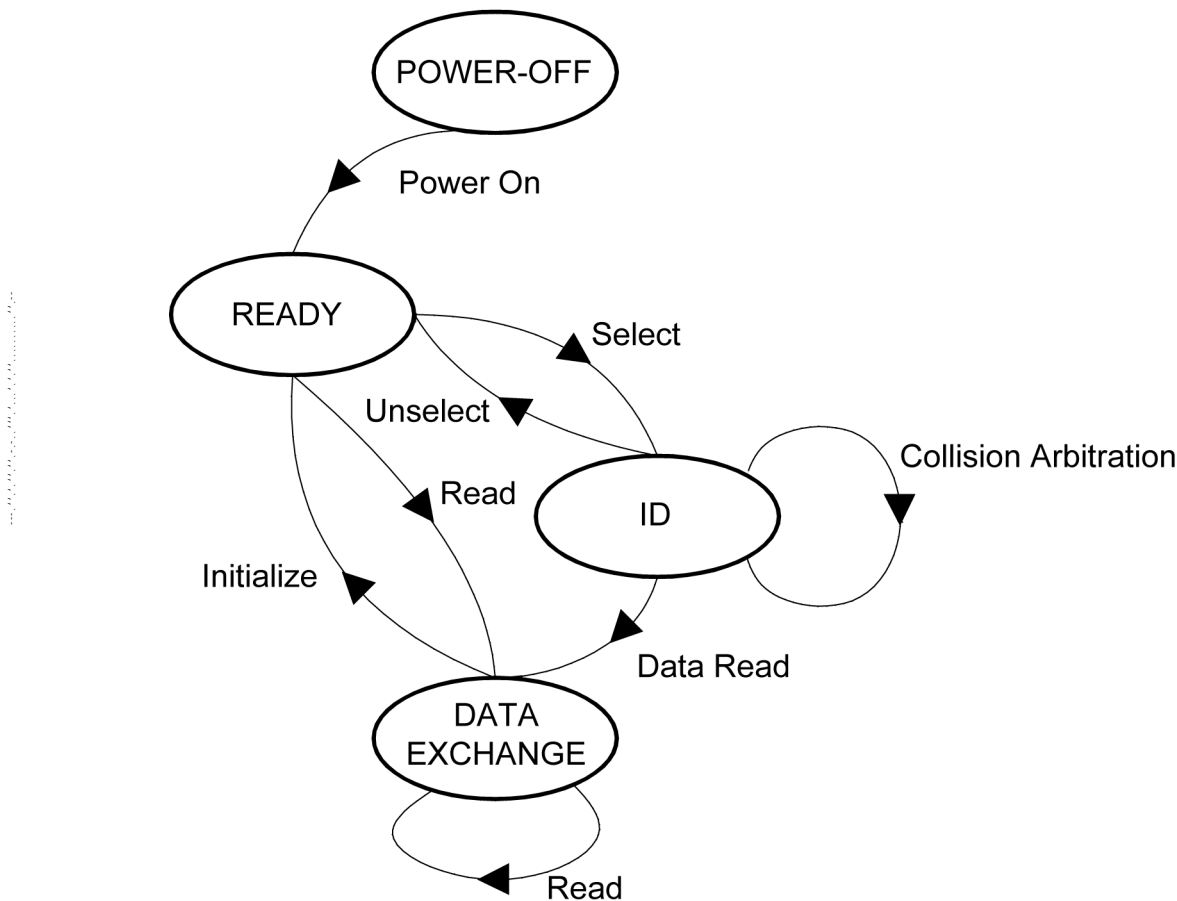


Figure 29 — State diagram

NOTE This diagram does not show that the tag goes into POWER-OFF, state from all other states when the interrogator field is off and the tag operation is no longer supported by the tag power buffer.

The state diagram only shows an overview of the possible transitions. Details are specified in Table 97.

Power-On:

State change when interrogator field is turned on.

Select:

State change due to selection of tag by GROUP_SELECT or READ commands

Unselect:

State change due to deselection of tag by GROUP_UNSELECT commands or INITIALIZE command

Collision_Arbitration:

No state change during collision arbitration until a single tag is identified.

Data_Read:

State change due to first read access in collision arbitration process

Read:

State change due to read access independent of collision arbitration process.

Initialize:

State change due to deselection of tag by INITIALIZE command

The transition between these states is specified in Table 97.

8.2.4.2 Detailed command processing

As shown in Table 96, commands shall be active in states marked with "X" and neither causes a state change, nor cause a response in the other states.

Table 96 — Detailed command processing

COMMAND	States		
	READY	ID	DATA EXCHANGE
GROUP_SELECT_EQ	X	X	
GROUP_SELECT_NE	X	X	
GROUP_SELECT_GT	X	X	
GROUP_SELECT_LT	X	X	
GROUP_SELECT_EQ_FLAGS	X	X	
GROUP_SELECT_NE_FLAGS	X	X	
GROUP_UNSELECT_EQ		X	
GROUP_UNSELECT_NE		X	
GROUP_UNSELECT_GT		X	
GROUP_UNSELECT_LT		X	
GROUP_UNSELECT_EQ_FLAGS		X	
GROUP_UNSELECT_NE_FLAGS		X	
MULTIPLE_UNSELECT		X	
FAIL		X	
SUCCESS		X	
RESEND		X	
INITIALIZE	X	X	X
READ	X	X	X
DATA_READ		X	X
READ_VERIFY	X	X	X
READ_VERIFY_4BYTE	X	X	X
WRITE	X	X	X
WRITE4BYTE	X	X	X
WRITE4BYTE_MULTIPLE		X	X
WRITE_MULTIPLE		X	X
LOCK			X
QUERY_LOCK	X	X	X
FAIL_O		X	
SUCCESS_O		X	
DATA_READ_O		X	X
READ_FLAGS	X	X	X
READ_VARIABLE	X	X	X
READ_PORT	X	X	X
READ_UNADDRESSED	X	X	X
RESEND_O		X	

Table 97 — State transition table

Current state	Command	Condition	New state
POWER-OFF	ANY COMMAND		POWER OFF
POWER-OFF	“Power up”		READY
READY	GROUP_SELECT_EQ	≠	READY
READY	GROUP_SELECT_NE	=	READY
READY	GROUP_SELECT_GT	≤	READY
READY	GROUP_SELECT_EQ_FLAGS	flag not set	READY
READY	GROUP_SELECT_NE_FLAGS	flag set	READY
READY	GROUP_SELECT_LT	≥	READY
READY	GROUP_SELECT_EQ	=	ID
READY	GROUP_SELECT_NE	≠	ID
READY	GROUP_SELECT_GT	>	ID
READY	GROUP_SELECT_LT	<	ID
READY	GROUP_SELECT_EQ_FLAGS	flag set	ID
READY	GROUP_SELECT_NE_FLAGS	flag not set	ID
READY	INITIALIZE		READY
READY	READ	ID no match	READY
READY	READ	ID match	DATA_EXCHANGE
READY	READ_VERIFY	ID no match or not WRITE_OK	READY
READY	READ_VERIFY	ID match and WRITE_OK	DATA_EXCHANGE
READY	READ_VERIFY_4BYTE	ID no match or not WRITE_OK	READY
READY	READ_VERIFY_4BYTE	ID match and WRITE_OK	DATA_EXCHANGE
READY	WRITE	ID no match	READY
READY	WRITE	ID match	DATA_EXCHANGE
READY	WRITE4BYTE	ID no match	READY
READY	WRITE4BYTE	ID match	DATA_EXCHANGE
READY	QUERY_LOCK	ID no match	READY
READY	QUERY_LOCK	ID match	DATA_EXCHANGE
READY	READ_FLAGS	ID no match	READY
READY	READ_FLAGS	ID match	DATA_EXCHANGE
READY	READ_VARIABLE	ID no match	READY
READY	READ_VARIABLE	ID match	DATA_EXCHANGE
READY	READ_PORT	ID no match	READY
READY	READ_PORT	ID match	DATA_EXCHANGE
READY	READ_UNADDRESSED		DATA_EXCHANGE
ID	GROUP_UNSELECT_EQ	≠	ID
ID	GROUP_UNSELECT_NE	=	ID
ID	GROUP_UNSELECT_GT	≤	ID

Table 97 (continued)

Current state	Command	Condition	New state
ID	GROUP_UNSELECT_LT	$\geq$	ID
ID	GROUP_UNSELECT_EQ_FLAGS	flag not set	ID
ID	GROUP_UNSELECT_NE_FLAGS	flag set	ID
ID	GROUP_UNSELECT_EQ	=	READY
ID	GROUP_UNSELECT_NE	$\neq$	READY
ID	GROUP_UNSELECT_GT	$>$	READY
ID	GROUP_UNSELECT_LT	$<$	READY
ID	GROUP_UNSELECT_EQ_FLAGS	flag set	READY
ID	GROUP_UNSELECT_NE_FLAGS	flag not set	READY
ID	MULTIPLE_UNSELECT	$\neq$ or not WRITE_OK	ID
ID	MULTIPLE_UNSELECT	= and WRITE_OK	READY
ID	GROUP_SELECT_EQ		ID
ID	GROUP_SELECT_NE		ID
ID	GROUP_SELECT_GT		ID
ID	GROUP_SELECT_LT		ID
ID	GROUP_SELECT_EQ_FLAGS		ID
ID	GROUP_SELECT_NE_FLAGS		ID
ID	FAIL		ID
ID	SUCCESS		ID
ID	RESEND		ID
ID	INITIALIZE		READY
ID	READ	ID no match	ID
ID	READ	ID match	DATA_EXCHANGE
ID	DATA_READ	ID no match	ID
ID	DATA_READ	ID match	DATA_EXCHANGE
ID	READ_VERIFY	ID no match or not WRITE_OK	ID
ID	READ_VERIFY	ID match and WRITE_OK	DATA_EXCHANGE
ID	READ_VERIFY_4BYTE	ID no match or not WRITE_OK	ID
ID	READ_VERIFY_4BYTE	ID match and WRITE_OK	DATA_EXCHANGE
ID	WRITE	ID no match	ID
ID	WRITE	ID match	DATA_EXCHANGE
ID	WRITE4BYTE	ID no match	ID
ID	WRITE4BYTE	ID match	DATA_EXCHANGE
ID	WRITE_MULTIPLE		ID
ID	WRITE4BYTE_MULTIPLE		ID
ID	QUERY_LOCK	ID no match	ID
ID	QUERY_LOCK	ID match	DATA_EXCHANGE

Table 97 (continued)

Current state	Command	Condition	New state
ID	RESEND_O		ID
ID	READ_FLAGS	ID no match	ID
ID	READ_FLAGS	ID match	DATA_EXCHANGE
ID	READ_VARIABLE	ID no match	ID
ID	READ_VARIABLE	ID match	DATA_EXCHANGE
ID	READ_PORT	ID no match	ID
ID	READ_PORT	ID match	DATA_EXCHANGE
ID	FAIL_O		ID
ID	SUCCESS_O		ID
ID	DATA_READ_O	ID no match	ID
ID	DATA_READ_O	ID match	DATA_EXCHANGE
ID	READ_UNADDRESSED		DATA_EXCHANGE
DATA_EXCHANGE	INITIALIZE		READY
DATA_EXCHANGE	READ		DATA_EXCHANGE
DATA_EXCHANGE	DATA_READ		DATA_EXCHANGE
DATA_EXCHANGE	READ_VERIFY		DATA_EXCHANGE
DATA_EXCHANGE	READ_VERIFY_4BYTE		DATA_EXCHANGE
DATA_EXCHANGE	WRITE		DATA_EXCHANGE
DATA_EXCHANGE	WRITE4BYTE		DATA_EXCHANGE
DATA_EXCHANGE	WRITE4BYTE_MULTIPLE		DATA_EXCHANGE
DATA_EXCHANGE	WRITE_MULTIPLE		DATA_EXCHANGE
DATA_EXCHANGE	LOCK		DATA_EXCHANGE
DATA_EXCHANGE	QUERY_LOCK		DATA_EXCHANGE
DATA_EXCHANGE	READ_UNADDRESSED		DATA_EXCHANGE
DATA_EXCHANGE	DATA_READ_O		DATA_EXCHANGE
DATA_EXCHANGE	READ_FLAGS		DATA_EXCHANGE
DATA_EXCHANGE	READ_VARIABLE		DATA_EXCHANGE
DATA_EXCHANGE	READ_PORT		DATA_EXCHANGE

8.2.5 Collision arbitration

8.2.5.1 Collision arbitration general

The interrogator may use the GROUP_SELECT and GROUP_UNSELECT commands to define all or a subset of tags in the field to participate in the collision arbitration. It then may use the identification commands to run the collision arbitration algorithm.

For the collision arbitration the tag shall support two pieces of hardware on the tag:

- An 8-bit counter COUNT, and
- A random generator (with two possible values 0 or 1).

In the beginning, a group of tags are moved to the ID state by GROUP_SELECT commands and shall set their internal counters to 0. Subsets of the group may be unselected by GROUP_UNSELECT commands back to the READY state. Other groups can be selected before the identification process begins. Simulation results show no advantage in identifying one large group or a few smaller groups.

After above described selection, the following loop should be performed:

- 1) All tags in the ID state with the counter COUNT at 0 shall transmit their ID. This set initially includes all the selected tags.
- 2) If more than one tag transmitted, the interrogator receives an erroneous response. The FAIL command shall be sent.
- 3) All tags receiving a FAIL command with COUNT not equal to 0 shall increment COUNT. That is, they move further away from being able to transmit.

All tags receiving FAIL command with a count of 0 (those that just transmitted) shall generate a random number. Those that roll a 1 shall increment COUNT and shall not transmit. Those that roll a zero shall keep COUNT at zero and shall send their UID again.

One of four possibilities now occurs:

- 4) If more than one tag transmits, the FAIL step 2 repeats. (Possibility 1)
- 5) If all tags roll a 1, none transmits. The interrogator receives nothing. It sends the SUCCESS command. All the counters decrement, and the tags with a count of 0 transmit. Typically, this returns to step 2. (Possibility 2)
- 6) If only one tag transmits and the ID is received correctly, the interrogator shall send the DATA_READ command with the ID. If the DATA_READ command is received correctly, that tag shall move to the DATA_EXCHANGE state and shall transmit its data.

The interrogator shall send SUCCESS. All tags in the ID state shall decrement COUNT.

- 7) If only one tag has a count of 1 and transmits, step 5 or 6 repeats. If more than one tag transmits, step 2 repeats. (Possibility 3)
- 8) If only one tag transmits and the ID is received with an error, the interrogator shall send the RESEND command. If the ID is received correctly, step 5 repeats. If the ID is received again some variable number of times (this number can be set based on the level of error handling desired for the system), it is assumed that more than one tag is transmitting, and step 2 repeats. (Possibility 4)

8.2.5.2 Special collision arbitration

In the case that parts of the user data are unique, or the probability of duplicated information is sufficient low, the commands FAIL_O, SUCCESS_O and DATA_READ_O may be used for collision arbitration. While the algorithm is equal to commands without _O, the ID used for the algorithm is either a 32 bit or 64 bit ID beginning at memory address '14'.

Additionally to the already mentioned use of GROUP_SELECT and GROUP_UNSELECT these commands may be used in combination with the 32-bit ID option of the _O command. This means in the case that a GROUP_SELECT only selects those tags that have all zeros in the upper 32 bits of a 64 bit UID and after handling those checks whether no tags with having not all zeros in the upper 32 bits of a 64 bit UID are still left.

8.2.6 Commands

Commands are divided into four functional groups:

- Selection commands,
- Identification commands,
- Data transfer commands, and
- Multiple commands.

Further, commands, as shown in Table 98, have one of the following types:

- Mandatory,
- Optional,
- Custom, and
- Proprietary.

Table 98 — Command classes

Code	Class	Number of possible codes
'00' - '0A', '0C', '15', '1E' - '3F'.	Mandatory	47
'0B', '0D' - '0F', '11' - '13', '17' - '1D', '40' - '9F'	Optional	110
'A0' - 'DF'	Custom	64
'10', '14', '16', 'E0' - 'FF'	Proprietary	35

8.2.7 Command types

8.2.7.1 Command types general

All tags with the same IC manufacturer code and same IC version number shall behave the same.

8.2.7.2 Mandatory

The command codes range from '00' to '0A', '0C', '15' and '1E' to '3F'.

A Mandatory command shall be supported by all tags that claim to be compliant. Interrogators which claim compliance shall support all mandatory commands. Mandatory commands shall be implemented as specified in this part of ISO/IEC 18000.

8.2.7.3 Optional

The command codes range from '0B', '0D' to '0F', '11' to '13', '17' to '1D' and from '40' to '9F'.

Optional commands are commands that are specified within this part of ISO/IEC 18000. Interrogators shall be technically capable of performing all optional commands that are specified this part of ISO/IEC 18000 (although need not be set up to do so). Tags may or may not support optional commands.

If an optional command is used, it shall be implemented in the manner specified in this part of ISO/IEC 18000.

If the tag does not support an optional command, it shall remain silent.

NOTE The command whose code ranges from '0B, 0D to 13 and '17' to '1D' are optional and not essential to operate the tag. However, their support by the tag is recommended for appropriate performance. To reflect this, they are reported as "recommended" in Table 99.

8.2.7.4 Custom

The command codes range from 'A0' to 'DF'.

Tags support them, at their option, to implement manufacturer specific functions. The only fields that can be customized are the parameters and the data fields.

Any custom command contains as its first parameter the IC manufacturer code. This allows IC manufacturers to implement custom commands without risking duplication of command codes and thus misinterpretation. Custom commands may be enabled by this part of ISO/IEC 18000, but they shall not be specified in this part of ISO/IEC 18000.

Custom commands may be enabled by this part of ISO/IEC 18000, but they shall not be specified in this part of ISO/IEC 18000. A custom command shall not solely duplicate the functionality of any mandatory or optional command defined in this part of ISO/IEC 18000 by a different method.

If the tag does not support a custom command it shall remain silent.

8.2.7.5 Proprietary

The command codes are '10', '14', '16' and the range from 'E0' to 'FF'.

These commands are used by IC and tag manufacturers for various purposes such as tests, programming of system information, etc. They are not specified in this part of ISO/IEC 18000. The IC manufacturer may at its option document them or not. It is allowed that these commands are disabled after IC and/or tag manufacturing.

Proprietary commands may be enabled by this part of ISO/IEC 18000, but they shall not be specified in this part of ISO/IEC 18000.

A proprietary command shall not solely duplicate the functionality of any mandatory or optional command defined in this part of ISO/IEC 18000 by a different method.

8.2.7.6 Command codes and format

8.2.7.6.1 Command codes and format general

Command codes and formats are shown in Table 99.

Table 99 — Command codes and format

Command code	Type	Command name	Parameters			
'00'	Mandatory	GROUP_SELECT_EQ	ADDRESS	BYTE_MASK	WORD_DATA	
'01'	Mandatory	GROUP_SELECT_NE	ADDRESS	BYTE_MASK	WORD_DATA	
'02'	Mandatory	GROUP_SELECT_GT	ADDRESS	BYTE_MASK	WORD_DATA	
'03'	Mandatory	GROUP_SELECT_LT	ADDRESS	BYTE_MASK	WORD_DATA	
'04'	Mandatory	GROUP_UNSELECT_EQ	ADDRESS	BYTE_MASK	WORD_DATA	
'05'	Mandatory	GROUP_UNSELECT_NE	ADDRESS	BYTE_MASK	WORD_DATA	
'06'	Mandatory	GROUP_UNSELECT_GT	ADDRESS	BYTE_MASK	WORD_DATA	
'07'	Mandatory	GROUP_UNSELECT_LT	ADDRESS	BYTE_MASK	WORD_DATA	
'08'	Mandatory	FAIL	none	none	none	
'09'	Mandatory	SUCCESS	none	none	none	
'0A'	Mandatory	INITIALIZE	none	none	none	
'0B'	Recommended	DATA_READ	ID	ADDRESS	none	
'0C'	Mandatory	READ	ID	ADDRESS	none	
'0D'	Recommended	WRITE	ID	ADDRESS	BYTE_DATA	
'0E'	Recommended	WRITE_MULTIPLE	none	ADDRESS	BYTE_DATA	
'0F'	Recommended	LOCK	ID	ADDRESS	none	
'10'	Proprietary	IC manufacturer dependant				
'11'	Recommended	QUERY_LOCK	ID	ADDRESS	none	
'12'	Recommended	READ_VERIFY	ID	ADDRESS	none	
'13'	Recommended	MULTIPLE_UNSELECT	ADDRESS	BYTE_DATA	none	
'14'	Proprietary	IC manufacturer dependant				
'15'	Mandatory	RESEND	none	none	none	
'16'	Proprietary	IC manufacturer dependant				
'17'	Recommended	GROUP_SELECT_EQ_FLAGS	none	BYTE_MASK	BYTE_DATA	
'18'	Recommended	GROUP_SELECT_NE_FLAGS	none	BYTE_MASK	BYTE_DATA	
'19'	Recommended	GROUP_UNSELECT_EQ_FLAGS	none	BYTE_MASK	BYTE_DATA	
'1A'	Recommended	GROUP_UNSELECT_NE_FLAGS	none	BYTE_MASK	BYTE_DATA	
'1B'	Recommended	WRITE4BYTE	ID	ADDRESS	BYTE_MASK	4BYTE_DATA
'1C'	Recommended	WRITE4BYTE_MULTIPLE	ADDRESS	BYTE_MASK	4BYTE_DATA	
'1D'	Recommended	READ_VERIFY_4BYTE	ID	ADDRESS	none	
'1E'-'3F'	Mandatory	RFU				
'40', '41'	Optional	FAIL_O	none	none	none	
'42', '43'	Optional	SUCCESS_O	none	none	none	
'44', '45'	Optional	DATA_READ_O	ID	ADDRESS	none	
'46', '47'	Optional	RESEND_O	none	none	none	

Table 99 (continued)

Command code	Type	Command name	Parameters		
'48' – '4F'	Optional	RFU			
'50'	Optional	READ_FLAGS	ID	ADDRESS	none
'51'	Optional	READ_VARIABLE	ID	ADDRESS	LENGTH
'52'	Optional	READ_PORT	ID	ADDRESS	none
'53'	Optional	READ_UNADDRESSED		ADDRESS	none
'54' – '9F'	Optional	RFU			
'A0' – 'DF'	Custom	IC Manufacturer dependent			
'E0' – 'FF'	Proprietary	IC Manufacturer dependent			

8.2.7.6.2 Command Fields

Command fields are shown in Table 100.

Table 100 — Command fields

Field name	Field size
COMMAND	1 byte
ADDRESS	1 byte
BYTE_MASK	1 byte
ID	8 bytes
WORD_DATA	8 bytes
BYTE_DATA	1 byte
4BYTE_DATA	4 bytes
LENGTH	1 byte
CRC-16	2 bytes

8.2.7.6.3 Tag responses

Tag responses are shown in Table 101.

Table 101 — Tag responses

Response code	Response name	Response size
'00'	ACKNOWLEDGE	1 byte
	ACKNOWLEDGE_NOK	1 byte
'01'	ACKNOWLEDGE_OK	1 byte
'FE'	ERROR_NOK	1 byte
'FF'	ERROR	1 byte
	ERROR_OK	1 byte
n/a	WORD_DATA	8 bytes
N/a	VARIABLE DATA	LENGTH bytes
n/a	BYTE_DATA	1 byte
	CRC-16	2 bytes
	ID	8 bytes

8.2.7.7 Selection commands

8.2.7.7.1 Selection commands general

Selection commands define a subset of tags in the field to be identified or written to and may be used as part of the collision arbitration.

8.2.7.7.2 Data comparison for selection command on memory

Each select command of the commands

- GROUP_SELECT_EQ,
- GROUP_SELECT_NE,
- GROUP_SELECT_GT,
- GROUP_SELECT_LT,
- GROUP_UNSELECT_EQ,
- GROUP_UNSELECT_NE,
- GROUP_UNSELECT_GT, or
- GROUP_UNSELECT_LT,

has 3 arguments (parameter and data)

- ADDRESS,
- BYTE_MASK, and
- WORD_DATA,

and the tag shall make one of 4 possible comparisons:

- EQ M EQUAL D,
- NE M NOT EQUAL D,
- GT M GREATER THAN D, or
- LT M LOWER THAN D.

The arguments of the comparison are shown in Table 102.

Table 102 — Comparison arguments

M7 (MSB)	M6	M5	M4	M3	M2	M1	M0 (LSB)
Tag memory content at ADDRESS+0	Tag memory content at ADDRESS+1	Tag memory content at ADDRESS+2	Tag memory content at ADDRESS+3	Tag memory content at ADDRESS+4	Tag memory content at ADDRESS+5	Tag memory content at ADDRESS+6	Tag memory content at ADDRESS+7

$$M = M0 + M1 * 2^8 + M2 * 2^{16} + M3 * 2^{24} + M4 * 2^{32} + M5 * 2^{40} + M6 * 2^{48} + M7 * 2^{56}$$

and the arguments of the command, are shown in Table 103.

Table 103 — Command arguments

D7 (MSB)	D6	D5	D4	D3	D2	D1	D0 (LSB)
First byte after command	...						Last byte after command

$$D = D0 + D1 * 2^8 + D2 * 2^{16} + D3 * 2^{24} + D4 * 2^{32} + D5 * 2^{40} + D6 * 2^{48} + D7 * 2^{56}$$

The argument BYTE_MASK defines what bytes to be considered for comparison, and are shown in Table 104.

Table 104 — Data masking for Group_Select and Group_Unselect commands

BYTE_MASK	WORD_DATA
Bit 7 (MSB) is set	Consider D7 and M7 for comparison
Bit 6 is set	Consider D6 and M6 for comparison
Bit 5 is set	Consider D5 and M5 for comparison
Bit 4 is set	Consider D4 and M4 for comparison
Bit 3 is set	Consider D3 and M3 for comparison
Bit 2 is set	Consider D2 and M2 for comparison
Bit 1 is set	Consider D1 and M1 for comparison
Bit 0 (LSB) is set	Consider D0 and M0 for comparison
Bit 7 (MSB) is cleared	Ignore D7 and M7 for comparison
Bit 6 is cleared	Ignore D6 and M6 for comparison
Bit 5 is cleared	Ignore D5 and M5 for comparison
Bit 4 is cleared	Ignore D4 and M4 for comparison
Bit 3 is cleared	Ignore D3 and M3 for comparison
Bit 2 is cleared	Ignore D2 and M2 for comparison
Bit 1 is cleared	Ignore D1 and M1 for comparison
Bit 0 (LSB) is cleared	Ignore D0 and M0 for comparison

8.2.7.7.3 Data comparison for selection command on flags

Each select command of the following commands have two arguments (parameter and data):

- GROUP_SELECT_EQ_FLAGS,
- GROUP_SELECT_NE_FLAGS,
- GROUP_UNSELECT_EQ_FLAGS, or
- GROUP_UNSELECT_NE_FLAGS.

The two arguments (parameter and data) are:

- BYTE_MASK, or
- BYTE_DATA.

The tag shall make two possible comparisons:

- EQ FLAGS EQUAL D, or
- NE FLAGS NOT EQUAL D.

The arguments of the comparison are FLAGS, as defined in clause 8.2.1.3 and the argument of the command D, consisting of the bits D7 (MSB) to D0 (LSB).

The argument BYTE_MASK defines what bits to be considered for comparison, as shown in Table 105.

Table 105 — Data masking for Group_Select_Flags and Group_Unselect_Flags

BYTE_MASK	BYTE_DATA
Bit 7 (MSB) is set	Consider D7 and FLAG7 for comparison
Bit 6 is set	Consider D6 and FLAG6 for comparison
Bit 5 is set	Consider D5 and FLAG5 for comparison
Bit 4 is set	Consider D4 and FLAG4 for comparison
Bit 3 is set	Consider D3 and FLAG3 for comparison
Bit 2 is set	Consider D2 and FLAG2 for comparison
Bit 1 is set	Consider D1 and FLAG1 for comparison
Bit 0 (LSB) is set	Consider D0 and FLAG0 for comparison
Bit 7 (MSB) is cleared	Ignore D7 and FLAG7 for comparison
Bit 6 is cleared	Ignore D6 and FLAG6 for comparison
Bit 5 is cleared	Ignore D5 and FLAG5 for comparison
Bit 4 is cleared	Ignore D4 and FLAG4 for comparison
Bit 3 is cleared	Ignore D3 and FLAG3 for comparison
Bit 2 is cleared	Ignore D2 and FLAG2 for comparison
Bit 1 is cleared	Ignore D1 and FLAG1 for comparison
Bit 0 (LSB) is cleared	Ignore D0 and FLAG0 for comparison

NOTE In the formulae below the following symbols are used, = is equal to, != ... is not equal to, ! ... Boolean not

Formula describing the EQUAL function:

The EQUAL comparison passes, if $(!B7+(D7=FLAG7)) * (!B6+(D6=FLAG6)) * (!B5+(D5=FLAG5)) * (!B4+(D4=FLAG4)) * (!B3+(D3=FLAG3)) * (!B2+(D2=FLAG2)) * (!B1+(D1=FLAG1)) * (!B0+(D0=FLAG0))$ is true.

Formula describing the UNEQUAL function:

The UNEQUAL comparison passes, if $B7*(D7!=FLAG7) + B6*(D6!=FLAG6) + B5*(D5!=FLAG5) + B4*(D4!=FLAG4) + B3*(D3!=FLAG3) + B2*(D2!=FLAG2) + B1*(D1!=FLAG1) + B0*(D0!=FLAG0)$ is true.

8.2.7.7.4 GROUP_SELECT_EQ

Command code = '00'

On receiving a GROUP_SELECT_EQ command, as shown in Table 106, a tag that is in the READY state shall read the 8-byte memory content beginning at the specified address and compare it with the WORD_DATA sent by the interrogator. In the case that the memory content is equal to WORD_DATA the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 107, and go into the state ID.

On receiving a GROUP_SELECT_EQ command, a tag that is in the ID state shall set its internal counter COUNT to 0, read its UID and send back the UID and stay in the ID state.

In all other cases the tag shall not send a reply.

Table 106 — GROUP_SELECT_EQ command

Preamble	Delimiter	Command	Address	Mask	WORD_DATA	CRC-16
		8 bits	8 bits	8 bits	64 bits	16 bits

Table 107 — GROUP_SELECT_EQ response in the case of NO error

Preamble	ID	CRC-16
	64 bits	16 bits

NOTE If the byte mask is zero, GROUP_SELECT_EQ selects all tags.

8.2.7.7.5 GROUP_SELECT_NE

Command code = '01'

On receiving a GROUP_SELECT_NE command, as shown in Table 108, a tag that is in the READY state shall read the 8-byte memory content beginning at the specified address and compare it with the WORD_DATA sent by the interrogator. In the case that the memory content is not equal to WORD_DATA the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 109, and go into the state ID.

On receiving a GROUP_SELECT_NE command, a tag that is in the ID state shall set its internal counter COUNT to 0, read its UID and send back the UID and stay in the ID state.

In all other cases the tag shall not send a reply.

Table 108 — GROUP_SELECT_NE command

Preamble	Delimiter	Command	Address	Mask	WORD_DATA	CRC-16
		8 bits	8 bits	8 bits	64 bits	16 bits

Table 109 — GROUP_SELECT_NE response in the case of NO error

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.7.6 GROUP_SELECT_GT

Command code = '02'

On receiving a GROUP_SELECT_GT command, as shown in Table 110, a tag that is in the READY state shall read the 8-byte memory content beginning at the specified address and compare it with the WORD_DATA sent by the interrogator. In the case that the memory content is greater than WORD_DATA the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 111, and go into the state ID.

On receiving a GROUP_SELECT_GT command, a tag that is in the ID state shall set its internal counter COUNT to 0, read its UID and send back the UID and stay in the ID state.

In all other cases the tag shall not send a reply.

Table 110 — GROUP_SELECT_GT command

Preamble	Delimiter	Command	Address	Mask	WORD_DATA	CRC-16
		8 bits	8 bits	8 bits	64 bits	16 bits

Table 111 — GROUP_SELECT_GT response in the case of NO error

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.7.7 GROUP_SELECT_LT

Command code = '03'

On receiving a GROUP_SELECT_LT command, as shown in Table 112, a tag that is in the READY state shall read the 8-byte memory content beginning at the specified address and compare it with the WORD_DATA sent by the interrogator. In the case that the memory content is lower than WORD_DATA the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 113, and go into the state ID.

On receiving a GROUP_SELECT_LT command, a tag that is in the ID state shall set its internal counter COUNT to 0, read its UID and send back the UID and stay in the ID state.

In all other cases the tag shall not send a reply.

Table 112 — GROUP_SELECT_LT command

Preamble	Delimiter	Command	Address	Mask	WORD_DATA	CRC-16
		8 bits	8 bits	8 bits	64 bits	16 bits

Table 113 — GROUP_SELECT_LT response in the case of NO error

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.7.8 GROUP_UNSELECT_EQ

Command code = '04'

On receiving a GROUP_UNSELECT_EQ command, as shown in Table 114, a tag that is in the ID state shall read the 8-byte memory content beginning at the specified address and compare it with the WORD_DATA sent by the interrogator. In the case that the memory content is equal to WORD_DATA the tag shall go into the state READY and not send any reply. In the case that the comparison fails, the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 115.

In all other cases the tag shall not send a reply.

Table 114 — GROUP_UNSELECT_EQ command

Preamble	Delimiter	Command	Address	Mask	WORD_DATA	CRC-16
		8 bits	8 bits	8 bits	64 bits	16 bits

Table 115 — GROUP_UNSELECT_EQ response in the case of NO error and comparison fails

Preamble	ID	CRC-16
	64 bits	16 bits

NOTE If the byte mask is zero, GROUP_UNSELECT_EQ unselects all tags.

8.2.7.7.9 GROUP_UNSELECT_NE

Command code = '05'

On receiving a GROUP_UNSELECT_NE command, as shown in Table 116, a tag that is in the ID state shall read the 8-byte memory content beginning at the specified address and compare it with the WORD_DATA sent by the interrogator. In the case that the memory content is not equal to WORD_DATA the tag shall go into the state READY and not send any reply. In the case the comparison fails, the tag shall set its internal counter COUNT to 0, read its UID and send back the UID as shown in Table 117.

In all other cases the tag shall not send a reply.

Table 116 — GROUP_UNSELECT_NE command

Preamble	Delimiter	Command	Address	Mask	WORD_DATA	CRC-16
		8 bits	8 bits	8 bits	64 bits	16 bits

Table 117 — GROUP_UNSELECT_NE response in the case of NO error and comparison fails

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.7.10 GROUP_UNSELECT_GT

Command code = '06'

On receiving a GROUP_UNSELECT_GT command, as shown in Table 118, a tag that is in the ID state shall read the 8-byte memory content beginning at the specified address and compare it with the WORD_DATA sent by the interrogator. In the case that the memory content is greater than to WORD_DATA the tag shall go into the state READY and not send any reply. In the case that the comparison fails, the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 119.

In all other cases the tag shall not send a reply.

Table 118 — GROUP_UNSELECT_GT command

Preamble	Delimiter	Command	Address	Mask	WORD_DATA	CRC-16
		8 bits	8 bits	8 bits	64 bits	16 bits

Table 119 — GROUP_UNSELECT_GT response in the case of NO error and comparison fails

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.7.11 GROUP_UNSELECT_LT

Command code = '07'

On receiving a GROUP_UNSELECT_LT command, as shown in Table 120, a tag that is in the ID state shall read the 8-byte memory content beginning at the specified address and compare it with the WORD_DATA sent by the interrogator. In the case that the memory content is lower than to WORD_DATA the tag shall go into the state READY and not send any reply. In the case that the comparison fails, the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 121.

In all other cases the tag shall not send a reply.

Table 120 — GROUP_UNSELECT_LT command

Preamble	Delimiter	Command	Address	Mask	WORD_DATA	CRC-16
		8 bits	8 bits	8 bits	64 bits	16 bits

Table 121 — GROUP_UNSELECT_LT response in the case of NO error and comparison fails

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.7.12 MULTIPLE_UNSELECT

Command code = '13'

On receiving a MULTIPLE_UNSELECT command, as shown in Table 122, a tag that is in the ID state shall read the 1-byte memory content beginning at the specified address and compare it with the BYTE_DATA sent by the interrogator. In the case that the memory content is equal to BYTE_DATA and the flag WRITE_OK is set, then the tag shall go into the state READY and not send any reply. In the case that the comparison fails, the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 123.

In all other cases the tag shall not send a reply.

Table 122 — MULTIPLE_UNSELECT command

Preamble	Delimiter	Command	ADDRESS	BYTE_DATA	CRC-16
		8 bits	8 bits	8 bits	16 bits

Table 123 — MULTIPLE_UNSELECT response in the case that WRITE_OK is reset or BYTE_DATA is not equal to memory content at ADDRESS

Preamble	ID	CRC-16
	64 bits	16 bits

This command may be used to unselect all tags that had a successful write, while tags that had a weak write or write problems stay selected.

8.2.7.7.13 GROUP_SELECT_EQ_FLAGS

Command code = '17'

On receiving a GROUP_SELECT_EQ_FLAGS command, as shown in Table 124, a tag that is in the READY state shall compare the FLAGS with the BYTE_DATA sent by the interrogator. In the case that the FLAGS are equal to BYTE_DATA the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 125, and go into the state ID.

On receiving a GROUP_SELECT_EQ_FLAGS command, a tag that is in the ID state shall set its internal counter COUNT to 0, read its UID and send back the UID and stay in the ID state.

In all other cases the tag shall not send a reply.

Table 124 — GROUP_SELECT_EQ_FLAGS command

Preamble	Delimiter	Command	Mask	BYTE_DATA	CRC-16
		8 bits	8 bits	8 bits	16 bits

Table 125 — GROUP_SELECT_EQ_FLAGS response in the case of NO error

Preamble	ID	CRC-16
	64 bits	16 bits

NOTE If the byte mask is zero, GROUP_SELECT_EQ_FLAGS selects all tags.

8.2.7.7.14 GROUP_SELECT_NE_FLAGS

Command code = '18'

On receiving a GROUP_SELECT_NE_FLAGS command, as shown in Table 126, a tag that is in the READY state shall compare the FLAGS with the BYTE_DATA sent by the interrogator. In the case that the FLAGS are not equal to BYTE_DATA the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 127, and go into the state ID.

On receiving a GROUP_SELECT_NE_FLAGS command, a tag that is in the ID state shall set its internal counter COUNT to 0, read its UID and send back the UID and stay in the ID state.

In all other cases the tag shall not send a reply.

Table 126 — GROUP_SELECT_NE_FLAGS command

Preamble	Delimiter	Command	Mask	BYTE_DATA	CRC-16
		8 bits	8 bits	8 bits	16 bits

Table 127 — GROUP_SELECT_NE_FLAGS response in the case of NO error

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.7.15 GROUP_UNSELECT_EQ_FLAGS

Command code = '19'

On receiving a GROUP_UNSELECT_EQ_FLAGS command, as shown in Table 128, a tag that is in the ID state shall compare the FLAGS with the BYTE_DATA sent by the interrogator. In the case that the FLAGS are equal to BYTE_DATA the tag shall go into the state READY and not send any reply. In the case that the comparison fails, the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 129.

In all other cases the tag shall not send a reply.

Table 128 — GROUP_UNSELECT_EQ_FLAGS command

Preamble	Delimiter	Command	Mask	BYTE_DATA	CRC-16
		8 bits	8 bits	8 bits	16 bits

Table 129 — GROUP_UNSELECT_EQ_FLAGS response in the case of NO error and comparison fails

Preamble	ID	CRC-16
	64 bits	16 bits

NOTE If the byte mask is zero, GROUP_UNSELECT_EQ_FLAGS unselects all tags.

8.2.7.7.16 GROUP_UNSELECT_NE_FLAGS

Command code = '1A'

On receiving a GROUP_UNSELECT_NE_FLAGS command, as shown in Table 130, a tag that is in the ID state shall compare the FLAGS with the BYTE_DATA sent by the interrogator. In the case that the FLAGS are not equal to BYTE_DATA the tag shall go into the state READY and not send any reply. In the case that the comparison fails, the tag shall set its internal counter COUNT to 0, read its UID and send back the UID, as shown in Table 131.

In all other cases the tag shall not send a reply.

Table 130 — GROUP_UNSELECT_NE_FLAGS command

Preamble	Delimiter	Command	Mask	BYTE_DATA	CRC-16
		8 bits	8 bits	8 bits	16 bits

Table 131 — GROUP_UNSELECT_NE_FLAGS response in the case of NO error and comparison fails

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.8 Identification commands

8.2.7.8.1 Identification commands general

Identification commands are used to perform the multiple tag identification protocol.

8.2.7.8.1.2 FAIL

Command code = '08'

The identification algorithm uses FAIL when more than one tag tried to identify itself at the same time. Some tags back off and some tags retransmit.

A tag shall only accept a FAIL command, as shown in Table 132, if it is in the ID state. In the case that its internal counter COUNT is not zero or the random generator result is 1, then COUNT shall be increased by 1, unless it is FF. If the count is at FF, then the count remains unchanged for further FAIL commands.

If the resulting COUNT value is 0, then the tag shall read its UID and sent back it in the response, as shown in Table 133.

Table 132 — FAIL command

Preamble	Delimiter	Command	CRC-16
		8 bits	16 bits

Table 133 — FAIL response in the case that COUNT stays zero

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.8.1.3 SUCCESS

Command code = '09'

SUCCESS initiates identification of the next set of tags. It is used in two cases:

- When all tags receiving FAIL backed off and did not transmit, SUCCESS causes those same tags to transmit again, and
- After a DATA_READ moves an identified tag to DATA_EXCHANGE, SUCCESS causes the next subset of selected but unidentified tags to transmit.

A tag shall only accept a SUCCESS command, as shown in Table 134, if it is in the ID state. In the case that the tag's internal counter COUNT is not zero, the internal counter shall be decreased by 1.

If the resulting COUNT value is 0, then the tag shall read its UID and sent back it in the response, as shown in Table 135.

Table 134 — SUCCESS command

Preamble	Delimiter	Command	CRC-16
		8 bits	16 bits

Table 135 — SUCCESS response in the case that COUNT is zero

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.8.1.4 RESEND

Command code = '15'

The identification algorithm uses RESEND when only one tag transmitted but the UID was received in error. The tag that transmitted resends its UID.

A tag shall only accept a RESEND command, as shown in Table 136, if it is in the ID state. If the COUNT value is 0, then the tag shall read its UID and sent back it in the response, as shown in Table 137.

Table 136 — RESEND command

Preamble	Delimiter	Command	CRC-16
		8 bits	16 bits

Table 137 — RESEND response in the case that COUNT is zero

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.8.1.5 INITIALIZE

Command code = '0A'

On receiving a INITIALIZE command, as shown in Table 138 a tag shall go into the READY state and reset the Data_Exchange_Status_Bit.

It shall not send any response.

Table 138 — INITIALIZE command

Preamble	Delimiter	Command	CRC-16
		8 bits	16 bits

8.2.7.8.1.6 FAIL_O

Command code = '40' or '41'

The identification algorithm uses FAIL_O when more than one tag tried to identify itself at the same time. Some tags back off and some tags retransmit.

A tag shall only accept a FAIL_O command, as shown in Table 139, if it is in the ID state. In the case that the tag's internal counter COUNT is not zero, or the random generator result is 1, the internal counter shall be increased by 1, unless COUNT is FF.

If the resulting COUNT value is 0, then the tag shall read the memory at address '14' sent back either 32 bit (for command code '40') or 64 bit (for command code '41') in the response, as shown in Table 140.

Table 139 — FAIL_O command

Preamble	Delimiter	Command	CRC-16
		8 bits	16 bits

Table 140 — FAIL_O response in the case that COUNT stays zero

Preamble	WORD_DATA	CRC-16
	32 or 64 bits	16 bits

8.2.7.8.1.7 SUCCESS_O

Command code = '42' or '43'

SUCCESS_O initiates identification of the next set of tags. It is used in two cases:

- When all tags receiving FAIL_O backed off and did not transmit, SUCCESS_O causes those same tags to transmit again, and
- After a DATA_READ_O moves an identified tag to DATA_EXCHANGE, SUCCESS_O causes the next subset of selected but unidentified tags to transmit.

A tag shall only accept a SUCCESS_O command, as shown in Table 141 if it is in the ID state. In the case that the tag's internal counter COUNT is not zero it shall be decreased by 1.

If the resulting COUNT value is 0, then the tag shall read the memory at address '14' sent back either 32 bit (for command code '40') or 64 bit (for command code '41') in the response, as shown in Table 142.

Table 141 — SUCCESS_O command

Preamble	Delimiter	Command	CRC-16
		8 bits	16 bits

Table 142 — SUCCESS_O response in the case that COUNT is zero

Preamble	WORD_DATA	CRC-16
	64 bits	16 bits

8.2.7.8.1.8 RESEND_O

Command code = '46' or '47'

The identification algorithm uses RESEND_O when only one tag transmitted but the ID was received in error. The tag that transmitted resends its ID.

A tag shall only accept a RESEND_O command, as shown in Table 143, if it is in the ID state. If the COUNT value is 0, then the tag shall read the memory at address '10' sent back either 32 bit (for command code '46') or 64 bit (for command code '47') in the response, as shown in Table 144.

Table 143 — RESEND_O command

Preamble	Delimiter	Command	CRC-16
		8 bits	16 bits

Table 144 — RESEND_O response in the case that COUNT is zero

Preamble	ID	CRC-16
	64 bits	16 bits

8.2.7.9 Data transfer commands

8.2.7.9.1 Data transfer commands general

Data Transfer commands are used to read or write data from or to the memory.

8.2.7.9.2 READ

Command code = '0C'

On receiving the READ command, as shown in Table 145, the tag shall compare the sent ID with its UID. In the case that the ID is equal to the UID, the tag shall from any state move to the DATA_EXCHANGE state, read the 8 byte memory content beginning at the specified address and send back its content in the response, as shown in Table 146. Further, the tag shall mark the byte at ADDRESS lockable. In the case that ID is not equal to UID or any other error the tag shall not send a reply.

The address is numbered from '00' to 'FF' (0 to 255).

Table 145 — Read command

Preamble	Delimiter	Command	ID	Address	CRC-16
		8 bits	64 bits	8 bits	16 bits

Table 146 — Read response in the case of NO error

Preamble	WORD_DATA	CRC-16
	64 bits	16 bits

8.2.7.9.3 DATA_READ

Command code = '0B'

On receiving the DATA_READ command, as shown in Table 147, the tag shall only if it is in either the state ID or the state DATA_EXCHANGE compare the sent ID with its UID. In the case that the ID is equal to the UID, the tag shall from any state except READY move to the DATA_EXCHANGE state, read the 8 byte memory content beginning at the specified address and send back its content in the response, as shown in Table 148. Further, the tag shall mark the byte at ADDRESS lockable. In the case that the tag is not in the state READY, or ID is not equal to UID or any other error the tag shall not send a reply.

The address is numbered from '00' to 'FF' (0 to 255).

Table 147 — DATA_READ command

Preamble	Delimiter	Command	ID	Address	CRC-16
		8 bits	64 bits	8 bits	16 bits

Table 148 — DATA_READ response in the case of NO error

Preamble	WORD_DATA	CRC-16
	64 bits	16 bits

8.2.7.9.4 DATA_READ_O

Command code = '44', or '45'

On receiving the DATA_READ_O command, as shown in Table 149, the tag shall compare the sent DATA with its memory contents starting at address '14' if the tag is in either the ID state or the DATA_EXCHANGE state. The amount of data compared shall be determined by the command code. If the received command code is '44', the tag shall logically compare the 32 bits. If command code is '45', the tag shall logically compare 64 bits. If the data strings (32 or 64 bits) are equal, then the tag shall transition from any state except READY to the DATA_EXCHANGE state, read the 8 byte memory content beginning at the specified address and send back its contents in the response, as shown in Table 150. Further, the tag shall mark the byte at ADDRESS lockable. If the tag is in the READY state, or the data strings are not equal, or any other error the tag shall not send a reply.

The address is numbered from '00' to 'FF' (0 to 255).

Table 149 — DATA_READ_O command

Preamble	Delimiter	Command	DATA	Address	CRC-16
		8 bits	32 or 64 bits	8 bits	16 bits

Table 150 — DATA_READ_O response in the case of NO error

Preamble	WORD_DATA	CRC-16
	64 bits	16 bits

8.2.7.9.5 READ_FLAGS

Command code = '50'

On receiving the READ_FLAGS command, as shown in Table 151, the tag shall from any state move to the DATA_EXCHANGE state, read the FLAGS and send back its content in the response, as shown in Table 152.

Table 151 — READ_FLAGS command

Preamble	Delimiter	Command	ID	CRC-16
		8 bits	64 bits	16 bits

Table 152 — READ_FLAGS in the case of NO error

Preamble	BYTE_DATA	CRC-16
	8 bits	16 bits

8.2.7.9.6 READ_VARIABLE

Command code = '51'

On receiving the READ_VARIABLE command, as shown in Table 153, the tag shall compare the sent ID with its UID. In the case that the ID is equal to the UID, the tag shall from any state move to the DATA_EXCHANGE state, read the specified length of the memory content beginning at the specified address and send back its content in the response, as shown in Table 154. In the case that ID is not equal to UID or any other error the tag shall not send a reply.

The address is numbered from '00' to 'FF' (0 to 255). Length is numbered from '00' to 'FF'.

Table 153 — READ_VARIABLE command

Preamble	Delimiter	Command	ID	Address	Length	CRC-16
		8 bits	64 bits	8 bits	8 bits	16 bits

Table 154 — READ_VARIABLE response in the case of NO error

Preamble	Length * BYTE_DATA	CRC-16
	Length * 8 bits	16 bits

8.2.7.9.7 READ_PORT

Command code = '52'

On receiving the READ_PORT command, as shown in Table 155, the tag shall compare the sent ID with its UID. In the case that the ID is equal to the UID, the tag shall from any state move to the DATA_EXCHANGE, read the 8 bit memory content beginning at the specified port address and send back the memory content in the response, as shown in Table 156. In the case that ID is not equal to UID or any other error the tag shall not send a reply.

The address is numbered from '00' to 'FF' (0 to 255). Length is numbered from '00' to 'FF'

Table 155 — READ_PORT command

Preamble	Delimiter	Command	ID	Address	CRC-16
		8 bits	64 bits	8 bits	16 bits

Table 156 — READ_PORT response in the case of NO error

Preamble	BYTE_DATA	CRC-16
	8 bits	16 bits

Selecting port 0 shall select the flags as specified in clause 8.2.1.3, and Table 95

Ports 1, 2, ... to 'FF' are reserved for future use.

8.2.7.9.8 READ_UNADDRESSED

Command code = '53'

On receiving the READ_UNADDRESSED command, as shown in Table 157, the tag shall from any state move to the DATA_EXCHANGE state, read the 16-byte memory content beginning at the specified address and send back its content in the response, as shown in Table 158.

The address is numbered from '00' to 'FF' (0 to 255).

Table 157 — READ_UNADDRESSED command

Preamble	Delimiter	Command	Address	CRC-16
		8 bits	8 bits	16 bits

Table 158 — READ_UNADDRESSED response in the case of NO error

Preamble	WORD_DATA	WORD_DATA	CRC-16
	64 bits	64 bits	16 bits

8.2.7.9.9 READ_VERIFY

Command code = '12'

On receiving the READ_VERIFY command, as shown in Table 159 the tag shall compare the sent ID with its UID. In the case that the ID is equal to the UID and the WRITE_OK flag is set, the tag shall from any state move to the DATA_EXCHANGE state, read the 1-byte memory content beginning at the specified address and send back its content in the response, as shown in Table 160. Further, the tag shall mark the byte at ADDRESS lockable. In the case that ID is not equal to UID, WRITE_OK is not set, or any other error the tag shall not send a reply.

The address is numbered from '00' to 'FF' (0 to 255).

Table 159 — READ_VERIFY command

Preamble	Delimiter	Command	ID	Address	CRC-16
		8 bits	64 bits	8 bits	16 bits

Table 160 — READ_VERIFY response in the case of NO error

Preamble	BYTE_DATA	CRC-16
	8 bits	16 bits

8.2.7.9.10 READ_VERIFY_4BYTE

Command code = '1D'

On receiving the READ_VERIFY_4BYTE command, as shown in Table 161, the tag shall compare the sent ID with its UID. In the case that the ID is equal to the UID and the WRITE_OK flag is set, the tag shall from any state move to the DATA_EXCHANGE state, read the 4-byte memory content beginning at the specified address and send back its content in the response, as shown in Table 162. Further, the tag shall mark the byte at ADDRESS lockable. Bytes at ADDRESS+1, ADDRESS+2 and ADDRESS+3 shall not be marked lockable.

In the case that ID is not equal to UID, WRITE_OK is not set, or any other error the tag shall not send a reply.

BYTE_MASK of the command

ADDRESS bit of BYTE_MASK to select whether byte should be written

[ADDR+0] B7

[ADDR+1] B6

[ADDR+2] B5

[ADDR+3] B4

The address is numbered from '00' to 'FF' (0 to 255).

Table 161 — READ_VERIFY_4BYTE command

Preamble	Delimiter	Command	ID	Address	CRC-16
		8 bits	64 bits	8 bits	16 bits

Table 162 — READ_VERIFY_4BYTE response in the case of NO error

Preamble	BYTE_DATA	CRC-16
	8 bits	16 bits

8.2.7.9.11 WRITE

Command code = '0D'

On receiving the WRITE command, as shown in Table 163, the tag shall compare the sent ID with its UID. In the case that the ID is equal to the UID, the tag shall from any state move to the DATA_EXCHANGE state, read the lock information for the byte on the specified memory content beginning at the specified address. In the case that the memory is locked, it shall send back the ERROR response, as shown in Table 165. Further, the tag shall mark the byte at ADDRESS lockable. In the case that the memory is unlocked, it shall send back the ACKNOWLEDGE, as shown in Table 164, and program the data into the specified memory address. In all other cases the tag shall not send a reply.

In the case that the write access was successful, the tag shall set the WRITE_OK bit. Otherwise it shall reset it.

The address is numbered from '00' to 'FF' (0 to 255).

Table 163 — Write command

Preamble	Delimiter	Command	ID	Address	BYTE_DATA	CRC-16
		8 bits	64 bits	8 bits	8 bits	16 bits

Table 164 — WRITE response in the case of NO error

Preamble	ACKNOWLEDGE	CRC-16
	8 bits	16 bits

Table 165 — WRITE response in the case of locked memory

Preamble	ERROR	CRC-16
	8 bits	16 bits

8.2.7.9.12 WRITE4BYTE

Command code = '1B'

On receiving the WRITE4BYTE command, as shown in Table 166, the tag shall compare the sent ID with its UID. In the case that the ID is equal to the UID, the tag shall from any state move to the DATA_EXCHANGE state, read the lock information for the 4 bytes on the specified memory content beginning at the specified address. In the case that one of the memory bytes is locked, it shall send back the ERROR response, as shown in Table 168. In the case that all bytes are unlocked, it shall send back the ACKNOWLEDGE, as shown in Table 167, and program the data into the specified memory. In all other cases the tag shall not send a reply.

Executing WRITE4BYTE a tags shall only write those bytes that are selected by the BYTE_MASK, which means that write could be done to 1 to 4 bytes(using the mask bits in the BYTE_MASK field).

In the case that the write access was successful, the tag shall set the WRITE_OK bit. Otherwise it shall reset it.

BYTE_MASK of the command

ADDRESS bit of BYTE_MASK to select whether byte should be written

[ADDR+0] B7

[ADDR+1] B6

[ADDR+2] B5

[ADDR+3] B4

The address is numbered from '00' to 'FF' (0 to 255). The starting address for the WRITE4BYTE command must be on a 4-byte page boundary.

Table 166 — WRITE4BYTE command

Preamble	Delimiter	Command	ID	Address	Byte_Mask	Data	CRC-16
		8 bits	64 bits	8 bits	8 bits	32 bits	16 bits

Table 167 — WRITE4BYTE response in the case of NO error

Preamble	ACKNOWLEDGE	CRC-16
	8 bits	16 bits

Table 168 — WRITE4BYTE response in the case that of locked memory

Preamble	ERROR	CRC-16
	8 bits	16 bits

8.2.7.9.13 LOCK

Command code = '0F'

On receiving a LOCK command, as shown in Table 169, a tag that is in the DATA_EXCHANGE state shall read its UID and compare it with the ID sent by the interrogator. In the case that the UID is equal to ID, the ADDRESS is within the valid address range and the byte at ADDRESS is marked lockable, then the tag shall send back the ACKNOWLEDGE, as shown in Table 170 and program the lock bit of the specified memory address. In case the ADDRESS is not in the valid range, or it is not marked as lockable, then the tag shall send back the ACKNOWLEDGE_NOK, as shown in Table 171.

In all other cases the tag shall not send a reply.

In the case that the write access was successful, the tag shall set the WRITE_OK bit. Otherwise it shall reset it.

The address is numbered from '00' to 'FF' (0 to 255).

Table 169 — LOCK command

Preamble	Delimiter	Command	ID	Address	CRC-16
		8 bits	64 bits	8 bits	16 bits

Table 170 — LOCK response in the case that locking was possible

Preamble	ACKNOWLEDGE	CRC-16
	8 bits	16 bits

Table 171— LOCK response in the case that locking was not possible

Preamble	ERROR	CRC-16
	8 bits	16 bits

8.2.7.9.14 QUERY_LOCK

Command code = '11'

On receiving a QUERY_LOCK command, as shown in Table 172, a tag shall read its UID and compare it with the ID sent by the interrogator. In the case that the UID is equal to ID and the ADDRESS is within the valid address range the tag shall move into the DATA_EXCHANGE state. Further, the tag shall read the lock bit for the memory byte at ADDRESS. Further, the tag shall mark the byte at ADDRESS lockable. In the case that this memory is not locked, then it shall either respond with ACKNOWLEDGE_OK, as shown in Table 173, if WRITE_OK is set, or ACKNOWLEDGE_NOK, as shown in Table 174 if WRITE_OK is cleared. In the case that this memory is locked, then it shall either respond with ERROR_OK, as shown in Table 175 if WRITE_OK is set, or ERROR_NOK, as shown in Table 176, if WRITE_OK is cleared.

In all other cases the tag shall not send a reply.

The address is numbered from '00' to 'FF' (0 to 255).

Table 172 — QUERY_LOCK command

Preamble	Delimiter	Command	ID	Address	CRC-16
		8 bits	64 bits	8 bits	16 bits

Table 173 — QUERY_LOCK response in memory address is not locked and WRITE_OK is set

Preamble	ACKNOWLEDGE_OK	CRC-16
	8 bits	16 bits

Table 174 — QUERY_LOCK response in memory address is not locked and WRITE_OK is cleared

Preamble	ACKNOWLEDGE_NOK	CRC-16
	8 bits	16 bits

Table 175 — QUERY_LOCK response in memory address is locked and WRITE_OK is set

Preamble	ERROR_OK	CRC-16
	8 bits	16 bits

Table 176 — QUERY_LOCK response in memory address is locked and WRITE_OK is cleared

Preamble	ERROR_NOK	CRC-16
	8 bits	16 bits

8.2.7.9.15 WRITE_MULTIPLE

Command code = '0E'

Write Multiple commands, as shown in Table 177, are used to write to and to verify multiple tags in parallel.

On receiving the WRITE_MULTIPLE command, a tag that is in the ID state or the DATA_EXCHANGE state shall read the lock information for the byte on the specified memory content beginning at the specified address. In the case that the memory is locked, it shall do nothing. In the case that unlocked, it shall program the data into the specified memory.

In the case that the write access was successful, the tag shall set the WRITE_OK bit. Otherwise it shall reset it.

Further, the tag shall mark the byte at ADDRESS lockable.

The address is numbered from '00' to 'FF' (0 to 255).

Table 177 — WRITE_MULTIPLE command

Preamble	Delimiter	Command	Address	Data	CRC-16
		8 bits	8 bits	8 bits	16 bits

8.2.7.9.16 WRITE4BYTE_MULTIPLE

Command code = '1C'

Write Multiple commands, as shown in Table 178, are used to write to and to verify multiple tags in parallel.

On receiving the WRITE4BYTE_MULTIPLE command, a tag that is in the ID state or the DATA_EXCHANGE state shall read the lock information for the 4 bytes on the specified memory content beginning at the specified address. In the case that one of the 4-byte block is locked, it shall do nothing. In the case that all bytes are unlocked, it shall program the data into the specified memory.

Executing WRITE4BYTE a tags shall only write those bytes that are selected by the BYTE_MASK, which means that write could be done to 1 to 4 bytes (using the mask bits in the byte_mask field).

In the case that the write access was successful, the tag shall set the WRITE_OK bit. Otherwise it shall reset it.

BYTE_MASK of the command

ADDRESS bit of BYTE_MASK to select whether byte should be written

[ADDR+0] B7

[ADDR+1] B6

[ADDR+2] B5

[ADDR+3] B4

The address is numbered from '00' to 'FF' (0 to 255). The starting address for the WRITE4BYTE command must be on a 4-byte page boundary.

Table 178 — WRITE4BYTE_MULTIPLE command

Preamble	Delimiter	Command	Address	Byte_Mask	Data	CRC-16
		8 bits	8 bits	8 bits	32 bits	16 bits

8.2.7.10 Response description (binary tree protocol type)**8.2.7.10.1 ACKNOWLEDGE**

ACKNOWLEDGE indicates a successful acceptance of the WRITE or LOCK.

8.2.7.10.2 ERROR

ERROR indicates an error in the WRITE. e.g. write to locked memory area

8.2.7.10.3 ACKNOWLEDGE_OK

ACKNOWLEDGE_OK is the response to a QUERY_LOCK and indicates an unlocked memory byte and a successful proceeding write command.

8.2.7.10.4 ACKNOWLEDGE_NOK

ACKNOWLEDGE_NOK is the response to a QUERY_LOCK and indicates an unlocked memory byte and an unsuccessful proceeding write command.

8.2.7.10.5 ERROR_OK

ERROR_OK is the response to a QUERY_LOCK and indicates a locked memory byte and a successful proceeding write command.

8.2.7.10.6 ERROR_NOK

ERROR_NOK is the response to a QUERY_LOCK and indicates as locked memory byte and an unsuccessful proceeding write command.

8.2.7.10.7 WORD_DATA

WORD_DATA is 8 bytes returned in response to a READ, DATA_READ, or DATA_READ_O command.

8.2.7.10.8 BYTE_DATA

BYTE_DATA is 1 byte returned in response to the READ_VERIFY and READ_PORT command.

8.2.7.10.9 4BYTedata

4BYTedata are 4 bytes used as argument for commands WRITE4BYTE, WRITE4BYTE_MULTIPLE and READ_VERIFY4BYTE.

8.2.8 Transmission errors

There are two types of transmission errors: modulation coding errors (detectable per bit) and CRC errors (detectable per command). Both errors cause any command to be aborted. The tag does not respond.

For all CRC errors, the tag returns to the ready state.

For all coding errors, the tag returns to the READY state if a valid start delimiter had been detected. Otherwise it maintains its current state.

Annex A
(informative)

Cyclic redundancy check (CRC)

A.1 Interrogator to tag CRC-5

The polynomial used to calculate the CRC-5 is x^5+x^3+1 .

A possible implementation is using a 5-bit shift register as defined in Figure A.1. The 5 bit CRC register is named Q4 to Q0, with Q4 being the MSB and Q0 being the LSB. The 5-bit register must be preloaded with '01001' (MSB to LSB) or in hexadecimal notation 0x09 (HEX), as shown in Table A.1

Table A.1 — 5 bit CRC register pre-load values

Register	Pre-load value
Q0	1
Q1	0
Q2	0
Q3	1
Q4	0

The 11 bits of data must be clocked through the CRC register, using the MSB first. The 5 CRC bits are then sent, MSB first. After the LSB of the CRC-5 bit is clocked through, the 5-bit CRC register should contain all zero's.

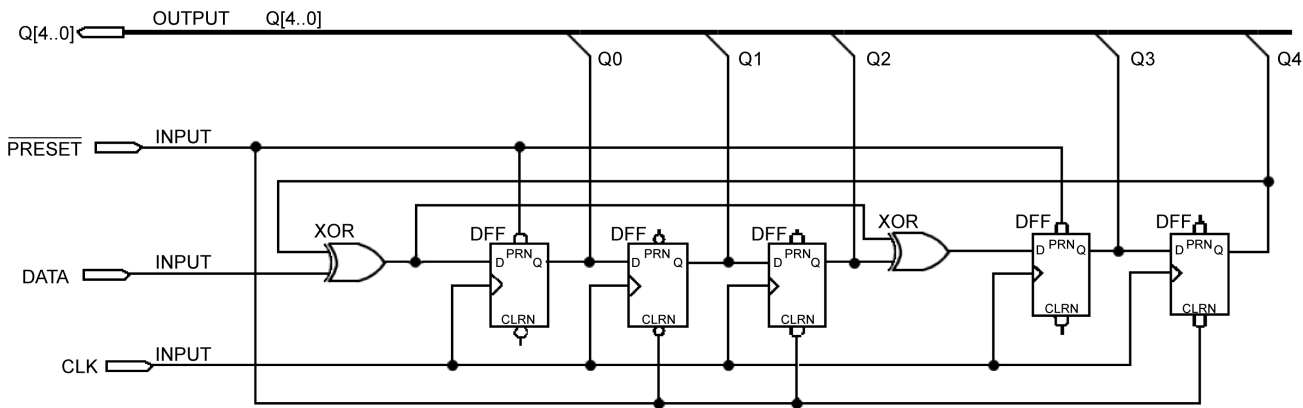


Figure A.1 — CRC-5

A.2 Interrogator to tag and tag to interrogator CRC-16

A.2.1 CRC-16 general

The polynomial used to calculate the CRC-16 is $X^{16}+X^{12}+X^5+1$, which is the CRC-CCITT International Standard. The Cyclic Redundancy Check (CRC) is calculated on all data contained in a message, from the start of the command through to the end of data. This CRC-16 is used from interrogator to tag and from tag to interrogator. The CRC-16 is defined in Table A.2

Table A.2 — CRC-16 definition

CRC definition					
CRC type	Length	Polynomial	Direction	Preset	Residue
CRC-CCITT	16 bits	$X^{16} + X^{12} + X^5 + 1$	Forward	0xFFFF (HEX)	'0'

The CRC algorithm is as follows:

For computing the CRC: (see example Table A.4)

- initialize the CRC accumulator to all ones – 0xFFFF (HEX),
- accumulate, MSB first, data using the polynomial $X^{16} + X^{12} + X^5 + 1$,
- invert the resulting CRC value, and
- attach the inverted CRC-16 to the end of the packet and transmit it MSB first.

For checking the CRC: (See example Table A.5)

- compute the CRC on the incoming packet,
- invert the received CRC data bits,
- accumulate the “inverted CRC bits” in the CRC registers, and
- verify that the accumulator is all zeroes.

An alternative can be used to check the CRC bits, which does not involve inverting the received CRC bits. The resulting CRC value will be 0x1D0F (HEX), as defined in CRC-CCITT. (See example Table A.6):

- compute the CRC on the incoming packet,
- accumulate the 16 “received CRC” in the CRC registers, and
- verify that the accumulator contains the value 1D0F.

Example A.1 — Example C code to generate the CRC bits for the Type B success command.

```
unsigned int Calc_CRC (unsigned int CRCacc, unsigned int cword)
```

```
{
/* Routine to calculate CRC for 1 byte (lower 8 bits of cword) */
/* Initially, CRCacc should have been set to 0xffff */
int i;
unsigned int xorval;
printf("\n");
for (i=0; i<8; i++)
{
xorval = ((CRCacc>>8) ^ (cword << i)) & 0x0080;
CRCacc = (CRCacc << 1) & 0xfffe;
if (xorval)
CRCacc ^= 0x1021;
printf("%04x\n",CRCacc);
}
return (CRCacc);
}
```

```
main()
{
unsigned int CRCacc = 0xffff;
int i;
unsigned char test_str[2];
test_str[0] = 0x09; /* Success Command */
test_str[1] = '\0';
for (i =0; i < strlen(test_str); i++)
CRCacc = Calc_CRC(CRCacc, test_str[i]);
printf("\nCRC = %04x\n",CRCacc);
}
```

A possible implementation is using a 16-bit shift register as defined in Figure A.2.

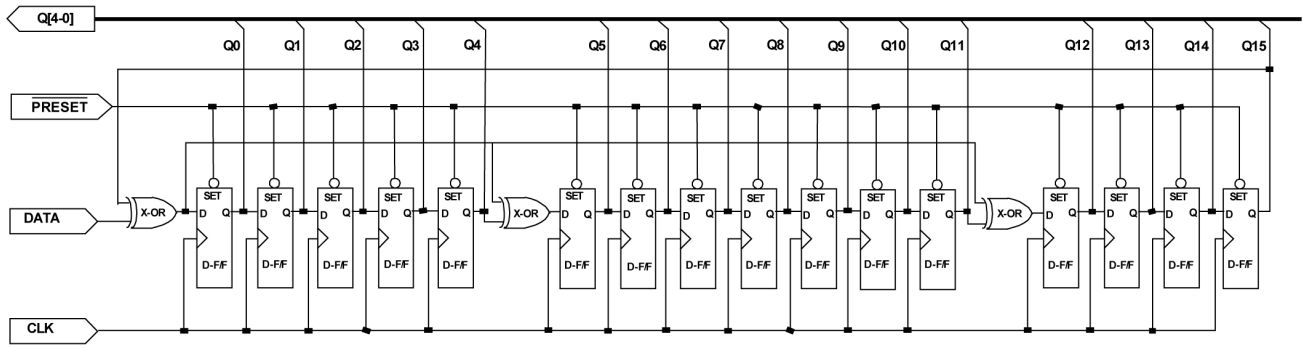


Figure A.2 — Shift register implementation of CRC-16 accumulator

A.2.2 CRC calculation examples

This example refers to a Type B interrogator sending a SUCCESS command.

Command code. '09'

The packet, as shown in Table A.3, sent from the interrogator to the tag consists of the following blocks, but only the SUCCESS command (09h), shown shaded, is used in the CRC calculation.

Table A.3 — CRC example Type B success command

Preamble detect	Preamble	Start delimiter	SUCCESS command	CRC-16
2 bytes field high	nine Manchester 0's	11 00 11 10 10	'09'	CRC-16

The CRC is calculated on the SUCCESS command as the field is transmitted MSB.

The following example, as shown in Table A.4, shows the values of the 16 CRC registers as the data is shifted through the CRC registers.

SUCCESS command 0 0 0 0 1 0 0 1 ('09')

Table A.4 — Practical example of CRC calculation for a 'SUCCESS' command in the interrogator

Step	Input (SUCCESS command)	Calculated CRC in interrogator (HEX)
1	0	'EFDF'
2	0	'CF9F'
3	0	'8F1F'
4	0	'0E1F'
5	1	'0C1F'
6	0	'183E'
7	0	'307C'
8	1	'70D9'

The CRC bits transmitted by the interrogator are inverted, 0x8F26 (HEX), MSB first.

The tag then inverts the received data bits and accumulates those 16 bits (i.e. 0x70D9 (HEX), as shown in Table A.5.

Table A.5 — Practical example of CRC checking for a 'SUCCESS' command in the tag

Step	Input (sent CRC-16)	Calculated CRC in tag (HEX)
0		'70D9'
1	0	'E1B2'
2	1	'C364'
3	1	'86C8'
4	1	'0D90'
5	0	'1B20'
6	0	'3640'
7	0	'6C80'
8	0	'D900'
9	1	'B200'
10	1	'6400'
11	0	'C800'
12	1	'9000'
13	1	'2000'
14	0	'4000'
15	0	'8000'
16	1	'0000'

Alternatively, the tag does not invert the received data bits and accumulates those 16 bits (i.e. 0x70D9 (HEX), as shown in Table A.6

Table A.6 — Practical example of CRC checking for a ‘SUCCESS’ command in the tag, without inversion of the received CRC bits, i.e. the received CRC bits 0x8F26 (HEX) are used.

Step	Input (sent CRC-16)	Calculated CRC in tag (HEX)
0		‘70D9’
1	1	‘F193’
2	0	‘F307’
3	0	‘F62F’
4	0	‘FC7F’
5	1	‘F8FE’
6	1	‘F1FC’
7	1	‘E3F8’
8	1	‘C7F0’
9	0	‘9FC1’
10	0	‘2FA3’
11	1	‘4F67’
12	0	‘9ECE’
13	0	‘2DBD’
14	1	‘4B5B’
15	1	‘8697’
16	0	‘1D0F’

Annex B (normative)

Memory mapping for Type B

B.1 Unique identifier (normative)

B.1.1 Unique identifier general

The tag serial number shall either be in accordance to clause B.1.2 and Table B.1, or clause B.1.3.

Differentiation is done by the leading bits in byte 0, which are 111 for unique identifiers as defined in chapter B.1.2 and 000 for unique identifier as defined in chapter B.1.3.

B.1.2 Unique identifier format

B.1.2.1 Unique identifier format general

Table B.1 — UID format

MSB							LSB
Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
M L	M L	M L	M L	M L	M L	M L	M L
'E0' 8 bits	IC Mfg code acc. ISO/IEC 7816-6	Chip Manufacturer Assigned 48 bits					

B.1.2.2 'E0' (byte 0)

E0 is the header for unique identifier followed by the manufacturer code according ISO/IEC 7816-6.

B.1.2.3 IC Mfg code according ISO/IEC 7816-6 (byte 1)

ISO/IEC 7816-6 defines an 8 bit code for chip manufacturers.

B.1.2.4 Chip manufacturer assigned (bytes 2 – 7)

This is a 48-bit field that is defined and managed by the chip manufacturer. Different chip manufacturers will have different manufacturer codes (see below), thus eliminating the potential for duplicated collision arbitration data (Tag UIDs). The numbering system employed by the chip manufacturer must ensure that all tags produced will have a unique and unambiguous number (used by the collision arbitration algorithm). This unique number will be “locked” prior to use. Maximum value for this field is $2^{48} - 1$.

Responsibility for ensuring uniqueness and for locking this unique number prior to use shall rest with the chip manufacturer.

B.1.3 Unique identifier according to ANSI 256

Table B.2 depicts bytes 0 to 7 of the tag serial number.

Table B.2 — Layout of tag serial number (bytes 0-7)

MSB								LSB	
Byte 0		Byte 1		Byte 2		Byte 3		Byte 4	
MSB	LSB	MSB	LSB	MSB	LSB	MSB	LSB	MSB	LSB
000 zero 3bits	Chip Manufacturer Assigned 47 bits MSB							Manufacturer ID 8 bits see NOTE 1 MSB ... LSB	FAB 4 bits see NOTE 2 MSB ... LSB
									CK 2 bits see NOTE 3 MSB ... LSB

NOTE 1 Manufacturer ID field is the same as the IC Manufacturer.

NOTE 2 FAB is allocated by the IC manufacturer, such that when combined with the IC manufacturer code and serial number the resulting number is unique.

NOTE 3 CK represents the truncated sum of the bits set to '1' of the 62 bits preceding the checksum, valid values are 0, 1, 2 and 3.

The Tag Serial Number shall be programmed and locked at the factory with a unique number for each tag.

B.1.4 Remaining system memory

B.1.4.1 Manufacturer ID (Bytes 8,9)

These two bytes, as shown in Table B.3, have been reserved for encoding the Tag Manufacturer ID to provide some conformance to anticipated RFID International Standards. These fields will initially be encoded with the following codes depending on the manufacturer of the tags.

Table B.3 — Manufacturer codes

Manufacturer	ASCII representation	Hexadecimal code
Reserved	'AT'	4154
Reserved	'HT'	4854
Reserved	'AA'	4141
Reserved	'AS'	4153
Reserved	'AN'	414E

B.1.4.2 Tag hardware type (bytes 10,11)

This is a 2-byte hexadecimal representation of the tag hardware design. This number shall be different for each type of hardware change made to the tag design that affects the function of the tag. This does not include differences in tag packaging, or colour, nor does it include differences in RF operational frequency. This field will be used to distinguish differences in commands or command structure, block size, and data capacity. It will also be used to distinguish differences in data protocol or optional features such as audio, or visual indicators.

B.1.4.3 Embedded application code (Byte 12)**B.1.4.3.1 Embedded application code general**

This is the top-level hierarchy of tag memory layout. This field, in conjunction with the Tag Memory Allocation allows an application to determine the format and content of the user data. This field can be used to represent various formats of the tag data content.

Current hexadecimal assignments of the Embedded Application Code are shown in Table B.4:

Table B.4 — Embedded application codes

Embedded application code	Description of embedded application codes
00, FF	Unformatted, programmed to "FF" at manufacturer.
01	Reserved
02	Customer Specific Memory Allocation
03	File Allocation Table (Long Directory) – TBD in future
04	Check Tag
05	RFID Interrogator Configuration Tag
06	Reserved for future use
07	Reserved for Engineering Development
08	Reserved for future use
09	Reserved for future use
0A	ISO 15962 Compliant Data Format
0B	ANSI MH10.8.4 Compliant Data Format
0C – 0E	Reserved for future used
0F	UCC.EAN GTAG Compliant Data Format (see NOTE)
0C – FE	To be allocated and registered by application based needs.

NOTE GTAG compliant data format is selected by the command WAKEUPGTAG:

GROUP_SELECT_EQ (ADDRESS = '12', BYTE_MASK = '01', WORD_DATA = '0F 00 00 00 00 00 00')

(0x00 0x12 0x01 0x0F 0x00 0x00 0x00 0x00 0x00 0x00 0x00)

B.1.4.3.2 Embedded application code '01' - reserved

All other Tag Memory Allocation Map combinations for Embedded Applications Code "01" are currently considered undefined. These remaining characters may be used to further specify functions and/or applications that have been appended to the primary application.

B.1.4.3.3 Embedded application code '02' - customer specific memory allocation

Customer specific data and file allocation tables are not detailed in this specification. However, it is envisioned that customers must register a Customer Specific Memory Allocation Code (CSMAC) with the manufacturer's marketing and obtain a configuration string to write their two byte CSMAC value as well as an embedded application code of "02" in tag memory location byte 12.

Table B.5 provides examples of specific tag memory allocation fields for Customer Specific Tag Memory applications already being used by customers. It should be noted that this table is not inclusive of all customer specific memory allocations, nor does it represent all the codes registered. It is recommended that the manufacturer be contacted for obtaining a current listing and registering any additional codes.

Table B.5 — Customer Specific memory allocation codes bytes 13 & 14

Hexadecimal code	ASCII representation
4141	'AA'
414E	'AN'
4143	'AS'
4154	'AT'
4650	'GP'
4854	'HT'
4D44	'MD'
5046	'PF'
5354	'ST'

Other Customer Specific Tag Memory applications that may be added in the future include (but are not limited to):

- Theft Detection and Deterrence, or
- Emergency Warning Systems.

B.1.4.3.4 Embedded application code '03' - file allocation table (long directory)

The Tag Memory Allocation for Embedded Applications Code '03', File Allocation Table, has not yet been defined, and is only a placeholder at current. This format will allow a user to view the tag memory similar to that of a floppy drive on a computer.

B.1.4.3.5 Embedded application code '04' - check tag

The Check Tag architecture is made available so the interrogator may selectively read the check tag to verify the type of antenna attached and determines duty cycle and/or output power. This check tag function may also be used for providing end-to-end system diagnostic operational verification. These bytes are programmed and locked at the factory.

B.1.4.3.6 Embedded application code '05' - RFID interrogator configuration tag

The Tag Memory Allocation for Embedded Applications Code '05', RFID Interrogator Configuration Tag, has been reserved to indicate a tag used for configuring a interrogator by means of a special diagnostic configuration mode. This tag can be used for setting various configuration parameters in a interrogator or group of interrogators simply by reading the tag in the configuration mode. Format of this data is to be defined by the interrogator specification or may be added in a future version of this document as an appendix.

B.1.4.3.7 Embedded application codes '06' through '09'

Reserved for future use.

B.1.4.3.8 Embedded applications code '0A' – ISO/IEC 15961 and ISO/IEC 15962 compliant data format

The Tag Memory Allocation for Embedded Applications Code '0A', ISO/IEC 15961 and ISO/IEC 15962 Compliant Data Format, has been reserved to indicate a tag used in a fashion as defined by this part of ISO/IEC 18000. Under the current ISO/IEC 15961 and ISO/IEC 15962 definition byte 13 (Tag Memory Allocation Map) is allocated to retain the Application Family Identifier (AFI) information. Byte 14 is allocated to retain the Data Storage Format (DSF).

B.1.4.3.9 Embedded application codes '0B' – ANSI MH10.8.4 compliant data format

The Tag Memory Allocation for Embedded Applications Code '0B', ANSI MH10.8.4 Compliant Data Format, has been reserved to indicate a tag used in a fashion as defined by that application standard.

B.1.4.3.10 Embedded application codes '0C through 0E'

Reserved for future use.

B.1.4.3.11 Embedded application codes '0F' – EAN.UCC GTAG compliant data format

The Tag Memory Allocation for Embedded Applications Code '0F', EAN.UCC GTAG Compliant Data Format, has been reserved to indicate a tag used in a fashion as defined by that application standard. Under this definition byte 13 (Tag Memory Allocation Map) is allocated to retain the Application Family Identifier (AFI) information

B.1.4.3.12 Embedded application codes '10' through 'FF'

Reserved for future use.

B.1.4.3.13 Tag memory allocation map (Bytes 13 through 17)

This field is based on a structured hierarchy that allows for an application to determine the format and content of the user data in conjunction with the Embedded Application Code in byte 12 defined above.

B.1.4.3.14 Tag application memory (bytes 18 and above)

These bytes represent the general application data storage area in tag memory.

Annex C
(informative)

Tag Memory Map for Type B

C.1 Tag memory map

Table C.1 describes the overall memory map of the tags.

Table C.1 — Layout of tag memory map

Bytes	Field name	Written	Locked
0-7	Tag ID	Manufacturing	Manufacturing
8,9	Tag Manufacturer	Manufacturing	Manufacturing
10,11	Tag Hardware Type	Manufacturing	Manufacturing
12-17	Tag Memory Layout	Manufacturing or Application	As Required by Application
18 and above	User Data*	Application	As Required

* Definition and Format of User Data determined by Tag Memory Layout

The first eight bytes of the tag memory shall be programmed with unique Tag ID numbers. This field is hard-coded in the ASIC to allow tag sort algorithms to function properly, therefore it is important that these Tag Serial Numbers be unique.

Bibliography

The following material, though not specifically referenced in the body of the present document (or not publicly available), gives supporting information.

- [1] ISO/IEC 18000-1, *Information technology — Radio frequency identification for item management — Part 1: Reference architecture and definition of parameters to be standardized*
- [2] ISO/IEC TR 18047-6, *Information technology — Radio frequency identification device conformance test methods — Part 6: Test methods for air interface communications at 860 MHz to 930 MHz*
- [3] ISO/IEC 15961, *Information technology — Radio frequency identification (RFID) for item management — Data protocol: application interface*
- [4] ISO/IEC 15962, *Information technology — Radio frequency identification (RFID) for item management — Data protocol: data encoding rules and logical memory functions*
- [5] ISO/IEC 15963, *Information technology — Radio frequency identification for item management — Unique identification for RF tags*

Copyright International Organization for Standardization

.....

© ISO/IEC 2004. All rights reserved.
Provided by IHS under license with ISO
No reproduction or networking permitted without license from IHS

ICS 35.040

Price based on 134 pages